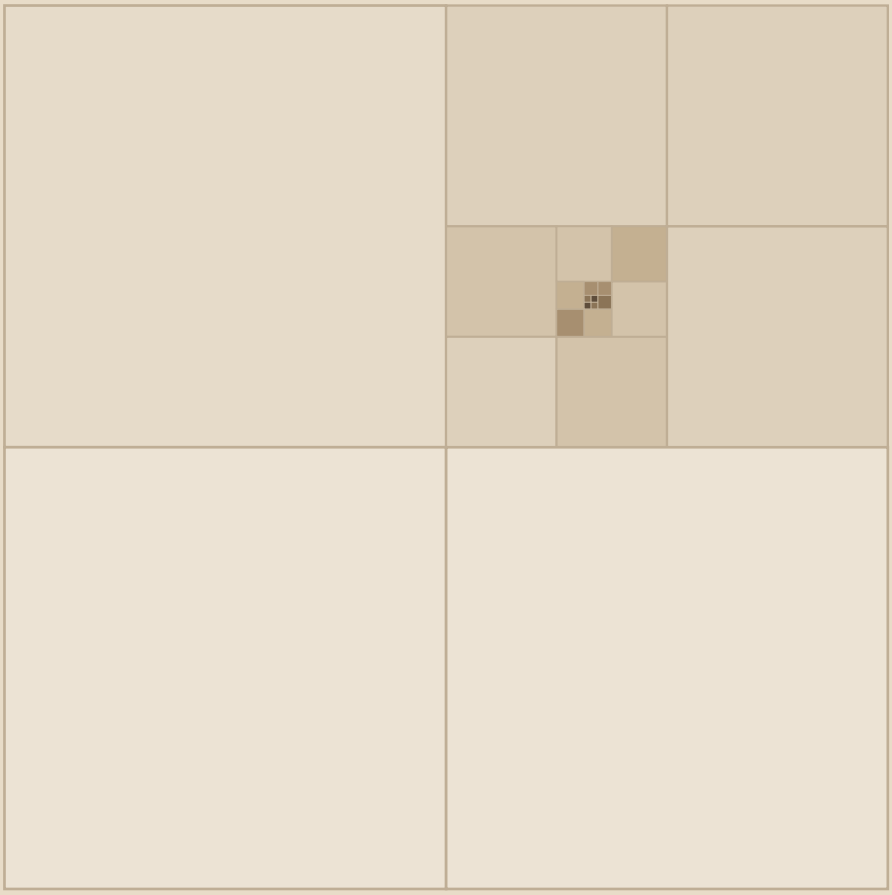
Tessera



The World *in Squares*

A PostGIS vector tile server in C# 14, with ASP.NET Core on .NET 10

‘The world cut into squares, and every square drawn only in the moment a map asks to see it’

Phase Oct Edition · October 2026
Visual Studio Code edition

Tessera at a glance

A PostGIS vector tile server in C# 14 with ASP.NET Core on .NET 10, rebuilt test-first, one story at a time, with the finished repository beside you as the worked answer.

THE JOURNEY OF ONE TILE

1. **The browser**
   MapLibre asks for one square of Macau
   `GET /food.cafes/12/3339/1788.pbf`
2. **Nginx, on port 80**
   office only, a cap for each machine, kept tiles
   `geo · limit_req · proxy_cache`
3. **Kestrel and ASP.NET Core**
   one of two copies matches the route
   `/{layer}/{z:int}/{x:int}/{y:int}.pbf`
4. **Three checks, then the query**
   list loaded, layer published, square on the grid
   `catalogue() · Find · TileAddress.Of`
5. **The query, built pure**
   names quoted, values as parameters
   `Queries.Tile · SqlName.Quote`
6. **The one door to the database**
   a pooled connection; a failure becomes a value
   `Database.Single → Result`
7. **PostGIS**
   finds, clips and packs the cafés
   `ST_TileEnvelope · ST_AsMVTGeom · ST_AsMVT`

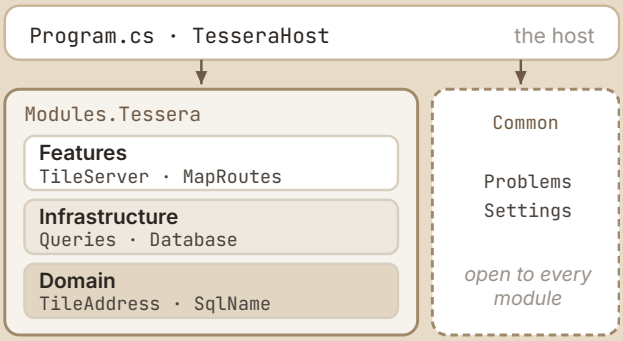


0/0/0, then 1/1/0, then 2/3/1:
each level splits every square into four

SEVEN IDEAS

1. **One program, walled modules**
   one module, three layers, six border rules
2. **A pure core, effects at the edge**
   the core decides; Database alone sends SQL
3. **Data is plain and never changes**
   records, immutable lists, closed cases
4. **A wrong value cannot be built**
   private constructors; settings checked at start
5. **Failures are values**
   a Result or a T?, never a throw
6. **A request is a chain of checks**
   each check returns early; the query comes last
7. **The host wires the parts through one door**
   AddTessera and MapTessera list every part

COMMON, ONE MODULE, THREE LAYERS

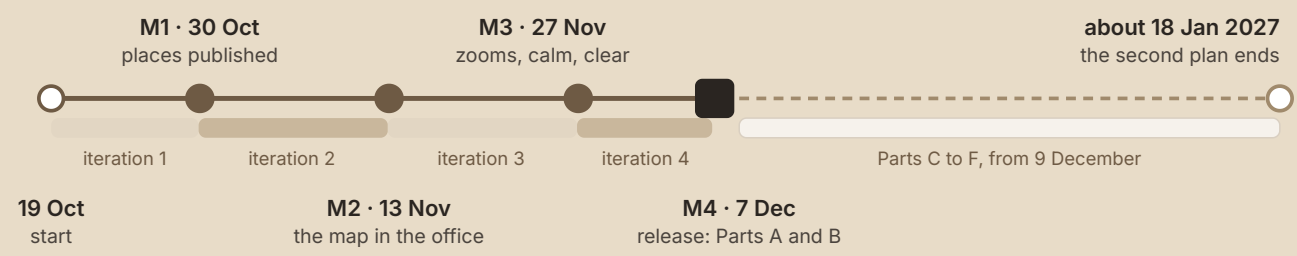


Six ArchUnitNET rules in BoundaryTests guard the borders.

THE BOOK IN FIVE NUMBERS

30	52	4	900	151
user stories	points, each about a day	iterations to the release	places in nine layers	requirements

FOUR MILESTONES, ENDING IN RELEASE



Sources: docs/user-stories/user-stories.md; docs/guides/delivery-plan.md; docs/requirements; ADR 0010; nginx/default.conf.template; Features/Tiles.cs.

How to read this book

> IN SHORT This book guides you as you rebuild Tessera, a map server, one small story at a time, with the finished code beside you as the worked answer. Every part opens with its answer, so you can read it at three depths: the answer, the argument and the proof.

On the way you learn seven ideas that keep a program safe, stable and quick as it grows. This page says whom the book is for, how each part works, what the boxes, story pages, numbers and type mean, what to have beside you and which way to read.

Who it is for. The book is for you: a school student of computer science, aged about 15 or 16. You know variables, selection and iteration, subroutines, arrays and records. You have written SQL with `SELECT`, `WHERE` and `JOIN`, turned numbers into binary and drawn a network of clients and servers. You have written some Python and perhaps a little C#. You have not yet met ASP.NET Core, generics, lambdas, nullable types, Docker, PostGIS or a test framework; the book explains each the first time it needs it, starting from what you know.

Answer first. Every chapter, section and story opens with its answer, then gives the reasons. This is Barbara Minto's ***pyramid principle***: the one idea to take away comes first, as a full sentence; three to five supporting points follow, in the order the opening names them; the detail comes last: code, a table, a drawing, a test. Read the first paragraph of a chapter for the answer, the first sentence of each paragraph for the argument, and the whole for the proof.

The boxes. Labelled boxes set each kind of help apart from the main text.

BOX	WHAT IT HOLDS
In short	A chapter's answer, in one to three sentences, at its start
Key idea	The point most worth remembering
In plain words	A story or an idea told in everyday words, before the detail
Foundations, From school	A link from Tessera to something you learnt at school
Watch out, Common mistake	A mistake that is easy to make and costly to find
Try it	A small task at the keyboard, with what you should see
The proof	A command you run to see for yourself that a claim holds
Note	A known limit, or a fact worth keeping beside the text
What to carry away	The points of a chapter or story to keep once you finish it
Think it over	One open question to carry with you as you move on

The story pages. Chapter 5 holds the work itself: 30 ***user stories***, each a short statement of one thing a person needs from Tessera. Each story has a page or two, in the order you build it. Under the heading stands the story's point in one sentence, then a small table of the requirements it meets and its size. A plain account follows, then the story itself, the criteria that say when it is done, and the tests to write first, named as the repository names them. Section 5.1 takes one page apart.

Numbers and type. Section 2.4 is the fourth section of chapter 2. Appendices carry letters, so F.2 is a section of appendix F. An *exhibit* is a table or drawing that makes one point; it takes its chapter's number and its place, such as Exhibit 2.4, and its title states the point as a sentence. Bold italic marks a term where the book explains it; the glossary in appendix G gathers them all. Code, file names and test names appear `like this`, exactly as the repository writes them.

This edition. This edition uses Visual Studio Code; appendix F sets it up and guides you through your first sessions at the keyboard.

What to have beside you. You need two things: the repository and a computer that runs Docker. The ***repository*** is the folder that holds Tessera's finished code, tests and documents. It is your ***worked answer***, like the answers at the back of a mathematics book: read it after you have tried a story, never before. ***Docker*** runs other programs in sealed boxes; the tests start a real database that way. Appendix E lists the rest: the .NET 10 SDK, Node.js 26, uv for the Python pipeline, and Visual Studio Code, all on Ubuntu 24.04 inside WSL2 on Windows 11. A companion book, *Sixteen Tiles*, explains each technology Tessera uses, one to a chapter; this book points to it for more depth.

Two ways through. You may read straight through, since the chapters come in the order you need them. Or you may read chapters 1 to 4, for the answer, the ideas and the method, then build the stories of chapter 5, turning to chapters 6 to 9 and the appendices when a story sends you there. Either way, read chapter 1 first: it holds the whole book in a few pages.

Contents

How to read this book 3

1 **The book in brief** 8
The whole book on a few pages: the product, the ideas, the path and the proof

1.1 What you will build 8
1.2 Seven ideas in one page 9
1.3 The path: 30 stories in four iterations and after 10
1.4 How you will know it works 10
1.5 Your first two sessions 11

2 **What Tessera does** 13
One question, asked all day: which places lie in this square of the world?

2.1 The world in squares 13
2.2 Who uses it 16
2.3 Tessera in the office 16
2.4 One tile's journey 18
2.5 What makes it hard: security, stability, speed 19

3 **Seven ideas, one program** 21
One module, three layers, and the habits that keep whole kinds of mistake out

3.1 One program, walled modules 21
3.2 A pure core, effects at the edge 23
3.3 Data is plain and never changes 25
3.4 A wrong value cannot be built 27
3.5 Failures are values 29
3.6 A request is a chain of checks 31
3.7 The host wires the parts through one door 32
3.8 The seven together 33

4 **How you will work** 36
Test first, a story at a time, in four iterations to the release

4.1 Test first, with few tests 36
4.2 Which tests to write 38
4.3 Every test names its requirement 40
4.4 A session at the keyboard 42
4.5 Stories, points and iterations 43
4.6 The definition of done 44

5 **The stories, one at a time** 46
Thirty stories, each test-first, in the order of the iterations

5.1 How a story page works 46
5.2 Who the stories speak for 46
5.3 Iteration 1 · The skeleton and the places 47
5.4 Iteration 2 · Safe tiles, and the map in the office 56
5.5 Iteration 3 · The map reads well 67
5.6 Iteration 4 · Clusters, names and the release 75
5.7 Part C · Tiles that cost less 83
5.8 Part D · Steady when the database is not 89
5.9 Part E · Nginx guards the door 103
5.10 Part F · Clean places 107
5.11 After the release 110

6 **Running it in the office** 111
Two containers, two copies of the service, two tasks that repeat

6.1 Two containers and two copies of the service 111
6.2 The first start 112

6.3 The scripts that repeat 113
6.4 A weekly check 114
6.5 What each error means 114

7 **Proving it is secure, stable and fast** 118
One model for every limit, fourteen checks at the edge, and the gaps left honest

7.1 One model for every limit 118
7.2 Load: what exists and what does not 120
7.3 Attack: a scanner, run by hand 121
7.4 The edge's fourteen checks 122
7.5 From requirement to evidence 123

8 **Decisions and risks** 126
Ten records of why, the stack part by part, the threats and the risks to your path

8.1 What a decision record is 126
8.2 The ten records behind Tessera 127
8.3 The stack, and why each part 128
8.4 The threat model in brief 130
8.5 Risks to your path, and the remedies 131

9 **What comes next** 133
Four gaps, seven study plans, and the habits that outlast the stories

9.1 The gaps 133
9.2 The study plans in `docs/study` 134
9.3 How to keep learning 135
9.4 A closing thought 136

A **The requirements** 137
What the service does, how well it does it, how it defends itself

A.1 How to read a requirement 137
A.2 What the service does 138
A.3 How well it does it 139
A.4 How it defends itself 140
A.5 Where the tests stand 141

B **Nginx, the front door** 143
Thirteen jobs at the door, the file that does them, and the check behind each

B.1 What Nginx does for Tessera 143
B.2 The parts no story page quotes 144
B.3 Which check proves which job 145

C **C# for Python programmers** 147
Records, nullable types, pattern matching, LINQ, async and await, and Result, each beside its Python cousin

C.1 Static types, and `var` 147
C.2 Records 148
C.3 Nullable types, in place of `None` 149
C.4 Pattern matching, and closed unions 150
C.5 Lambdas and LINQ 150
C.6 `async` and `await` 151
C.7 `Result`, and the generics behind it 152
C.8 Immutable collections 153
C.9 What Tessera leaves out 153

D **.NET attributes, and how ASP.NET Core wires everything together** 154
Labels that tools read, and a wiring that reads as plain code

D.1 What an attribute is, and who reads it 154
D.2 Every attribute in the code, counted 155
D.3 Settings: limits the validator reads at start-up 156
D.4 [Closed]: a label for the analyser 157
D.5 Test attributes 157
D.6 Attributes the build writes for you 158
D.7 From Program.cs to TesseraHost.Build 159
D.8 The service collection: every registration and its lifetime 160

D.9 Options: binding tessera.json and the environment, and checking them 163
D.10 Hosted services: work that starts and stops with the program 164
D.11 Endpoints, filters and the order of the middleware 165
D.12 One request for 12/3339/1788, through the objects DI built 166
D.13 Attributes Tessera avoids, and why 168

E **The toolchain** 170
Four runtimes on one Ubuntu, and a version named in every file that uses it

E.1 The toolchain in one picture 170
E.2 The .NET SDK and global.json 171
E.3 The shared build files, and the analysers they switch on 172
E.4 The test platform: xUnit v3 on Microsoft.Testing.Platform 173
E.5 Docker: the images and what each runs 174
E.6 Node.js and uv, for the parts outside the service 175
E.7 Every version in one place 176
E.8 What stops when a tool is missing 177

F **Ten sessions in VS Code: set up and rebuild the skeleton** 179
One session builds the machine, eight rebuild three stories test-first, and the tenth reviews them

F.1 The ten sessions at a glance 179
F.2 Three facts about VS Code 180
F.3 The shape of every session 180
F.4 Session 1 · The machine 181
F.5 Session 2 · US-00, an empty solution that answers 183
F.6 Session 3 · US-00, the build rules, the door and the first wall 184
F.7 Session 4 · US-01, one database image and two roles 186
F.8 Session 5 · US-01, the same database for every test 186
F.9 Session 6 · US-00, the settings and the start refused 187
F.10 Session 7 · US-07, a tile that cannot be off the grid 188
F.11 Session 8 · US-07, the SQL and the one door to the database 188
F.12 Session 9 · US-07, the route, the seed and the walking skeleton 189
F.13 Session 10 · Review, and compare with the worked answer 191
F.14 Where your rebuild may differ 191

G **Glossary** 193
Every concept the book explains, in a sentence or two

1
The book in brief

You can build a safe, stable and quick map server yourself, one small story at a time, if you hold to seven ideas and write each test before its code. This chapter gives the whole book in a few pages, so you know where every later page fits.

Five points support that claim, in the order you meet them. First, the product is simple to state: Tessera turns the address of one square of the world into one database query and one small file of shapes. Second, seven ideas keep the code small, safe and easy to test, each preventing one kind of trouble. Third, the work splits into 30 stories: 16 of them, in four iterations, reach the office on 7 December 2026, and the other 14 follow. Fourth, six kinds of check show at each step whether the service works, and each of 151 written requirements names the check that proves it. Fifth, two sessions at the keyboard start you off. Chapters 2 to 5 give the service, the ideas, the method and the stories; chapters 6 to 9 give the running, the proof, the decisions and what comes next.

1.1 What you will build

Tessera is a web service that serves a map in squares to every machine on one office's network. Each square, a ***vector tile***, is a small file that holds the shapes inside one square of the world at one level of zoom. The browser loads the squares it needs and draws them. ***PostgreSQL***, a database, stores the shapes, and ***PostGIS***, an extension to it, teaches it to work with shapes on the globe. Tessera asks PostGIS to pack the shapes of each square into a tile, and sends the tile back over HTTP.

The service is written in C# 14 on .NET 10 with ***ASP.NET Core***, Microsoft's library for web services, which answers each address with a short function of your own. The .NET tools build it into one program, `Tessera.dll`, which the command `dotnet Tessera.dll` runs. The repository holds three more parts, shared with JTessera, the Java build of the same service: the map, written in TypeScript (JavaScript with types) with React and MapLibre GL JS; a Python ***pipeline***, which cleans files of places and turns them into SQL scripts; and TypeScript scripts that build, start and test the whole. Tessera borrows one rule from pg_tileserv, an open-source tile server: the database decides what the map shows, so a table becomes a map layer only when the database lets the service read it.

Tessera runs on one Windows 11 machine in the office, in Ubuntu 24.04 inside WSL2, which runs Linux within Windows, and serves the office network alone. ***Nginx***, a program that receives every request first and passes it on, stands at the front door on port 80. It refuses machines outside the office, caps what each machine may ask, keeps each tile for a minute, and shares the work between two copies of the service. The service publishes 900 places of Macau, in nine layers, and the map shows them on a plain background.

EXHIBIT 1.1 Tessera answers five kinds of request, and every one only reads

ADDRESS	WHAT IT RETURNS
`/{layer}/{z}/{x}/{y}.pbf` (or `.mvt`)	One vector tile of one layer; an empty body where the square holds nothing
`/`	The map of the area, Macau, with all nine layers

`/{layer}.html`, such as `/food.cafes.html`	The same map, opened on one layer
`/assets/…`	The map's scripts, styles and fonts, built by `npm run build`
`/health`	`{"status":"UP"}` when the service can serve tiles, or 503 ('Service Unavailable') with `DOWN` or `OUT_OF_SERVICE`

Source: `Features/Tiles.cs`, `Features/Map.cs`, `Features/Health.cs`. The tile, page and health routes answer GET and HEAD alone; any other method gets 405.

1.2 Seven ideas in one page

Seven ideas hold Tessera's code together, and each prevents one kind of trouble that grows with a program. Chapter 3 takes them in this order, from the shape of the whole program down to the way its parts join. Exhibit 1.2 gives each in one line, with the trouble it prevents and one place in Tessera that shows it.

EXHIBIT 1.2 Each idea prevents one kind of trouble, and Tessera shows each in a place you can read

#	IDEA	THE TROUBLE IT PREVENTS	WHERE TESSERA SHOWS IT
1	One program, walled modules	A change in one corner breaks another far away	`Common` and one module, `Modules.Tessera`, in three layers; six rules in `BoundaryTests`
2	A pure core, effects at the edge	Rules you can test only with a database running	`TileAddress.Of`, `SqlName.Quote` and `Queries.Tile` decide; `Database` alone sends SQL
3	Data is plain and never changes	Data changed under a reader, or a case forgotten	Records such as `Layer`, with immutable lists; `DatabaseError`'s four closed cases; `LiveCatalogue` swaps the whole list of layers at once
4	A wrong value cannot be built	A check forgotten in one of many places	Private constructors: `TileAddress.Of` gives a square only on the grid, `SqlName.Quote` a name only quoted; every setting checked as the program starts
5	Failures are values	An exception nobody catches, or a null nobody checks	`Result` holds the answer or the reason; `Database` turns the database's exceptions into values; a `T?` says a value may be missing
6	A request is a chain of checks	One long handler nobody can test in parts	`TileServer.Answer`: the list, the layer, the square, each refusing early, then one query
7	The host wires the parts through one door	A program whose shape is spread across every file	`TesseraModule`'s `AddTessera` and `MapTessera` list every part; `TesseraHost` calls them

Source: chapter 3; `docs/adr/0001`, `0004` and `0010`; `src/Tessera`.

One method binds the seven: ***test first***. You write a test, a small method that checks one answer of your code, and watch it fail before you write the code that makes it pass: one or two tests for each rule, each chosen to fail when the rule breaks. Each test's name states its rule, such as `Domain_stands_alone`, and each requirement names the test that proves it. ***The ideas make the code easy to test, and the tests keep the ideas honest.*** Chapter 4 shows the method at work.

1.3 The path: 30 stories in four iterations and after

You build Tessera in 30 stories, and the calendar decides which of them reach the office first. A story is one thing a person needs, written in a sentence, with criteria that show when it is done. A ***point*** measures its size: about one working day, tests included. The 30 stories come to 52 points, but the calendar from 19 October to 8 December 2026 holds only 30 days for stories. So the plan builds Parts A and B, the service and the map, worth 29 points, in four ***iterations***, blocks of about two weeks that each end in a review, and keeps one day in hand. Parts C to F, worth 23 points, follow from 9 December.

EXHIBIT 1.3 Four iterations take you from an empty folder to a release on 7 December

ITERATION	DATES	THEME	STORIES, IN ORDER	POINTS
1	19–30 Oct	The skeleton and the places	US-00, US-01, US-03, US-24; US-07 begun	8
2	2–13 Nov	Safe tiles, and the map in the office	US-07, S-01, S-06, US-35, US-23	8
3	16–27 Nov	The map reads well	UI-09, UI-15, UI-12, UI-19; UI-10 begun	9
4	30 Nov–7 Dec	Clusters, names and the release	UI-10, UI-11, UI-17; S-07 if the day in hand is free	4
After	9 Dec to about 18 Jan	Parts C to F	N-18 to N-29, S-07 if still to build, S-30, US-36	22 or 23
			30 stories	52

Source: `docs/guides/delivery-plan.md`; `docs/user-stories/user-stories.md`. *A story that crosses a review counts in both iterations.*

The first iteration builds a ***walking skeleton***: the thinnest version of the whole service that works from end to end. Every later story adds flesh to that skeleton and never breaks it. The service comes before the map, since the map has nothing to draw until the service serves a tile. The defences come before the office, so no office machine ever meets an unguarded service. Each later story of the map teaches it one more thing to show.

Four ***milestones*** mark the way, each a state of the product that someone could see and test. M1, on 30 October, reads 'The places are published by grant'. M2, on 13 November, reads 'The map opens in the office, on safe tiles'. M3, on 27 November, reads 'The map zooms, keeps calm and tells places apart'. M4, on 7 December, reads 'Parts A and B released to the office'. At each review you compare the points of the finished stories with the points planned, so a slow week shows early, while you still have time to act.

1.4 How you will know it works

You will know Tessera works because six kinds of check say so, each from a different distance. They reach from the rules inside the program, through the map and the pipeline, to the whole service behind Nginx in a real browser. One command, `npm test`, runs all six. GitHub runs them again whenever new code reaches the branch `main`, and on every ***pull request***, a request to merge new code into it.

xUnit, a test framework for .NET, runs the service's tests, and they are few on purpose: one or two for each rule. They live in three test projects, each a small program that runs its own tests. In the run this book was checked against, they ran 167 cases, and all passed: 6 architecture tests, which guard the walls between the parts; 94 unit tests, which need no database; and 67 integration tests, which start the packaged

database in Docker and ask the running service for real tiles. Together they ran 96.1 per cent of the service's lines and 85.5 per cent of its branches. That share is the *coverage*, and `npm test` fails below a floor of 94 and 84 per cent; chapter 4 shows how.

EXHIBIT 1.4 Six kinds of check prove Tessera from the inside out

CHECK	WHAT IT PROVES	HOW YOU RUN IT
Three xUnit projects	The borders between the layers; the rules of each layer, with no database; the service against the packaged PostGIS over real HTTP; together, a floor of coverage	`dotnet run --project tests/Tessera.ArchitectureTests`, then `UnitTests` and `IntegrationTests` the same way
The map's tests	Its types and style rules, its model and pure `update`, its style, controls and colours, without a browser	`npm test` in `web/`
The pipeline's checks	Its types, its style, that its pure code reads no file, each filter, the stop rules and the SQL it writes	`uv run pyright`, `ruff`, `lint-imports` and `pytest -q` in `etl/`
The scripts' tests	The scripts' types, and their pure parts, such as the settings and the restart waits	`npx tsc --noEmit`; `node --test "scripts/**/*.test.ts"`
The browser tests	The map opens through Nginx, draws its tiles and matches approved screenshots	`npm run e2e`
The Nginx checks	Office only, the caps, one query for a crowd, kept tiles, 304, gzip, HEAD, malformed addresses, and a second copy taking over	`scripts/lib/edge.ts`, inside `npm run e2e`

Source: `scripts/test.ts`; `.github/workflows/build.yml`; `scripts/lib/edge.ts`; `README.md`. *GitHub runs the screenshot comparisons inside Playwright's own Docker image, whose browser draws text as the approved images do; no GitHub run has proved that step yet.*

A written list ties the checks to what Tessera must do. The folder `docs/requirements` holds 151 ***requirements***, sentences the finished service must make true. Of these, 58 are functional (FR-) and say what it does; 67 are non-functional (NFR-) and say how well; 26 are for security (SR-). Each comes from one acceptance criterion of a story and names the test that proves it. Some still lack a full test: 19 say 'No test yet', and 21 more are proved only 'in part'. Where a chapter meets one, it turns the gap into an exercise.

> KEY IDEA A check you cannot run is a hope. Every claim this book makes about Tessera ends in a command you can type and an answer you can see, or in a gap it names.

1.5 Your first two sessions

Your first two ***sessions***, each one sitting at the keyboard, set up your machine and start the first story; appendix F guides you through both and the sessions after them. By the end of the second you will have run the worked answer's tests and written your own first failing test.

1. **Session 1: the machine and the worked answer.** Install WSL2 with Ubuntu 24.04, Docker Engine, the .NET 10 SDK, Node.js 26, uv and Visual Studio Code. Then, in the repository, run `npm install` and `npm test` with Docker running. The session ends when every check passes, which proves your machine before you write any code.

2. **Session 2: your own solution and its first test.** Start your own .NET solution for US-00, the walking skeleton, with one project for the service and one for its tests. Write the story's first test, 'The skeleton answers', watch it fail, and write the least code that makes it pass.

> TRY IT Start the worked answer with `npm start`, then ask Nginx for one tile of cafés:
>
> ```bash
> curl -s -o tile.pbf -w '%{http_code}\n' http://127.0.0.1/food.cafes/12/3339/1788.pbf
> ```
>
> You should see `200`, HTTP's code for success, and a file `tile.pbf` beside you. Now ask for `/food.cafes/1/2/0.pbf`. You should see `404` ('Not Found'), because at level 1 the columns run only from 0 to 1.

How you use the worked answer decides what you learn. Open it only after you have tried a story. At the end of each day, compare your code with Tessera's, keep the better of the two, and write down what the comparison taught you. If a story runs a day past its points, stop, read the worked answer's version, take what you need, note the difference and go on.

> WATCH OUT Your tests can slide into copying the repository's, line for line. The story pages give you the test names on purpose; the bodies must be yours. Close the repository's file before you write a test.

> THINK IT OVER The calendar holds 30 days for 52 points, so 14 stories wait for a second plan. If you could move one story from that second plan into the first, which would you choose, and which story of Parts A and B would you give up for it?

2
What Tessera does

Tessera answers one question all day: which places lie in this square of the world? It turns the square's address into one database query and one small file of shapes. The hard part is to do it safely, steadily and quickly for every machine in an office.

Five points explain what Tessera does. First, a web map loads the world in squares, and three numbers name each square. Second, five kinds of people use Tessera, and while you build it you are two of them. Third, Tessera runs on one office machine, and only Nginx faces the office. Fourth, one tile passes seven stops between the browser and PostGIS, and pure functions, which compute and touch nothing else, make every decision on the way. Fifth, hostile input, sudden crowds and a failing database make the job hard, and the seven ideas of chapter 3 answer them.

2.1 The world in squares

A web map never downloads the whole world. It loads only the squares it needs, called ***tiles***, and three numbers, z, x and y, name each one. This section shows how the world becomes one square, how that square splits into tiles, what a tile holds and how PostGIS makes one.

2.1.1 One square for the whole world

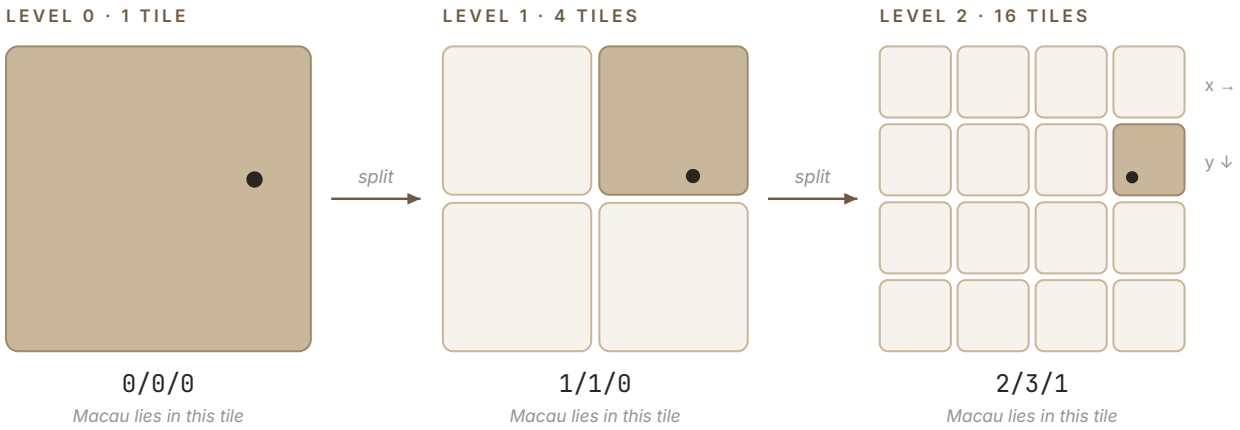
The globe is round and the screen flat, so every web map must stretch the globe. Nearly all use ***Web Mercator***, a projection: a rule for drawing the round globe on a flat page. It unrolls the globe like a label peeled from a tin, and keeps the shapes of small areas true at the price of stretching land near the poles: Greenland looks as large as Africa. It cuts the map off at about 85 degrees north and south, where the map becomes exactly as tall as it is wide. The whole world then fits one square, measured in metres east and north of the point where the equator crosses the meridian of Greenwich. Its code number is EPSG:3857.

Tessera's tables store each place in a different system, ***WGS 84***: longitude and latitude in degrees. Its code number, its ***SRID*** (spatial reference identifier), is 4326. The tile query converts between the two: it turns each place into Web Mercator as it packs the tile, and the tile's square into degrees to search the table. The last part of this section shows why that second turn matters.

2.1.2 Each level splits every square into four

At level 0 one tile covers the whole world. Each step of zoom halves the width of a tile, so each tile splits into four. At ***zoom level*** z the world is therefore 2^z tiles wide and 2^z tiles tall. The tiles stack by level into a ***tile pyramid***: one at the top, four beneath it, sixteen beneath those.

EXHIBIT 2.1 The world is one square at level 0, four at level 1 and sixteen at level 2



Source: the tile grid; Macau's home view, 113.55° E, 22.19° N, from `tessera.json`*. Each address reads level, then column from the west, then row from the north.*

The address of a tile is z/x/y. The level, z, says how many times the world has split. The column, x, counts tiles from the west edge, and the row, y, from the north edge, both starting at 0. At level 1, tile 1/0/0 is the north-west quarter and tile 1/1/1 the south-east. Follow Macau down the pyramid in Exhibit 2.1: it lies in tile 0/0/0, then 1/1/0 in the north-east quarter, then 2/3/1. At level 12 it lies in tile 12/3339/1788, the square drawn on this book's cover. The map of Macau opens at level 15.

EXHIBIT 2.2 Every step of zoom multiplies the tiles by four

LEVEL Z	TILES ACROSS (2^z)	TILES IN ALL (4^z)	WIDTH OF ONE TILE AT THE EQUATOR
0	1	1	The whole world, about 40,075 km
2	4	16	About 10,000 km
12	4,096	16,777,216	About 9.8 km: the square on the cover
15	32,768	about 1.07 billion	About 1.2 km: the map of Macau opens here
22	4,194,304	about 17.6 trillion	About 9.6 m: the deepest level Tessera serves by default
30	1,073,741,824	about 1.15×10^{18}	About 3.7 cm: the most Tessera will ever allow

Source: the tile grid, with the equator's 40,075 km; `Tiles.MaxZoom` *in* `tessera.json`*;* `Domain/Tiles.cs`*.*

> FROM SCHOOL The tile grid is binary at work. At level 12 the world is 2^{12} = 4,096 tiles across, so x runs from 0 to 4,095: exactly the numbers that fit in twelve bits. `TileAddress.Of` computes 2^z as `1 << z`, a left shift of the bit pattern 1 by z places, and refuses any x or y outside 0 to $2^z - 1$. Level 30 is the limit because 1 << 30 still fits in a 32-bit `int`; one more shift would reach the sign bit and turn the number negative. The tests `Accepts_a_tile_that_exists` and `Refuses_a_tile_outside_the_grid` check the rule at its edges.

2.1.3 A tile holds shapes, not pixels

A tile can hold a picture or a list of shapes, and Tessera's tiles hold shapes. A ***raster tile*** is a picture, a grid of coloured pixels. The server chooses every colour as it draws, so a change to the map's style means

drawing every tile again. A vector tile holds the shapes instead, each with its properties, such as a name. Tessera packs them in the ***Mapbox Vector Tile*** format, a compact binary format whose files end in `.pbf` or `.mvt`. Inside a tile, each shape's position sits on a grid of 4,096 by 4,096 cells, the setting `Tiles.Resolution`.

The browser draws the shapes itself, with a library called MapLibre GL JS. One set of tiles can therefore serve any style, text stays sharp at every size, and the files stay small. Tessera's places are all points: cafés, restaurants, markets, parks, playgrounds, museums, libraries, train stops and bus stops. Each carries some twenty properties, such as its name in three languages, its address and its opening hours, and the tile carries them all.

2.1.4 PostGIS does the hard work

PostGIS stores shapes and answers questions about them, such as 'which cafés lie in this square?' (*Sixteen Tiles*, chapter 8). Tessera's own work is to choose the right question and send it. One SQL statement, built from three PostGIS functions, makes each tile. `ST_TileEnvelope` gives the square of a tile address, in Web Mercator metres. `ST_AsMVTGeom` moves each shape onto the tile's grid and clips it at the edge. `ST_AsMVT` packs the rows into one tile. Here is the statement as `Queries.Tile` writes it for a table of cafés with three properties and a key, the text a test keeps as its approved copy:

```sql
SELECT ST_AsMVT(tile.*, @layer, @resolution, 'tessera_shape') FROM (
  SELECT ST_AsMVTGeom(ST_Transform(t."geom", 3857), ST_TileEnvelope(@z, @x, @y),
                      @resolution, @buffer, true) AS tessera_shape, "group", "layer", "name"
  FROM "food"."cafes" AS t
  WHERE t."geom" && ST_Transform(ST_TileEnvelope(@z, @x, @y, margin => @margin), @srid)
  ORDER BY t."id"
  LIMIT @limit
) AS tile
```

Two kinds of gap wait in that text, and the service fills each in a different way. Each word that begins with `@`, such as `@z`, is a ***parameter***: a slot the database fills with a value sent beside the statement, never pasted into its text. Each name in double quotes, such as `"food"."cafes"`, sits in the text itself, since a name cannot travel as a parameter; the service takes it from the database's own list, quoted by `SqlName.Quote`, never from the request.

The rest of the statement keeps each tile quick and steady. The `&&` operator asks whether the boxes around two shapes overlap, and the table's ***spatial index*** answers fast: like the index at the back of a book, it records where each shape lies, so the database need not read every row. That is why the query turns the tile's square into the table's system and leaves the shapes alone; turn every shape instead, and the index no longer applies, so every tile reads the whole table. The `margin` widens the square by a buffer of 256 cells, so a symbol near an edge shows on both tiles. `ORDER BY` keeps the *features*, the shapes with their properties, in the order of the table's key, so the same tile always comes out the same, and `LIMIT` caps a tile at 10,000 of them. For `food.cafes` at 12/3339/1788, the tile carries 54 cafés: 39 inside the square and 15 in its margin.

Publishing a layer needs no code. The service signs in to the database as `tessera_reader`, a ***role*** that may only read. A table becomes a ***layer*** only when the data publisher ***grants*** that role the right to read it. The service asks the database for the list of tables it may read, its ***catalogue***, when it starts and again every 30 seconds, the setting `Catalogue.RefreshSeconds`. A grant therefore shows on the map within half a minute, and a withdrawn grant takes the layer away as quickly.

2.2 Who uses it

Five kinds of people use Tessera, and while you build it you are two of them: the maintainer who keeps it running and the data publisher who feeds it. The stories call each kind a ***persona***, and every story begins 'As …', naming one. Exhibit 2.3 takes them from the outside in: the people who look at the map, those who build on it, and those who feed, run and repair it.

EXHIBIT 2.3 Five personas use Tessera, and the maintainer asks the most of it

PERSONA	WHO THEY ARE, AND WHAT THEY NEED	THEIR STORIES	POINTS
Stakeholder	One of the office's staff, who judges the map as a whole: a map of the area, with nothing to install	US-23	3
Visitor	Anyone who opens the map, a visitor who sees colour poorly among them: a calm map that zooms as other web maps do, tells places apart, keeps names readable and stays quick on a busy day	UI-09, UI-15, UI-12, UI-19, UI-10, UI-11, UI-17, N-20, S-30, N-29 (10)	18
Map developer	Whoever writes a map against the tile address: a tile by layer and coordinates, and a polite 503 when the database cannot answer	US-07, N-22 (2)	4
Data publisher	Whoever owns the places, signing in as `tessera_publisher`: tables published by grant, files of places loaded as SQL scripts, a new grant served without a restart	US-03, US-24, N-23, US-36 (4)	10
Maintainer	Whoever builds, runs and repairs Tessera, which while you rebuild it means you: one build, one database, safe SQL, a door for the office, kept tiles, one settings file, a health check and a service that stays stable	US-00, US-01, S-01, S-06, US-35, N-18, N-19, N-21, N-24, N-25, N-26, N-28, S-07 (13)	17
		30 stories	52

Source: `docs/user-stories/user-stories.md`, each story's 'As …' and its size.

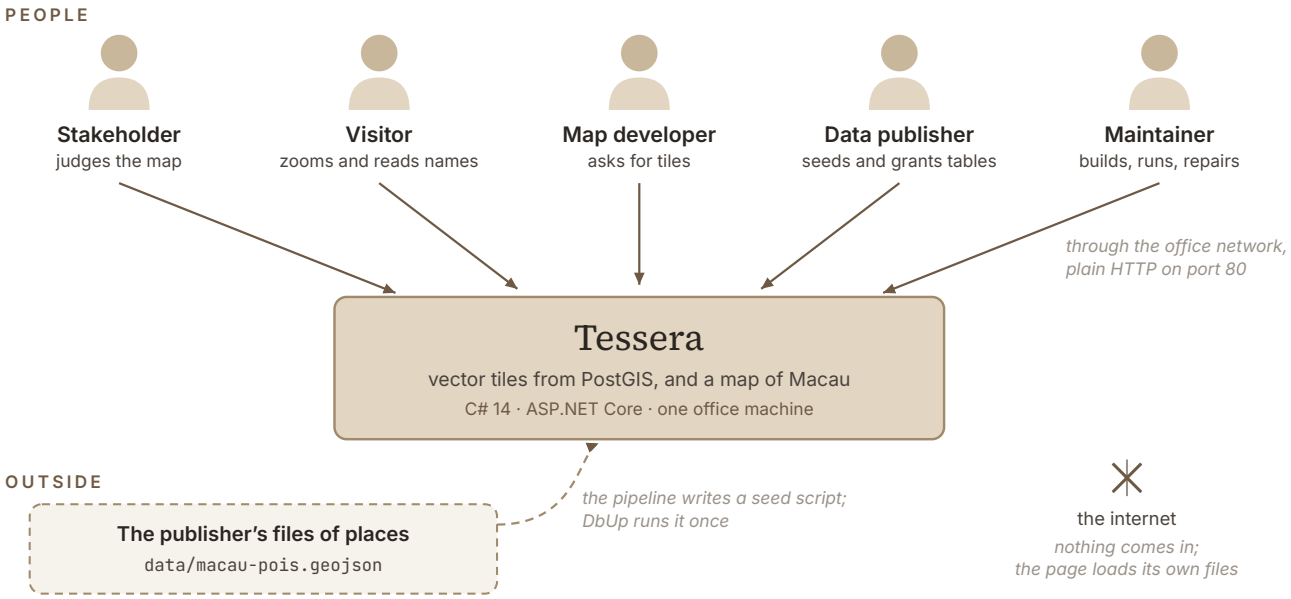
2.3 Tessera in the office

Tessera runs on one Windows 11 machine in the office, and only one of its parts, Nginx, faces the office network. This section draws the service at two levels of zoom, after the ***C4 model***: Simon Brown's way of describing software in diagrams that zoom in step by step, like a map that shows first a city, then a district. The first diagram, the ***context***, shows who uses the system and what it relies on. The second shows its ***containers***: the parts that run on their own or hold data.

2.3.1 The context: people and the world outside

Tessera sits between five kinds of user and one outside source of data, as Exhibit 2.4 shows. The users reach it through the office network; nothing reaches it from the internet. The places arrive as files the data publisher owns, such as `data/macau-pois.geojson`. The pipeline turns each file into a SQL script, and DbUp, a library that runs database scripts in order, runs each script once, when the service's `seed` command starts it. Nothing else comes from outside while the map runs: its scripts, fonts and icons ship inside the build, and the page's rules let it load only its own files, so the map draws its places on a plain background, with no base map from another site.

EXHIBIT 2.4 Five kinds of user reach Tessera through the office network, and one source of data feeds it

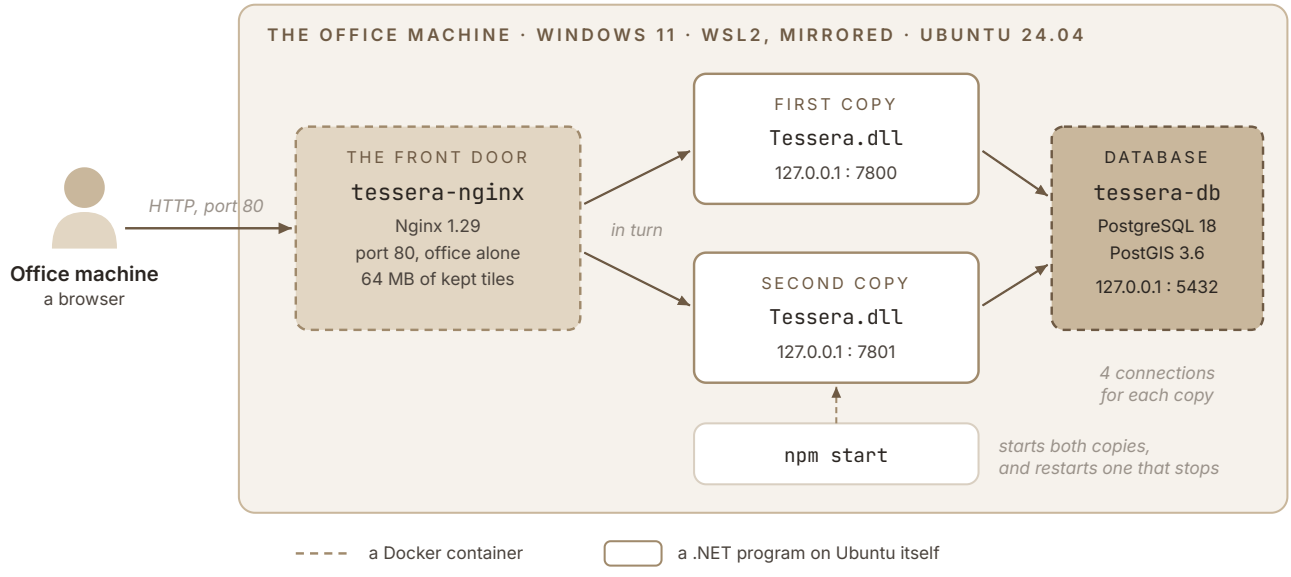


Source: the personas, `docs/user-stories/user-stories.md`*;* `Features/Map.cs`*, its Content Security Policy; ADR 0009.*

2.3.2 The parts on the machine

Four parts run on the office machine, and only one of them listens on the office network (Exhibit 2.5). The machine runs Windows 11, which hosts Ubuntu 24.04 through ***WSL2***, the Windows Subsystem for Linux: a light virtual machine (a computer that Windows imitates in software). WSL2 runs in ***mirrored networking***, in which Ubuntu shares the Windows machine's own network address. A request to the Windows machine on port 80 therefore reaches Nginx in Ubuntu directly. Nginx also sees the address of the office machine that sent it, which it needs to count each machine's requests fairly.

EXHIBIT 2.5 Four parts run on one machine, and only Nginx listens on the office network



Only Nginx listens on the office network; every other part answers on 127.0.0.1 alone.

Source: `scripts/start.ts`*;* `scripts/lib/plan.ts`*;* `nginx/Dockerfile`*;* `db/Dockerfile`*;* `tessera.json`*.*

The two copies are the same program, `src/Tessera/bin/Release/net10.0/Tessera.dll`, run twice by `dotnet`. If one crashes, Nginx sends its requests to the other while `npm start` starts the crashed copy

again after a second, and doubles the wait after each quick failure, up to 30 seconds. Each copy keeps a pool of four connections to the database, so the two together hold at most eight. In this book's examples the office's network is `192.168.10.0/24`, the setting `TESSERA_OFFICE_NETWORK`. Chapter 6 says how each part runs and recovers after a failure.

2.4 One tile's journey

One tile passes seven stops between the browser and PostGIS, and the order of the stops is the design. Follow the request `GET /food.cafes/12/3339/1788.pbf`, the cafés in the square on the cover. Each stop either decides or acts. ***The deciding stops are pure functions, and the acting stops sit at the edges, where the tests can replace them.***

The 'At a glance' page at the front of the book draws the seven stops. Stops 1 to 3, the network and the web server, form one edge. Stops 4 and 5, the core, decide and touch nothing. Stops 6 and 7, the other edge, act on the database. The tile travels back with headers, labelled lines sent before its bytes, that say how long anyone may keep it.

1. The browser asks. MapLibre works out which squares cover the screen and asks for each square of each layer switched on. The map builds each address on the page's own site, `/{layer}/{z}/{x}/{y}.pbf`, from a list of nine layers in its own code, so it never asks the service for a list. The path carries the layer's name and the tile's address, and nothing else.

2. Nginx receives it. Nginx first checks the sender: a machine outside the office's network hears 403 ('Forbidden') before any other rule runs. It then counts each machine's requests by its address. Each office machine may ask for 100 tiles a second, with room for a sudden burst of 300, and may keep 32 requests open at once. Past either limit it hears 429 ('Too Many Requests'). If Nginx holds a copy of the tile built within the last minute, it answers at once from its store, runs no query, and says so in the header `X-Cache-Status: HIT`. Otherwise it passes the request to one of the two copies, in turn. If many machines ask for the same new tile at once, Nginx lets the first request through and holds the rest until its answer arrives, so the crowd costs one query.

3. Kestrel and ASP.NET Core take it. ***Kestrel***, the web server inside every ASP.NET Core program, receives the request on `127.0.0.1:7800`. The service first reads the address exactly as sent and refuses, with 400, one that hides a dot folder as `%2e` (S-06). ASP.NET Core then matches the path to the tile route, `/{layer}/{z:int}/{x:int}/{y:int}.pbf`. The `:int` marks let a part match only when it reads as a whole number that fits a 32-bit `int`, so `/food.cafes/x/0/0.pbf` or a level of ten digits matches no route at all, and answers 404 as a problem document. The route hands four values to `TileServer.Answer`: `"food.cafes"`, 12, 3339 and 1788, with a token that tells the code if the browser gives up. `Answer` is an ***async*** method: where it waits for the database it writes `await`, and its thread serves other requests meanwhile (*Sixteen Tiles*, chapter 6).

4. Three checks decide. Each check either lets the request go on or answers at once with a problem, and the rest never run. The first asks whether the list of layers has loaded; until it has, the answer is 503. The second, `Find`, looks the layer up by its exact name; a layer nobody published is 404, whatever square it names. The third lets `TileAddress.Of` check the three numbers against the grid and the levels the settings allow: a square off the grid is 404 too. Only when all three pass does a query exist. Section 3.6 shows the checks as code.

5. The pure core builds the query. `Queries.Tile` fills the statement shown in section 2.1. Its names come from the `Layer` that the catalogue made, each already quoted as a `SqlName`. Its parameters take the numbers, gathered in one small object beside the text: the layer's name, z, x, y, the resolution, the buffer,

the margin, the SRID and the limit. Nothing in this step touches the database, the clock or the network, so a test can check it with no database at all.

6. The edge acts, through one door. `Database` is the one class that sends SQL. Its method `Single` borrows a connection from the pool, sends the statement through Dapper, a small library that runs a query and reads its answer, and returns the tile's bytes inside a `Result`. A request waits at most three seconds for a connection, and the database stops any query still running after ten seconds. Whatever error the database raises, `Database` catches it at once and turns it into a `DatabaseError`, one of four kinds: a missing table, a missing column, a database that cannot answer now, or anything else.

7. PostGIS packs the tile. `ST_TileEnvelope`, `ST_AsMVTGeom` and `ST_AsMVT` find, clip and pack the features into one tile. Tessera answers 200 with the type `application/vnd.mapbox-vector-tile`, a `Cache-Control` header that lets anyone keep the tile for 60 seconds, and a weak `ETag`, a short fingerprint made from the tile's bytes. A square that holds nothing still answers 200, with an empty body. On the way out, Nginx keeps the tile and shrinks it with gzip.

Every failure on the way takes a second road: the check that fails returns a problem, and the steps after it never run. The visitor receives a ***problem document***: a short answer in JSON, a plain text format for data, in the shape that the standard RFC 9457 sets. It holds a type, a title, the status and, where it helps, a sentence of detail. It never repeats the visitor's input, and it carries `Cache-Control: no-store`, so no browser keeps it. One class, `Common/Api/Problems.cs`, gives every problem document its shape, whichever route or failure made it.

2.5 What makes it hard: security, stability, speed

Tessera's task is plain: one square in, one query, one file out. Yet any office machine may send anything, at any rate; the database behind it can serve only a few queries at once; and the staff watch the map fill in while they wait. You build the service alone, and each of its 151 requirements needs evidence that it holds. How, then, can one student build a service that stays safe, stable and quick, and prove it? ***Let the code enforce each defence, so that none depends on memory, and let a test show that each defence holds.***

Security begins at the door. A ***hostile input*** is any request built to make the service misbehave. The most dangerous is ***SQL injection***, in which text from a request ends up inside a SQL statement, where the database runs it as a command. Tessera makes it impossible to write. It uses a request's layer name only to look the layer up; the name that enters the SQL comes from the database's own list, quoted by `SqlName.Quote`. `SqlName`'s constructor is private, so the compiler refuses `new SqlName(…)` anywhere else, and no unquoted name can reach a query. The tile's numbers travel as parameters. A test of the program's shape, `Only_Infrastructure_talks_to_the_database`, fails if any code outside the Infrastructure layer uses the database's libraries. S-01's page lists the tests that prove it. Nginx refuses a machine outside the office and an address it cannot read. One gap remains: the office reaches Tessera in plain HTTP, with no encryption, and the study plans in `docs/study` make adding it an exercise.

Stability means saying no early and clearly. A database serves a few queries well and many queries badly. Tessera therefore gives each copy four connections, makes other requests wait their turn for at most three seconds, and answers 503 with a `Retry-After` header when the wait runs out. Every limit is a setting, and the limits nest. Nginx waits 20 seconds for a copy, longer than the service's worst case, a 3-second wait and a 10-second query together, which the service holds to on a clock of its own; so the visitor always hears Tessera's own answer before Nginx gives up. If a copy crashes, Nginx sends its requests to the other; if both stop, Nginx still serves every tile it holds. `DatabaseErrorTests` checks that eight of PostgreSQL's error codes each land in the right one of the four kinds, and so earn the right answer.

Speed comes from doing less. One statement makes each tile, and PostGIS does the heavy work close to the data. Nginx keeps each tile for a minute, so a visitor who moves the map back costs nothing, and a crowd that opens the map at nine o'clock costs one query a tile. Tessera sets no measured target for speed yet, and no test measures it; section 7.2 makes measuring it an exercise.

The defences share a few shapes: a value that cannot be wrong, a failure that travels as a value, a pure core with its effects at the edge, and a chain of checks that stops at the first failure. Chapter 3 names these shapes as seven ideas and shows each one at work.

> THINK IT OVER Nginx keeps each tile for 60 seconds. A publisher who grants a new table sees it on the map within 30 seconds of the grant, but a tile already kept still shows the old places for up to a minute. Which number would you change first if the office asked for quicker updates, and what would that cost the database?

Check your understanding

1. How many tiles cover the world at level 3, and what range of values can x take there?
2. Why does the tile query turn the tile's square into the table's own system, rather than turning every place into Web Mercator before it searches?
3. A request names a layer `food.secret` that exists in the database but was never granted. What does the visitor receive, and which check decides it?
4. Which part of Tessera listens on the office network, and on which addresses do the other three parts listen?
5. Name the two kinds of gap in the tile statement, and say how each is filled.

Answers. 1. $2^3 \times 2^3 = 64$ tiles; x runs from 0 to 7. 2. So that the spatial index on the table's shapes applies; turning every shape would make each tile read the whole table. 3. A 404 problem document; `Find`, the second check, decides, since the catalogue lists only granted tables. 4. Nginx, on port 80; the two copies on 127.0.0.1, ports 7800 and 7801, and the database on 127.0.0.1, port 5432. 5. Parameters, such as `@z`, take values sent beside the statement; names, such as `"food"."cafes"`, enter the text quoted by `SqlName.Quote`, from the database's own list.

3
Seven ideas, one program

Seven ideas hold Tessera together, and each makes one kind of mistake impossible or cheap to find: one program, walled modules; a pure core, effects at the edge; data is plain and never changes; a wrong value cannot be built; failures are values; a request is a chain of checks; and the host wires the parts through one door. When an idea breaks, a test, the compiler or a review catches the slip. This chapter takes the ideas in order of structure: two shape the program, three govern its values, and two join its parts. Each section states the rule, the trouble it prevents, how C# 14 says it, Tessera's own code, a trap, and the stories that use it. The last section follows one tile through all seven and names two decisions beside them. *Sixteen Tiles*, chapters 1 to 3, explains the language behind each idea at greater length.

3.1 One program, walled modules

Build one program, cut it into modules with walls between them, and let a test fail the build whenever code crosses a wall. Tessera is a ***modular monolith***: one program, run as one process and shipped as one file, `Tessera.dll`, divided inside into ***modules***. It takes its shape from a published modular-monolith template. `Common` holds what every module may share, and `Modules/Tessera` holds the one module, which serves tiles, the map and `/health`. Three ***layers*** divide the module again: `Domain` for plain values and rules, `Infrastructure` for the database, and `Features` for the web routes.

Without walls, any class may call any other, and a change in one corner breaks another far away. Think of a block of flats: one building, one roof and one water main, yet each flat has a front door. A modular monolith keeps the shared building and adds the doors. Separate programs talking over a network would give stronger walls, at the price of several releases on one machine and a network call wherever a method call stood (ADR 0001, the written record of that decision).

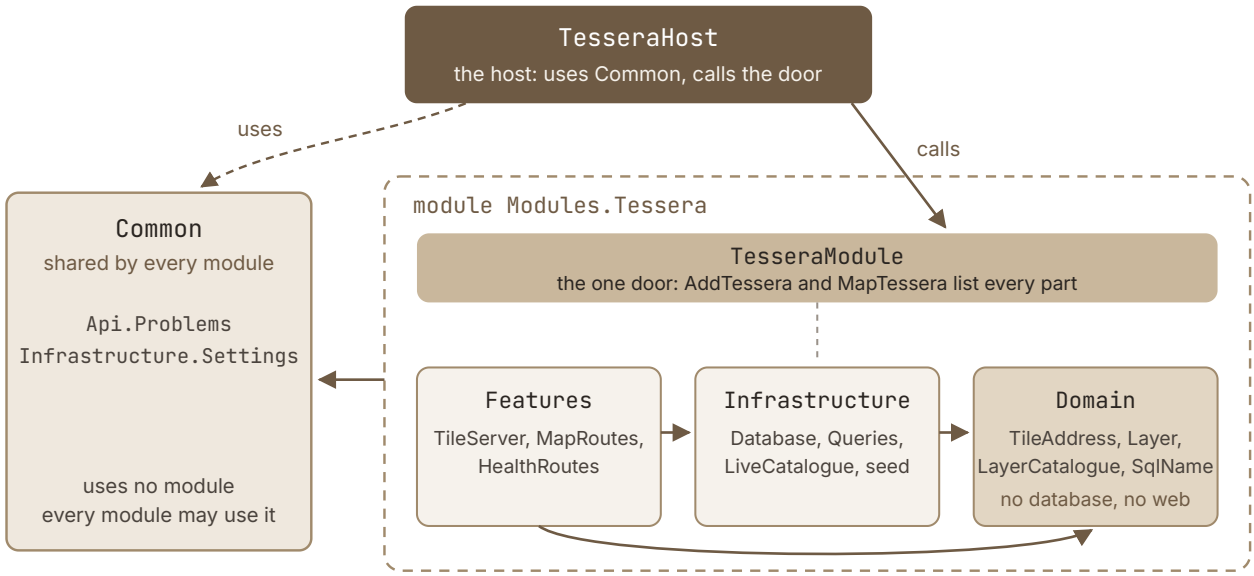
C# groups classes in ***namespaces***, dotted names such as `Modules.Tessera.Domain`. Its word `internal` means 'any code in this program may use this', and Tessera is one program, so the language draws no wall between its layers; two habits draw them. First, each namespace matches its folder: `Tessera.csproj` sets no root namespace, and the build treats rule IDE0130, 'namespace does not match folder', as an error. Second, six tests read the built program and check which namespace uses which. ***ArchUnitNET***, a library that tests rules about which classes use which, writes each rule as a sentence of code:

```csharp
// tests/Tessera.ArchitectureTests/BoundaryTests.cs
// The types in a namespace and in every namespace inside it, so
// 'Modules.Tessera.Infrastructure' covers 'Modules.Tessera.Infrastructure.Sql' too.
private static GivenTypesConjunction In(string space) =>
    Types().That().ResideInNamespaceMatching($@"^{Regex.Escape(space)}(\..+)?$");

// Rule 1: the Domain holds records and pure functions, so it needs no database, no web, and no
// other layer of its module (ADR 0004).
[Fact]
public void Domain_stands_alone() =>
    OursIn("Modules.Tessera.Domain").Should().NotDependOnAny(Workings)
        .AndShould().NotDependOnAny(Database).AndShould().NotDependOnAny(Web)
        .Check(Program);
```

`Workings` names the module's Infrastructure and Features, `Database` the libraries Npgsql, Dapper and DbUp, and `Web` ASP.NET Core. Exhibit 3.1 draws the host, `Common`, the module and its three layers, and the only ways one part may lean on another.

EXHIBIT 3.1 Common, one module, three layers, and the only ways one may lean on another



An arrow means 'may use'. Six ArchUnitNET rules fail the tests on any other.

Source: `src/Tessera`; `tests/Tessera.ArchitectureTests/BoundaryTests.cs`; *ADR 0010.*

Start at the right: the domain leans on no other layer, no database and no web, and every other layer may lean on it. Infrastructure may use the domain but never the web; features may use both. `Common` serves every module and reaches none. The six rules live in `BoundaryTests`, the first of the service's three test projects to run, so a broken wall shows before any other test of the service fails.

EXHIBIT 3.2 Six rules in one class keep the program's shape

TEST IN `BOUNDARYTESTS`	WHAT FAILS IT
`Domain_stands_alone`	A domain type that uses Infrastructure, Features, Npgsql, Dapper, DbUp or ASP.NET Core

`Only_Infrastructure_talks_to_the_database`	Any type outside Infrastructure that uses Npgsql, Dapper or DbUp (S-01 c2)
`Infrastructure_knows_no_web`	An infrastructure type that uses ASP.NET Core or Features
`Common_never_reaches_a_module`	A type in `Common` that uses anything under `Modules`
`Other_modules_reach_Tessera_through_PublicApi_alone`	Another module that reaches Tessera past its `PublicApi`; it waits for a second module
`PublicApi_shows_no_workings`	A type in `PublicApi` that uses Infrastructure or Features; it waits for the folder's first type

Source: `tests/Tessera.ArchitectureTests/BoundaryTests.cs`*; ADR 0010, 'The rules, and what guards each'.*

The last two rules guard against a quiet failure of their own. With one module, no type yet falls under them, and ArchUnitNET fails a rule that checks nothing, so each says `.WithoutRequiringPositiveResults()`: it passes empty now, and starts to bite the day a second module joins.

> WATCH OUT The rules read the built program, not the source. A `using Npgsql;` line that nothing uses leaves no trace in the built program, so it breaks no rule. A field, a parameter or a call does leave a trace, and the rule catches it. A rule also sees only the types it has loaded, which is why `BoundaryTests` loads the program together with the libraries it uses.

> TRY IT Add one method inside `SqlName`, in the domain, that builds an `Npgsql.NpgsqlConnectionStringBuilder` from a string and returns its `Host`. It compiles. Run `dotnet run --project tests/Tessera.ArchitectureTests`: two of the six tests turn red, `Domain_stands_alone` and `Only_Infrastructure_talks_to_the_database`, each naming `Modules.Tessera.Domain.SqlName does depend on "Npgsql.NpgsqlConnectionStringBuilder"`. Delete the method and the six pass again.

The walls stand from the first story. US-00 lays out the folders, and S-01 c2 rests on the second rule of Exhibit 3.2. ADR 0010 records why one project with tests won over a project for each layer, which would let the compiler guard the walls: the tests guard the same borders without three or four project files to keep.

3.2 A pure core, effects at the edge

Let pure functions make every decision, and keep the code that touches the world thin, at the edge, and free of choices. A ***pure function*** gives the same output for the same inputs and changes nothing outside itself: no database, no clock, no network, no file. Tessera follows a style named ***functional core, imperative shell***: pure functions decide, and a thin edge of code that gives orders acts on their decisions. In the tile route the core checks the address, finds the layer and writes the SQL; the edge sends that one query.

You can test code that decides and acts at once only with its world around it: to check that the service refuses level 23, you would need a database, a seeded table and a running server, so the tests stay slow, few and fragile. Think of a satnav. It plans the route, and a driver turns the wheel; you can test the planning at a desk against a thousand made-up trips without moving a car. The planner is the core, the driver the edge.

> FROM SCHOOL A flowchart from school has two kinds of box that matter here: the diamond, which decides, and the parallelogram, which reads input or writes output. Tessera's core holds every diamond. The edge holds the parallelograms, and as few of them as the job allows.

C# has no keyword for purity, so Tessera keeps it by habit and checks part of it by rule. The domain is pure by construction: `Domain_stands_alone` forbids it any library that could reach the database or the web. Beyond the domain, pure decisions live in `static` methods, which belong to a class rather than an object and take and return values: `Queries.Tile`, `CatalogueReader.FromRows` and `DatabaseError.Of`. The one dangerous step, the database, reaches the tile route as a function handed in from the edge. A ***delegate*** names the shape of such a function:

CSHARP

```csharp
// src/Tessera/Modules/Tessera/Features/Tiles.cs
public delegate Task<Result<byte[]?, DatabaseError>> TileReader(
    Query query, CancellationToken cancel);

public sealed class TileServer(
    Func<LayerCatalogue?> catalogue, TileReader read, TileSettings settings, ILogger log)

// … and in TileRoutes.MapTiles, the edge hands in the real database and log:
var server = new TileServer(
    () => catalogue.Current,
    database.Single<byte[]>,
    settings,
    routes.ServiceProvider.GetRequiredService<ILoggerFactory>().CreateLogger("Tessera.Tiles"));
```

The program hands `TileServer` the real database's `Single` method and a real log; a test hands it a stand-in that answers whatever the test needs, with no database at all, and a log that writes nowhere. `TileTests` does so to prove that a dropped column answers 503:

CSHARP

```csharp
// tests/Tessera.UnitTests/TileTests.cs
var server = new TileServer(
    () => CatalogueReader.FromRows(
        [new CatalogueRow("food", "cafes", "geom", 4326, "POINT", "[]")]),
    (_, _) => Task.FromResult(Result.Failure<byte[]?, DatabaseError>(
        DatabaseError.Of("42703", "column does not exist"))),
    Limits,
    NullLogger.Instance);
```

The edge itself, `Database`, does one thing: it opens a connection, runs the work on a clock of its own, and turns whatever the database throws, or a clock that runs out, into a value. It never chooses a layer, a level or a column:

CSHARP

```csharp
// src/Tessera/Modules/Tessera/Infrastructure/Sql/Database.cs (line breaks moved)
private async Task<Result<T, DatabaseError>> Run<T>(
    Func<NpgsqlConnection, CancellationToken, Task<T>> work, CancellationToken cancel)
{
    try
    {
        return Result.Success<T, DatabaseError>(
            await Within(_limit, limit => RunOnce(work, limit), cancel).ConfigureAwait(false));
    }
    catch (NpgsqlException error)
    {
        return Result.Failure<T, DatabaseError>(Error(error));
    }
    // …
    catch (Exception error)
        when (error is TimeoutException or OperationCanceledException
            && !cancel.IsCancellationRequested)
    {
        return Result.Failure<T, DatabaseError>(
            DatabaseError.Of(PostgresErrorCodes.QueryCanceled, "The query ran past its time."));
    }
}
```

`RunOnce` borrows a connection from the pool and runs the work on it. `Within` gives the work at most `_limit`, the wait for a connection and the query's own limit together, 13 seconds as shipped: a database frozen in mid-query can hold Npgsql's own limits for longer, and Nginx stops waiting at 20 seconds.

The same split runs through every layer: `CatalogueReader.FromRows` and `DatabaseError.Of` decide, while `LiveCatalogue` acts on a timer; `TileServer.CacheControl`, `TileServer.Tag` and `MapRoutes.Render` decide, while the routes answer.

The split pays in test time: the 94 unit cases, which need no database, finish in a few seconds, while the 67 integration cases, which start PostgreSQL in Docker, take about a minute.

> WATCH OUT The rules see which types a class uses, not what it does. A call to `DateTime.Now` breaks no rule, yet two runs could then give two answers. `LiveCatalogue` needs time, so it takes its two waits, `every` and `retry`, and its way to load the list, `load`, as values passed in. The program passes 30 seconds, 5 seconds and the real database; the unit tests pass a stand-in load and a retry of 10 milliseconds, so `Tries_until_the_database_answers` ends in a moment. Pass the clock and the world in wherever you can (ADR 0004, 'referential transparency').

US-03 sets the pattern: `CatalogueReader.FromRows` turns rows into layers, tested with no database by `Publishes_neither_of_two_tables_whose_ids_clash`, while `CatalogueReader.Load` fetches the rows through `Database`. US-07 and S-01 repeat the split for the tile query, and N-23 for the timer.

3.3 Data is plain and never changes

Hold data in plain records that never change, name every case of a choice in one closed list, and let the build check that each `switch` handles each case. Data is plain values with no hidden behaviour, and functions take those values and return new ones.

Data that can change breeds two kinds of bug. Shared, it changes under a reader, and a request sees the list of layers half old and half new. Grown, a choice gains a fifth case that one of ten places forgets to

handle. Think of a printed bus timetable. When the buses change, the company prints a new one, and each old copy stays true for its own dates; nobody rubs out a line on a copy someone else is reading.

A ***record*** is a class whose main job is to hold named values, like a row in a school database. Written with its fields in brackets after its name, a positional record gets from C# a constructor, a property for each field, such as `layer.Id`, comparison by value, and a `with` expression that makes a changed copy and leaves the first alone. Its properties can be set only while it is built. Tessera's lists inside records are ***immutable collections***, `ImmutableArray` and `ImmutableDictionary`, which offer no way to add, remove or replace an item:

CSHARP

```
// src/Tessera/Modules/Tessera/Domain/Layers.cs
public sealed record Layer(
    string Id, SqlName Table, SqlName Geometry, int Srid, string GeometryType,
    ImmutableArray<SqlName> Properties, ImmutableArray<SqlName> Order);

public sealed record LayerCatalogue(ImmutableDictionary<string, Layer> Layers)
{
    public Layer? Find(string id) => Layers.GetValueOrDefault(id);
}
```

Some data is a choice between a fixed set of cases. F#, the language Tessera's maintainer comes from, calls that a union; C# 14 has none yet. Tessera stands in for one with an abstract record, whose cases are records nested inside it, and a `[Closed]` attribute that lists them. An ***analyser***, a small program that runs inside the compiler and adds checks of its own, does the rest: ExhaustiveMatching stops the build when a `switch` over the record misses a listed case. `DatabaseError` has four:

CSHARP

```
// src/Tessera/Modules/Tessera/Infrastructure/Sql/Database.cs
[Closed(typeof(MissingTable), typeof(MissingColumn), typeof(Unavailable), typeof(Other))]
public abstract record DatabaseError(string SqlState, string Message)
{
    public sealed record MissingTable(string SqlState, string Message)
        : DatabaseError(SqlState, Message);
    public sealed record MissingColumn(string SqlState, string Message)
        : DatabaseError(SqlState, Message);
    public sealed record Unavailable(string SqlState, string Message)
        : DatabaseError(SqlState, Message);
    public sealed record Other(string SqlState, string Message)
        : DatabaseError(SqlState, Message);
    // … Of(sqlState, message) sorts each refusal into one case
}
```

Add a fifth record to the list, and `TileServer.Answer` stops building, with the analyser's error EM0003 'Subtype not handled by switch', until its `switch` names the new case: the build remembers each place for you. Inside the `switch`, a pattern such as `DatabaseError.MissingTable _` matches a case by its type, and the ***discard*** `_` says the code needs nothing else from it. One gap remains, and a test closes it: a record could derive from `DatabaseError` without joining the `[Closed]` list, and the analyser would never see it. `Lists_every_kind_as_a_case`, in `DatabaseErrorTests`, compares the list with every record that derives from `DatabaseError`.

One value in the service must change: the list of layers, which `LiveCatalogue` rereads every 30 seconds (N-23). It never changes a list in place: it builds a new `LayerCatalogue` and swaps it in whole with `Interlocked.Exchange`, which replaces a reference in one step that no other thread can see half done;

requests read it with `Volatile.Read`, which always fetches the latest. Each request therefore reads one whole list.

> WATCH OUT A record compares field by field, but each field compares in its own way. An `ImmutableArray` compares by the array it wraps, not by its contents, so two `Layer` records built from the same row, with equal names in two arrays, are not equal; a run of two such records printed `False`. Tessera never compares two layers, so the gap costs nothing today; before you compare records that hold collections, check how each collection compares.

US-03 builds `Layer` and `LayerCatalogue`; S-01 builds `SqlName`; N-22 sorts failures into `DatabaseError`'s four cases; N-23 builds `LiveCatalogue` and its swap.

3.4 A wrong value cannot be built

Shape each type so that a value breaking its rule cannot exist, give the type one door that checks the rule, and check every setting once, as the program starts. The aim has a name, ***making illegal states unrepresentable***: no variable can hold a tile off the grid or an unquoted table name. The door is a ***smart constructor***, a function that checks its inputs and returns either a valid value or no value.

Without such types, code must check a rule wherever a value travels. Suppose a request asks for x = 300 at level 8, where the grid runs from 0 to 255. If only the route checks it, every later step must check again or trust that someone did, and one forgotten check sends a bad number to PostGIS. Think of passport control at an airport: once you pass the desk, nobody in the departure lounge asks for your passport again, because the building has no other way in.

C# shuts the other doors with a ***private constructor***: a constructor that only code inside the type may call. A positional record cannot be a smart constructor, since its constructor is always public. So `TileAddress` is a record with its constructor private, properties that can only be read, and one static function that checks the rule:

CSHARP

```csharp
// src/Tessera/Modules/Tessera/Domain/Tiles.cs
public sealed record TileAddress
{
    public int Z { get; }

    public int X { get; }

    public int Y { get; }

    private TileAddress(int z, int x, int y) => (Z, X, Y) = (z, x, y);

    public static TileAddress? Of(int z, int x, int y, int minZoom = 0, int maxZoom = 22) =>
        z < minZoom || z > maxZoom || z > 30 || x < 0 || y < 0 || x >= 1 << z || y >= 1 << z
            ? null
            : new TileAddress(z, x, y);
}
```

`Of` returns a tile only when the level lies inside the settings and at most 30, and x and y lie on the grid; otherwise it returns `null`, and the `?` on `TileAddress?` makes every caller check. Write `new TileAddress(31, -1, 7)` anywhere else, even in a test, and the compiler refuses it with error CS0122, 'inaccessible due to its protection level'. The properties have no `init`, so a `with` expression cannot copy an address off the grid either. Section 4.1 builds `Of`, test-first.

`SqlName` works the same way. A name reaches the text of a query only as a `SqlName`, and `Quote` wraps it in double quotes and doubles any quote inside, so `x"; DROP TABLE cafes` stays one odd name (S-01 c3):

CSHARP

```
// src/Tessera/Modules/Tessera/Domain/SqlName.cs
public sealed record SqlName
{
    public string Sql { get; }

    private SqlName(string sql) => Sql = sql;

    public static SqlName Quote(string name) =>
        new($"\"{name.Replace("\"", "\"\"", StringComparison.Ordinal)}\"");

    public static SqlName Qualified(string schema, string table) =>
        new($"{Quote(schema).Sql}.{Quote(table).Sql}");
}
```

Where JTessera, the Java build, needs a test to forbid `new SqlName(…)` outside the type, C# lets the compiler do it. A third type follows the pattern with a reason attached: `ConnectionString.FromUrl` turns `DATABASE_URL` into a connection string, or into the reason it could not, such as 'missing or empty'. Holding a `ConnectionString` proves the address was well formed; the code's comment calls the habit 'parse, don't validate'.

Settings: checked once, as the program starts. A setting is a value too, and a wrong one should stop the program before it serves a tile. Each section of `tessera.json` binds to a record with limits on its fields, and a record's own `Validate` checks values that must agree:

CSHARP

```
// src/Tessera/Modules/Tessera/Domain/Settings.cs
public sealed record TileSettings : IValidatableObject
{
    [Range(1, int.MaxValue)]
    public int Resolution { get; init; } = 4096;
    // … Buffer, MaxFeatures

    [Range(0, 30)]
    public int MinZoom { get; init; }

    [Range(0, 30)]
    public int MaxZoom { get; init; } = 22;

    [Range(0, int.MaxValue)]
    public int CacheSeconds { get; init; } = 60;

    public IEnumerable<ValidationResult> Validate(ValidationContext validationContext) =>
        MaxZoom >= MinZoom ? [] : [new ValidationResult("MaxZoom must not be below MinZoom.")];
}
```

The square brackets hold ***attributes***, labels that tools read; `[Range(0, 30)]` tells .NET's checks the lowest and highest value allowed. `Settings.Section`, in `Common`, binds each section, names any key no field reads and any value of the wrong kind, and asks for every check to run at start; `TesseraHost.Build` then runs them all before the service starts. Run the program with `Tiles__MinZoom=5`, `Tiles__MaxZoom=3` and no `DATABASE_URL`, and it prints both problems, 'MaxZoom must not be below MinZoom.' and 'DATABASE_URL: missing or empty', and stops with exit code 1. `SettingsTests` proves the rules:

`Refuses_values_that_disagree`, `Names_every_value_out_of_range_at_once`, `Refuses_a_key_it_does_not_know` and `Names_a_misspelt_key_with_every_other_problem`.

> NOTE A misspelt key, such as `Tiles__CacheSecond`, joins the same list: the program names it with its section, `Tiles: 'CacheSecond' is not a setting`, beside every other problem, so 'every such problem at once' (N-21 c4) holds for keys and values alike; N-21's page gives the run. One limit stands: a misspelt section name, such as `Tile`, passes unseen and keeps its defaults, as `Leaves_a_misspelt_section_at_its_defaults` pins.

S-01 builds `SqlName`; US-07 builds `TileAddress`; US-00 and N-21 build the settings and their checks at start-up.

3.5 Failures are values

Return every expected failure as a value, so its type tells the caller that a step may fail, and join steps so that the first failure carries the work past every step after it. A step that can fail returns a `Result`: either a success, holding the value, or a failure, holding the reason. A step that may find nothing returns a ***nullable type***, such as `Layer?`, which the compiler makes every caller check before use.

C#'s usual way to report a failure is an ***exception***, an object thrown out of a method that jumps up the chain of callers until some code catches it. Exceptions stay out of a method's signature, its first line, so a caller can forget them, and an expected failure, such as an unknown layer, travels the same road as a bug. Picture a railway with points instead. When a signal turns red, the points switch the train to a parallel track, and it runs past every later station to the same terminus, which deals with both kinds of train. Scott Wlaschin named this ***railway-oriented programming***.

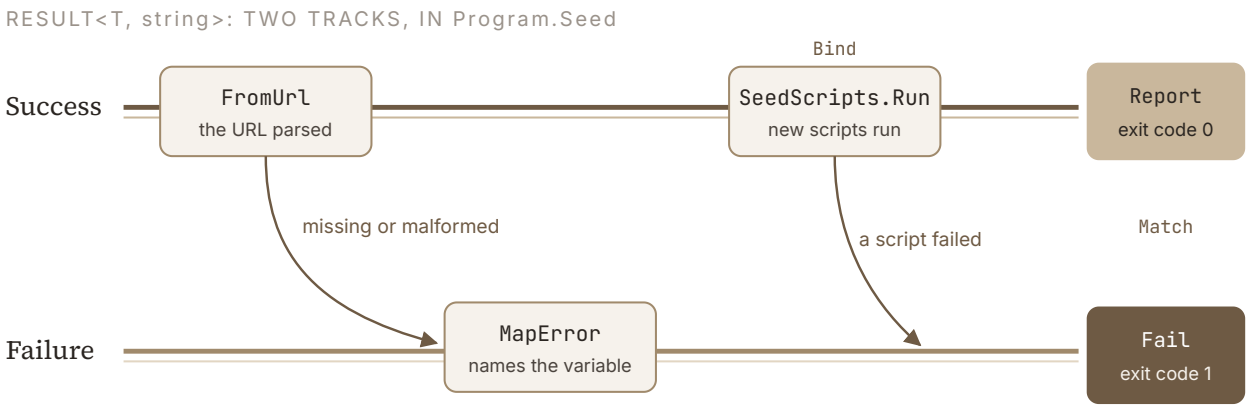
`Result<T, E>` comes from a small library, CSharpFunctionalExtensions, so Tessera need not write or test its own (ADR 0004). It is ***generic***: `T` stands for the value a success carries and `E` for the reason a failure carries. Four of its methods do the work. `Map` applies a function that cannot fail. `Bind` applies a step that can, so its failure joins the second track. `MapError` changes the reason on the second track. `Match` ends the line, with one function for each track. The `seed` command runs its whole job as one such chain:

CSHARP

```csharp
// src/Tessera/Program.cs
return ConnectionString.FromUrl(configuration.Build()[variable])
    .MapError<ConnectionString, string, string>(why => $"{variable}: {why}")
    .Bind(connection => SeedScripts.Run(connection.Text, SeedScripts.Folder))
    .Match(Report, why => Fail([why]));
```

Each `=>` above writes a ***lambda***, a function without a name: `why => $"{variable}: {why}"` takes a reason and returns it with the variable's name in front. Exhibit 3.3 draws the two tracks of this chain.

EXHIBIT 3.3 Railway-oriented programming: the seed's success track, its failure track and the points between them



The first failure throws the points; every later station on the success track is passed by.

Source: `src/Tessera/Program.cs`, `Seed`; *ADR 0004.*

Every failure of a tile request ends as a problem document, with `Cache-Control: no-store` so no browser keeps it. Exhibit 3.4 lists every way a tile request can leave the success track.

EXHIBIT 3.4 Eight ways a tile request can fail, each with one status

WHERE	FAILURE	STATUS	THE PROBLEM'S DETAIL
the route	A level, column or row that is no whole number of 32 bits: no route matches	404	None
`catalogue()`	The list of layers has not loaded yet (N-23 c3)	503	'The list of layers is not ready yet.'
`Find`	No published layer has the name	404	'No published layer has that name.'
`TileAddress.Of`	The square lies off the grid, or outside the levels served	404	'No tile has those coordinates.'
`read`	`MissingTable`: 42501 or 42P01, a grant withdrawn (US-03 c6)	404	'No published layer has that name.'
`read`	`MissingColumn`: 42703, a column dropped since the list was read	503	'The layer has changed. Try again shortly.'
`read`	`Unavailable`: class 08, 57014, 57P01 to 57P03, 53300 (N-22)	503	'The database cannot be reached. Try again shortly.'
`read`	`Other`: anything else, the service's own fault	500	None

Source: `Features/Tiles.cs`; `Infrastructure/Sql/Database.cs`; `Common/Api/Problems.cs`. *A 503 also carries* `Retry-After: 5`.

Most failures arise in the database, and `Database.Run`, shown in section 3.2, is the one place that catches its exceptions. It catches `NpgsqlException`, the database client's own kind, and its own clock running out, and nothing else, so a bug in Tessera's code still escapes as the fault it is. `Error` then sorts each refusal once, by PostgreSQL's five-character ***SQLSTATE*** code, so no caller reads a code again:

```csharp
// src/Tessera/Modules/Tessera/Infrastructure/Sql/Database.cs
private static DatabaseError Error(NpgsqlException error) => error switch
{
    PostgresException refusal => DatabaseError.Of(refusal.SqlState, refusal.MessageText),
    { InnerException: TimeoutException } =>
        DatabaseError.Of(PostgresErrorCodes.QueryCanceled, error.Message),
    _ => DatabaseError.Of(PostgresErrorCodes.ConnectionFailure, error.Message),
};
```

`DatabaseErrorTests` pins the sorting with eight codes, in `Asks_the_browser_to_wait_for_a_failure_that_may_pass` and `Sorts_any_other_failure_into_its_kind`. The 500 names no cause, so a stranger learns nothing, and `TileServer.Fault` writes the cause to the log instead: the layer, the tile, the SQLSTATE code and the database's message, at error level. `Logs_the_cause_of_a_500`, in `TileTests`, proves it with a log that keeps each line.

> WATCH OUT A step that returns a `Result` belongs in `Bind`; any other belongs in `Map`. And watch the library choose for you. The comment above `Program.Seed`'s `MapError` says why it names three types: given a `string`, the library would otherwise pick the version that returns its plain `Result<T>`, and `Bind` would then find no step to run. Never throw for a failure you expect: a step that throws leaves the railway altogether.

US-07 builds the tile route's two tracks and the problem document; US-03 c6 adds the withdrawn grant; N-22 sorts the database's failures; N-23 adds the 503 before the first list loads; US-24 builds the seed's chain.

3.6 A request is a chain of checks

Write each request as a short chain of checks, each refusing early when it fails, so the code reads in the order the request travels and the query comes last. A request begins as text in an address and ends as an HTTP answer, and between the two it passes a handful of checks, each a line you can name, read and test. A check that returns at once when it fails is a ***guard clause***.

The alternative is one long handler that parses, checks, queries and answers in nested `if` statements, where any line can reach any variable and you can test no step alone. Think of a car wash. The car rolls through soak, scrub, rinse and dry, each station doing one job, and a car turned away at the entrance never reaches the dryer.

> FROM SCHOOL In a flowchart from school, a decision can lead to another decision inside its 'yes' branch, and that one to another, until the chart drifts off the right edge of the page. Turn each 'no' into an exit instead, and the chart reads straight down: each diamond either ends the run or lets it fall to the next.

Tessera's seed reads as a railway of `Bind` steps, since it passes one value along. The tile route refuses for several reasons, and ADR 0004 judges that it 'reads better top to bottom, one `if` and one `return` for each refusal':

CSHARP

```csharp
// src/Tessera/Modules/Tessera/Features/Tiles.cs (comments trimmed)
public async Task<IResult> Answer(string layer, int z, int x, int y, CancellationToken cancel)
{
    if (catalogue() is not { } layers)
        return Problems.Unavailable("The list of layers is not ready yet.");
    if (layers.Find(layer) is not { } found)
        return Problems.NotFound("No published layer has that name.");
    if (TileAddress.Of(z, x, y, settings.MinZoom, settings.MaxZoom) is not { } tile)
        return Problems.NotFound("No tile has those coordinates.");

    var built = await read(Queries.Tile(found, tile, settings), cancel).ConfigureAwait(false);
    return built.Match(
        bytes => new TileResult(bytes ?? [], CacheControl(settings.CacheSeconds)),
        error => error switch
        {
            DatabaseError.MissingTable _ =>
                Problems.NotFound("No published layer has that name."),
            DatabaseError.MissingColumn _ =>
                Problems.Unavailable("The layer has changed. Try again shortly."),
            DatabaseError.Unavailable _ =>
                Problems.Unavailable("The database cannot be reached. Try again shortly."),
            DatabaseError.Other _ => Fault(found, tile, error),
            _ => throw ExhaustiveMatch.Failed(error),
        });
}
```

Each of the three checks returns a `T?`, and `is not { } name` both checks it and names the value for the lines after it. The three checks decide; only `read` touches the database, and only after all three pass. US-07's first test asks for that: an unknown layer is refused before any query runs. The database's answer then arrives as a `Result`, and `Match` turns each track into an answer. Exhibit 3.4, in the section before, names each check's way off the success track.

> WATCH OUT The order of the checks is part of the design. `Find` comes before `Of`, so an unknown layer answers 'No published layer has that name.' whatever square it asks for, and a stranger learns nothing about the grid from a name that does not exist. No answer repeats the name the stranger sent. Swap two checks, and every test of a single failure still passes; only a request that breaks two rules at once shows the change.

US-07 lays the tile route's checks; S-01 fixes what `Queries.Tile` may write into the text; N-18 and N-19 add the cache time and the tag in `TileResult`.

3.7 The host wires the parts through one door

Let ASP.NET Core build each part and hand it to the parts that need it, but list every part of a module in one class, so the program's shape is written down in one place. Handing each part of a program the other parts it needs, rather than letting it build them itself, has a name: ***dependency injection***. ASP.NET Core keeps a ***service container***, a table of 'when code asks for this type, give it this object'. A ***singleton*** is an object the container makes once and hands to every part that asks.

The host, `TesseraHost.Build`, builds the program in a dozen lines: it adds the shape of every problem document, calls the module's door, checks every setting, adds the steps every request passes through, and calls the door again for the routes. Appendix D, section D.7, quotes it whole. The door, `TesseraModule`,

holds two methods, named as ASP.NET Core's own `AddHealthChecks` and `MapHealthChecks` are. `AddTessera` adds every service the module needs:

```csharp
// src/Tessera/Modules/Tessera/TesseraModule.cs (trimmed)
public static IHostApplicationBuilder AddTessera(this IHostApplicationBuilder builder)
{
    AddSettings(builder.Services);
    DatabaseConnections.Register(builder.Services, builder.Configuration,
        DatabaseConnections.ReaderDatabase);
    builder.Services.AddSingleton(services =>
        Database.Open(/* … the URL, the pool's limits */));
    builder.Services.AddSingleton(services => new LiveCatalogue(/* … load, 30 s, 5 s, log */));
    builder.Services.AddHostedService(services =>
        services.GetRequiredService<LiveCatalogue>());
    builder.Services.AddHealthChecks()
        .AddCheck<DatabaseAnswers>("database")
        .AddCheck<CatalogueReady>("catalogue");
    return builder;
}
```

Read the door from the top. It binds the module's five sections of `tessera.json` and the database's address, makes the one `Database`, which holds the one pool of connections, and the one `LiveCatalogue`. It runs the catalogue as a ***hosted service***, a loop that starts and stops with the program, and adds the two health checks. `MapTessera` then adds the routes: `MapHealth`, `MapTiles` and `UseMap`, each handed the parts it needs. The `seed` command builds no host at all: it reads one setting, runs the scripts and stops, so none of the module starts. Appendix D follows each call in turn, and Exhibit D.4 lists what each adds to the container.

> WATCH OUT A new route file in `Features` compiles, and its unit tests pass, yet the running service answers its address with a 404 problem document. ASP.NET Core's routes exist only when code adds them, and nothing scans the folders for them. Call the route's `Map` method from `MapTessera`; the door is the one place that says what the module serves.

The style keeps the framework's machinery small and in sight. No class marks a field for the container to fill, no route lives outside the door, and each route takes what it needs as parameters. US-00 starts the host and the door; US-23 adds `UseMap`; N-23 adds the hosted `LiveCatalogue`; N-25 the two health checks.

3.8 The seven together

The seven ideas are one design seen from seven sides, and each has a guard that fails the build, or a review that must catch the slip, when the idea breaks. Follow one request, `/food.cafes/12/3339/1788.pbf`, the square on the cover, which holds the area's home point.

The request reaches Nginx on port 80, which passes it to the service on 127.0.0.1, port 7800. ASP.NET Core hands it to `TileServer.Answer`, the route the module's door added (idea 7). The route is a chain of checks (idea 6): it reads the live list, a `LayerCatalogue` swapped in whole and never changed (idea 3), and finds `food.cafes` in it. It turns 12, 3339 and 1788 into a `TileAddress` through the one checked door (idea 4), and `Queries.Tile`, a pure function (idea 2), writes the SQL with every name a quoted `SqlName` and every number a parameter. Had any check failed, a problem document would have left at once, and a database failure would have come back as a value (idea 5). Only now does the edge act: `Database.Single` sends one

query, from the only layer allowed to (idea 1), and the answer leaves with its type, `Cache-Control: public, max-age=60` and a weak tag. Exhibit 3.5 sets the seven side by side, with the guard on each.

EXHIBIT 3.5 Each idea prevents one kind of trouble, and a test, the compiler or review guards each

IDEA	THE TROUBLE IT PREVENTS	THE C# 14 TOOL	TESSERA'S EXAMPLE	GUARDED BY
1 One program, walled modules	A change in one corner breaks another	Namespaces that match folders	`Common`; `Modules.Tessera` in three layers	`BoundaryTests`, six rules; IDE0130
2 A pure core, effects at the edge	Rules testable only with a database	`static` functions over records; delegates	`Queries.Tile` against `Database`; `TileReader`	`BoundaryTests`; the unit tests, which need no database
3 Data is plain and never changes	Data changed under a reader; a case forgotten	Records; immutable collections; `[Closed]`	`Layer`, `LayerCatalogue`, `DatabaseError`, `LiveCatalogue`'s swap	ExhaustiveMatching; `DatabaseErrorTests`
4 A wrong value cannot be built	A check forgotten in one of many places	Private constructors; `T?`; `[Range]` and `Validate`	`TileAddress.Of`, `SqlName.Quote`, `ConnectionString.FromUrl`, `TileSettings`	The compiler; `TileTests`, `SqlTests`, `SettingsTests`
5 Failures are values	An exception nobody catches; a null nobody checks	`Result` with `Map`, `Bind` and `Match`; nullable types	`Database.Run`; `DatabaseError.Of`; `Program.Seed`	The compiler's null checks; `DatabaseErrorTests`; the integration tests
6 A request is a chain of checks	One long handler nobody can test in parts	Guard clauses; `is not { } name`	`TileServer.Answer`	`TileTests`; `ServiceTests`
7 The host wires the parts through one door	A program whose shape is spread across every file	The service container; extension methods	`TesseraHost`, `TesseraModule`	`ServiceTests`, which start the real host; review

Source: this chapter; `tests/`; ADR 0001, ADR 0004, ADR 0010.

The seven lock together. Purity is cheap because the data never changes; the checks are short because the values cannot be wrong; the walls hold because the door hides nothing. Where C# leaves a gap, as with a record that derives from `DatabaseError` outside its list, the design names it and closes it with a test.

Two decisions sit beside the seven, and keep the service small enough for them to hold. The first gives the door to Nginx, which refuses machines outside the office, caps each machine, keeps tiles, packs them and shares the work between two copies, so the service keeps no limits, retries or cache of its own (ADR 0007, ADR 0008). The second gives the drawing to the browser: the tiles carry plain points, and the map clusters crowded places itself, from the tiles it has loaded (ADR 0005). Chapter 8 weighs both decisions, and *Sixteen Tiles*, chapter 4 and mini-guide M2, shows how each works. Chapter 4 adds the method that keeps the seven in place: write the test first, and write few tests, each chosen to fail when its rule breaks.

> THINK IT OVER Suppose Tessera were a tenth of its size. Which of the seven ideas would you drop first, and which kind of mistake would then come back?

Check your understanding

1. Which tests fail if a class in `Domain` builds an `NpgsqlConnectionStringBuilder`, and why does an unused `using Npgsql;` line fail no test?
2. Why can the unit tests prove that a dropped column answers 503 with no database running?
3. What happens to the build if you add a fifth record to `DatabaseError`'s `[Closed]` list?
4. Why does the compiler refuse `new TileAddress(31, -1, 7)`, and what must a caller write instead?
5. A new route file in `Features` answers 404 in the running service. What did its author forget?

Answers. 1. `Domain_stands_alone` and `Only_Infrastructure_talks_to_the_database`; the rules read the built program, and an unused `using` leaves no trace in it. 2. `TileServer` takes the database as a function, a `TileReader`, and the test hands it a stand-in that returns the failure 42703. 3. It stops: ExhaustiveMatching reports EM0003 in `TileServer.Answer` until its `switch` names the new case. 4. The constructor is private; the caller writes `TileAddress.Of(31, -1, 7)`, which returns `null`, and must check for it. 5. To call the route's `Map` method from `TesseraModule.MapTessera`; nothing scans the folders for routes.

4

How you will work

You build Tessera one story at a time, and each story test first: a test that fails, the least code that passes it, then tidier code that still passes. You write few tests, one or two for each rule, each chosen to fail when its rule breaks and each citing the story criterion it proves. Sixteen stories worth 29 points, in four iterations, bring Parts A and B to the office on Monday 7 December 2026; Parts C to F follow in a second plan.

Professional teams know this chapter's loop well: write a failing test, make it pass, then tidy the code while the test stays green. It keeps each change small and leaves a proof of every claim the service makes. The sections run from the smallest unit of work to the largest: one turn of the loop, the tests worth writing, the thread from each test to its requirement, one session at the keyboard, the stories and iterations, and the checklist that decides when a story is finished.

4.1 Test first, with few tests

Write the test before the code and watch it fail, because only a test you have seen fail can prove it catches a mistake. A ***test*** is a small method that calls your code with chosen inputs and checks the answer. You met the idea at school as a test plan: normal, boundary and erroneous data, each with the result you expect. In Tessera a test is a C# method marked `[Fact]`, which ***xUnit***, the test framework, finds and runs. Each check inside it is an ***assertion***, a line such as `Assert.Null(TileAddress.Of(z, x, y))`, which stops the test and turns it red when the answer is wrong.

Test-first development turns the test plan into a loop of three steps: red, green and refine.

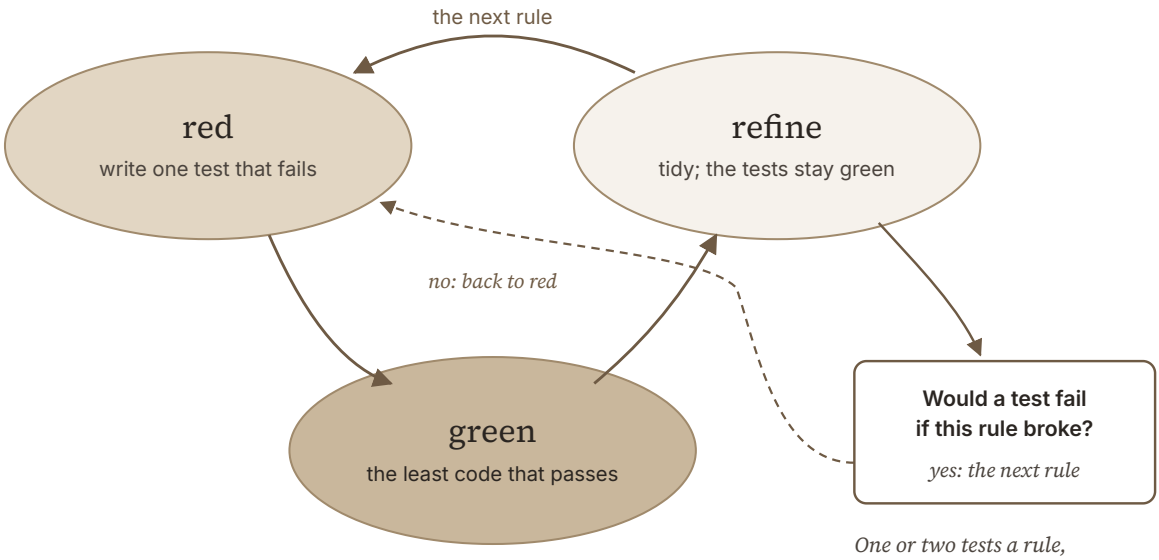
- **Red.** Write one test for one rule, run it and watch it fail. A test you have never seen fail may test nothing at all.
- **Green.** Write the least code that makes the test pass. Leave the next rule alone; it gets its own test.
- **Refine.** Tidy the code: better names, no repetition, a clearer shape. Run the test again. If it turns red, the tidying changed what the code does, and you undo it.

Follow the arrows of Exhibit 4.1 round once for each rule, then answer the question in the box at the right. Yes leads on to the next rule; no leads back to red.

4.1.1 One turn of the loop: a tile's address

A tile's address is three numbers, z/x/y, and one rule governs them. At level z the grid is 2^z tiles a side, so x and y each run from 0 to $2^z - 1$. The level must also lie inside the settings' range and at most 30. The rule has a boundary at each end, the boundary data of your school test plan. The tests that guard it, in `TileTests`, are two ***theories***: methods marked `[Theory]`, which xUnit runs once for each `[InlineData]` row above them.

EXHIBIT 4.1 The test-first loop: red, green, refine, and the question that ends it



One or two tests a rule, each citing its story.

Source: `docs/guides/delivery-plan.md`, *'The weekly and daily rhythm'.*

CSHARP

```csharp
// tests/Tessera.UnitTests/TileTests.cs
// Web-map levels run from 0 to 22 (UI-09), and level z has 2^z columns and rows, numbered
// from 0; so 4194303, 2^22 - 1, is the last column at level 22. Anything past these edges
// names no tile.
[Theory]
[InlineData(0, 0, 0)]
[InlineData(15, 26719, 14311)]
[InlineData(22, 4194303, 0)]
public void Accepts_a_tile_that_exists(int z, int x, int y) =>
    Assert.NotNull(TileAddress.Of(z, x, y));

[Theory]
[InlineData(-1, 0, 0)]
[InlineData(23, 0, 0)]
[InlineData(1, 2, 0)]
[InlineData(1, 0, -1)]
public void Refuses_a_tile_outside_the_grid(int z, int x, int y) =>
    Assert.Null(TileAddress.Of(z, x, y));
```

Red. The first time you run them, the tests fail to build, because `TileAddress` does not yet exist; a test that cannot build counts as red. Read them row by row: each row is z, x and y, and the method's name says whether a tile should exist. The range of levels comes from `Of`'s own defaults, 0 to 22, the shipped `MinZoom` and `MaxZoom`.

Green. The least code that passes is a check of the level and of x and y against the side of the grid, and a value returned only when all hold:

CSHARP

```csharp
// src/Tessera/Modules/Tessera/Domain/Tiles.cs
public static TileAddress? Of(int z, int x, int y, int minZoom = 0, int maxZoom = 22) =>
    z < minZoom || z > maxZoom || z > 30 || x < 0 || y < 0 || x >= 1 << z || y >= 1 << z
        ? null
        : new TileAddress(z, x, y);
```

`1 << z` is a binary shift, the quick way to compute 2^z: it moves the single 1 bit z places to the left. The check `z > 30` comes before the shift, and `||` stops at the first true part, so the shift never runs past the 31 value bits an `int` holds.

Refine. Two refinements make the rule hard to break. The first makes the constructor private, as section 3.4 shows, so no code can build an address past `Of`. The second adds a ***property test***: a test that states a rule for every input and lets a library, FsCheck, try a hundred made-up inputs against it. `Gives_only_addresses_on_the_grid` asks that any address `Of` returns, from any three numbers, lies on its level's grid. Text that is no whole number never reaches `Of`: the route's `{z:int}` refuses it first, as section 2.4 showed. `Answers_a_tile_address_with_a_word_for_a_number_with_404`, in `ServiceTests`, proves it: `zero`, `0x1` and `1.5` in place of a number each get the 404 problem document of any unknown address.

4.1.2 When to stop

Stop when every rule of the story has a test that would fail if the rule broke. The grid rule has seven rows, each guarding one edge or one corner. An eighth row inside the grid, such as `3, 4, 5`, would fail for the same reasons as `0, 0, 0`, adding time to every run and nothing to the proof. ***One or two tests for each rule, each chosen to fail when its rule breaks, prove more than twenty tests chosen by habit.***

4.2 Which tests to write

Test behaviour, test few things, and choose each test for the mistake it catches. The school test plan gives the first sort. ***Normal data*** is a value the rule should accept. ***Boundary data*** sits at the edge of what the rule accepts: the last value inside, or the first outside. ***Erroneous data*** is a value of the wrong kind altogether. Exhibit 4.2 sorts the seven rows of the two tests above.

EXHIBIT 4.2 Seven rows of two tests, sorted as a school test plan sorts them

Z, X, Y	EXPECTED	KIND	WHAT IT PINS
`0, 0, 0`	a tile	Boundary, inside	The one tile of level 0, the lowest level
`15, 26719, 14311`	a tile	Normal	A square in the middle of a level's grid: Macau's home point at level 15
`22, 4194303, 0`	a tile	Boundary, inside	The last column of the deepest level, x = $2^{22} - 1$
`-1, 0, 0`	none	Boundary, outside	A level one below `MinZoom`
`23, 0, 0`	none	Boundary, outside	A level one above `MaxZoom`, 22
`1, 2, 0`	none	Boundary, outside	A column one past the grid, x = 2^1
`1, 0, -1`	none	Boundary, outside	A row one below 0

Source: `tests/Tessera.UnitTests/TileTests.cs`, `Accepts_a_tile_that_exists` *and* `Refuses_a_tile_outside_the_grid`.

> TRY IT The table holds one row of normal data, 15/26719/14311, the square at level 15 that holds Macau's home point, 113.55° E and 22.19° N, as `tessera.json` sets it; normal data comes from the area Tessera serves. Add the square on the cover to `Accepts_a_tile_that_exists`: `[InlineData(12, 3339, 1788)]`, which holds the same point. Predict whether it passes before you run it, then decide whether it guards against any mistake the other rows miss. The table also holds no erroneous data, since a C# `int` cannot carry 'x'; where would such a test belong?

Three questions then decide whether a test earns its place.

Does it test behaviour, not code? ***Behaviour*** is what a caller can see: the answer, the status code, the text of a SQL query. The tests of `TileAddress` call `Of` and check its answer; none looks at how `Of` computes it, so you may rewrite the check however you like. A test tied to the inner workings turns red whenever you refine, even when nothing a caller sees has changed; it fails for the wrong reason, and soon you learn to ignore it.

Which mistake would turn it red? Ask this of every test before you keep it. If no likely mistake would, delete the test. `Keeps_every_tile_address_out_of_the_text` shows the question answered well. It is a property test: for a hundred random tiles it builds the query and checks that the text never changes from the approved copy in `approved/tile-query.sql`. One slip in `Queries.Tile` that pasted a level or a column into the SQL would turn it red.

Is it the cheapest kind that can catch the mistake? A rule decided by a pure function needs a ***unit test***, which calls one function with plain values and needs nothing but .NET. A rule that only the database can keep needs the database. Exhibit 4.3 lists the kinds Tessera holds.

Each kind has its own job. An ***architecture test***, such as `BoundaryTests`, reads the built program and checks its shape. An ***integration test*** runs Tessera's code against a real PostgreSQL, and the fullest of them start the whole service and talk to it over HTTP. The database runs in a ***container***, a packaged and isolated program that Docker starts in seconds. ***Testcontainers*** is the library that starts it from the test and removes it afterwards. `DatabaseFixture` builds that container from `db/Dockerfile`, the same image, or packaged program, that the office runs, and the tests sign in as the same role, `tessera_reader`.

EXHIBIT 4.3 The service's sixteen test classes, and the tests beside them, by what each needs

PROJECT AND CLASS	WHAT IT PROVES	TESTS (CASES)	NEEDS
`ArchitectureTests`: `BoundaryTests`	The walls between the layers	6 (6)	The built program
`UnitTests`: `TileTests`	Tile addresses, the tile query's text, a view with no order, a tile's cache time and tag, a dropped column, the log of a 500	11 (17)	.NET alone
`UnitTests`: `SettingsTests`	Each section of `tessera.json`; the database's address; bad values, misspelt keys and words where numbers belong	12 (16)	.NET alone
`UnitTests`: `CatalogueTests`	A row becomes a layer; the live list loads, retries and keeps the last good one	10 (10)	.NET alone
`UnitTests`: `MapTests`	The area written into the page; the nine pages; the 404s	5 (10)	.NET alone

`UnitTests`: `DatabaseErrorTests`, `SqlTests`	Sorting failures; quoting names	3 (9), 2 (2)	.NET alone
`UnitTests`: `SourceTests`, `TargetTests`	The source read as text: no machine named, SQL in `Queries.cs` alone, the map's nine layers; addresses in disguise	7 (13), 3 (17)	.NET and the repository's files
`IntegrationTests`: `ServiceTests`	The whole service over HTTP, seeded and granted; disguised addresses sent byte for byte	22 (42)	Docker, through Testcontainers
`IntegrationTests`: `CatalogueTests`, `SeedingTests`, `PoolTests`, `DatabaseTests`, `FrozenDatabaseTests`	Grants, the seed and its order, the pool's queue, the shipped roles, a frozen database	8, 5, 3, 3, 1	Docker, through Testcontainers
`IntegrationTests`: `StartTests`	The program refuses to start without its settings, names a misspelt key with the rest, and starts with the database down	5 (5)	The built program; no database
`web/src`, `web/e2e`	The map's logic, under Vitest; the map in a browser, under Playwright	Not counted here	Node; a browser
`scripts/lib/*.test.ts`	The scripts' pure parts, under `node --test`	Not counted here	Node
`etl/tests`	The Python pipeline, under pytest	254, as pytest collected them	Python, through uv

Source: `tests/`; xUnit's report of one run on 18 October 2026: 6 + 94 + 67 = 167 cases, all passed; `scripts/test.ts`; `etl/tests`.

The balance follows from the second idea of chapter 3. The core is pure, so you can prove most rules without a container; the edge is thin, so it needs few tests of its own. The 94 unit cases finish in a few seconds. The 67 integration cases take about a minute, mostly spent starting databases in Docker. The six architecture tests take 8 to 13 seconds on a quiet machine and up to 45 on a busy one, as ADR 0010 records, mostly spent loading the program and its libraries. Run the unit tests after every change and the rest before every commit.

4.3 Every test names its requirement

Every test cites, in a comment, the story criterion it proves, and every requirement names, in a table, the test class that proves it. A requirement is a sentence the finished service must make true. Tessera keeps 151 of them in three files under `docs/requirements`: 58 functional requirements, FR-01 to FR-58, say what the service, the map, the pipeline and the scripts do; 67 non-functional requirements, NFR-01 to NFR-67, say how fast, how stable and how easy to use; and 26 security requirements, SR-01 to SR-26, say whom to serve and whom to refuse. Appendix A lists them.

Each requirement comes from one acceptance criterion of a story. Its row names that criterion as its source, written as the story and the criterion's number: 'S-01 c3' is S-01, criterion 3. The row's last column, 'Checked by', names the test that proves it, or says 'No test yet'. Exhibit 4.4 shows the four rows that come from S-01.

EXHIBIT 4.4 Four security requirements, each traced back to a criterion and forward to its tests

ID	THE REQUIREMENT, SHORTENED	SOURCE	CHECKED BY
SR-05	Every value of a tile address reaches the database as a bound parameter, and a property test proves it	S-01 c1	`TileTests`
SR-06	Every query's text lives in `Queries.cs`, and `Database` alone sends it	S-01 c2	`BoundaryTests`, `SourceTests`
SR-07	Each table or column name comes from the catalogue, through the one function that quotes names	S-01 c3	`SqlTests`, `CatalogueTests` (unit), `TileTests`
SR-08	A layer named `x; DROP TABLE cafes` answers 404, and every table keeps its rows	S-01 c4	`ServiceTests`

Source: `docs/requirements/security-requirements.md`, *section 2.*

The thread runs the other way too. A test's comment names the criterion it proves, so a reader who asks 'why does this test exist?' finds the answer above it:

CSHARP

```
// tests/Tessera.UnitTests/CatalogueTests.cs
// N-23 c4: a database that blinks does not unpublish every layer.
[Fact]
public async Task Keeps_the_last_good_list_through_a_failure()
{
    // …
}
```

The same habit runs through the code itself: `TesseraModule` says the list of layers loads in the background 'so the service answers from its first moment (N-23 c3)'. A search finds every place that serves one criterion; `-w` matches the whole word, so 'US-01' does not count as 'S-01':

BASH

```
grep -rnw "S-01" tests --include=*.cs     # 14 lines, in five files, cite S-01
grep -rnw "N-23" src --include=*.cs       # the code that keeps N-23, in six files
```

A test with no criterion behind it is a candidate for deletion, and a requirement with no test behind it is a promise nobody has checked. The files are honest about the second: 19 of the 151 rows say 'No test yet', and 21 more say a test proves the requirement 'in part'.

> WATCH OUT Nothing checks the thread but you. No script reads the comments or the 'Checked by' column, and the column names a class, not a method. The `ServiceTests` test behind NFR-47 reads `no-store` on the `UP` answer of `/health` alone, never on `DOWN`, which is why that row says 'in part'. Copy each ID exactly, in the form 'S-01 c3', so a search finds it.

> TRY IT Write the missing check as a script in TypeScript with zx, as the repository's other scripts are written. Read the three files in `docs/requirements`, take each class named in a 'Checked by' cell, such as `TileTests`, and fail when no file of that name exists under `tests/`. Then extend it: fail when no test in that class cites the row's source criterion.

4.4 A session at the keyboard

A session follows the loop: take the next criterion of a story, run one test class until it turns green, then run everything before you commit. The ***.NET SDK*** is the set of tools that builds the code, runs the tests and packs the program; its command is `dotnet`, and `Tessera.slnx` lists the projects it builds. `npm` runs the project's own scripts, which call `dotnet` among other tools. Five commands carry almost every session:

```bash
dotnet run --project tests/Tessera.UnitTests -- --filter-class Tessera.UnitTests.TileTests
                                # one test class: seconds
dotnet run --project tests/Tessera.UnitTests    # every unit test, after each change
npm test                        # scripts, pipeline, map, service, browser
npm run build                   # the map, then the service in Release
git commit -am "Refuse a tile one step outside the grid (US-07)"
```

A session runs in eight steps.

1. **Read the story** in chapter 5, and take the next criterion from its tests to write first.
2. **Write the test**, then run its class alone. It must fail, and for the reason you expect: a missing type or a wrong answer, never a typing slip.
3. **Write the least code** that passes, and run the class again until it is green.
4. **Refine**, running the class after each change.
5. **Repeat** for the next criterion of the story.
6. **Run everything** with `npm test`, with Docker running.
7. **Commit.** A ***commit*** is a saved snapshot of the project in Git, the tool that keeps the project's history, with a message saying what changed. Make one for each green step. Write the message as an order, as the repository's own history does ('Run the seed command's steps as one Result chain'), and add the story it serves, as the plan asks and as only 26 of the repository's 328 commits do.
8. **Compare with the worked answer.** Open the repository's version of the file only when your own tests pass, and note what differs and why.

> TRY IT Break a rule on purpose and watch the right test catch it. In `TileAddress.Of`, change `x >= 1 << z` to `x > 1 << z`, then run `TileTests` alone. One case of 17 turns red every time: `Refuses_a_tile_outside_the_grid(z: 1, x: 2, y: 0)`. The property test `Gives_only_addresses_on_the_grid` failed in four of nine runs for this book and passed in the rest: its hundred random numbers land on the one edge only some of the time, which is why the boundary row exists. Undo the change and run the class again; all 17 pass.

> WATCH OUT Three traps stop a first session. First, `global.json` asks for the .NET 10 SDK, from 10.0.100 upwards; without one, every `dotnet` command stops with 'A compatible .NET SDK was not found.' (appendix E, section E.2), so install .NET 10 before anything else. Second, Docker must be running before the integration tests or `npm test`, since they start PostgreSQL in a container; with Docker stopped, they fail at their first line with a complaint about Docker, not a broken rule. Third, `DatabaseFixture` finds `db/Dockerfile` by climbing to the folder that holds `.git`; run `git init` in your own project on its first day, or the integration tests cannot find the database to build.

4.5 Stories, points and iterations

The work is cut into 30 stories worth 52 points; the first plan builds 16 of them, worth 29 points, in four iterations, and four milestones mark the dates by which something must work. A user story states one need in one sentence of three parts: who wants it (‘As a map developer’), what they want (‘I want a tile by layer and coordinates’) and why (‘so that my map shows the layer’s features’). Its acceptance criteria say what must be true when it is done, each in the form ‘Given a situation, when something happens, then a result follows’, and each becomes at least one test.

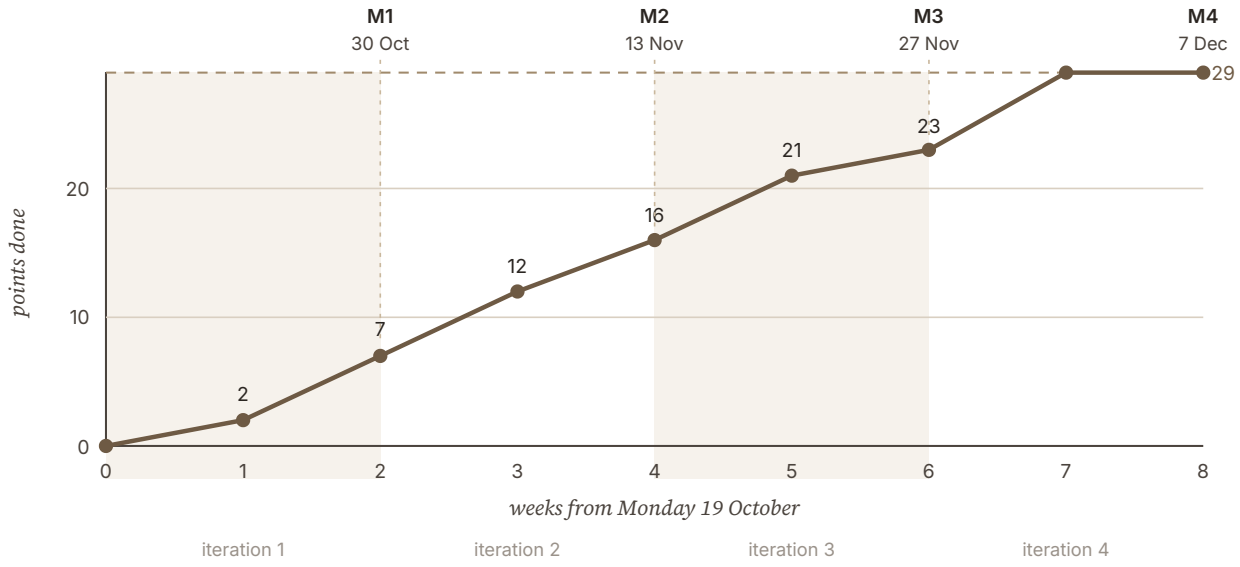
A story point measures size: one point is about one working day, tests included. No story exceeds 3 points, because a larger one hides its risk. The stories fall into six parts, from A, the service, to F, clean places. An iteration is a block of about two weeks that ends with a review on its last working day. A milestone is a state of the product that someone could see and test, named in a sentence.

The calendar decides how much fits. From Monday 19 October to Tuesday 8 December 2026 lie 37 weekdays; two holidays, four review days and one set-up day leave 30 days for stories. Parts A and B, the service and the map, need 29. The other four parts, worth 23 points, cannot fit, so the plan moves whole parts, not half-built stories, past 8 December, and keeps one day in hand. Exhibit 1.3, in chapter 1, lists the four iterations and their stories, and each ends at one of the four milestones.

The order of the stories obeys one rule: each builds only on stories above it. Nothing can run before US-00 sets up the build, and no tile can exist before US-03 publishes the layers. Within that rule, the service comes before the map, the database before the tiles, and S-01 and S-06 guard the tile route before US-35 opens it to the office. The first fortnight lays the walking skeleton, so a wrong assumption about any part shows early, while it is cheap to fix.

You track progress with a ***burn-up chart***, which Exhibit 4.5 draws: two lines climbing towards a ceiling of 29 points. One line shows the points you planned to finish by each Friday; the other, which you add each week, shows the points you did finish.

EXHIBIT 4.5 The planned burn-up: 29 points over seven weeks, with four milestones



Source: `docs/guides/delivery-plan.md`, *‘Tracking progress with a burn-up’. Week 8 holds only the review and the release.*

Read the line against the milestones above it. The steepest climb is week 7, when the clusters and the names finish together; week 6 climbs two points, since UI-19 alone finishes and UI-10 only begins. Read the gap between your line and the plan in days, since one point is about one day. A gap of one day uses the day in hand; a gap of two at a review calls for the first cut: S-07, then UI-17, then UI-11, never a test and never a defence.

> KEY IDEA Measure progress in stories done, never in stories begun. A story at 90 per cent counts as nothing, since half a story cannot ship. At each review, count only the points whose stories meet the definition of done.

4.6 The definition of done

A story is done when its own criteria pass and a short list of checks holds for the whole repository; the build runs most of the checks itself. The list is the ***definition of done***. No story page repeats it, since it holds the habits that keep every story safe. The delivery plan names four checks, and the build and the requirement files add the rest. Exhibit 4.6 sets them out.

EXHIBIT 4.6 A story is done when nine checks hold, and five of them run by themselves

	CHECK	WHAT PROVES IT
☐	Each criterion has a test, written first and seen to fail; for US-35, a check on the real machine	Your own discipline, and one commit for each green step
☐	Every test passes, and the tests run at least 94 per cent of the service's lines and 84 per cent of its branches	`npm test`, through `scripts/test.ts` and `scripts/coverage.ts`; the `build` job on GitHub
☐	The compiler and its analysers report no warning, and no value may be null unmarked	`TreatWarningsAsErrors`, `Nullable` and `AnalysisMode` All, with Meziantou, in `Directory.Build.props`; ExhaustiveMatching, in `Tessera.csproj`
☐	The layers keep their walls, and each namespace matches its folder	The six tests of `BoundaryTests`; IDE0130 as an error
☐	Every value reaches SQL as a parameter, and every name quoted	`Keeps_every_tile_address_out_of_the_text`, `Quotes_every_name_safely`; `SqlName`'s private constructor
☐	The map, the scripts and the pipeline pass their own type and style checks	`tsc` and ESLint; `pyright`, `ruff` and `lint-imports`, inside `npm test`
☐	The code is committed, with messages that name the story	Your own review
☐	The requirement rows the story touches name their tests, and the tests cite their criteria	Your own review of `docs/requirements`
☐	The comparison with the worked answer is written down	Your own notes

Source: `docs/guides/delivery-plan.md`, *'The weekly and daily rhythm'*; `Directory.Build.props`; `scripts/test.ts`; `.github/workflows/build.yml`; `tests/`.

The four checks no machine runs are the easiest to skip, so give each a fixed place in the session: the failing test at step 2, the commit message at step 7, and the requirement rows and the comparison at step 8. A story that passes its tests but fails the checklist stays open; finish it before you start the next.

> NOTE The coverage floor is the check behind the second row: *coverage* is the share of lines and branches the tests run. Each test project carries Microsoft.Testing.Extensions.CodeCoverage, so each suite writes a record of the lines it ran; ReportGenerator merges the three, and `scripts/coverage.ts` fails `npm test` and the GitHub build below 94 per cent of lines or 84 of branches. The run for this book covered 96.1 per cent of the lines (648 of 674) and 85.5 per cent of the branches (267 of 312), and the report in `coverage/report/` shows each line the tests miss.

> THINK IT OVER Pick any test you wrote this week. If you deleted the code it tests, would it fail? If you broke the rule its comment names, would it fail for that reason, and no other?

Check your understanding

1. Sort these rows of the tile tests as normal, boundary or erroneous data: `22, 4194303, 0`; `15, 26719, 14311`; `23, 0, 0`; and a level of `x` in the address.
2. What does ‘S-01 c3’ name, and where would you find the tests that prove it?
3. Time runs short. In what order does the plan cut stories, and what does it never cut?

Answers. 1. Boundary, inside; normal; boundary, outside; erroneous, which the route refuses before `Of` sees it. 2. Story S-01, criterion 3; the row SR-07 in `docs/requirements/security-requirements.md` names its tests in ‘Checked by’, and `SqlTests`’s comment cites ‘S-01 c3’. 3. S-07, then UI-17, then UI-11; never a test and never a defence.

5
The stories, one at a time

Build the 30 stories in the order of the iterations, each test first, and open the worked answer only when your own tests pass. Every story has a page of the same shape: who wants it, what must be true, which tests to write first and which of the seven ideas it uses. The delivery plan fits 16 of the stories, worth 29 points, into four iterations from 19 October to 7 December 2026, and leaves the other 14 to a second plan that starts on 9 December. Two short sections apply to every page and come first: how a page works, and whom the stories speak for. The iterations follow, from the walking skeleton to the release.

5.1 How a story page works

Every ***story page*** answers the same questions in the same order, from where the story sits in the plan to what most decides whether it holds. The order follows this book's pyramid: the answer, the need, the work, then the idea behind the code. Read a page once from top to bottom, then come back to its list of tests at the keyboard.

Under the heading stands the story's point, its main thought in one sentence. A small card follows, giving the requirements the story meets (**Traces**); the ideas of chapter 3 it puts to work, by number and name (**Ideas**); its points, the stories it builds on and its iteration (**Size**); the milestone it counts towards; what you learnt at school that it leans on (**You know**); and the files that hold the worked answer. Next come the story in everyday words, then the story itself and its criteria, word for word in meaning from `docs/user-stories/user-stories.md`, then the tests to write first. 'The idea at work' quotes up to 16 lines of the worked answer and shows the story's ideas at work in them. 'What to carry away' closes the page with the points that most decide whether the story holds.

A few rules about tests keep the pages short. A C# test is an xUnit method whose name is a sentence with underscores for spaces, such as `Answers_404_for_a_table_revoked_after_start_up`, so a failing test reads as the rule it guards. A Python test's name follows the same rule in lower case. The map and the scripts name their tests with a sentence in quotation marks, such as 'names every missing variable at once'. Every test a page names is in the repository today. Where a story asks for a test the repository lacks, the page says 'No test yet', as the requirements files do, and turns the gap into a task for you. A page explains an idea of chapter 3 only where the story adds to it; elsewhere it names the idea and moves on.

> KEY IDEA The excerpt comes after the tests that call for it, and so should your reading of it. Write your own tests from the page and make them pass with your own code. Then compare your version with the excerpt and with the whole file, as the delivery plan sets out for each day.

5.2 Who the stories speak for

Every story speaks for one of the five personas of section 2.2. The visitor and the maintainer between them speak for 23 of the 30. Each story names its persona in its first words, 'As a …', and its 'so that' clause says what that person gains. The visitor's stories change what a person sees on the screen; the maintainer's

change what holds the service up. Many of the maintainer's, such as US-00 and US-01, are enablers: they give no client anything new, but every later story stands on them. While you rebuild Tessera you are the maintainer, and on the office machine you do the publisher's work as well. When a story's 'so that' no longer holds for its persona, the story has drifted. Put it right before you write its tests.

5.3 Iteration 1 · The skeleton and the places

The first iteration turns an empty folder into a service that reads the places from a database, where a grant publishes each table. It runs from Monday 19 to Friday 30 October 2026. Of its ten working days, the set-up day and the review take two and leave eight for stories. Four stories, worth 7 points, fill seven of them, and US-07 begins on the eighth, Thursday 29 October, with its first test. The plan finishes US-00 and US-01 in the first week, and US-03 and US-24 in the second. The iteration ends with milestone M1, 'The places are published by grant'. Each C# path in a page's 'Files' row starts at `src/Tessera/` unless it says otherwise.

5.3.1 Monday 19 October · Set up the machine

Spend one day, before any code, installing the tools the worked answer names and proving them with its whole test suite, so that a later failure is a fault in your code, never in your machine.

The set-up day carries no points and no story of its own. Your machine matches the office machine: Ubuntu 24.04 inside WSL2 on Windows 11. In Ubuntu you install Docker Engine, the .NET 10 SDK (the Software Development Kit, which holds the compiler), Node.js 26, uv (Python's project tool) and your editor. The worked answer's `global.json` names SDK 10.0.100 and lets it move to a newer feature band of .NET 10, never to .NET 11, so check that `dotnet --version` starts with 10. Appendix F, session 1, gives each step. The day ends when the worked answer's `npm test` passes on your machine.

THE CHECK THAT ENDS THE DAY

1. **Every check of the worked answer.** `npm test` runs the scripts' type check and tests, the pipeline's four checks, the map's tests, the build, the service's three suites of tests with their coverage check, and the browser tests. It needs Docker running and the .NET 10 SDK on your path.

`scripts/test.ts`, whose checks Exhibit 1.4 lists, runs the cheapest first, so a broken machine shows itself in seconds, not after the slow database tests. A failed step stops the run, because zx throws an exception when a command exits with an error. Each test project is a program of its own, which `dotnet run` starts, since `global.json` names Microsoft.Testing.Platform as the runner. If `uv` or Docker is missing, the script stops at the step that needs it and names that step.

> WHAT TO CARRY AWAY **Copy the version pins.** Take the versions from the worked answer's `global.json`, `Directory.Packages.props`, `package.json`, `web/package.json` and `etl/pyproject.toml`, not from a tutorial. xUnit v3, for one, changed how a test project runs, and older answers on the web still show the old way.
>
> **Prove the machine before the code.** Once the worked answer's tests pass on your machine, every later red test points at your own work.

5.3.2 US-00 Set up the walking skeleton

One command builds the map and the service, one command runs `Tessera.dll` and one command runs every test, so every later story adds to a program that already answers.

Traces	FR-01, FR-02, FR-03, SR-25
Ideas	7 The host wires the parts through one door · 1 One program, walled modules
Size	1 point · builds on nothing · iteration 1, week 1
Milestone	Counts towards M1 at the end of iteration 1. US-01 builds on it, and every later story stands on it.
You know	Translators: a compiler checks the whole program before it runs · subroutines grouped into modules · input from outside the program
Files	`Directory.Build.props` (at the root) · `Program.cs` · `TesseraHost.cs` · `Modules/Tessera/Infrastructure/Sql/Connection.cs` · `scripts/build.ts` · `scripts/test.ts`

> IN PLAIN WORDS A builder pegs out the site before laying a brick. This story lays out the codebase so that one command builds the service and another runs it. The walking skeleton is the thinnest service that works from end to end: an answer from the service, on your screen. `npm run build` builds the map and then the solution, with the map copied inside the service as `wwwroot/`; `dotnet src/Tessera/bin/Release/net10.0/Tessera.dll` runs it; `npm test` runs every test. The service learns where its database lives from an ***environment variable***, a named value the operating system hands a program as it starts, so the code itself names no machine.

> **As** the maintainer, **I want** a codebase that builds and runs from one command, **so that** every later story starts from working ground.

ACCEPTANCE CRITERIA

1. **Given** a fresh copy of the repository, **when** I run the build command, **then** the service builds.
2. **Given** the build, **when** I run the run command, **then** the service starts and answers an HTTP request on its port.
3. **Given** the test command, **when** I run it, **then** every test passes.
4. **Given** the run command, **then** the service reads its database's name, address and port from the environment variable `DATABASE_URL`, and the code names no machine. **Given** that variable missing, empty or malformed, **then** the service refuses to start and names it.

TESTS TO WRITE FIRST

1. **The build is the first test.** `Directory.Build.props` sets `TreatWarningsAsErrors`, `Nullable` and every rule of .NET's own analysers (`AnalysisMode All`), with Meziantou's rules besides, so a warning stops the build. The automatic build, `.github/workflows/build.yml`, runs it on a clean Ubuntu 24.04 at every push to `main` and every pull request (FR-01).
2. **The skeleton answers.** `Starts_from_the_run_command_and_answers`, in `ServiceTests`, starts `Tessera.dll` as a process of its own on a free port and asks it for a tile until it answers 200. Write it first and watch it fail (FR-02).
3. **Machines come from the environment.** `Refuses_to_serve_and_names_each_missing_variable`, in `StartTests`, starts the program with an empty environment and expects exit code 1 and the words

`DATABASE_URL`. `Names_a_missing_or_bad_database_url`, in the unit tests' `SettingsTests`, tries no address, an empty one and `http://db/tessera`, and expects 'missing or empty' or 'expected postgresql://' beside the variable's name; `Reads_the_database_url_into_a_connection_string` reads a password holding `%40`, an encoded `@`. `Names_no_machine_in_the_code`, in the unit tests' `SourceTests`, reads every C# file, project file and settings file of the service as text and finds no machine but the loopback address, by which a computer reaches itself (SR-25).

4. **The walls stand from the first day.** The six rules of `BoundaryTests`, such as `Domain_stands_alone`, which section 3.1 quotes.

THE IDEA AT WORK

CSHARP

```csharp
// Program.cs: one program, two commands
return args switch
{
    [] => await Serve().ConfigureAwait(false),
    ["seed"] => Seed(),
    _ => Usage(),
};
// …
// Modules/Tessera/Infrastructure/Sql/Connection.cs: a bad address names its variable
public IEnumerable<ValidationResult> Validate(ValidationContext validationContext) =>
    Parsed.IsFailure ? [new ValidationResult($"{Variable}: {Parsed.Error}")] : [];
```

A `switch` on the arguments chooses between the program's two commands. With no argument it serves tiles and the map; with `seed` it runs each new SQL script once and stops; anything else prints how to use the program and returns exit code 2. `Serve` asks `TesseraHost.Build` for the web service. That is idea 7, which section 3.7 explains: the host calls the module's one door, `AddTessera` and `MapTessera`, and knows nothing of what lies behind it. Before anything runs, the host checks every setting. `ConnectionString.FromUrl` parses `DATABASE_URL` once, and `Validate` turns a failed parse into a problem that names the variable, so a bad address stops the start with one clear line. Idea 1 starts on the same day, so a wall broken today fails a test today, while the change is still small.

> WHAT TO CARRY AWAY **Turn warnings into errors in the first commit.** Warnings let in later arrive by the hundred, and nobody fixes a hundred at once.
>
> **Name no machine in the code.** The address comes from `DATABASE_URL`, so the same program runs on your desk, in the tests and in the office.

5.3.3 US-01 Start the same database for the service and its tests

One image, built from `db/Dockerfile`, gives the service, its tests and the office the same PostgreSQL, the same two roles and the same places, so a test that passes on your desk means the same in the office.

Traces	FR-04, FR-05, SR-01
Ideas	2 A pure core, effects at the edge
Size	1 point · after US-00 · iteration 1, week 1
Milestone	Counts towards M1. US-03, US-24 and every integration test build on it.
You know	SQL tables · users and access levels, from network security · client and server
Files	`db/Dockerfile` · `db/initdb/20-roles.sql` · `db/seed/0001-macau-places.sql` · `data/macau-pois.geojson` · `tests/Tessera.IntegrationTests/DatabaseFixture.cs`

> IN PLAIN WORDS Every layer lives in a database, and the service and its tests should meet the same one. `db/Dockerfile` packs PostgreSQL 18 with PostGIS 3.6, the spatial add-on, into one image. On its first start the image runs `db/initdb/20-roles.sql`, which makes two roles, or database users. `tessera_reader` is the service's: it may read, and it sees only the tables granted to it. `tessera_publisher` is the data publisher's: it owns the places' tables, loads them and grants them. The reader's role works like a library card that lets you read every book on the open shelves but never take one home. Each password comes from an environment variable, never from the file. The places arrive as a ***seed script***, a SQL file made from `data/macau-pois.geojson`, which US-24 loads.

> **As** the maintainer, **I want** one packaged spatial database with the service's role and the reference data, **so that** the service and its tests meet the same database and the same places.

ACCEPTANCE CRITERIA

1. **Given** the packaged database, **when** it starts, **then** the role script has made the service's role.
2. **Given** the package, **then** it carries the seed script, `db/seed/0001-macau-places.sql`, made from `data/macau-pois.geojson`, ready for US-24's seed.
3. **Given** the integration tests, **when** they run, **then** they start the same packaged database, run the same role script and read the same seed.

TESTS TO WRITE FIRST

1. **The role, from the script you ship.** `Signs_in_as_the_shipped_reader_role`, in `DatabaseTests`, signs in as `tessera_reader` on a fresh copy of the image and loads the list of layers: the load proves the sign-in, and the empty list proves the role starts with no grant (SR-01). `Keeps_the_reader_read_only` finds `default_transaction_read_only=on` among the role's settings.
2. **The seed, as the repository holds it.** `Ships_the_seed_made_from_the_places` works out the SHA-256 fingerprint of `data/macau-pois.geojson`, a 64-digit value that changes whenever one byte of the file changes, and compares it with the one recorded in the test. It then checks that the shipped script begins '-- Made from data/macau-pois.geojson.' (FR-04).
3. **The same image everywhere.** `DatabaseFixture` builds the image from the folder `db`, and every integration class that needs a database starts its own copy of it; `scripts/e2e.ts` builds the same image

for the browser tests and runs `Tessera.dll seed` on the shipped scripts (FR-05).

THE IDEA AT WORK

CSHARP

```csharp
private static readonly Lazy<Task<IFutureDockerImage>> Image = new(async () =>
{
    var image = new ImageFromDockerfileBuilder()
        .WithDockerfileDirectory(CommonDirectoryPath.GetGitDirectory(), "db")
        .WithName("tessera-db-test:latest")
        .WithCleanUp(false)
        .Build();
    await image.CreateAsync().ConfigureAwait(false);
    return image;
});
// …
_container = new PostgreSqlBuilder(await Image.Value.ConfigureAwait(false))
    .WithDatabase(Name)
    .WithUsername("postgres")
    .WithPassword(OwnerPassword)
    .WithEnvironment("TESSERA_READER_PASSWORD", ReaderPassword)
    .WithEnvironment("TESSERA_PUBLISHER_PASSWORD", PublisherPassword)
    .Build();
```

Idea 2, a pure core with effects at the edge, shapes the tests as well as the code. The unit tests test the core and need no database; only the integration tests, which test the edge, start one. `DatabaseFixture` builds the image from `db/Dockerfile` rather than pulling a plain PostGIS image, so the tests run the shipped role script, never a hand-made copy. `Lazy` builds the image once for all the classes, and Docker reuses its layers after the first build. A test class asks xUnit for the fixture through `IClassFixture<DatabaseFixture>`, so each class gets a database of its own and no two classes share rows. The two passwords go in as the variables the role script reads, and the fixture's `Reader` signs the tests in as `tessera_reader`, the role the service uses in the office, never as the database's owner.

> WATCH OUT The second line of `db/seed/0001-macau-places.sql` reads 'Never change its SQL after it has run'. DbUp knows a script by its name alone and keeps no checksum, so it never notices a changed script: the database keeps the old rows, and the repository no longer tells what the database holds. Write a new script instead.

> WHAT TO CARRY AWAY **Grants are the wall; read-only is the second lock.** `20-roles.sql` sets `default_transaction_read_only = on` for `tessera_reader`, so even a table granted for writing by mistake stays read-only.
>
> **Pin one database version.** `FROM postgis/postgis:18-3.6` means every machine, and every test, meets the same database until someone changes that line on purpose.

5.3.4 US-03 Publish a table by granting it

One query, run as the service's own role, asks the database which tables and views that role may read, and pure functions turn each row into a layer with every name quoted, so a grant publishes a table and a revoke withdraws it.

Traces	FR-06, FR-07, FR-08, FR-09, SR-02, SR-03
Ideas	2 A pure core, effects at the edge · 3 Data is plain and never changes · 4 A wrong value cannot be built · 5 Failures are values
Size	3 points · after US-01 · iteration 1, week 2
Milestone	Counts towards M1. US-24, US-07, S-01 and N-23 build on it.
You know	SQL SELECT with JOIN and WHERE · records with named fields · strings, and a quote inside a string
Files	`Infrastructure/Sql/Queries.cs` · `Infrastructure/Catalogue.cs` · `Domain/Layers.cs` · `Domain/SqlName.cs`, each under `src/Tessera/Modules/Tessera/`

> IN PLAIN WORDS The database keeps the guest list; the service only reads it. A table becomes a map layer when the data publisher grants the service's role the right to read it, and leaves when the publisher withdraws that right. The service builds its list of layers, the catalogue, from the database. Each layer also needs an SRID, the number that names a coordinate system, such as 4326 for longitude and latitude. Every name in the catalogue, such as `food.cafes`, comes from the database, so a visitor can never slip a name of their own into a query.

> "**As** a data publisher, **I want** every table or view I grant to the service's role to become a layer, **so that** publishing needs no change to the service.

ACCEPTANCE CRITERIA

1. **Given** a granted table with a typed geometry column and an SRID above 0, **when** the catalogue loads, **then** it is a layer with its SRID and geometry type.
2. **Given** a granted view, or a view that stores its rows, **when** the catalogue loads, **then** it is a layer.
3. **Given** a table never granted, **when** the catalogue loads, **then** it is absent.
4. **Given** a table whose name holds a space and a double quote, **when** the catalogue loads, **then** the layer is present and the service serves its tiles.
5. **Given** a granted function, **when** the catalogue loads, **then** it is absent, since Phase Oct publishes tables and views alone.
6. **Given** a table revoked after start-up, **when** a tile is requested, **then** the answer is 404, as a problem document (US-07).

TESTS TO WRITE FIRST

1. **One way to quote a name.** `Quotes_a_name_and_doubles_each_inner_quote`, in `SqlTests`, turns `odd "name"` into `"odd ""name"""`. `Quotes_every_name_safely` is a property test (section 4.1): FsCheck makes up a hundred strings, and the test checks for each that the quoted form opens and closes with a quote and holds no lone quote between.

2. **Rows become layers, purely.** In the unit tests' `CatalogueTests`, `Turns_a_row_into_a_layer_whose_id_is_schema_dot_name` hands `CatalogueReader` a row written by hand and finds `food.cafes` and `"food"."cafes"`; `Finds_a_layer_exactly_and_case_sensitively` finds `food.cafes` and refuses `Food.Cafes` and `food`; `Publishes_neither_of_two_tables_whose_ids_clash` drops schema `a.b` with table `c` and schema `a` with table `b.c`, which both read `a.b.c`.
3. **The catalogue query.** `Names_each_column_after_a_field_of_the_row` checks that `Queries.Layers` names one column after each field of `CatalogueRow`, since Dapper fills the record by matching names. `Lists_tables_and_views_alone` finds `c.relkind IN ('r', 'p', 'v', 'm')` in the query's text (FR-09).
4. **Load it as the real role.** The integration tests' `CatalogueTests` make their own stage before each test: a granted table, a table in Web Mercator, a granted view, a view that stores its rows, a table never granted, a column with no type, a table named `odd "name"` and a granted function. `Publishes_what_is_granted_and_nothing_else` expects exactly six layers (FR-07, SR-02, FR-09). `Gives_each_layer_its_srid_and_type` reads 4326 and 3857 (FR-06). `Serves_the_awkward_name` asks for its tile and gets 200 (FR-08). `Answers_404_for_a_table_revoked_after_start_up` revokes a table after the list has loaded and expects 404 as `application/problem+json` (SR-03).

THE IDEA AT WORK

CSHARP

```csharp
public static Layer ToLayer(CatalogueRow row) => new(
    $"{row.Schema}.{row.Name}",
    SqlName.Qualified(row.Schema, row.Name),
    SqlName.Quote(row.GeometryColumn),
    row.Srid,
    row.GeometryType,
    Names(row.Columns),
    Names(row.KeyColumns));
// …
public static async Task<Result<LayerCatalogue, DatabaseError>> Load(
    Database database, CancellationToken cancel = default) =>
    (await database.Rows<CatalogueRow>(Queries.Layers, cancel).ConfigureAwait(false))
        .Map(rows => FromRows(rows));
```

Four ideas meet in these lines. `ToLayer` is pure (idea 2): a row in, a `Layer` out, with no database in sight. The `Layer` is a record that never changes (idea 3), and every name in it but the id is a `SqlName`, whose private constructor leaves quoting the one way to make one (idea 4). `Load` is the thin edge: `Database.Rows` returns a `Result`, and `Map` runs `FromRows` only when the query succeeded (idea 5). The query itself asks PostgreSQL's own `has_schema_privilege` and `has_table_privilege`, and keeps the kinds `'r'`, `'p'`, `'v'` and `'m'`: a table, a table split into parts, a view and a view that stores its rows. Functions live elsewhere in PostgreSQL, so they never appear. The query also leaves out a geometry column whose SRID is 0.

> WHAT TO CARRY AWAY **The database decides what is visible.** The privilege checks run inside the database as the service's own role, so a revoke takes effect at the next tile query.
>
> **The integration tests use the real role.** A test signed in as the database's superuser would pass criterion 3 for the wrong reason, since a superuser sees every table, granted or not.

5.3.5 US-24 Publish the places of the area in nine layers

A Python pipeline turns each file of places into a numbered SQL script, DbUp runs each new script once as `tessera_publisher`, and only a grant publishes the nine tables, so the seed loads the places and the publisher decides when the office sees them.

Traces	FR-10, FR-11, FR-12, FR-19, FR-20, SR-04
Ideas	5 Failures are values · 2 A pure core, effects at the edge
Size	2 points · after US-01 and US-03 · iteration 1, week 2
Milestone	Completes M1, 'The places are published by grant', on Friday 30 October. US-07 cuts tiles from these tables; UI-12 and US-36 build on them.
You know	Latitude and longitude · SQL CREATE TABLE and INSERT · primary keys · text files and records
Files	`etl/src/tessera_etl/core/table_sql.py` · `db/seed/0001-macau-places.sql` · `Modules/Tessera/Infrastructure/Seeding/SeedScripts.cs` · `Program.cs` · `scripts/publish.ts`

> IN PLAIN WORDS Tessera serves the places of the area in nine layers, gathered in four groups: `food` (cafes, restaurants, markets), `outdoors` (parks, playgrounds), `culture` (museums, libraries) and `transport` (trains, buses). Each group becomes a ***schema***, a named folder of tables, so the layer `food.cafes` carries its group in its name. The pipeline in `etl/` reads each file through GDAL, a library that reads and converts map files. It writes a SQL script that stores every shape in longitude and latitude, SRID 4326. `Tessera seed` runs each new script once, and `npm run publish` grants the tables. A library buys a book (the seed), but the book reaches the open shelf only when the librarian says so (the grant).

> "**As** a data publisher, **I want** each file of places I own turned into a SQL script, each new script loaded once, and the tables published by grant, **so that** office machines can load the places as tiles, layer by layer.

ACCEPTANCE CRITERIA

1. **Given** a file of places and a pipeline file that lists its tables, each with a layer or a filter on its rows, **when** `npm run etl` runs, **then** it writes one SQL script that makes each table, with a numbered `fid` as its primary key, one column for each property, a spatial index and fresh statistics.
2. **Given** a file in another coordinate system, such as EPSG:3857, **then** the script stores its shapes in longitude and latitude, SRID 4326.
3. **Given** a script name already used, or a table left with no rows, **then** the pipeline refuses, names the fault, and writes no script.
4. **Given** the scripts in `db/seed`, **when** `Tessera seed` runs, **then** it runs each script it has not run before, once, in the order of their names, and records it; run again, it runs nothing.
5. **Given** no grant, **then** no table is published; after the grant of each schema and each table, all nine are.
6. **Given** a script that fails, **then** the database keeps what it held before, and the script runs again once fixed. A script that replaces tables replaces them in one transaction, so no tile ever carries half a script.

TESTS TO WRITE FIRST

1. **Nine tables from the shipped script.** `Loads_the_shipped_seed_once`, in `SeedingTests`, seeds a fresh database as `tessera_publisher` and counts 900 places in the nine tables, 100 in `transport.buses` and

nine spatial indexes. It finds `0001-macau-places.sql` in the ***journal***, the table `tessera.schemaversions` where DbUp records each script it has run, and a second run runs nothing (FR-19). `Runs_the_scripts_in_the_order_of_their_names` writes three scripts, the last first, with `0010` after `0002`, and reads back the order they ran in.

2. **Publish by grant, and only then.** `Publishes_nothing_until_the_grant` finds no `food.cafes` in the list before the grant; after it, `transport.buses` reads SRID 4326 and `food.cafes` keeps the order of `fid` (SR-04).
3. **A failed script changes nothing.** `Undoes_a_script_that_fails` runs a script that drops a table and then calls a function that does not exist. The error starts '0002-trial-a-broken.sql failed', the table keeps its row, and the journal holds no record of the script (FR-20).
4. **A replaced table waits for its grant.** `Withdraws_a_replaced_table_until_the_grant` grants a table, runs a new script that drops and makes it again, and finds it gone from the list until the next grant.
5. **The pipeline's own rules.** In `etl/tests`: `test_writes_a_table_as_GDAL_wrote_it` expects the statements GDAL writes, a `fid SERIAL` primary key, SRID 4326 and the statistics, with GDAL's own `BEGIN` and `COMMIT` taken out (FR-10). `test_extracts_one_layer_flat_and_in_longitude_and_latitude` checks the arguments handed to GDAL, and `test_stores_a_source_in_web_map_metres_in_longitude_and_latitude`, in `test_run.py`, runs the pipeline on a place in EPSG:3857 and reads it back from the script at 113.55° E, 22.19° N in SRID 4326 (FR-11). `test_refuses_a_table_with_no_rows` and `test_matches_its_recorded_outcome`, whose golden case `output-exists` refuses a script name in use, cover the refusals (FR-12). `test_refuses_a_misspelt_setting` refuses a misspelt pipeline file.

THE IDEA AT WORK

CSHARP

```csharp
var upgrade = DeployChanges.To.PostgresqlDatabase(connection)
    .WithScriptsFromFileSystem(folder)
    .WithTransactionPerScript()
    // DbUp would otherwise read $name$ in a script as a value to fill in, and could change
    // a place's text.
    .WithVariablesDisabled()
    .JournalToPostgresqlTable(JournalSchema, JournalTable)
    // The program prints its own short report, so DbUp's step-by-step log goes nowhere.
    .LogToNowhere()
    .Build();
var result = upgrade.PerformUpgrade();
```

`SeedScripts.Run` sets DbUp up in code, one step to a line. DbUp reads every `.sql` file in the folder, sorts them by name, runs each one its journal lacks in a transaction of its own and records it in `tessera.schemaversions`, so a script takes effect whole or not at all. The method returns a `Result` (idea 5): the names of the scripts it ran, or the name of the script that failed and why, and `Program.cs` prints one or the other. `Tessera seed` reads one setting, `TESSERA_PUBLISHER_DATABASE_URL`, and never falls back on `DATABASE_URL`, so the service never holds the right to write. The pipeline puts idea 5 to work in Python as well: `table_sql.statements` returns `Ok` with the statements or `Err` with the reason, such as 'food.cafes: no rows to write.', and the run stops with no script written.

> WHAT TO CARRY AWAY **A grant is a publication.** The seed creates the tables; the data publisher alone publishes them, with `npm run publish`.
>
> **Typed, indexed, analysed.** The catalogue publishes only a typed geometry column (US-03), and without an index and fresh statistics each tile query reads the whole table.

> TRY IT Meet M1 on your own machine. Run `npm start`, then `npm run publish`. Within 30 seconds, the default `Catalogue.RefreshSeconds`, the log of each copy of the service reads 'The service publishes 9 layers.'

5.4 Iteration 2 · Safe tiles, and the map in the office

Iteration 2 finishes the tile, guards it against two attacks, opens it to the office and draws the first map on it. It runs from Monday 2 to Friday 13 November 2026. Monday 2 November is All Souls' Day, a public holiday in Macau, and Friday 13 November holds the review, which leaves eight days for stories. Its five stories are worth 9 points, but US-07 began on Thursday 29 October, in iteration 1, so 8 of them land here. The iteration ends at milestone M2, 'The map opens in the office, on safe tiles', with 16 of the 29 planned points done. Each C# path in a page's 'Files' row starts at `src/Tessera/` unless it says otherwise.

5.4.1 US-07 Serve a tile

One request for a tile passes three checks in a fixed order, and only then does one query ask PostGIS for the tile, so every failure on the way comes back to the client as a problem document of the same shape.

Traces	FR-21, FR-22, FR-23, FR-24, SR-13
Ideas	6 A request is a chain of checks · 5 Failures are values · 7 The host wires the parts through one door
Size	2 points · after US-24 · iterations 1 and 2, weeks 2–3
Milestone	Begun on Thursday 29 October, finished in week 3; counts towards M2. S-01, S-06 and US-35 build on it, and US-23 draws its tiles.
You know	Powers of two and binary shifts · SQL SELECT with WHERE · HTTP status codes, such as 200 and 404
Files	`Modules/Tessera/Features/Tiles.cs` · `Modules/Tessera/Domain/Tiles.cs` · `Common/Api/Problems.cs` · `TesseraHost.cs`

> IN PLAIN WORDS This story is the heart of the product. Asked for tile z/x/y of a layer, the service sends PostGIS one query. PostGIS cuts the layer's places to that square and packs them into a small binary file. Whenever the service cannot serve, it answers with a problem document (section 2.4), so every client reads every error the same way, as a form at the post office has the same boxes whoever fills it in.

> "**As** a map developer, **I** want a tile by layer and coordinates, **so that** my map shows the layer's features.

ACCEPTANCE CRITERIA

1. **Given** a published layer, **when** I request `/{layer}/{z}/{x}/{y}.pbf`, **then** I receive 200 ('OK') with the content type `application/vnd.mapbox-vector-tile`.
2. **Given** the same request ending `.mvt`, **then** the body is identical.

3. **Given** a layer stored in WGS 84 (US-24), **when** I decode its tiles, **then** its features sit at the right positions in the tile's grid.
4. **Given** an unknown layer, **when** I request a tile, **then** I receive 404.
5. **Given** any error answer from the HTTP API, such as the 404 above and the 400 of S-06, **then** its body is a problem document as RFC 9457 defines it: the content type `application/problem+json`, with `type`, `title` and `status`, and `detail` where a sentence helps. `status` matches the HTTP status code, and with `type` as `about:blank`, `title` is the status's own phrase, such as 'Not Found'.

TESTS TO WRITE FIRST

1. **An address on the grid, or none.** In `TileTests`, `Accepts_a_tile_that_exists` and `Refuses_a_tile_outside_the_grid` try the edges of the grid, such as column 4194303 at level 22 and column 2 at level 1; `Gives_only_addresses_on_the_grid`, a property test, hands `TileAddress.Of` any three numbers and finds every address it makes on its level's grid. Section 4.1 builds them test-first. None needs a database.
2. **An unknown layer is not found** (FR-24, SR-13). `Answers_an_unknown_layer_with_a_problem_document`, in `ServiceTests`, asks for `/food.zoos/0/0/0.pbf` and expects 404, `application/problem+json`, `about:blank`, 'Not Found' and 404 in the body. `Answers_a_tile_outside_the_grid_with_404` asks for column 5 at level 1 and gets the same shape.
   `Answers_a_tile_address_with_a_word_for_a_number_with_404` puts `zero`, `0x1` and `1.5` where a number belongs; no route matches, and the answer is the 404 problem document of any unknown address.
3. **Through the real database** (FR-21, FR-22, FR-23). `Serves_a_tile_as_a_mapbox_vector_tile` asks for the level-15 tile that holds the café `cafes-001` and expects 200 and the tile's content type; its line on `Cache-Control` belongs to N-18, so leave it for Part C. `Serves_the_same_body_for_mvt` expects the same bytes from `.mvt`. `Places_each_feature_where_it_lies` decodes the tile with NetTopologySuite, a geometry library, and finds the café within 0.00001 degrees of its longitude and latitude, from a layer in WGS 84 and from the same cafés in Web Mercator.

THE IDEA AT WORK

CSHARP

```csharp
// TesseraHost.cs
builder.Services.AddProblemDetails(
    options => options.CustomizeProblemDetails = Problems.Shape);
// …
app.UseExceptionHandler();
app.UseStatusCodePages();
// Common/Api/Problems.cs
public static void Shape(ProblemDetailsContext context)
{
    context.ProblemDetails.Type = "about:blank";
    context.ProblemDetails.Extensions.Remove("traceId");
    context.HttpContext.Response.Headers.CacheControl = "no-store";
    if (context.ProblemDetails.Status == StatusCodes.Status503ServiceUnavailable)
        context.HttpContext.Response.Headers.RetryAfter =
            RetryAfterSeconds.ToString(System.Globalization.CultureInfo.InvariantCulture);
}
```

The route itself, `TileServer.Answer`, is the chain of checks that section 3.6 shows (ideas 6 and 5). It checks that the list of layers has loaded, that the layer is published and that the address lies on the grid, and only then runs one query, so an unknown layer gets 404 before any query runs. The query's `Result` then splits two ways: the bytes become a tile, and each kind of `DatabaseError` becomes its own refusal. The excerpt shows the other half of the story, criterion 5. ASP.NET Core writes every problem document,

from a route's refusal, from a crash that `UseExceptionHandler` catches, or from an empty 404 that `UseStatusCodePages` fills, and each passes through `Shape` on its way out, so no handler writes an error body of its own (idea 7). `Shape` sets the type to `about:blank` and removes the `traceId` that ASP.NET Core adds, which would tell a stranger about the service's insides. Every problem carries `no-store`, and a 503 adds `Retry-After: 5` for N-22. The refusals use fixed words, such as 'No published layer has that name.', and never repeat the name a stranger sent.

WHAT TO CARRY AWAY

- **One shape for every error.** `AddProblemDetails` with `Problems.Shape`, and the two steps that catch what no route answered, give every client one shape to read.
- **Decode the tile.** A test that finds a name among the bytes proves little; `Places_each_feature_where_it_lies` decodes the tile and checks where the café sits, for a layer in each coordinate system.

TRY IT Ask your own service for a tile of the cafés at level 0, the whole world in one square: `curl -i http://127.0.0.1:7800/food.cafes/0/0/0.pbf`. Expect `200`. Then ask for `/food.nothing/0/0/0.pbf` and read the problem document that comes back.

5.4.2 S-01 Keep SQL out of every request

Every value from a request travels beside the query as a bound parameter, and every name in the query's text comes from the catalogue, quoted, so no address can change what the service asks the database.

Traces	SR-05, SR-06, SR-07, SR-08
Ideas	4 A wrong value cannot be built · 1 One program, walled modules · 3 Data is plain and never changes
Size	2 points · after US-07 · iteration 2, week 3
Milestone	Counts towards M2. No later story builds on it, but US-35 waits for it: the office meets the service only once it serves tiles safely.
You know	SQL injection, from cyber security · strings and escape characters · parameters of a subroutine
Files	`Modules/Tessera/Infrastructure/Sql/Queries.cs` · `…/Sql/Database.cs` · `Modules/Tessera/Domain/SqlName.cs` · `tests/Tessera.UnitTests/approved/tile-query.sql` · `tests/Tessera.ArchitectureTests/BoundaryTests.cs`

IN PLAIN WORDS SQL injection is an attack that hides SQL inside a request, such as a layer named `x; DROP TABLE cafes`, in the hope that the database will run it. Two defences stop it. First, each value from the request travels beside the query as a ***bound parameter***, like an answer written in a form's box, so the database never reads it as SQL. Second, the only names that enter the SQL's text come from the catalogue, which the database itself wrote, and each is wrapped in double quotes.

> **As** the maintainer, **I want** no request to change the SQL the service sends, **so that** no office machine can read, change or drop a table through the tile API.

ACCEPTANCE CRITERIA

1. **Given** any tile address, **when** the tile query runs, **then** every value reaches the database as a bound parameter, and a property test proves it.
2. **Given** any query, **then** its text lives in `Queries.cs`, the one file that holds SQL, and only `Database` sends it, save the seed's journal schema (ADR 0009); a search at each review finds SQL text nowhere else.
3. **Given** a table or column name in a query, **then** it comes from the catalogue, through the one function that quotes names (US-03).
4. **Given** a request for a layer named `x; DROP TABLE cafes`, **then** the service answers 404, as a problem document (US-07), and every table keeps its rows.

TESTS TO WRITE FIRST

1. **Every address, one text** (SR-05). `Keeps_every_tile_address_out_of_the_text`, in `TileTests`, is a property test: it folds three random numbers onto a real tile, from level 0 to 22, builds the tile query and expects the approved text every time. `Sends_the_layer_and_the_tile_as_parameters` finds no `food.cafes` in the text and every value, from the layer's id to the limit of 10,000 features, among the parameters.
2. **The tile query's text** (SR-07). `Matches_the_approved_tile_query` compares the query for one tile with `approved/tile-query.sql`, an approved snapshot: a change to the query fails the test until someone reads the new text and copies it over the old file. The snapshot shows the quoted table, the quoted columns and `ORDER BY t."fid"`, the key the seed gives every table. `Quotes_every_name_safely`, written for US-03, guards the quoting itself.
3. **One door to the database** (SR-06). `Only_Infrastructure_talks_to_the_database`, in `BoundaryTests`, fails on any type outside `Modules.Tessera.Infrastructure` that uses Npgsql, Dapper or DbUp. `Keeps_every_sql_text_in_queries_cs`, in the unit tests' `SourceTests`, reads every C# file of the service as text and fails on the start of a SQL statement, such as `SELECT` or `DROP TABLE`, outside `Queries.cs`; comments may name SQL. `Finds_the_sql_in_queries_cs` proves the search sees SQL where it lives, so the test cannot pass by finding nothing.
4. **An attack changes nothing** (SR-08). `Answers_an_attack_with_404_and_keeps_every_row`, in `ServiceTests`, asks for the layers `x; DROP TABLE cafes` and `food.cafes'; --`, each written with its spaces and semicolon percent-encoded. It expects 404 as `application/problem+json`, and counts the rows of every table before and after: the two counts must match.
   `Serves_a_granted_table_named_as_an_attack_and_keeps_every_row` grants a real table named `x"; DROP TABLE food.cafes; --`, gets its one place in a tile, and finds every table whole.

THE IDEA AT WORK

CSHARP

```csharp
// Modules/Tessera/Infrastructure/Sql/Queries.cs, Tile (trimmed and rewrapped)
var properties = string.Concat(layer.Properties.Select(name => $", {name.Sql}"));
// …
return new(
    $"""
    SELECT ST_AsMVT(tile.*, @layer, @resolution, 'tessera_shape') FROM (
      SELECT ST_AsMVTGeom(ST_Transform(t.{layer.Geometry.Sql}, 3857), …)
             AS tessera_shape{properties}
      FROM {layer.Table.Sql} AS t
      -- … WHERE, ORDER BY and LIMIT @limit
    ) AS tile
    """,
    new
    {
        layer = layer.Id, resolution = limits.Resolution, z = tile.Z, x = tile.X, y = tile.Y,
        buffer = limits.Buffer, margin = (double)limits.Buffer / limits.Resolution,
        srid = layer.Srid, limit = limits.MaxFeatures,
    });
```

`Queries.Tile` writes the statement of section 2.1 in two parts: the names go into the text, and the values travel beside it. The text holds names alone, each the `.Sql` of a `SqlName` that the catalogue read from the database and `SqlName.Quote` wrapped (idea 4). Every value, from `z` to `limit`, and even the layer's id inside the tile, travels in the anonymous object beside the text, and Dapper hands each to PostgreSQL as a parameter, matched by name. `Query` is a record of a text and its parameters that never changes once made (idea 3). Idea 1's boundary rule shuts the door to the database on every type outside Infrastructure. A request's layer name is a lookup key and nothing more: `LayerCatalogue.Find` answers null for `x; DROP TABLE cafes`, and the route answers 404 before any query exists.

WHAT TO CARRY AWAY

- **Test every table, as the story asks.** An attack may aim at any table, so the attack tests count the rows of every table, not of `food.cafes` alone; one statement, built with `query_to_xml`, counts them all.
- **A person reads an approved text.** When `Matches_the_approved_tile_query` fails, read the new text line by line before you copy it over the old one; the test proves only that nothing changed unseen.

5.4.3 S-06 Refuse malformed URLs

Nginx refuses what it cannot read and any address that hides a dot as `%2e`, and the service refuses a hidden dot folder too, before it picks a route, so no request reaches a route in disguise and no refusal repeats the address.

Traces	SR-09, SR-10, SR-11, SR-12
Ideas	6 A request is a chain of checks · 2 A pure core, effects at the edge · 7 The host wires the parts through one door
Size	1 point · after US-07 · iteration 2, week 3
Milestone	Counts towards M2. Nginx enters the build here; US-35 opens it to the office, and S-07 and S-30 later add rules to it.
You know	The parts of a URL: scheme, host, path · characters and their codes, such as ASCII · validation of input
Files	`nginx/default.conf.template` · `Common/Api/Targets.cs` · `TesseraHost.cs` · `Common/Api/Problems.cs` · `scripts/lib/edge.ts`

> IN PLAIN WORDS A ***malformed URL*** is an address disguised so that it looks clean only after a web server tidies it, such as a path that climbs above the root with `..`, or the same `..` hidden as `%2e%2e`. Nginx stands in front of the service and refuses an address it cannot read, or one that hides a dot folder. A copy of the service reached directly refuses a hidden dot folder too, before it chooses a route, because Kestrel, ASP.NET Core's own web server, would otherwise tidy `/x/%2e%2e/food.cafes/…` into a tile's address. Each guard turns the visitor away; none rewrites the ticket.

> "As the maintainer, I want every malformed or disguised request refused, so that no request reaches a route in disguise.

ACCEPTANCE CRITERIA

1. **Given** a request target that climbs above the root, holds a stray or invalid `%`, an encoded null (`%00`) or a raw control character, **then** Nginx answers 400 before the service sees it.
2. **Given** that 400, **then** its body is a problem document (US-07) whose `title` reads 'Bad Request', and it never repeats the target.
3. **Given** a target that hides a dot folder as `%2e`, such as `/x/%2e%2e/`, **then** Nginx and the service alike answer 400 as a problem document; **given** an encoded slash (`%2F`) or a backslash, **then** it matches no route, and the service answers 404 as a problem document.
4. **Given** a clean tile target, **then** it reaches US-07's route unchanged.

TESTS TO WRITE FIRST

1. **Malformed targets are refused** (SR-09, SR-10, SR-11). The S-06 loop of `checkEdge` in `scripts/lib/edge.ts` writes twelve raw requests through Nginx, byte for byte, each with the status criteria 1 and 3 name. Ten must answer exactly 400: five dot folders hidden as `%2e`, in either case, `../..`, `%zz`, `%00` and two raw control characters. Two must answer exactly 404: an encoded slash and a backslash. Each answer is `application/problem+json`, its title reads 'Bad Request' or 'Not Found', and its body holds no `food.cafes`. Then `/malformed`, asked for by name, must answer the 404 of any unknown address.

2. **The answer hides the target** (SR-10). The same loop finds no part of the target in a 400's body, and so do the service's own tests below.
3. **A disguised address is refused** (SR-11). In `ServiceTests`, `Http.Raw` writes each request byte for byte straight to a copy, since `HttpClient` would tidy `%2e%2e` away first. `Refuses_a_dot_folder_in_disguise_with_400` sends five targets, from `/x/%2e%2e/food.cafes/0/0/0.pbf` to `/x/%2e%2e/health`, and expects 400 with the title 'Bad Request'; `Answers_a_backslash_with_404` sends three and expects a 404 problem document. `Answers_a_raw_control_character_with_400` and `Answers_an_encoded_null_with_an_empty_400` cover criterion 1 for a copy reached directly. In the unit tests, `TargetTests` tries the rule alone, with no web server: `Spots_a_dot_folder_in_disguise`, `Spots_a_raw_control_character` and `Lets_a_clean_target_pass`.
4. **Clean targets pass** (SR-12). The first check of `checkEdge` asks Nginx for `/food.cafes/0/0/0.pbf` and expects 200 and the tile's content type; `Lets_a_clean_address_reach_the_tile_route` sends the same address, and `/food%2Ecafes/0/0/0.pbf`, straight to a copy and expects 200. `scripts/e2e.ts` runs `checkEdge` after the browser tests, against a real Nginx and the built service.

THE IDEA AT WORK

CSHARP

```csharp
// Common/Api/Targets.cs
public static bool IsDisguised(string target) =>
    target.Any(char.IsControl) || PathOf(target).Split('/').Any(IsDotFolderInDisguise);
// …
private static bool IsDotFolderInDisguise(string folder) =>
    folder.Contains('%', StringComparison.Ordinal)
    && folder.Replace("%2e", ".", StringComparison.OrdinalIgnoreCase) is "." or "..";
// TesseraHost.cs
app.UseStatusCodePages();
// S-06: a target that hides a '..' as '%2e%2e', or holds a raw control character, answers
// 400 here, before routing, since Kestrel has already tidied the disguise out of the path
// that routing reads.
app.UseDisguisedTargetRefusal();
app.UseRouting();
```

The whole path is idea 6 at the scale of the machine: Nginx, then the service's own check, then the routes, and the first that refuses ends the request. Nginx refuses an address it cannot read before any `location` block runs, and its `error_page 400 /malformed` turns its HTML page into the problem document every other error carries. A `map` on `$request_uri`, the address exactly as sent, sets `$disguised` for any `%2e`, in any case, and the two locations that pass requests on answer it with 400. Behind Nginx, `IsDisguised` is a pure function (idea 2): a target in, true or false out, so `TargetTests` needs no web server. A folder counts as disguised only when it held a `%` and reads `.` or `..` once each `%2e` becomes a dot, so `food%2ecafes` passes. `UseDisguisedTargetRefusal` is the edge: it reads `RawTarget` from `IHttpRequestFeature`, the target before Kestrel tidied it, and answers `Problems.BadRequest()`, a 400 with no detail. `TesseraHost` adds it before `UseRouting` (idea 7), so no route ever sees a disguise; `UseStatusCodePages` turns the empty 404 of an address no route knows into a problem document.

> WATCH OUT Kestrel tidies an address before routing reads it: it decodes `%2e%2e` and removes the dot segment with the folder before it, as a browser would. So `/x/%2e%2e/food.cafes/0/0/0.pbf` once reached the tile route as `/food.cafes/0/0/0.pbf` and answered 200, though criterion 3 says a disguised target must reach no route. The routes see only the tidied path; the refusal reads the target as sent, where the disguise still shows. Kestrel refuses `%00` on its own with an empty 400, not a problem document, which repeats nothing; Nginx refuses it first, so a browser never sees that answer.

WHAT TO CARRY AWAY

- **Never repair an address.** Nginx and the service refuse what hides a dot folder or cannot be read; neither tidies a path into one that matches.
- **Two guards, two rules.** Nginx refuses every `%2e`, the service only a dot folder in disguise, so `/food%2Ecafes/0/0/0.pbf` gets 400 through Nginx and a tile from a copy reached directly. The story leaves open which rule is right.

TRY IT With Nginx running on the office machine, send a path that climbs above the root, and stop curl from tidying it first: `curl --path-as-is -i http://localhost/food.cafes/../../1/2/3.pbf`. Read the status and the body, and check that neither names the café layer.

5.4.4 US-35 Open the service to the office on port 80

WSL2's mirrored networking lets Ubuntu share the Windows machine's address, so office machines reach Nginx on port 80 by the machine's office name, and Nginx sees each one's own address.

Traces	FR-55, FR-56, FR-57, FR-58, SR-14, SR-15
Ideas	2 A pure core, effects at the edge · 3 Data is plain and never changes
Size	1 point · after US-07 · iteration 2, week 4
Milestone	Counts towards M2, whose map opens 'in the office'. US-23 builds on it, and so does S-07, which tells office machines from strangers by the address this story keeps.
You know	IP addresses and ports · firewalls, from network security · a LAN, and a name that points at an address
Files	`scripts/office-port.ts` · `scripts/lib/office.ts` · `scripts/lib/office.test.ts` · `.env.example`

IN PLAIN WORDS The service runs in Ubuntu 24.04, inside WSL2, on one Windows 11 machine in the office. WSL2 is a light virtual machine. In its mirrored networking mode, Ubuntu shares the Windows machine's own address, so a request to the Windows machine on port 80 reaches Nginx in Ubuntu directly. Nginx then sees the address of the office machine that sent it, and passes each request to the service on 127.0.0.1, port 7800. Two ***firewalls***, programs that decide which requests may enter a machine, stand in the way: Windows' own, and the one in front of WSL2 that belongs to Hyper-V, the part of Windows that runs virtual machines. Each must let port 80 in. The Windows machine keeps a reserved address, so its office name always points at it.

> **As** the maintainer, **I want** each office machine to reach the service on port 80 by the Windows machine's office name, with Nginx seeing each machine's own address, **so that** office machines can load tiles, and later stories can tell one office machine from another.

ACCEPTANCE CRITERIA

1. **Given** an office machine, **when** it opens `http://` and the Windows machine's office name, **then** it reaches Nginx in Ubuntu on port 80 and receives tiles in plain HTTP.

2. **Given** a request from an office machine, **then** Nginx sees that machine's own address, not the Windows machine's.
3. **Given** a reserved address for the Windows machine, **when** it restarts, **then** it keeps its office address, and the office name still points at it.
4. **Given** WSL2 restarting, **then** port 80 still reaches the service after `npm start`, with no step run again, since Ubuntu shares the Windows machine's address.
5. **Given** the Windows firewall and Hyper-V's firewall in front of WSL2, **then** each lets in requests on port 80, and on no other port.
6. **Given** WSL2 in any mode but mirrored, **when** `npm run office-port` runs, **then** it says so and prints the two steps that switch it.

TESTS TO WRITE FIRST

1. **The steps, without Windows** (FR-58, SR-15 in part). `scripts/lib/office.test.ts` holds three tests that `node --test` runs on any machine: 'asks nothing more of a machine already in mirrored mode', 'tells a machine in NAT mode, or an older WSL, how to switch', and 'opens the one port in both firewalls, and never through a port proxy'.
2. **Port 80 serves** (FR-55, SR-14). No test yet: it needs the office machine. From another office machine, run the check the script prints, `curl -I http://<this machine's office name>:80/food.cafes/0/0/0.pbf`, and expect 200; then read `docker logs --tail 5 tessera-nginx` and find that machine's address.
3. **A restart changes nothing** (FR-56, FR-57). No test yet. After `wsl --shutdown` and a fresh `npm start`, test 2 passes again; after a restart of Windows, the office name still reaches the machine.

THE IDEA AT WORK

```ts
export function networkingSteps(mode: string): readonly string[] {
  if (mode.trim() === mirrored) return [];
  return [
    // … one line that names the mode found and the mode needed
    '  1. Add these two lines to %UserProfile%\\.wslconfig:',
    '       [wsl2]',
    `       networkingMode=${mirrored}`,
    '  2. In PowerShell, run "wsl --shutdown", then open Ubuntu again and rerun this script.',
    '',
  ];
}
```

`networkingSteps` is the pure core of idea 2: a mode in, a list of lines out, nothing touched on the way, so the tests read its answer on a machine with no Windows. Its answer is idea 3, a `readonly` list that no caller may change. The thin edge is `scripts/office-port.ts`. It checks `/proc/version` for Microsoft's name, so it runs only inside WSL2. It asks `wslinfo --networking-mode` for the mode, prints these lines and then `firewallSteps`, and ends with exit code 1 when the mode must change first. The script prints the firewall commands rather than running them, since both need Windows and an administrator. Its Hyper-V rule names WSL's fixed `-VMCreatorId`, so port 80 opens to WSL alone, not to every virtual machine on the computer.

WHAT TO CARRY AWAY

- **Check it on the real machine.** WSL2, Windows and the two firewalls each keep their own settings, so tests 2 and 3 need a run on the office machine itself. Mirrored networking needs Windows 11 22H2 or later and WSL 2.0 or later.
- **No port proxy.** A Windows port proxy would also reach Ubuntu, but it hides every office machine behind the Windows machine's own address, which defeats S-07 and S-30.
- **'No other port' is unproved.** The firewall test checks that each printed rule opens port 80, not that nothing else is open. On the office machine, list the inbound rules once and record what you find.

5.4.5 US-23 Open the map of the area

The service writes the area into one built page and serves it at `/` and at each of the nine layers' pages, and the page keeps everything it shows in one model that only a pure update may replace, so tests check every rule of the page with values alone, before any browser opens.

Traces	FR-25, FR-26, FR-27, FR-28, FR-29, FR-30, FR-31, FR-32, NFR-61
Ideas	2 A pure core, effects at the edge · 3 Data is plain and never changes · 5 Failures are values
Size	3 points · after US-07 and US-35 · iteration 2, week 4
Milestone	Completes M2 at the review of Friday 13 November. UI-09, UI-15 and UI-12 build on it, and every later map story adds messages to its model.
You know	Records with named fields · events, and the code that handles them · client and server: a page and the files it loads
Files	`Modules/Tessera/Features/Map.cs` · `web/src/map/map.ts` · `web/src/app/App.tsx` · `web/src/layers/layers.ts` · `web/src/map/tiles.ts`

IN PLAIN WORDS The service learns to serve a map. The page is a fixed frame, and the tiles arrive later, like photographs slid into it. The map knows the nine layers from its own code, so it loads no list of layers. The front end keeps all it knows in one ***model***, a single value that only a pure ***update*** function may replace. Given the old model and a message, a value that says what happened, `update` returns the new model. Work with the outside world, such as moving the map, leaves through ***ports***. Here a port is no network port but a function passed in from outside, which a test swaps for a fake that records each call.

> **As** a stakeholder, **I want** one 2D map of the area at `/` and the same map opened on one layer at `/{layer}.html`, **so that** I can look at the data without installing anything.

ACCEPTANCE CRITERIA

1. **Given** `/`, **then** the map fills the window and opens on the area's home view, with the places of the nine layers (US-24) on a plain background.
2. **Given** `/food.cafes.html`, **then** the map opens with the cafés alone, in view.
3. **Given** the front end's state, **then** it is one immutable model changed only by a pure `update` function, and its tests run without a browser.

4. **Given** the front end, **then** it ships inside the service's own release.
5. **Given** the preview switched off in the settings, **then** `/` answers 404.
6. **Given** a pin, **then** it is a sign, not a button: a click on it opens nothing.
7. **Given** the map, **then** it names the nine layers in its own code and builds each tile's address, `/{layer}/{z}/{x}/{y}.pbf`, on the page's own origin; it loads no list of layers.
8. **Given** a tile request answers with a problem document, **then** the page shows its `detail`, or its `title` where it has none, as text over the map.
9. **Given** a browser that cannot draw the map, because it has no WebGL or has taken it away, **then** the page says so in words in place of the controls, and shows them again once the browser can draw; **and given** an error no part of the page catches, **then** the page says so in words rather than going blank.

TESTS TO WRITE FIRST

1. **A build that fails first.** The automatic build, in `.github/workflows/build.yml`, runs the map's `npm test` before the service's build. It checks the types, runs the linter (a checker of style and common mistakes) and runs Vitest, and it fails until the front end exists.
2. **One model, one pure update** (FR-27). In `web/src/app/app.test.ts`, 'starts from one model and changes it only through the update's handlers'. In `web/src/architecture.test.ts`, 'keeps React, the DOM and MapLibre out of every update file (ADR 0004, a pure core)'.
3. **The opening view** (FR-25, FR-26, FR-31). In `web/src/map/map.test.ts`, 'opens on the area's home view with the nine layers on, and asks for no list of layers' and 'opens /food.cafes.html with the cafés alone, framing the area that holds them'; in `web/src/layers/layers.test.ts`, 'builds each tile's address on the page's own origin'.
4. **A problem shows** (FR-32). In `map.test.ts`, 'shows a problem document's detail, or its title where it has none', 'leaves a failed tile's detail in the model, read from the answer's body' and 'makes words from the status alone when the body is no problem document'.
5. **A page that cannot draw says so** (NFR-61). In `web/src/app/App.test.tsx`, 'says so in words, in place of the controls, when MapLibre cannot start'; in `web/src/app/mount.test.tsx`, 'replaces a page that throws with words that say what to do'; in `web/e2e/cannot-draw.spec.ts`, a browser started with WebGL turned off: 'US-23 criterion 9: says so in words, with no zoom buttons over a blank map'.
6. **The service serves the page** (FR-26, FR-28, FR-29). In the unit tests' `MapTests`: `Writes_the_area_block_before_the_head_closes`, `Matches_the_block_in_adr_0006`, `Answers_404_when_there_is_no_page_to_show`, `Opens_a_page_named_for_one_of_the_nine_layers` and `Refuses_any_other_name`. In `ServiceTests`, `Serves_the_map_page_with_the_area_written_in` fetches `/` and `/food.cafes.html` from the built service, then the script the page names. In `scripts/lib/build.test.ts`, 'empties the release's map folder, so no file of an older map ships': the copy into `wwwroot/` never deletes a file, so `npm run build` empties the folder first (FR-28).
7. **In a real browser** (FR-25, FR-26, FR-30, FR-32). In `web/e2e/map.spec.ts`: 'opens / on the area's home view with the nine layers, asking only for the page, its files and tiles', 'opens /food.cafes.html with the cafés alone, all in view', 'treats a pin as a sign: a click on it opens nothing and moves nothing' and 'shows a tile's problem document over the map'.

THE IDEA AT WORK

CSHARP

```csharp
internal static IResult Answer(string? page, MapSettings settings) =>
    !settings.ShowPreview ? Problems.NotFound("The map preview is switched off.")
    : page is null ? Problems.NotFound("This release carries no map.")
    : TypedResults.Text(page, "text/html; charset=utf-8");
// …
public static string? LayerOf(string file) =>
    file.EndsWith(".html", StringComparison.Ordinal)
        && LayerIds.Contains(file[..^".html".Length])
        ? file[..^".html".Length]
        : null;
```

`MapRoutes` is the service's half of the story. Both functions belong to idea 2's pure core, values in and an answer out, so `MapTests` calls them with no web server. `UseMap` reads the built page once, at start-up, and `Render` writes the area into it as JSON in a `<script id="tessera-area">` before `</head>`, in the shape ADR 0006 fixes, so the map opens on the office's own area without a second request. `Answer` gives the page, or a 404 that says why there is none. `LayerOf` opens a page only for one of the nine layers, letter for letter, so `/index.html` and `/food.zoos.html` answer 404 rather than an empty map. The browser's half uses react-elmish, a library that gives a React page one model and a pure update. `init` in `map.ts` builds the first model from the page's props alone, every field `readonly` (idea 3). A refused tile arrives as a `Problem` value inside a `tileFailed` message, never as an exception (idea 5), and `tileProblem` in `tiles.ts` turns any refusal into that value, even a proxy's HTML page.

WHAT TO CARRY AWAY

- **Fake the ports, not the library.** The map's tests swap `fitBounds` and `jumpTo` for `vi.fn()` and read what the update asked of the map; MapLibre itself appears only in the browser tests.
- **Two lists must agree.** `MapRoutes.LayerIds` in C# and the map's own list in `web/src/layers/layers.ts` both name the nine layers. `Lists_the_same_nine_layers_as_the_map`, in `SourceTests`, reads the map's file, sorts both lists and fails when they differ (FR-31).

5.5 Iteration 3 · The map reads well

Iteration 3 teaches the map to move, to keep calm and to tell places apart. It runs from Monday 16 to Friday 27 November, with nine days and nine points for stories: seven for UI-09, UI-15, UI-12 and UI-19, and two for the start of UI-10, which the next iteration finishes. The review on Friday 27 November checks milestone M3, 'The map zooms, keeps calm and tells places apart'. Every story here changes the browser alone; no line of C# changes. The tests are Vitest files beside the code in `web/src`, which run with no browser, and Playwright tests in `web/e2e/map.spec.ts`, which drive a real Chrome through Nginx. The map carries the ideas of chapter 3 in their TypeScript forms: one model changed only by a pure `update` (idea 2), read-only values (idea 3), branded types (idea 4) and the six walled modules that `web/src/architecture.test.ts` guards (idea 1).

5.5.1 UI-09 Zoom by web-map levels 0 to 22

Two buttons move the map one whole level at a time, and a branded type holds every level the model keeps within 0 to 22, so the map behaves as every other web map does.

Traces	FR-33, FR-34, FR-35, FR-36, FR-37, NFR-62
Ideas	2 A pure core, effects at the edge · 4 A wrong value cannot be built
Size	2 points · after US-23 · iteration 3, the first story of week 5
Milestone	Counts towards M3 on 27 November. UI-19 reads its levels, and UI-10 and UI-11 build on them.
You know	Powers of two · selection with if · boundary data: the values at each end of a range
Files	`web/src/shared/geo.ts` · `web/src/map/levels.ts` · `web/src/map/map.ts` · `web/src/map/MapControls.tsx` · `web/src/map/maplibre.ts`

> IN PLAIN WORDS Every web map counts its levels the same way. Level 0 shows the whole world in one square 256 pixels wide, and each level up doubles the detail, so level 22 shows a single doorway. MapLibre, the library that draws the map, counts one lower, since it draws the world 512 pixels wide at its zoom 0. The map keeps the two counts apart with a ***branded type***: a number carrying a mark that exists for the compiler alone, so code that hands a level where MapLibre's zoom belongs fails to compile.

> "**As** a visitor, **I want** zoom levels from 0 to 22 that behave as on other web maps, **so that** the map feels familiar.

ACCEPTANCE CRITERIA

1. **Given** '+' or '−', **then** the map moves one whole level, doubling or halving the scale about the centre of the view, and stops at 0 and at 22.
2. **Given** the end of the range, **then** the button at that end turns grey.
3. **Given** a screen reader, **then** each button's label states the current level, such as 'Zoom in, now at level 15 of 22'. The level shows nowhere on screen.
4. **Given** a click on a cluster (UI-10), **then** the map may stop between levels, such as 16.4.
5. **Given** a pan, **then** the map never slides far enough to show its edge.
6. **Given** a zoom, **then** the pins keep their size on screen and move in step with the map.

TESTS TO WRITE FIRST

1. **The scale doubles.** 'doubles and halves the scale one level at a time', in `readouts.test.ts`, checks that a pixel spans half the ground at level 16 that it spans at 15. The test of the same name in `map.test.ts`, which ends '... through the map's port', sends `zoomIn` at level 16.4 and expects a fake `jumpTo` to receive 17.4.
2. **The ends hold.** 'keeps the level within 0 to 22', in `map.test.ts`, expects no command from '+' at 22 or '−' at 0, and a view reported at level 24 kept as 22.
3. **Two counts, never mixed.** 'keeps the two scales apart, as F#'s units of measure would: the compiler refuses one for the other' marks two lines `@ts-expect-error`, a comment that says the next line must not compile, so the test fails the day the compiler accepts them.

4. **The buttons speak.** ‘states the current level in each label, and nowhere on screen’ and ‘turns the button grey at the end of the range, and it does nothing there’, in `controls.test.tsx`.
5. **In a real browser.** ‘moves one level per button, and says so in the labels’ and ‘keeps the edge of the map out of view’, in `web/e2e/map.spec.ts`.

THE IDEA AT WORK

TS

```ts
// shared/geo.ts
export type Level = number & { readonly unit: "level" };
export function clampZoom(zoom: number): Level {
  return level(Math.min(MAX_ZOOM, Math.max(MIN_ZOOM, zoom)));
}
// map/map.ts
export const canZoomIn = (model: Model): boolean => model.view.zoom < MAX_ZOOM;
function zoomBy(step: 1 | -1, model: Model, props: Props): UpdateReturnType<Model, Message> {
  const allowed = step > 0 ? canZoomIn(model) : canZoomOut(model);
  return allowed
    ? [{}, jump(props, { ...model.view, zoom: clampZoom(model.view.zoom + step) })]
    : [{}];
}
// …
  viewChanged({ view }) {
    return [{ view: { ...view, zoom: clampZoom(view.zoom) } }];
  },
```

`zoomBy` is idea 2 in TypeScript. It reads the model and returns the fields that change, here none, with a command to move the map; it never touches MapLibre. The command calls the `jumpTo` port, which `map-handle.ts` carries out as a movement of 0.3 seconds, so a test fakes the port and checks the level it was handed. `Level` is idea 4. `clampZoom` returns one, pulling any number back within 0 to 22, and the update clamps each view the map reports rather than trusting the sender, so the model never holds a level outside the range. At the edge, `baseOptions`, in `maplibre.ts`, keeps the map’s border out of view: it sets `maxBounds` to the world and turns off copies of the world beside it (criterion 5).

WHAT TO CARRY AWAY

- **Clamp in the core, not in the button.** The button greys itself, yet `viewChanged` clamps whatever arrives, from a scroll wheel or a cluster as well.
- **Name each count by its type.** Two counts one apart, levels and MapLibre’s zoom, invite off-by-one faults; the brand turns each mix-up into a compile error.

TRY IT Criterion 6 has no test yet (FR-37). Write a browser test that zooms from level 15 to 16 and checks that one pin’s box keeps its width on screen.

5.5.2 UI-15 Keep the page calm in Material 3

One theme in `styles.css` holds every colour, one module holds every duration, and tests that read the source as text refuse any colour or movement outside them, so the page stays calm while later stories add controls.

Traces	NFR-49, NFR-50, NFR-51, NFR-52, NFR-53, NFR-54, NFR-55, NFR-56, NFR-57
Ideas	3 Data is plain and never changes · 2 A pure core, effects at the edge
Size	2 points · after US-23 · iteration 3, week 5
Milestone	Counts towards M3. It comes early because every later control, UI-19's scale bar, UI-10's clusters and UI-11's tooltips, follows its rules.
You know	Colours written as RGB in hexadecimal · constants · a file read as text and searched line by line
Files	`web/src/styles.css` · `web/src/design/theme.ts` · `web/src/design/motion.ts` · `web/src/map/map-handle.ts` · `web/src/map/landing.ts`

> IN PLAIN WORDS Material 3 is Google's design system: a set of rules for colour, type, shape and movement. Its ***colour roles*** name each colour by its job, such as `surface` or `on-surface`, and all of them grow from one seed, here the blue `#1F5FA8`. The page keeps quiet so the places stand out. Movement follows the visitor, lasts 0.3 seconds or less, and starts fast and settles slowly. The curve of a movement's speed is its ***easing***, and Material calls this one emphasised easing. A visitor who asks their system to reduce motion sees nothing move.

> **As** a visitor, **I want** a familiar, calm page on which only the places stand out, **so that** I can use it without learning it and look at the map for a long time without strain.

ACCEPTANCE CRITERIA

1. **Given** the page, **then** its colours come from Material 3's colour roles, built from the action blue `#1F5FA8`; the group colours appear only on pins and clusters.
2. **Given** any text, **then** it uses Roboto in Material's type scale; **given** any icon, **then** it comes from Material Symbols, rounded.
3. **Given** a surface, **then** it follows Material's elevation: the scale bar and the coordinates sit at level 1; the zoom buttons, the compass and clusters at level 2. Every surface is solid, with no see-through effect.
4. **Given** a shape, **then** it follows Material's corners: 12 pixels for the zoom buttons, the scale bar and the coordinates, and 4 for the name tooltips.
5. **Given** any control, **when** I hover over it or press it, **then** a state layer tints it by 8 or 12 per cent.
6. **Given** the page loading, **then** each pin drops into place, one after another, in 0.3 seconds.
7. **Given** any other movement, **then** it follows an action of mine, uses Material's emphasised easing and lasts 0.3 seconds or less. Nothing loops.
8. **Given** a visitor who has asked their system to reduce motion, **then** nothing moves.
9. **Given** the map's background, **then** it stays plain and quiet, with nothing drawn beneath the places (US-24), so the places stand out.

TESTS TO WRITE FIRST

1. **One theme.** ‘keeps the theme in styles.css’ and ‘lists every colour written outside the theme’s Material roles, and finds none’, in `web/src/rules.test.ts`, which reads every source file as text. Then ‘takes each group’s colour from the theme, never from code’, in `design.test.ts`.
2. **Short movement.** ‘moves for no longer than 300 ms, and not at all for a visitor who asks for less’, in `design.test.ts`; ‘makes every transition and animation last 300 ms or less’ and ‘lets nothing loop’, in `rules.test.ts`; ‘eases as Material’s emphasised curve: fast away, slow to rest’.
3. **One font of icons.** ‘draws every icon from Material Symbols’, in `rules.test.ts`, and ‘draws the zoom icons from Material Symbols, one glyph each’, in `web/e2e/map.spec.ts`, which expects two icons 24 pixels wide.
4. **Stillness on request.** ‘stops all movement when the visitor asks for reduced motion’, once in `rules.test.ts` against the stylesheet and once in `map.spec.ts`, where Chrome asks for less motion and MapLibre’s fade must be 0.
5. **The drop.** ‘drops each pin in 300 ms, one after another, 30 ms apart’, in `design.test.ts`; the seven tests of ‘UI-15 the pins’ landing, as a pure state machine’, in `landing.test.ts`; and ‘drops the pins in as the page opens, then hands over to the map’s own’, in the browser.

THE IDEA AT WORK

TS

```ts
// design/motion.ts
export const MOTION = { short: 150, exit: 200, medium: 300 } as const;

export function movementMs(reduced: boolean): number {
  return reduced ? 0 : MOTION.medium;
}

// map/map-handle.ts
const motion = () => {
  const duration = movementMs(prefersReducedMotion());
  return { duration, easing: emphasised, essential: duration > 0 };
};
```

Every duration the code animates lives in `MOTION`, plain data that `as const` freezes (idea 3). `movementMs` is a pure function of one yes-or-no answer (idea 2), so the test checks it with two values. The edge asks the browser the one impure question, `prefersReducedMotion`, and hands the result to MapLibre on every move. CSS keeps the same rule for what it animates: a `@media (prefers-reduced-motion: reduce)` block in `styles.css` sets `animation: none` and `transition: none` on every element. The colours follow the same pattern. They live in one place, the `@theme` block, and `readTheme` reads them at run time for MapLibre and for the canvas, a bitmap on which the page paints each pin; neither can read CSS.

WHAT TO CARRY AWAY

- **Test the source, not only the screen.** A test that reads every file for ‘#’, `rgb(` or a named colour stops a stray colour before any reviewer sees it.
- **Honour reduced motion twice.** CSS stops its own animations, and the map handle gives MapLibre a duration of 0; either alone leaves something moving.

> WATCH OUT Criteria 3, 4 and 5, the elevation, the corners and the state layer, have no test yet (NFR-51 to NFR-53). `styles.css` meets them through `shadow-level-1`, `--radius-medium` and `.state-layer`, but nothing fails if someone changes them. Criterion 2 is proved in part (NFR-50): the tests find Roboto and Material Symbols, and none checks the type scale.

5.5.3 UI-12 Tell place types apart without relying on colour

Each group carries a shape as well as a colour, both fixed in one table the compiler checks for every group, so a visitor who sees colour poorly can still tell the groups apart.

Traces	NFR-64, NFR-65
Ideas	4 A wrong value cannot be built · 3 Data is plain and never changes
Size	1 point · after US-23 and US-24 · iteration 3, week 5
Milestone	Counts towards M3, whose words 'tells places apart' it fulfils. UI-10 rings each cluster in the same group colours.
You know	Records and lookup tables · colours in hexadecimal · ratios
Files	`web/src/design/theme.ts` · `web/src/design/shapes.ts` · `web/src/design/canvas.ts` · `web/src/styles.css`

> IN PLAIN WORDS Some visitors see colours poorly, most often red against green, and a map that marks groups by colour alone leaves them guessing. So each pin carries a white shape inside its coloured head: a circle for Food and drink, a triangle for Outdoors, a square for Culture and a diamond for Transport. A ***contrast ratio*** compares the lightness of two colours, from 1:1 for the same colour to 21:1 for black on white; text needs at least 4.5:1 to read with ease.

> "**As** a visitor who sees colour poorly, **I want** each group marked by a shape as well as a colour, **so that** I can tell the groups apart.

ACCEPTANCE CRITERIA

1. **Given** a pin, **then** it shows its group's colour, with the group's shape in white inside: a circle for Food and drink, a triangle for Outdoors, a square for Culture and a diamond for Transport.
2. **Given** text in a group's colour on white, **then** its contrast is at least 4.5:1.

TESTS TO WRITE FIRST

1. **Four shapes, all different.** 'gives each group its own shape', in `design.test.ts`, checks the table and that the four shapes draw four different paths.
2. **Readable colours.** 'meets 4.5:1 for text in each group colour on white' reads each group colour from `styles.css` and measures it against `surface-lowest`, `#FFFFFF`.
3. **The pin the map draws.** 'draws each place with its group's pin', in `style.test.ts`, expects the image name `pin:` plus the group, and that `pin:bakeries`, no group at all, names no image.

THE IDEA AT WORK

```ts
// design/shapes.ts
export type Shape = "circle" | "triangle" | "square" | "diamond";
// design/theme.ts
export type GroupKey = "food" | "outdoors" | "culture" | "transport";

export interface GroupLook {
  readonly shape: Shape;
  readonly colour: `--color-group-${GroupKey}`;
}

export const GROUP_LOOKS: Readonly<Record<GroupKey, GroupLook>> = {
  food: { shape: "circle", colour: "--color-group-food" },
  outdoors: { shape: "triangle", colour: "--color-group-outdoors" },
  culture: { shape: "square", colour: "--color-group-culture" },
  transport: { shape: "diamond", colour: "--color-group-transport" },
};
```

The table is idea 4 three times over. `Record<GroupKey, GroupLook>` demands one entry for each of the four groups, so a group left out fails to compile. `Shape` admits four words alone. The colour is a template literal type, a string that must read `--color-group-` and then a group's key, so a misspelt CSS variable never reaches the page. The table is plain, read-only data (idea 3), and `drawPin`, in `canvas.ts`, paints each pin from it with the paths of `shapes.ts`. On white, the four group colours in the theme reach from about 5.5:1, for Food and drink, to 7.4:1, for Culture.

WHAT TO CARRY AWAY

- **Never let colour carry meaning alone.** A shape, a word or a pattern must say the same thing.
- **Measure contrast in a test.** A colour chosen by eye passes on one screen and fails on another; the ratio is the same everywhere.

5.5.4 UI-19 Read distance, direction and position

Pure functions turn a level and a latitude into a round length, and a point into words, so the scale bar, the compass and the coordinates tell a visitor how far, which way and where.

Traces	FR-49, FR-50, FR-51, FR-52, FR-53, FR-54, NFR-60, NFR-66, NFR-67
Ideas	2 A pure core, effects at the edge · 3 Data is plain and never changes
Size	2 points · after UI-09 · iteration 3, the first two days of week 6
Milestone	Completes the stories of M3, reviewed on Friday 27 November.
You know	Cosine from trigonometry · units and their conversion · rounding and formatting numbers as text
Files	`web/src/map/scale.ts` · `web/src/map/coordinates.ts` · `web/src/map/ScaleBar.tsx` · `web/src/map/MapControls.tsx` · `web/src/map/map.ts`

> IN PLAIN WORDS A printed map has three helpers: a ***scale bar*** that shows how far a stretch of the map spans, a compass that shows north, and a grid of latitude and longitude. The page has all three. The bar's length depends on the level and on the latitude, because Web Mercator, the flat projection every web map uses, stretches the world more and more towards the poles. At Macau's latitude, 22.19° N, one pixel spans 4.42 m at level 15.

> "As a visitor, I want a scale bar, a compass and the coordinates under my pointer, so that I can judge distances, keep my bearings and say where a point lies.

ACCEPTANCE CRITERIA

1. **Given** any level, **then** a scale bar at the bottom left shows a round length, 1, 2 or 5 of a unit, no longer than 140 pixels: 500 m at level 15 and 50 m at level 18 over the area, at Macau's latitude.
2. **Given** the bar, **then** it has four sections, filled and empty in turn, marked '0', the halfway figure and the full length with its unit.
3. **Given** a zoom, **then** the bar changes its length and figures within 0.3 seconds.
4. **Given** I click the bar, **then** it switches between metric (metres and kilometres) and imperial (feet and miles), and its label names the units it will switch to.
5. **Given** the map, **then** a compass at the top of the controls on the right points north, with a red needle and a red 'N'.
6. **Given** a map that never turns, **then** the compass is a sign, not a button, and screen readers hear 'Compass: north is up'.
7. **Given** the pointer over the map, **then** the latitude and longitude under it show to the right of the scale bar, latitude first, to two decimal places: '22.19° N, 113.55° E'. They change as the pointer moves.
8. **Given** the pointer off the map, or over the scale bar, the compass or a zoom button, **then** the readout goes.
9. **Given** a screen reader, **then** the readout stays silent, since it follows the pointer alone.

TESTS TO WRITE FIRST

1. **Two known lengths.** 'chooses 500 m at level 15 and 50 m at level 18', in `readouts.test.ts`, also requires both bars to be 140 pixels or shorter. 'takes the longest length of 1, 2 or 5 of a unit that fits' tries the boundaries: 0.9 gives 0.5, 1.99 gives 1, 2 gives 2.
2. **Four sections.** 'draws four sections, filled and empty in turn, under "0", the halfway figure and the full length', in `controls.test.tsx`, reads four quarters from the bar's gradient in `styles.css`, and 0, 250 and 500 m under them (FR-50).
3. **The units switch.** 'switches between metric and imperial on a click, and names the units it will switch to', in `readouts.test.ts`: 2,000 ft at level 15, and the label 'Switch to metric units'. The tests of the same name in `map.test.ts` and `controls.test.tsx` check the message and the click.
4. **North, spoken.** 'names north for screen readers, as a sign rather than a button', in `controls.test.tsx`.
5. **The point in words.** 'writes the point under the pointer as 22.19° N, 113.55° E, latitude first, to two decimal places', then 'names the southern and western hemispheres, and takes a value that rounds to zero as north or east'.
6. **In a real browser.** 'reads 500 m at level 15 and 50 m at level 18, and switches units on a click' and 'shows the point under the pointer while it is over the map, and not over a control', in `web/e2e/map.spec.ts`.

THE IDEA AT WORK

```ts
export function metresPerPixel(level: number, latitude: number): number {
  return (EQUATOR_METRES * Math.cos((latitude * Math.PI) / 180)) / (256 * 2 ** level);
}
// …
export function scaleBar(level: number, latitude: number, units: Units): ScaleReading {
  const perPixel = metresPerPixel(level, latitude);
  const { length, unit, metres } = roundLength(perPixel * SCALE_MAX_PX, units);
  const [one, many] = UNIT_WORDS[unit];
  const other = units === "metric" ? "imperial" : "metric";
  return {
    widthPx: Math.round(metres / perPixel),
    half: figure(length / 2),
    full: `${figure(length)} ${unit}`,
    // …
  };
}
```

Both functions in `scale.ts` are pure (idea 2), so the tests run them with numbers alone. At level 15 over Macau a pixel spans 4.42 m, so 140 pixels span 619 m; the longest round length below that, 500 m, takes 113 pixels. `scaleBar` returns one `ScaleReading`, a read-only record (idea 3). It holds the width, both figures and the spoken label (cut from the excerpt) together, so the drawing and the words always agree. `ScaleBar.tsx`, at the edge, draws the record and sends `toggleUnits` on a click, and does nothing more. The model keeps the units, and `pointerMoved` keeps the point, or `null`, which hides the readout.

WHAT TO CARRY AWAY

- **Measure where the reader looks.** The bar uses the latitude of the map's centre, since Web Mercator's scale changes from north to south.
- **Compute once, show twice.** One record feeds both the bar and its label, so a screen reader hears the length the eye sees.

TRY IT Work out the bar at level 18 by hand: 0.553 m a pixel, 140 pixels, then the longest of 1, 2 or 5 of a unit below the result. Check your answer against the browser test, which reads the bar's three figures run together, 0, 25 and 50 m, as '02550 m'. Three criteria are proved only in part. No test checks the compass's place or its red needle (FR-52, criterion 5); the 0.3 seconds of criterion 3 rest on a rule that caps every duration written in the source, and no test watches the bar change (NFR-60); and no test moves the pointer over the scale bar or the compass to see the readout go (FR-54, criterion 8). Write a test for any one of them.

5.6 Iteration 4 · Clusters, names and the release

Iteration 4 finishes the map and releases Parts A and B to the office on Monday 7 December. It runs from Monday 30 November: six working days, one for the review, four for the points of UI-10, UI-11 and UI-17, and one in hand. If nothing has slipped, the day in hand, Friday 4 December, goes to S-07, which lets Nginx answer office machines alone. Monday 7 December holds the review and the release, milestone M4, 'Parts A and B released to the office': `npm test` from a clean copy, `npm start` on the office machine, and the map opened from another office machine.

Tuesday 8 December is a holiday, and the release never slips past the Monday. If time runs short, the plan cuts S-07 first, then UI-17, then UI-11, and never a test.

5.6.1 UI-10 See crowded places merge into clusters that open as I zoom

MapLibre clusters the places in the browser from the tiles it has loaded, a pure update decides how far a click zooms, and the service stays as it was, so the map stays readable at every level.

Traces	FR-38, FR-39, NFR-58, NFR-59, NFR-63
Ideas	2 A pure core, effects at the edge · 3 Data is plain and never changes
Size	3 points · after UI-09 · begun in iteration 3 on Wednesday 25 November, finished on Monday 30 November
Milestone	Counts towards M4. UI-11 builds on it, since names stay hidden inside a cluster.
You know	Distance between two points · iteration over a list · events: code that runs when something happens
Files	`web/src/map/style.ts` · `web/src/map/places.ts` · `web/src/map/maplibre.ts` · `web/src/map/targets.ts` · `web/src/map/labels.ts` · `docs/adr/0005-cluster-in-the-browser.md`

> IN PLAIN WORDS Zoomed out, Macau's 900 places would pile into one blot of pins. A ***cluster*** stands for several places close together: one round marker with their number, ringed in the colours of their groups. Zoom in and it breaks back into its pins. MapLibre can cluster only GeoJSON, places written as JSON, but the service sends vector tiles. So the map reads the places of every tile it has loaded, joins them into one list, and hands the list to a clustered source (ADR 0005).

> "As a visitor, I want crowded places shown as one marker that opens as I zoom in, so that the map stays readable at every level.

ACCEPTANCE CRITERIA

1. **Given** pins that would overlap at the current level, **then** they show as one cluster with their number, ringed in the colours of their groups in proportion.
2. **Given** I zoom in far enough for them to part, **then** the pins move out from the cluster to their own places within 0.3 seconds; zooming out draws them back.
3. **Given** I click a cluster, **then** the map zooms no further than it must to open it, and at least to level 16.
4. **Given** a screen reader, **then** a cluster's label names its places and says that zooming in shows them.
5. **Given** a cluster on screen, **then** it rests still, with no pulse or loop.

TESTS TO WRITE FIRST

1. **The merge distance.** 'merges pins that sit closer than the distance for the level', in `clusters.test.ts`, loads two places into Supercluster, the library MapLibre clusters with, at the same radius: 0.8 times the distance apart gives one cluster at level 14, and 1.2 times gives two.
2. **Each level halves it.** 'halves that distance at each level up' checks `mergeDistanceMetres`, then the same pair at level 15.
3. **A click opens the cluster.** 'zooms at least to level 16' and 'zooms no further than the cluster needs to open', in the same file, run the update with a fake `jumpTo`.

4. **The label.** ‘UI-10 c4: names a cluster’s places and says that zooming in shows them’ expects ‘3 places: A, B, C. Zoom in to see them.’, and ten names then ‘and 30 more’ for a cluster of 40.
5. **In a real browser.** ‘opens a cluster on a click, at least to level 16’ and ‘names a cluster’s places for screen readers’, in `web/e2e/map.spec.ts`. ‘lets nothing loop’, in `rules.test.ts`, covers criterion 5.

THE IDEA AT WORK

TS

```ts
// map/style.ts
export const CLUSTER_RADIUS_PX = 48;
// …
    [SOURCE.places]: {
      type: "geojson",
      data: empty(),
      cluster: true,
      clusterMaxZoom: toMapZoom(LAST_CLUSTER_LEVEL),
      clusterRadius: CLUSTER_RADIUS_PX,
      clusterProperties: CLUSTER_PROPERTIES,
    },
// map/map.ts
  clusterOpened({ center, expansionLevel }, _model, props) {
    return [{}, jump(props,
      { center, zoom: clampZoom(Math.max(OPEN_CLUSTER_LEVEL, expansionLevel)) })];
  },
```

The clustered source is plain data (idea 3) that the pure `buildStyle` returns. Places closer than 48 pixels join, up to level 15, and `clusterProperties` counts each group’s places, so that `drawCluster`, in `design/canvas.ts`, can ring the marker in proportion. The edge does the rest. `feedPlaces`, in `maplibre.ts`, reads the loaded tiles and feeds the source only when the fingerprint of the places’ ids changes. On a click, `openCluster`, in `targets.ts`, asks MapLibre at which level the cluster parts and sends `clusterOpened`; the update (idea 2) takes the larger of that level and 16. Over Macau at level 14, 48 pixels span about 425 m on the ground.

WHAT TO CARRY AWAY

- **Cluster where the pins are drawn.** ADR 0005 keeps the service’s one API, the tile, and pays with an edge effect: a cluster at the screen’s edge counts only the places in loaded tiles.
- **Give a canvas a voice.** MapLibre draws on a canvas, a picture made of pixels that no screen reader can read, so `clusterLabel` names the first ten places in an invisible button over each cluster.

WATCH OUT Criterion 2, pins moving out of a cluster within 0.3 seconds, has no test yet (NFR-58). MapLibre’s `fadeDuration` is set from `movementMs`, but no test watches the pins part. Two more criteria are proved in part: no test checks the coloured ring of criterion 1 (FR-38), and ‘lets nothing loop’ reads the page’s own styles, not the clusters MapLibre draws (NFR-59).

5.6.2 UI-11 Read place names when zoomed in

One layer of the style draws each place's name in a dark tooltip from level 16, and a filter keeps the names of clustered places hidden, so a visitor reads the map close in without clicking.

Traces	FR-40, FR-41, FR-42
Ideas	2 A pure core, effects at the edge · 3 Data is plain and never changes
Size	1 point · after UI-10 · iteration 4, Tuesday 1 December
Milestone	Counts towards M4. Third in the cut order; UI-17 builds on it.
You know	Selection with if · comparison operators · drawing in layers, back to front
Files	`web/src/map/style.ts` · `web/src/map/levels.ts` · `web/src/design/canvas.ts`

> IN PLAIN WORDS At level 16 a street fills the screen, and each pin has room for its name. A ***symbol layer*** is a layer of MapLibre's style that draws an icon or a text at each point. The map gives names a symbol layer of their own, which starts at level 16 and skips any point that is a cluster. Each name sits in a small dark box, a tooltip, with light text.

> **As** a visitor, **I want** names beside the pins once I zoom in, **so that** I can read the map without clicking every pin.

ACCEPTANCE CRITERIA

1. **Given** level 16 or above, **then** each pin on screen shows its name beside it in a dark plain tooltip, one after another.
2. **Given** a level below 16, **then** no pin shows its name.
3. **Given** a cluster, **then** its pins show no names until it opens.

TESTS TO WRITE FIRST

1. **The style decides.** 'shows names from level 16, and none inside a cluster', in `style.test.ts`, reads the names layer's `minzoom`, its text and its filter from the style value.
2. **The tooltip.** 'writes names in light text on a dark tooltip, in the self-hosted Roboto' checks the colour role, the tooltip image and the two font files.
3. **In a real browser.** 'shows names from level 16, and none below', in `web/e2e/map.spec.ts`, finds no drawn name at level 15 over the busiest street and some at level 16.

THE IDEA AT WORK

```ts
// map/levels.ts
export const NAMES_FROM_LEVEL = level(16);
// map/style.ts
const notCluster: ExpressionSpecification = ["!", ["has", "point_count"]];
// …
    {
      id: LAYER.names,
      type: "symbol",
      source: SOURCE.places,
      filter: notCluster,
      // UI-11, criterion 2: below level 16 no name shows.
      minzoom: toMapZoom(NAMES_FROM_LEVEL),
      layout: nameLayout(),
      paint: { "text-color": theme.inverseOnSurface, ...paintOf(LAYER.names, landed) },
    },
```

The whole story is data in a pure function's answer (ideas 2 and 3), so a test reads it with no browser and no map. MapLibre gives only a cluster a `point_count`, so `notCluster` keeps names off clustered places. `minzoom` takes MapLibre's count, one below the level, which `toMapZoom` works out; the branded types of UI-09 stop anyone forgetting the conversion. The text takes the `inverse-on-surface` colour role, and `drawTooltip`, in `canvas.ts`, paints the dark box that MapLibre stretches around each name.

WHAT TO CARRY AWAY

- **Name each number once.** `NAMES_FROM_LEVEL`, `LAST_CLUSTER_LEVEL` and `OPEN_CLUSTER_LEVEL` sit together in `levels.ts`, so the stories' 15 and 16 stay in step.
- **Ship the font with the page.** Roboto comes from the page's own site, so the office never waits on a font service outside it.

5.6.3 UI-17 Keep place names from overlapping

A fixed order ranks every place, each name tries four sides of its pin, and a name with no room gives way while its pin stays, so a crowded street reads at a glance and the same name wins every time.

Traces	FR-43, FR-44, FR-45, FR-46, FR-47, FR-48
Ideas	2 A pure core, effects at the edge · 3 Data is plain and never changes
Size	2 points · after UI-11 · iteration 4, Wednesday 2 and Thursday 3 December
Milestone	The last story of M4. Second in the cut order, since nothing in the plan builds on it.
You know	Sorting · rectangles that overlap · a priority order
Files	`web/src/map/style.ts` · `web/src/map/places.ts` · `src/Tessera/Modules/Tessera/Infrastructure/Sql/Queries.cs` · `tests/Tessera.UnitTests/approved/tile-query.sql` · `web/e2e/__screenshots__/`

> IN PLAIN WORDS MapLibre places symbols one at a time. Each icon or name claims a box on screen, its ***collision box***, and a later name whose box would cover a claimed one must move or give way. So the order of placing decides who wins. The map places clusters and pins first, then names, and ranks the places by a ***sort key***: layers in the map's order, then ids within a layer. Each name tries the right of its pin, the left, above and below before it gives way.

> "As a visitor, I want place names that never overlap, so that I can read a crowded street at a glance.

ACCEPTANCE CRITERIA

1. **Given** any level, **then** no name overlaps another name or another icon, across every layer.
2. **Given** two places whose names would clash, **then** a fixed order decides which keeps its name: between layers, the order set in the map's style; within a layer, the places' ids, so the same name wins every time.
3. **Given** a name with no room on its first side, **then** the map tries the right, the left, above and below before it leaves the name out.
4. **Given** a name left out, **then** its icon still shows, and the name appears once zooming in makes room.
5. **Given** a crowded area, **then** places stay in clusters below level 16 (UI-10); from level 16 every place shows its icon, and the fixed order decides only which names win the space.
6. **Given** a name near the edge of a tile, **then** it gives way to its neighbours as one in the middle does.

TESTS TO WRITE FIRST

1. **Four sides, and a pin that stays.** 'gives the names four anchors, and lets a name give way while its pin stays', in `style.test.ts`, the name the story gives it.
2. **Icons first.** 'places every icon before any name, so no name covers an icon' expects the last three layers to be names, pins and clusters, since MapLibre places from the top down.
3. **A fixed order.** 'orders names between layers by the map's order, then by id, whatever order the tiles bring' feeds the cafés out of order and twice over. On the service's side, `Matches_the_approved_tile_query`, in `TileTests`, compares the tile query with its approved text, `approved/tile-query.sql`, which ends `ORDER BY t."fid"`, since the test's table is keyed by `fid`, as every shipped table is. `Gives_a_view_no_order` checks that a view, which has no key, gets no `ORDER BY` at all: an order on a column the view lacks would fail every tile of it.
4. **Clusters below 16.** 'keeps every place apart from level 16 (UI-17, criterion 5)', in `clusters.test.ts`.
5. **Pictures that must match.** 'matches the approved image at level 16 over the busiest street, collision boxes shown', and the same at 18 and 20, in `web/e2e/map.spec.ts`, against `UI-17-level-16.png`, `-18` and `-20`.

THE IDEA AT WORK

```ts
// map/places.ts
const LAYER_STEP = 1_000_000;
// …
const byId = (a: { values: { id: string } }, b: { values: { id: string } }): number =>
  a.values.id < b.values.id ? -1 : a.values.id > b.values.id ? 1 : 0;
// …
    return [...unique.values()].toSorted(byId).map(
      ({ point, values }, rank): PlaceFeature => ({
        type: "Feature",
        geometry: { type: "Point", coordinates: point },
        properties: { ...values, layerId, group: layer.group, one: layer.one,
          sort: layerIndex * LAYER_STEP + rank },
      }),
    );
```

`placeCollection` is pure (idea 2): it takes the tiles' features and returns new read-only records (idea 3), sorted with `toSorted`, which leaves the old list as it was. A place's key is its layer's position times a million plus its rank by id, so every place of the first layer outranks every place of the second. `byId` compares ids character code by character code, not by the rules of the browser's language, so every browser ranks them the same way. The style then hands MapLibre the rest: `symbol-sort-key` reads `sort`, `text-variable-anchor-offset` lists the four sides, and the pins' `icon-allow-overlap` keeps every pin on show. The service helps too: `Queries.Tile` orders each table's features by its primary key, so tiles arrive the same every time, and `LIMIT` keeps the same features when a square holds more than `Tiles.MaxFeatures`.

WHAT TO CARRY AWAY

- **Decide clashes by rule, not by chance.** Without a fixed order, the name that wins depends on which tile arrived first, and the map flickers as a visitor pans.
- **Let the label give way, never the place.** A missing name returns at the next level; a missing pin would hide a place.

WATCH OUT A screenshot may differ from its approved image in 1 pixel in 100, the margin `playwright.config.ts` sets, since Chrome's software drawing smooths edges differently from run to run. A moved name changes far more. When the test fails, look at the new picture before you approve it. The approved images come from Playwright's own Chromium and its fonts, and another browser draws text a few per cent differently. So the automatic build in `.github/workflows/build.yml` sets `TESSERA_PLAYWRIGHT_IN_DOCKER`, and `scripts/e2e.ts` runs Playwright inside its own Docker image, named after the version `web/package.json` pins, which brings the same Chromium and fonts to every machine. No run on GitHub has yet proved that step (FR-43). `npm test` runs the pictures on your own machine with Playwright's browser, and `TESSERA_PLAYWRIGHT_IN_DOCKER=1 npm run e2e` runs them in the image. Criterion 6, names at a tile's edge, has no test yet (FR-48).

5.6.4 S-07 Answer office machines alone

Nginx refuses every machine outside the office's network with 403 before any other rule runs, and `npm start` refuses a network setting that would let in the world, so a stranger cannot use the service.

Traces	SR-16, SR-17, SR-18, SR-19
Ideas	6 A request is a chain of checks · 5 Failures are values · 4 A wrong value cannot be built
Size	1 point · after US-35 · iteration 4, on the day in hand, Friday 4 December
Milestone	Joins M4 only if the day is free; first in the cut order. S-30 builds on it after 8 December.
You know	IP addresses and networks · binary: the bits of an address · validation: range and format checks
Files	`nginx/default.conf.template` · `scripts/lib/plan.ts` · `scripts/start.ts` · `scripts/lib/edge.ts` · `scripts/e2e.ts`

> IN PLAIN WORDS Every office machine sits on one network, a range of addresses written as an address, a slash and a ***prefix***, such as `192.168.10.0/24`. The prefix counts the leading bits every address in the range shares, so `/24` means the first three numbers. Nginx answers a machine inside the range and the machine Tessera runs on, and refuses any other with 403, 'Forbidden'.

> "As the maintainer, I want Nginx to answer office machines alone, so that a machine outside the office cannot use the service.

ACCEPTANCE CRITERIA

1. **Given** `TESSERA_OFFICE_NETWORK`, such as `192.168.10.0/24`, **when** a machine inside it asks, **then** Nginx answers it.
2. **Given** a machine outside it, **when** it asks for anything, **then** Nginx answers 403 as a problem document (US-07), with `Cache-Control: no-store`, and never passes the request to the service.
3. **Given** the machine Tessera runs on, **then** Nginx answers it too, so `npm start` can ask `/health`.
4. **Given** `TESSERA_OFFICE_NETWORK` missing, misspelt, or with a prefix below 8, such as `0.0.0.0/0`, which lets in every address, **then** `npm start` refuses to start and names it.

TESTS TO WRITE FIRST

1. **The setting, checked purely.** 'accepts an office network, and refuses one too wide or misspelt', in `scripts/lib/plan.test.ts`, which `node --test` runs with no Docker: `192.168.10.0/24` and `127.0.0.4/30` pass; `0.0.0.0/0`, `10.0.0.0/7`, a range with no prefix and `192.168.300.0/24` fail, and `readStartSettings` names the variable.
2. **The network reaches Nginx.** 'runs nginx on the machine's network, with both ports filled in', in the same file, finds `TESSERA_OFFICE_NETWORK=192.168.10.0/24` among the arguments of `docker run`.
3. **A stranger is refused.** `checkEdge`, in `scripts/lib/edge.ts`, which `npm run e2e` runs with the office set to `127.0.0.4/30`: a tile asked from `127.0.0.2` answers 403 as `application/problem+json` with `no-store`, and the same tile from `127.0.0.5` answers 200. Linux answers every `127.x.x.x` address on the machine itself, so one machine plays both.

THE IDEA AT WORK

NGINX

```
geo $office {
    default 0;
    127.0.0.1 1;
    ${TESSERA_OFFICE_NETWORK} 1;
}
# …
    if ($office = 0) {
        return 403;
    }
    error_page 403 = @forbidden;
# …
    location @forbidden {
        default_type application/problem+json;
        add_header Cache-Control no-store always;
        return 403 '{"type":"about:blank","title":"Forbidden","status":403}';
    }
```

Nginx's `geo` block turns the caller's address into one number: 1 inside the office or on the machine itself, 0 elsewhere. The Nginx image fills in the network from the environment as it starts. The `if` sits at the top of the `server` block, the first link of the request's chain (idea 6), so a stranger meets the refusal before the limits, the tile store and the service, and no kept tile leaks out. Like every error, the refusal is a problem document with no address in it. The setting gets a pure check of its own. `isOfficeNetwork`, in `plan.ts`, accepts four numbers up to 255 and a prefix from 8 to 32. `readStartSettings` gathers every fault into one `Result` (idea 5), so `npm start` names them all and starts nothing.

WHAT TO CARRY AWAY

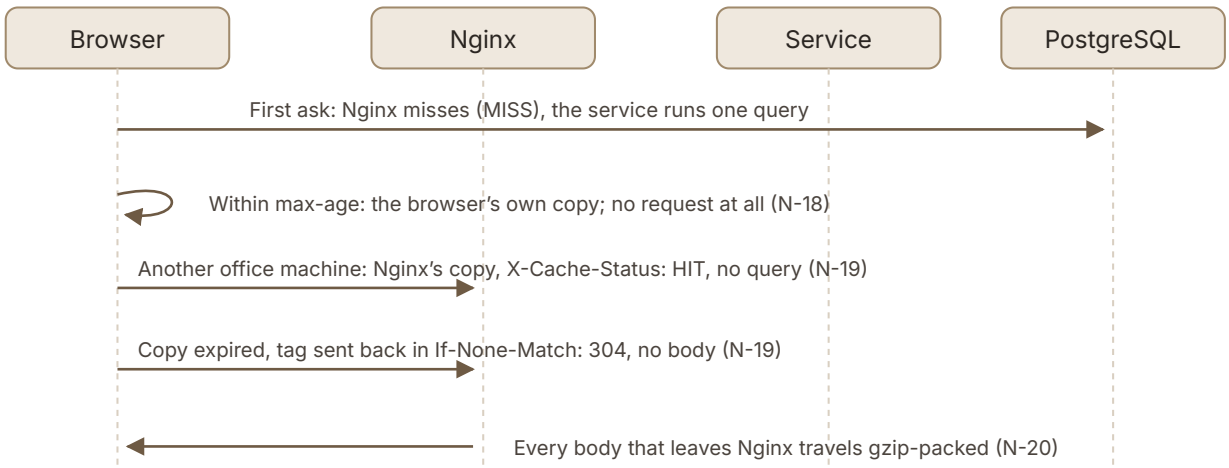
- **Check the setting before it reaches Nginx.** Nginx would accept `0.0.0.0/0` and open the door to the world; the start-up script refuses it.

THINK IT OVER The office may one day reach the map through a VPN, a second network of its own. `TESSERA_OFFICE_NETWORK` holds one range. What would you change, in the template and in `isOfficeNetwork`, to accept two?

5.7 Part C · Tiles that cost less

Once S-07 is built, the release of 7 December leaves thirteen stories, worth 22 points, for a second plan that starts on 9 December and ends about 18 January 2027. Part C comes first, because each of its three stories makes a working tile cheaper to serve again and lighter to send. Its four points rest on one idea: a tile the service has built should cost it nothing the second time. Nginx does most of the work; the service sets the headers that tell Nginx and the browser what they may keep. From here on, a service file named without its folder, such as `Features/Tiles.cs`, lives in `src/Tessera/Modules/Tessera/`, and one that starts `Common/` lives in `src/Tessera/`.

EXHIBIT 5.1 One tile asked for again and again: who answers, and what it costs



Source: `nginx/default.conf.template`, `Features/Tiles.cs`, `scripts/lib/edge.ts`

As Exhibit 5.1 shows, only the first ask of each minute reaches PostgreSQL. Each of the three stories below removes one kind of cost, in the order the tile meets them.

5.7.1 N-18 Let a browser keep a tile

One header, set from one setting, tells every browser how long to keep a tile, so a visitor who pans back and forth costs the service one query a tile a minute, not one a pan.

Traces	NFR-01, NFR-02, NFR-03, NFR-04, NFR-05
Ideas	2 A pure core, effects at the edge · 4 A wrong value cannot be built
Size	1 point · builds on US-07 · Part C, second plan
You know	Client and server · a request and its response · units of time
Files	`Features/Tiles.cs` · `Domain/Settings.cs` · `Common/Api/Problems.cs` · `tessera.json`

> IN PLAIN WORDS A browser asks for the same tile every time the visitor pans back over it, and every request costs a query. HTTP lets the service say how long an answer stays good: a ***Cache-Control*** header with ***max-age***, a number of seconds. For that long the browser keeps its copy and asks nothing. The places change only when a new seed script runs, so a minute is safe: new places show on every screen within a minute.

> **As** the maintainer, **I want** every tile to tell the browser how long to keep it, **so that** a visitor who pans back and forth costs the service one query a tile a minute, not one a pan.

ACCEPTANCE CRITERIA

1. **Given** a tile answered 200, **then** it carries `Cache-Control: public, max-age=N`, where N is `Tiles.CacheSeconds` in `tessera.json`; the shipped file and the default both set 60.
2. **Given** `Tiles.CacheSeconds` set to 0, **then** a tile carries `Cache-Control: no-cache`, so a browser checks before it reuses a copy, and Nginx keeps nothing.

3. **Given** any error answer, such as the 404 of US-07, **then** it carries `Cache-Control: no-store`, so no browser keeps a problem document.
4. **Given** a new seed script, **when** `max-age` seconds pass, **then** every browser asks again and sees the new places.
5. **Given** a setting below 0, or not a whole number, **then** the service refuses to start and names the setting (US-00 c4).

TESTS TO WRITE FIRST

1. **The header follows the setting.** `Tells_the_browser_and_nginx_how_long_to_keep_a_tile`, in the unit tests' `TileTests` (NFR-01, NFR-02), turns 60 into `public, max-age=60` and 0 into `no-cache`, with no server and no database.
2. **The check refuses -1.** `Names_every_value_out_of_range_at_once`, in `SettingsTests` (NFR-05), sets `CacheSeconds` to -1 and finds it named; `Refuses_values_that_disagree` sets it to `"soon"` and finds it named again.
3. **Through the real service.** `Serves_a_tile_as_a_mapbox_vector_tile`, in `ServiceTests` (NFR-01), reads `public, max-age=60` on a real tile; `Answers_an_unknown_layer_with_a_problem_document` (NFR-03) reads `no-store` on a 404. `checkEdge`, in `scripts/lib/edge.ts`, reads the same header through Nginx.
4. **The map asks once.** In `web/e2e/map.spec.ts`, 'asks once for a tile when the visitor pans away and back within the minute' opens the home view, pans to Coloane, pans back, and expects each home tile asked for once.

> WATCH OUT **No test yet** for criterion 4 (NFR-04): no test runs a new seed script and waits out `max-age`. Criterion 2 is proved in part (NFR-02): the test checks the header, never Nginx.

THE IDEA AT WORK

CSHARP

```csharp
// Features/Tiles.cs, inside TileServer
public static string CacheControl(int seconds) =>
    seconds > 0 ? $"public, max-age={seconds}" : "no-cache";
// …
public sealed record TileResult(ReadOnlyMemory<byte> Bytes, string CacheControl) : IResult
{
    public Task ExecuteAsync(HttpContext httpContext)
    {
        httpContext.Response.Headers.CacheControl = CacheControl;
        return TypedResults.Bytes(Bytes, ContentType, entityTag: Tag(Bytes.Span))
            .ExecuteAsync(httpContext);
    }
}
```

The decision lives in `CacheControl`, a pure function of one number (idea 2), so the first test needs no server. The edge, `TileResult`, copies its result into a header as ASP.NET Core writes the answer, and decides nothing. The number comes from `TileSettings`, whose `[Range(0, int.MaxValue)]` on `CacheSeconds` means a negative time never reaches this code: the start-up check refuses it first (idea 4). Errors take another road. `Problems.Shape` stamps `no-store` on every problem document the service writes, so no error borrows a tile's header.

WHAT TO CARRY AWAY

- **One number everywhere.** Browsers and Nginx keep a tile for the same `max-age`, so new places reach every screen within that time.
- **Never keep an error.** A kept 404 would hide a table published a minute later.

5.7.2 N-19 Answer a kept tile without a query

Each tile carries a weak tag made from its bytes, and Nginx keeps each tile for `max-age`, so a crowd opening the map costs the database one query a tile, not one a visitor.

Traces	NFR-06, NFR-07, NFR-08, NFR-09, NFR-10
Ideas	2 A pure core, effects at the edge · 6 A request is a chain of checks
Size	2 points · builds on N-18 and US-07 · Part C, second plan
You know	A hash as the fingerprint of a file · hexadecimal · a cache as fast storage close to where it is needed
Files	`Features/Tiles.cs` · `nginx/default.conf.template` · `scripts/lib/edge.ts`

IN PLAIN WORDS When `max-age` passes, the browser asks again, though it still holds its copy. HTTP lets it say so: it sends the copy's label, an ***ETag***, in an `If-None-Match` header, and the answer may be 304 ('Not Modified') with no body, which means 'yours is still good'. The service makes each tile's ETag from its bytes. Nginx keeps each tile for `max-age`, answers a second request from its kept copy, and answers 304 when the tags match.

> **As** the maintainer, **I want** a tile a browser already holds to cost nothing, and each tile to be built once a minute at most, **so that** a crowd opening the map costs the database one query a tile, not one a visitor.

ACCEPTANCE CRITERIA

1. **Given** a tile answered 200, **then** it carries a weak `ETag`, `W/"…"`, made from the tile's bytes alone, so the same bytes carry the same tag on any machine and after a restart. It is weak because N-20's packed form is the same tile.
2. **Given** a request whose `If-None-Match` equals the tile's tag, **then** the answer is 304 with no body.
3. **Given** a tile built within the last `Tiles.CacheSeconds` seconds, **when** any request for it arrives, **then** Nginx answers from its kept copy, says `X-Cache-Status: HIT`, and runs no query.
4. **Given** kept tiles, **then** together they take no more than 64 MB, Nginx's `max_size`; when the room is full, the tiles used least go first.
5. **Given** a new seed script, **when** `max-age` seconds pass, **then** every kept tile has expired, and the next request builds it again.

TESTS TO WRITE FIRST

1. **The tag follows the bytes.** `Tags_a_tile_by_its_bytes`, in the unit tests' `TileTests` (NFR-06), gives the bytes 1, 2, 3 the same tag twice, gives 1, 2, 4 another, and expects every tag to start `W/"`. A test of the same name in `ServiceTests` asks the running service for one tile twice and reads two equal weak tags.

2. **A second ask is a hit.** `checkEdge`, in `scripts/lib/edge.ts` (NFR-08), asks Nginx for `/food.cafes/0/0/0.pbf` twice and expects `X-Cache-Status: HIT` and the same bytes the second time.
3. **A matching tag is 304.** The same check sends the first answer's tag back and expects 304 and no body. `Answers_a_matching_tag_with_304`, in `ServiceTests` (NFR-07), proves the service does the same without Nginx, and that the 304 still carries `public, max-age=60`, so the browser keeps its copy another minute.

> WATCH OUT **No test yet** for criteria 4 and 5 (NFR-09, NFR-10). The 64 MB ceiling and the expiry stand in `nginx/default.conf.template`, but no test fills the store or waits out a minute after a new seed script.

THE IDEA AT WORK

CSHARP

```csharp
// Features/Tiles.cs, inside TileServer
public static EntityTagHeaderValue Tag(ReadOnlySpan<byte> bytes) =>
    new($"\"{Convert.ToHexStringLower(SHA256.HashData(bytes).AsSpan(0, 16))}\"",
        isWeak: true);
```

`Tag` reads its bytes and nothing else, so two machines, or one machine before and after a restart, give one tile one tag (idea 2). It keeps 16 of SHA-256's 32 bytes, written as 32 hexadecimal characters; an empty tile's tag reads `W/"e3b0c44298fc1c149afbf4c8996fb924"`. The service writes no 304 itself. `TileResult` hands the tag to `TypedResults.Bytes`, and ASP.NET Core compares it with the browser's `If-None-Match` and answers 304 on its own; Nginx does the same from its store. The request passes Nginx's store before it reaches C# (idea 6), which is why a hit runs no query.

> WHAT TO CARRY AWAY
>
> - **Read the cache's own word.** `X-Cache-Status` says whether a query ran; timing alone proves nothing.
> - **Tag the bytes, not the time.** A tag made from a clock would change on every restart and send every browser's copy back for nothing.

5.7.3 N-20 Send tiles and the map's files packed

Nginx packs every tile and every file of the map with gzip for a browser that accepts it, so the map opens sooner and each pan costs the network less.

Traces	NFR-11, NFR-12, NFR-13, NFR-14
Ideas	6 A request is a chain of checks
Size	1 point · builds on US-07, US-23 and N-19 · Part C, second plan
You know	Lossless compression · run-length encoding · kilobytes and megabytes
Files	`nginx/default.conf.template` · `scripts/lib/edge.ts` · `scripts/e2e.ts`

> IN PLAIN WORDS The map's own code is about a megabyte of JavaScript, and a tile runs from about 4 to 18 KB. Both shrink when ***gzip*** packs them: a tile by about half, text by two thirds or more. A browser says `Accept-Encoding: gzip` in every request. Nginx then answers with the packed body and `Content-Encoding: gzip`, and the browser unpacks the body before it reads it.

> **As** a visitor, **I want** the map and its tiles to arrive packed, **so that** the map opens sooner and each pan costs the network less.

ACCEPTANCE CRITERIA

1. **Given** a request with `Accept-Encoding: gzip` for a tile, the page or one of the map's files, **when** the answer is 200, **then** its body is gzip-packed, it carries `Content-Encoding: gzip`, and the unpacked bytes equal the plain answer's.
2. **Given** a request without `Accept-Encoding`, **then** the body travels plain, with no `Content-Encoding`.
3. **Given** either answer, **then** it carries `Vary: Accept-Encoding`, so no browser or proxy hands a packed body to a client that asked for plain.
4. **Given** a 304 (N-19), **then** the weak ETag matches whichever form the browser holds, since both forms are the same tile.

TESTS TO WRITE FIRST

1. **A tile packs and unpacks.** `checkEdge`, in `scripts/lib/edge.ts` (NFR-11), asks for the tile with `Accept-Encoding: gzip`, expects `Content-Encoding: gzip` and `Vary: Accept-Encoding`, and unpacks the body with `gunzipSync` to the plain answer's bytes.
2. **The map's script shrinks.** The same check finds the main script in the page and expects it packed at under half its plain size, and unpacked to the same bytes.
3. **Plain on request.** The check asks with `Accept-Encoding: identity` and compares the plain bytes (NFR-12, in part).
4. **The map still opens.** `scripts/e2e.ts` runs every Playwright test through Nginx, and a browser always sends `Accept-Encoding`.

> WATCH OUT **Proved in part,** all four criteria. No check packs the page itself (NFR-11); the plain requests send `identity` rather than no header, and none reads `Content-Encoding` on the answer (NFR-12); only the packed answer's `Vary` is read (NFR-13); and the 304 check sends back the plain answer's tag alone (NFR-14). Each gap is one more `assert` in `checkEdge`: a good first exercise.

THE IDEA AT WORK

NGINX

```
# nginx/default.conf.template; the last line is one line in the file
# N-20: tiles and the map's files travel packed with gzip. gzip_proxied any packs answers to
# requests that came through another proxy too, such as an office's web proxy, which Nginx
# would otherwise send unpacked. Level 1 is the fastest: a tile shrinks nearly as much as at
# level 9, for a fraction of the work. gzip_vary adds 'Vary: Accept-Encoding', so a cache along
# the way never hands a packed copy to a client that cannot unpack it. gzip_types lists the
# kinds worth packing; images are packed already.
gzip on;
gzip_proxied any;
gzip_comp_level 1;
gzip_vary on;
gzip_types application/vnd.mapbox-vector-tile application/problem+json
           application/json text/css text/javascript application/javascript;
```

Five lines of Nginx do the whole story, and the C# service never learns that gzip exists: `TesseraHost` adds no compression of its own. That is idea 6 at the scale of the office: a request passes a chain of stages, and

packing is one stage that Nginx owns. The weak tag from N-19 lets the stage stand apart: Nginx may change the bytes on the wire, and the tag still names the same tile.

> WHAT TO CARRY AWAY
>
> - **Pack lightly.** Level 1 is fast, and a tile gains little from more.
> - **Compare bytes, not sizes.** The check unpacks the body and compares it byte for byte; a smaller body alone could be a broken one.

5.8 Part D · Steady when the database is not

Part D holds seven stories, worth 11 points, that keep the service steady when the database is slow, busy, late or down, so the service and the database may start in any order. Its settings story comes first, because the later stories read the settings. Most of Part D's rules turn a database failure into a value the code can name, and then into a 503 that asks the browser to wait. Most of its criteria have a test, several against a real PostgreSQL in Docker; each page names those still waiting.

5.8.1 N-21 Read every setting from one file

Every setting but the database's address lives in `tessera.json`, each named with its unit and checked as the program starts, so the maintainer can change the service's behaviour without reading its code.

Traces	NFR-41, NFR-42, NFR-43, NFR-44, SR-26
Ideas	3 Data is plain and never changes · 4 A wrong value cannot be built · 7 The host wires the parts through one door
Size	2 points · builds on US-00 and N-18 · Part D, second plan
You know	Records and their fields · validation: presence, range and type checks · constants
Files	`tessera.json` · `Common/Infrastructure/Settings.cs` · `Domain/Settings.cs` · `Infrastructure/Settings.cs` · `TesseraModule.cs`

> IN PLAIN WORDS `tessera.json` sits beside the program and groups the settings in sections. ASP.NET Core reads it, then the environment, and each module binds its sections to records of its own. Each length of time carries its unit in its name, such as `Tiles.CacheSeconds`, so no one reads seconds as milliseconds. The database's address comes from `DATABASE_URL`, the name pg_tileserv gives it, so no password sits in the file.

> "As the maintainer, **I want** every setting in one file, each named with its unit, and any of them overridden from the environment, **so that** I can read and change the service's behaviour without reading its code.

ACCEPTANCE CRITERIA

1. **Given** `tessera.json`, **then** its keys sit in five sections, `Database`, `Tiles`, `Map`, `Catalogue` and `Area`, beside ASP.NET Core's own `Urls` and `Logging`, and each time carries its unit in its name.
2. **Given** a key left out, or the file missing, **then** each key takes its default.

3. **Given** an environment variable that names a key, with `__` for each step down, such as `Tiles__CacheSeconds=0`, **then** it overrides the file.
4. **Given** a key a section does not read, or a value outside its limits, **then** the service refuses to start and names every such problem at once (US-00 c4).
5. **Given** the shipped file, **then** it holds no password and listens on 127.0.0.1 alone.

TESTS TO WRITE FIRST

1. **The shipped file reads.** `Reads_the_shipped_file`, in `SettingsTests` (NFR-41, in part), reads the real `tessera.json`, copied beside the tests, and finds 60 in `TileSettings`, 15 in `AreaSettings` and 30 in `CatalogueSettings`.
2. **Every key left out takes its default.** `Gives_each_key_left_out_its_default` (NFR-42, in part) runs on `{}`, an empty file, and finds a pool of 4 and the area named Macau.
3. **A variable overrides a key.** `Lets_a_variable_override_a_key` (NFR-43) lays `Tiles:CacheSeconds` = 0 over the shipped file and reads 0. `Names_a_misspelt_key_with_every_other_problem`, in `StartTests`, starts the built program with real variables, `Tiles__MaxZoom=31` and `Database__PoolSize=0`, and finds both named, though the file beside it sets 22 and 4.
4. **An unknown key is refused, and named.** `Refuses_a_key_it_does_not_know` (NFR-44) writes `CacheSecond`, one letter short, and finds `'CacheSecond'` in the problem. `Names_a_misspelt_key_with_every_other_problem`, in `SettingsTests`, writes the same slip beside a `MaxZoom` of 31 and a `PoolSize` of 0, with no `DATABASE_URL`, and finds all four named, the key as `Tiles: 'CacheSecond' is not a setting`; its namesake in `StartTests` finds the same in one failed start. `Names_a_value_it_cannot_read_with_every_other_problem` writes 'soon' for `CacheSeconds` and finds `Tiles: CacheSeconds must be a whole number, not 'soon'` beside the zoom. `Names_every_value_out_of_range_at_once` sets three values out of range and leaves out `DATABASE_URL`, and finds all four named; `Refuses_values_that_disagree` sets `MinZoom` 10 and `MaxZoom` 5 and finds 'MaxZoom must not be below MinZoom'.
5. **No password, and this machine alone.** `Ships_a_file_with_no_password_that_listens_on_this_machine_alone` (SR-26) finds no 'assword' and no `postgresql://` in the shipped file, and `"Urls": "http://127.0.0.1:7800"`.

> WATCH OUT **Proved in part,** criteria 1 and 2. No test checks the five sections or the units in the names (NFR-41), and no test starts the program with no file at all (NFR-42). A misspelt key joins the list of problems, named with its section, so criterion 4 holds for keys and values alike. Started with `Tiles__CacheSecond=5`, `Database__PoolSize=0` and no `DATABASE_URL`, the program printed three lines and stopped with exit code 1:
>
> ```
> DataAnnotation validation failed for 'DatabaseSettings' members: 'PoolSize' with the error:
> 'The field PoolSize must be between 1 and 1000.'.
> Tiles: 'CacheSecond' is not a setting; Tiles holds Resolution, Buffer, MaxFeatures, MinZoom,
> MaxZoom, CacheSeconds.
> DataAnnotation validation failed for 'DatabaseConnection' members: '' with the error:
> 'DATABASE_URL: missing or empty'.
> ```
> TEXT
>
> A misspelt section name, such as `Tile`, still passes in silence and leaves its section at the defaults: `Leaves_a_misspelt_section_at_its_defaults` proves it, and the story lists it under 'Open to talk'.

THE IDEA AT WORK

JSON

```
{
  "Urls": "http://127.0.0.1:7800",
  "Database": { "PoolSize": 4, "ConnectionLifetimeSeconds": 3600,
                "QueryTimeoutSeconds": 10, "ConnectionWaitSeconds": 3 },
  "Tiles": { "Resolution": 4096, "Buffer": 256, "MaxFeatures": 10000,
             "MinZoom": 0, "MaxZoom": 22, "CacheSeconds": 60 },
  "Map": { "ShowPreview": true },
  "Catalogue": { "RefreshSeconds": 30 },
  "Area": { "Name": "Macau", "MinLongitude": 113.52, "MinLatitude": 22.10,
            "MaxLongitude": 113.61, "MaxLatitude": 22.22,
            "HomeLongitude": 113.55, "HomeLatitude": 22.19, "HomeLevel": 15 }
}
```

CSHARP

```
// Common/Infrastructure/Settings.cs; line breaks moved to fit the page
public static void Section<T>(IServiceCollection services, string name) where T : class
{
    services.AddOptions<T>()
        .Configure<IConfiguration>((options, configuration) =>
            Bind(options, configuration.GetSection(name)))
        .ValidateDataAnnotations()
        .ValidateOnStart();
    services.AddSingleton<IValidateOptions<T>>(provider =>
        new KeyCheck<T>(provider.GetRequiredService<IConfiguration>().GetSection(name)));
}
```

The shipped file, laid out more tightly here and with its `Logging` section cut, holds the whole of the service's behaviour in one place: plain data (idea 3) that an operator reads and Git keeps. `Section` binds one section to a record, such as `TileSettings`, whose attributes give each key its limits and whose initial values give its defaults, so a value outside them cannot be built (idea 4). `Bind` hands .NET's binder only the keys that read cleanly, so it never throws, and `KeyCheck`, a check of Tessera's own, names each key the record lacks and each value of the wrong kind. It runs beside the record's own checks, so its problems join theirs in one list. .NET's binder could refuse an unknown key itself, but it throws at once, and the first slip would hide every other. `ValidateOnStart` makes the checks run as the program starts, not when a request first reads the setting. `TesseraModule.AddSettings` calls `Section` once for each of the five sections (idea 7), so no class that uses a setting ever checks it again.

WHAT TO CARRY AWAY

- **Put the unit in the name.** `CacheSeconds` cannot be read as minutes.
- **Refuse to start rather than guess.** A misspelt key that passed in silence would leave its default in place, and nobody would know why the change did nothing.

5.8.2 N-22 Ask the browser to wait when the database cannot answer

A closed set of records sorts every database failure into one of four kinds, and the kinds that waiting may cure answer 503 with `Retry-After: 5`, so the map waits and tries again rather than showing a fault.

Traces	NFR-18, NFR-19, NFR-20, NFR-21
Ideas	5 Failures are values · 6 A request is a chain of checks
Size	2 points · builds on N-21 and US-07 · Part D, second plan
You know	Selection with many cases · error codes · client and server
Files	`Infrastructure/Sql/Database.cs` · `Infrastructure/Settings.cs` · `Features/Tiles.cs` · `Common/Api/Problems.cs`

> IN PLAIN WORDS When the database is down, busy or slow, the fault lies outside the service and will pass. HTTP's 503, 'Service Unavailable', with `Retry-After`, says exactly that: 'try again in 5 seconds'. A 500 says 'something is broken'. PostgreSQL names every refusal with a five-character SQLSTATE code, so the service can tell the two apart.

> "As a map developer, I want a passing database problem to answer 503 with `Retry-After`, and no query to run for ever, so that my map waits and tries again rather than showing a fault.

ACCEPTANCE CRITERIA

1. **Given** a database that cannot be reached, refuses a connection, has no room for another, or is stopping or starting, **when** a tile is asked for, **then** the answer is 503 with `Retry-After: 5` and `Cache-Control: no-store`, as a problem document.
2. **Given** a query that runs past `Database.QueryTimeoutSeconds` (10), or a wait for a connection past `Database.ConnectionWaitSeconds` (3), **then** the answer is 503.
3. **Given** a failure, **then** no copy of it is kept; the next request asks again.
4. **Given** any other database failure, **then** the answer stays 500.

TESTS TO WRITE FIRST

1. **Which failures wait.** `Asks_the_browser_to_wait_for_a_failure_that_may_pass`, in `DatabaseErrorTests` (NFR-18), sorts `08006`, `57014`, `57P01` and `53300` as `Unavailable`. `Sorts_any_other_failure_into_its_kind` sorts `42501` and `42P01` as `MissingTable`, `42703` as `MissingColumn` and `XX000` as `Other` (NFR-21). `Lists_every_kind_as_a_case` makes sure the four records and the list in `[Closed]` agree.
2. **An unreachable database answers 503.** `Answers_503_when_the_database_cannot_be_reached`, in the integration tests' `CatalogueTests` (NFR-18, NFR-20), points a pool at a port nothing holds and reads 503, `Retry-After: 5` and `no-store`.
3. **A slow query answers 503 within its time.** `Answers_503_when_a_query_runs_past_its_time` (NFR-19) asks for a tile of a view that sleeps three seconds, under a limit of one, and reads 503 in under three. `Gives_up_on_a_frozen_database_within_the_settings_limits`, in `FrozenDatabaseTests`, freezes a real

database with `docker pause` under a wait of one second and a query limit of two, and gets `Unavailable` back within three and a half.

4. **A 500 names its cause in the log.** `Logs_the_cause_of_a_500`, in `TileTests` (NFR-21), hands `TileServer` a refusal waiting cannot cure, 42883, and a log that keeps each line. It reads 500, and finds one line at error level that names the layer, the tile `15/26766/14284`, the code and the database's message.

> WATCH OUT A 500 tells a stranger nothing, so its cause must go somewhere the maintainer looks. `TileServer` takes an `ILogger`, under the category `Tessera.Tiles`, and its `Fault` method writes the layer, the tile, the SQLSTATE code and the database's message at error level before it answers 500. No test fills the pool and waits out `ConnectionWaitSeconds`; that such a wait counts as 57014 rests on Npgsql's behaviour and on the comment in `Database.Error`.

THE IDEA AT WORK

CSHARP

```csharp
// Infrastructure/Sql/Database.cs; line breaks moved to fit the page
public static DatabaseError Of(string sqlState, string message) => sqlState switch
{
    PostgresErrorCodes.InsufficientPrivilege or PostgresErrorCodes.UndefinedTable
        => new MissingTable(sqlState, message),
    PostgresErrorCodes.UndefinedColumn => new MissingColumn(sqlState, message),
    PostgresErrorCodes.QueryCanceled or PostgresErrorCodes.AdminShutdown
        or PostgresErrorCodes.CrashShutdown or PostgresErrorCodes.CannotConnectNow
        or PostgresErrorCodes.TooManyConnections => new Unavailable(sqlState, message),
    _ when sqlState.StartsWith("08", StringComparison.Ordinal)
        => new Unavailable(sqlState, message),
    _ => new Other(sqlState, message),
};
```

A failure is a value with a name (idea 5). `Database.Run` catches whatever a query throws, once, and `Error`, which section 3.5 quotes, turns it into a code: the database's own, 57014 for a time limit, or 08006 for a database that never answered. `Run` also keeps a clock of its own, the connection wait and the query limit together, 13 seconds as shipped, since a frozen database can hold Npgsql's limits longer; when that clock runs out, the answer is 57014 too. `Of` then sorts the code into one of four records, and the guard `when` catches the whole class 08, a failed connection, in one line. `TileServer.Answer` switches on the four: `Unavailable` and `MissingColumn` become a 503, `MissingTable` a 404, and `Other` a 500 with no detail, whose cause `Fault` writes to the log. The ExhaustiveMatching analyser stops the build if that switch misses a record. `Problems.Shape` then adds `Retry-After: 5` to every 503 and `no-store` to every problem, and the failure leaves through the same chain as a tile does (idea 6).

> WHAT TO CARRY AWAY
>
> - **A fault that waiting may cure is a 503; anything else is a 500.** A 503 tells the map to ask again shortly; a 500 tells the maintainer that something in the service is wrong.

5.8.3 N-23 Serve a newly granted table without a restart

The service rereads its list of layers every 30 seconds, keeps the last good list through a failure, and starts before the database answers, so a data publisher never restarts it by hand.

Traces	NFR-22, NFR-23, NFR-24, NFR-25
Ideas	2 A pure core, effects at the edge · 5 Failures are values · 7 The host wires the parts through one door
Size	2 points · builds on US-03 and N-22 · Part D, second plan
You know	Iteration on a timer · threads that share one value · SQL GRANT and REVOKE
Files	`Infrastructure/LiveCatalogue.cs` · `TesseraModule.cs` · `Infrastructure/Settings.cs` · `src/Tessera/Program.cs`

> IN PLAIN WORDS The service rereads its list of layers every `Catalogue.RefreshSeconds`, 30 by default, so a grant shows within half a minute. It also starts before the database answers, and asks each browser to wait until the list loads, so the two may start in any order. .NET runs the rereading as a hosted service (section 3.7), a loop that starts and stops with the program.

> "As a data publisher, **I want** a table I grant to be served within half a minute, and the service to start whether or not the database is up, **so that** I never restart it by hand.

ACCEPTANCE CRITERIA

1. **Given** a table granted after start-up, **then** it is served within `Catalogue.RefreshSeconds` seconds.
2. **Given** a grant revoked, **then** the layer leaves the list within the same time, and a new tile of it answers 404; tiles already kept stay in browsers and in Nginx for up to `Tiles.CacheSeconds`.
3. **Given** the database down at start, **then** the service starts, serves the map, and answers each tile 503 until the list loads; it tries every 5 seconds.
4. **Given** a reload that fails, **then** the last good list stays.

TESTS TO WRITE FIRST

1. **No list until the first load; the last good list through a failure.** In the unit tests' `CatalogueTests`, `Holds_no_list_until_the_first_load_succeeds` feeds a `LiveCatalogue` a failure, then a good list, and finds no list after the first; `Keeps_the_last_good_list_through_a_failure` feeds a good list, then a failure, and still finds the good list (NFR-25). `Tries_until_the_database_answers` and `Loads_once_when_refresh_is_zero` run the loop itself.
2. **A grant and a revoke within one refresh,** through the real database. `Sees_a_grant_and_a_revoke_within_one_refresh`, in the integration tests' `CatalogueTests` (NFR-22, NFR-23), refreshes every 200 ms, grants `stage.late`, sees it join the list, revokes it and sees it leave. `Answers_404_for_a_table_revoked_after_start_up` revokes a table under a loaded list and reads 404 as a problem document, through PostgreSQL's own refusal, 42501.
3. **The program starts with the database down.** `Starts_with_the_database_down_and_answers_503`, in `StartTests` (NFR-24), runs the built program against a port nothing holds, and reads 503 with `Retry-After: 5` while the program keeps running.

> WATCH OUT **Proved in part,** criterion 3 (NFR-24): no test asks for the map's page while the database is down, so 'serves the map' rests on `UseMap`, which reads no database.

THE IDEA AT WORK

CSHARP

```csharp
// Infrastructure/LiveCatalogue.cs
public async Task Refresh(CancellationToken cancel)
{
    var loaded = await load(cancel).ConfigureAwait(false);
    if (loaded.IsFailure)
    {
        log.LogWarning("Could not read the list of layers ({SqlState}): {Message}",
            loaded.Error.SqlState, loaded.Error.Message);
        return;
    }
    var old = Interlocked.Exchange(ref _current, loaded.Value);
    // … the log speaks only when the set of layers changes
}

protected override async Task ExecuteAsync(CancellationToken stoppingToken)
{
    // … inside a try that ends the loop quietly when the program stops
    while (!stoppingToken.IsCancellationRequested)
    {
        await Refresh(stoppingToken).ConfigureAwait(false);
        if (Current is not null && every <= TimeSpan.Zero)
            return;
        await Task.Delay(Current is null ? retry : every, stoppingToken).ConfigureAwait(false);
    }
}
```

`Refresh` reads a `Result`, never an exception (idea 5). It swaps a good list in whole with `Interlocked.Exchange`, one step no other thread can see half done; it logs a failure with the database's code and leaves the old list in place. Requests read the list through `Current`, which reads it afresh each time with `Volatile.Read`. The loop makes one choice on each turn: before the first list, wait `retry`; after it, wait `every`, or stop when `every` is zero. `LiveCatalogue` takes its `load` as a function, so a unit test hands it a queue of answers with no database (idea 2). `TesseraModule.AddTessera` wires the real parts together (idea 7): the real load, `RefreshSeconds` as `every`, `Problems.RetryAfterSeconds`, 5, as `retry`, and the same object as a hosted service, so the loop and the tiles share one list. Until that list loads, `TileServer.Answer` answers 503, 'The list of layers is not ready yet.'

> WHAT TO CARRY AWAY
>
> - **A failed refresh changes nothing.** A database that blinks for a second must not unpublish every layer.
> - **One wait, two places.** The loop retries every 5 seconds, the same 5 that `Retry-After` asks of each browser.

5.8.4 N-24 Serve a crowd from one query

Nginx lets the first request build a new tile and holds the rest until its answer arrives, so a crowd opening the map at once cannot fill the database's pool.

Traces	NFR-15, NFR-16, NFR-17
Ideas	6 A request is a chain of checks
Size	1 point · builds on N-19 · Part D, second plan
You know	Queues · many clients and one server · a lock that lets one process at a time through
Files	`nginx/default.conf.template` · `scripts/lib/edge.ts` · `scripts/e2e.ts`

> IN PLAIN WORDS At nine o'clock a crowd opens the map, and hundreds of requests for the same tile arrive before Nginx keeps any copy. Nginx lets the first build it, and the rest wait for its answer. This is a ***cache lock***: one request holds the right to fill the store, and the others queue behind it.

> "As the maintainer, **I want** one query for each tile however many ask at once, **so that** a crowd cannot fill the database's pool.

ACCEPTANCE CRITERIA

1. **Given** many requests for one unbuilt tile at once, **then** one query runs, and every request receives the tile.
2. **Given** one browser that leaves while it waits, **then** the query goes on for the rest (`proxy_ignore_client_abort`).
3. **Given** a query that has ended, **then** a later request finds the kept copy.

TESTS TO WRITE FIRST

1. **Twenty requests, one query.** `checkEdge`, in `scripts/lib/edge.ts` (NFR-15, NFR-17), sends twenty requests at once for a tile of `food.slow_cafes`, a view that `scripts/e2e.ts` makes with `pg_sleep(2)`, and expects every answer to be 200 within 1,800 to 4,000 milliseconds. Without the lock, twenty queries would share the two copies' eight connections and take three rounds, six seconds or more.

> WATCH OUT **No test yet** for criterion 2 (NFR-16). No check cuts off one waiting browser and proves the query runs on for the rest.

THE IDEA AT WORK

NGINX

```
location ~ \.(pbf|mvt)$ {
    # S-30: past the burst, a visitor's tiles answer 429 until the rate catches up.
    limit_req zone=visitor burst=300 nodelay;
    # …
    proxy_pass http://tessera;
    proxy_cache tiles;
    # Many people asking for one new tile at once cost one query (N-24). The others wait as
    # long as the first request may take, 20 seconds, the same as proxy_read_timeout.
    proxy_cache_lock on;
    proxy_cache_lock_timeout 20s;
    # …
    # A browser that leaves does not cancel a tile others may be waiting for.
    proxy_ignore_client_abort on;
    # …
}
```

The tile `location` writes out the chain of idea 6 in order: the visitor's rate first, then S-06's refusal of a disguised address, cut here, then the store, then the lock, and the service last. Two lines carry this story. `proxy_cache_lock on` lets one request through to the service and holds the others; `proxy_ignore_client_abort on` keeps that one request going when its own browser leaves, since others are waiting for it. The test reads time, not a count of queries, because only the clock shows that twenty requests shared one two-second query.

WHAT TO CARRY AWAY

- **Match the waits.** The lock holds a request for 20 seconds, the same as `proxy_read_timeout`, so no one gives up on a query that is still allowed to finish.
- **Make the slow case on purpose.** A view that sleeps two seconds turns a race nobody can see into a time anybody can measure.

5.8.5 N-25 Tell a watcher the service is well

`/health` says `UP` only when the database answers within a second and the list of layers has loaded, so a script can wait for the service and a watcher can see it fail.

Traces	NFR-45, NFR-46, NFR-47, NFR-48
Ideas	3 Data is plain and never changes · 7 The host wires the parts through one door
Size	1 point · builds on N-22 and N-23 · Part D, second plan
You know	Boolean conditions with AND · status codes · polling
Files	`Infrastructure/HealthChecks.cs` · `Features/Health.cs` · `TesseraModule.cs` · `scripts/start.ts`

IN PLAIN WORDS A ***health check*** is an address that a watcher, such as a script or a person, asks to learn whether the service can do its job. pg_tileserv answers at `/health`, and so does Tessera, through ASP.NET Core's own health checks. It counts two checks: the database answers, and the list of layers has loaded.

> **As** the maintainer, **I want** `/health` to say `UP` only when the service can serve tiles, **so that** a script can wait for it and a watcher can see it fail.

ACCEPTANCE CRITERIA

1. **Given** the list of layers loaded and the database answering within a second, **then** `/health` answers 200 `{"status":"UP"}`.
2. **Given** the database failing, **then** it answers 503 `{"status":"DOWN"}`; **given** only the list unloaded, 503 `{"status":"OUT_OF_SERVICE"}`. It names no fault; the log holds the detail.
3. **Given** either answer, **then** it carries `Cache-Control: no-store`.
4. **Given** `npm start`, **then** it says the service is ready only once `/health`, asked through Nginx, says `UP`.

TESTS TO WRITE FIRST

1. **`UP` on the running service.** `Says_up_when_the_service_can_serve_tiles`, in `ServiceTests` (NFR-45, NFR-47), reads exactly `{"status":"UP"}`, status 200, `application/json` and `no-store`. `Answers_head_on_health` asks with `HEAD`, as a watcher may, and reads 200; `Answers_any_method_but_get_and_head_with_405` finds `POST`, `PUT` and `DELETE` refused with 405 and `Allow: GET, HEAD`, since `/health` is read, never written.
2. **`DOWN` with the database down, and no fault named.** `Starts_with_the_database_down_and_answers_503`, in `StartTests` (NFR-46), reads 503 and exactly `{"status":"DOWN"}`.
3. **Out of service until the list loads.** `Reports_degraded_until_the_first_list_loads`, in the unit tests' `CatalogueTests`, finds `CatalogueReady` degraded before a load and healthy after one.
4. **`UP` through Nginx with one copy stopped.** `checkEdge` reads `/health` through Nginx after N-29 stops a copy, and expects `"UP"`.

THE IDEA AT WORK

CSHARP

```csharp
// Infrastructure/HealthChecks.cs; line breaks moved to fit the page
public sealed class DatabaseAnswers(Database database) : IHealthCheck
{
    public async Task<HealthCheckResult> CheckHealthAsync(
        HealthCheckContext context, CancellationToken cancellationToken = default) =>
        await database.Ping(cancellationToken).ConfigureAwait(false)
            ? HealthCheckResult.Healthy()
            : HealthCheckResult.Unhealthy();
}

public sealed class CatalogueReady(LiveCatalogue catalogue) : IHealthCheck
{
    public Task<HealthCheckResult> CheckHealthAsync(
        HealthCheckContext context, CancellationToken cancellationToken = default) =>
        Task.FromResult(catalogue.Current is null
            ? HealthCheckResult.Degraded()
            : HealthCheckResult.Healthy());
}
// Features/Health.cs, inside the answer's writer
var status = report.Status switch
{
    HealthStatus.Healthy => "UP",
    HealthStatus.Degraded => "OUT_OF_SERVICE",
    _ => "DOWN",
};
```

Each check holds the one part it asks about and answers with one expression. Neither knows the other, `/health` or the 503. `TesseraModule.AddTessera` registers both with `AddHealthChecks` (idea 7), and ASP.NET Core runs them and keeps the worst result: `Unhealthy` outranks `Degraded`. `MapHealth` turns that result into plain data (idea 3): a status code from a fixed table, 200 for healthy and 503 for either of the others, and one word in the words of Spring Boot Actuator, which many watchers read. The writer sets `no-store` on every answer. The body names no fault; `LiveCatalogue`'s warning in the log names the database's code.

> WATCH OUT **No test yet** for criterion 4 (NFR-48): `scripts/start.ts` waits for `UP` through Nginx before it prints 'Tessera is ready', but no test proves it. Criteria 2 and 3 are proved in part: no test reads `OUT_OF_SERVICE` from `/health` itself (NFR-46), and `no-store` is read on `UP` alone (NFR-47).

> WHAT TO CARRY AWAY
>
> - **Ask what a tile needs.** A health check that said `UP` without a list of layers would let `npm start` print 'ready' while every tile answers 503.
> - **Say little.** The answer names no fault, so a stranger learns nothing from it.

5.8.6 N-26 Wait in the order of arrival

Requests wait for a database connection first come, first served, and the health check gives up after a second in the same queue, so no request waits far longer than the rest.

Traces	NFR-26, NFR-27, NFR-28
Ideas	2 A pure core, effects at the edge · 7 The host wires the parts through one door
Size	1 point · builds on N-22 and N-25 · Part D, second plan
You know	Queues, first in, first out · threads · timeouts
Files	`Infrastructure/Sql/Database.cs` · `tessera.json` (`Database`) · `tests/Tessera.IntegrationTests/PoolTests.cs`

> IN PLAIN WORDS When many people pan the map at once, requests for different tiles wait for a free database connection. The first to arrive should be the first served, or an unlucky request loses its turn again and again and runs out of time. The service leaves this order to its ***connection pool***: a set of open connections that requests borrow in turn and give back. Npgsql's pool keeps the order and holds four connections for each copy of the service, so the service needs only tests that prove it. The health check's ping waits in the same queue, for a second at most.

> "As the maintainer, **I want** queries to wait for a connection in the order they arrive, and the health check to give up after a second in the same queue, **so that** no request waits far longer than the rest, and `/health` says `DOWN` when no tile could be served promptly.

ACCEPTANCE CRITERIA

1. **Given** every connection busy, **then** the pool lends the next free one to the request that has waited longest.
2. **Given** a query sent with its request's own cancel signal, and that request cancelled while the query waits, **then** the query leaves the queue at once. A tile query is such a query, but behind Nginx its request is never cancelled: see the known limit below.
3. **Given** every connection busy for more than a second, **then** the ping gives up within about a second, `/health` says `DOWN`, and the ping leaves the queue, so it never takes a connection a tile is waiting for.

TESTS TO WRITE FIRST

1. **Five tiles, one connection.** `Runs_queries_in_the_order_they_arrive`, in `PoolTests` (NFR-26), asks a pool of one connection for five different tiles, at levels 0 to 4, of a view that sleeps 0.4 seconds, 50 ms apart, and expects them to finish in the order 0, 1, 2, 3, 4.
2. **The ping behind a busy pool.** `Gives_up_the_ping_while_every_connection_is_busy` (NFR-28) holds the only connection for three seconds and expects the ping to give up within a second and a half, the held query still running; once the hold ends, the next ping answers.
3. **A cancelled wait.** `Lets_a_cancelled_request_leave_the_queue` (NFR-27) cancels a waiting query after 200 ms and expects it gone within a second, not after the 30 seconds the test's pool would wait.

> WATCH OUT **Proved in part,** criterion 3 (NFR-28): the test calls `Ping`, not `/health`.

THE IDEA AT WORK

CSHARP

```csharp
// Infrastructure/Sql/Database.cs; line breaks moved to fit the page
public async Task<bool> Ping(CancellationToken stop)
{
    try
    {
        await Within(PingLimit, PingOnce, stop).ConfigureAwait(false);
        return true;
    }
    // …
    catch (Exception error)
        when (error is TimeoutException or OperationCanceledException or NpgsqlException
            && !stop.IsCancellationRequested)
    {
        return false;
    }
}
// …
private static async Task<T> Within<T>(
    TimeSpan limit, Func<CancellationToken, Task<T>> work, CancellationToken cancel)
{
    var clock = CancellationTokenSource.CreateLinkedTokenSource(cancel);
    clock.CancelAfter(limit);
    var run = work(clock.Token);
    // … the clock is released once the work ends, by a continuation
    return await run.WaitAsync(limit, cancel).ConfigureAwait(false);
}
```

`PingLimit` is one second. `Within` keeps two clocks on the work: `WaitAsync` stops the caller waiting once the limit passes, and the token, cancelled at the same moment, takes the ping out of the pool's queue, so a ping that gave up never borrows a connection later. Every query runs through the same `Within`, on a limit of the connection wait and the query limit together. A cancel from the caller's own token escapes, since the database is not at fault; every other failure means 'not well' and answers `false`. Tessera writes no code for the fairness of the queue: `Database.Open` hands the pool its size and its waits from the `Database` section, and Npgsql does the rest. That is idea 2's edge at its thinnest: the service writes one rule, the one-second limit, and borrows the rest, with three tests to hold the lender to its word.

NOTE **Known limit.** A tile query carries its request's own cancel signal, the `CancellationToken` that `TileServer.Answer` receives, which ASP.NET Core raises when the connection that asked for the tile closes. Behind Nginx that connection stays open. When a person pans away before a tile arrives, the map cancels its request to Nginx, but `proxy_ignore_client_abort on` keeps Nginx's own request to the service alive, so the query still waits its turn, runs and lands in Nginx's store. That is safe, and on purpose: other people may be waiting on the same tile (N-24), and a person who pans back gets the tile at no cost. A client that asks the service directly, with no Nginx between, does cancel its query when it leaves (criterion 2). Under overload, abandoned queries hold connections that tiles still on screen need; the story places the cure in a limit on load (ADR 0007, S-30), not in a cancel.

WHAT TO CARRY AWAY

- **Borrow a rule only with a test.** A promise the pool makes is still the service's promise; `PoolTests` fails the build the day a newer Npgsql breaks it.

5.8.7 N-28 Serve kept tiles while the database hangs

Nginx serves every tile it holds while the service or the database cannot answer, and answers any other tile 503 with `Retry-After: 5`, so the map stays usable and browsers retry rather than hang.

Traces	NFR-29, NFR-30, NFR-31, NFR-32, NFR-33
Ideas	5 Failures are values · 6 A request is a chain of checks
Size	2 points · builds on N-19 and N-22 · Part D, second plan
You know	Backup copies · timeouts · client and server
Files	`nginx/default.conf.template` · `Infrastructure/Sql/Database.cs` · `scripts/lib/edge.ts` · `tests/Tessera.IntegrationTests/StartTests.cs` · `…/FrozenDatabaseTests.cs`

> IN PLAIN WORDS A database that hangs, behind a broken network or a stuck disk, refuses nothing: each request waits out its limits. Nginx keeps the map working meanwhile from its store. A kept copy older than `max-age` is a ***stale tile***, and still far better than a blank square.

> **As** the maintainer, **I want** kept tiles served at once while the database hangs or the service is down, and every other tile to answer 503, **so that** the map stays usable and browsers retry rather than hang.

ACCEPTANCE CRITERIA

1. **Given** the service or the database down or hanging, **then** Nginx serves every tile it holds at once, even past its `max-age`.
2. **Given** an expired tile, **then** Nginx serves it while it fetches a fresh copy behind it.
3. **Given** the service down and a tile Nginx never held, **then** Nginx answers 503 with `Retry-After: 5`, as a problem document.
4. **Given** the database hanging and a tile Nginx never held, **then** the service gives up once `Database.ConnectionWaitSeconds` and `Database.QueryTimeoutSeconds` together have passed, 13 seconds as shipped, then answers 503 (N-22).
5. **Given** the database down, **then** `/health` answers `DOWN` within a second (N-25).

TESTS TO WRITE FIRST

1. **Kept tiles with the service down.** At its end, `checkEdge` stops both copies of the service (NFR-29, NFR-31). The tile it asked for first still arrives, byte for byte, and `/food.cafes/22/0/0.pbf`, never kept, answers 503 as `application/problem+json` with `Retry-After: 5`.
2. **No database at start.** `Starts_with_the_database_down_and_answers_503`, in `StartTests` (NFR-33), times `/health` and a second tile: `DOWN` within one second, and the tile's 503 within half a second. The first tile bears the program's warm-up, so the clock times `/health` alone; runs on 18 October 2026 took 25 to 40 ms.
3. **A frozen database.** `Gives_up_on_a_frozen_database_within_the_settings_limits`, in `FrozenDatabaseTests` (NFR-32), pauses a real database and finds the query given up within the two limits together; the two 503 tests of N-22, in the integration tests' `CatalogueTests`, cover a database that is slow or gone.

> WATCH OUT **No test yet** for criterion 2 (NFR-30): no check waits for a kept tile to expire. Criterion 1 is proved in part: the edge check stops the service, not the database, and its kept tile is younger than its `max-age` (NFR-29).

THE IDEA AT WORK

NGINX

```
    # A tile already held is served while the service or the database cannot answer, and an
    # expired one is served while a fresh copy is fetched behind it.
    proxy_cache_use_stale error timeout updating http_500 http_502 http_503 http_504;
    proxy_cache_background_update on;
# …
# With the service down and no copy held, the browser gets the service's own kind of answer:
# a problem document that asks it to try again in five seconds (N-22, N-28).
error_page 502 504 = @unavailable;
location @unavailable {
    default_type application/problem+json;
    add_header Cache-Control no-store always;
    add_header Retry-After 5 always;
    return 503 '{"type":"about:blank","title":"Service Unavailable","status":503}';
}
```

`http_503` in the first list is the link to N-22: when the database is down, the service answers 503, and Nginx treats that answer as a reason to serve its stale copy instead. When no copy exists, the failure still leaves as a value of one known shape (idea 5). Nginx turns its own 502 or 504 into the same problem document the service writes, so the map reads one kind of 503 whoever sent it. The stages of idea 6 now run both ways: a request falls back from the service to the store before it falls back to a refusal. With the database frozen, `Database.Run` gives up on its own clock after 13 seconds as shipped, well inside Nginx's 20-second `proxy_read_timeout`, so the browser hears the service's own 503; chapter 7 draws the nest of waits.

> WHAT TO CARRY AWAY
>
> - **An old tile beats no tile.** The places change only when a seed script runs, so a tile a few minutes old is nearly always right.

> NOTE **Known limit.** Requests already waiting for a connection when the database freezes wait out their limits, so the first seconds bring a wave of slow 503s for tiles Nginx never held.

5.9 Part E · Nginx guards the door

Part E closes the door to strangers, shares the service fairly and runs a spare copy. Its first story, S-07, which answers office machines alone, took iteration 4's day in hand. If that day went to a slipped story, S-07 joins this part and comes first. The two stories here, worth 4 points, need US-35's mirrored networking, so that Nginx sees which machine asks. Nginx does the work (ADR 0007, ADR 0008).

5.9.1 S-30 Cap what one visitor may ask

Nginx counts each office machine's tile requests by its address, lets a fresh view's burst through, and answers anything beyond 100 a second, or beyond 32 at once, with 429, so no machine crowds out the rest.

Traces	SR-20, SR-21, SR-22, SR-23, SR-24
Ideas	6 A request is a chain of checks
Size	2 points · builds on S-07 and N-19 · Part E, second plan
You know	IP addresses · counting with variables · fairness among many users of one resource
Files	`nginx/default.conf.template` · `scripts/lib/edge.ts` · `docs/adr/0007-limits-in-nginx.md`

> IN PLAIN WORDS One busy browser, or a script in a loop, can ask for tiles faster than the database can build them, and every other visitor waits. Nginx sets a ***rate limit*** on each office machine, counted by its address. It lets a ***burst***, the rush of requests a fresh view makes, through at once, then holds each machine to a steady rate and answers anything beyond with 429 ('Too Many Requests'), which asks the browser to wait a second.

> "**As** a visitor, **I want** no other office machine to crowd me out, **so that** my map stays quick on a busy day.

ACCEPTANCE CRITERIA

1. **Given** one office machine, **then** Nginx lets it ask for 100 tiles a second, with a burst of 300 at once: one fresh view of nine layers asks for about a hundred.
2. **Given** a machine past that burst, **then** its extra tile requests answer 429 as a problem document, with `Retry-After: 1` and `Cache-Control: no-store`, until its rate falls back within the limit.
3. **Given** one machine, **then** Nginx handles at most 32 of its requests at once, and answers the 33rd with 429.
4. **Given** one machine at its limit, **then** every other machine's requests pass.
5. **Given** the machine Tessera runs on, **then** no limit applies to it, so its own checks never meet one.

TESTS TO WRITE FIRST

1. **A flood meets the limit.** `checkEdge` (SR-20, SR-21) sends 600 requests for a kept tile from `127.0.0.6` over eight connections: at least 300 answer 200, the rest 429 as a problem document with `Retry-After: 1`.
2. **Others pass.** Straight after the flood, the same check asks from `127.0.0.5`, another office address, and gets 200 (SR-23).
3. **Thirty-two at once.** Forty requests at once from `127.0.0.7`, each on a connection of its own, for a tile of the slow view that no check has asked for yet, leave at least eight answered 429 and the rest 200 (SR-22).

> WATCH OUT **No test yet** for criterion 5 (SR-24). Every check that runs through Nginx comes from an address other than 127.0.0.1, so none proves the machine's own requests pass free. Criterion 2 is proved in part (SR-21): no check reads `Cache-Control` on the 429.

THE IDEA AT WORK

NGINX

```
map $remote_addr $visitor {
    127.0.0.1 "";
    default $binary_remote_addr;
}
# …
limit_req_zone $visitor zone=visitor:10m rate=100r/s;
# …
limit_conn_zone $visitor zone=connections:10m;
# …
    limit_conn connections 32;
    limit_conn_status 429;
    limit_req_status 429;
    error_page 429 = @busy;
# …
        limit_req zone=visitor burst=300 nodelay;
```

The key, `$visitor`, is each machine's address, or nothing for 127.0.0.1; Nginx counts nothing for an empty key, which is how the machine's own checks pass free. Two counters share that key: `limit_req_zone` counts requests a second, `limit_conn_zone` counts requests open at once. The cap on requests at once covers every path in the server, while the rate covers tiles alone. Both refusals become 429 and meet `@busy`, which writes the problem document. In the order of idea 6, the office check of S-07 runs first, then these limits, then the store, so a stranger never reaches a count.

> WHAT TO CARRY AWAY
>
> - **Count by machine, not by connection.** A browser opens several connections; the limit follows the address, so opening more cannot dodge it.
> - **Give each check its own address.** The flood comes from 127.0.0.6 and the forty from 127.0.0.7, so a limit one check meets touches no other.

5.9.2 N-29 Keep serving when one copy of the service fails

`npm start` runs two copies of the service behind Nginx and starts each again when it stops, so a crash in one copy never blanks the map.

Traces	NFR-34, NFR-35, NFR-36, NFR-37, NFR-38, NFR-39, NFR-40
Ideas	2 A pure core, effects at the edge · 5 Failures are values
Size	2 points · builds on N-25 and N-28 · Part E, second plan
You know	Doubling, as a shift left does · a `while` loop with a condition · redundancy
Files	`scripts/lib/supervise.ts` · `scripts/lib/plan.ts` · `scripts/start.ts` · `nginx/default.conf.template`

> IN PLAIN WORDS A program can crash, or hang on a fault in its own code. With one copy of the service, every tile Nginx does not hold then fails until someone restarts the copy. `npm start` runs two copies instead, on two ports, and Nginx shares the requests between them in turn, a plan called ***round robin***. When one copy fails, Nginx sends its requests to the other; when a copy stops, `npm start` starts it again after a wait that doubles each time, a ***back-off***. Both copies run on the one machine, so they guard against a failed program, not a failed machine.

> “ **As** a visitor, **I want** tiles to keep coming when one copy of the service fails, **so that** a crash in the service never blanks my map.

ACCEPTANCE CRITERIA

1. **Given** `npm start`, **then** it runs two copies of the service, on `TESSERA_SERVICE_PORT`, 7800 by default, and the next port, and Nginx shares the requests between them in turn.
2. **Given** one copy stopped or crashed, **when** a request reaches it, **then** Nginx passes the request to the other copy, and the visitor gets the answer.
3. **Given** three failed requests to one copy within ten seconds, **then** Nginx rests that copy for ten seconds and sends everything to the other.
4. **Given** a copy that stops on its own, **then** `npm start` starts it again after a second, and doubles the wait after each quick failure, up to 30 seconds. A copy that ran for a minute counts as healthy, and its next restart waits a second again.
5. **Given** the database down, **then** neither copy is rested: both answer 503 (N-22), which counts as an answer, not a failure.
6. **Given** both copies down, **then** Nginx serves every tile it holds and answers the rest 503 (N-28).
7. **Given** Ctrl+C, **then** `npm start` stops both copies, and starts neither again.

TESTS TO WRITE FIRST

1. **The wait, purely.** ‘waits a second before a restart, doubling after each quick failure up to 30’, in `scripts/lib/plan.test.ts` (NFR-37), maps 0, 1, 2, 3, 4, 5, 6 and 10 quick failures to 1, 1, 2, 4, 8, 16, 30 and 30 seconds; ‘puts the two copies on ports 7800 and 7801 unless told otherwise’ (NFR-34) checks the ports, and refuses 80, 65535 and ‘abc’.
2. **One copy stopped.** `checkEdge` (NFR-35) stops the first copy, then asks for six tiles Nginx never held: each answers 200 with `X-Cache-Status: MISS`, and `/health` still says `UP`.
3. **Both stopped.** The check then stops the second copy (NFR-39): the kept tile still arrives, and a tile never kept answers 503.
4. **The restart, with fakes.** `scripts/lib/supervise.test.ts` (NFR-37, NFR-40) drives fake copies on a fake clock: ‘starts a copy again after each stop, doubling the wait after quick failures’ expects waits of 1, 2, 4 and 8 seconds; ‘waits a second again after a copy that ran for over a minute, then doubles afresh’ expects 1, 2, 1, 2; ‘starts nothing again once told to stop’ expects one start and no wait.

> WATCH OUT **No test yet** for criteria 3 and 5 (NFR-36, NFR-38). The three-failure rest and the rule that a database outage rests no copy stand in `nginx/default.conf.template` (`max_fails=3 fail_timeout=10s`, `proxy_next_upstream error timeout`), but no check proves either. Criterion 7 is proved in part (NFR-40): the test proves a stopped loop starts nothing, and no test presses Ctrl+C.

THE IDEA AT WORK

```ts
export async function keepRunning(name: string, world: Supervision): Promise<void> {
  let quickFailures = 0;
  while (!world.stopping()) {
    const began = world.now();
    const ended = await world.start();
    if (world.stopping()) return;
    quickFailures = world.now() - began > 60_000 ? 1 : quickFailures + 1;
    const wait = restartDelay(quickFailures);
    world.say(
      `The copy on ${name} stopped (${ended}); starting it again in ${wait / 1000} s.`);
    await world.wait(wait);
  }
}
```

Every effect the loop needs arrives in `world`: starting a copy, the clock, the wait, the words it prints (idea 2). `scripts/start.ts` passes the real ones, and each copy it starts runs `dotnet` on the built `Tessera.dll` with its own `Urls`. The test passes a fake copy that 'runs' for 100 milliseconds of fake time and a wait that adds to a number and sleeps not at all, so four restarts take no time. The doubling itself is `restartDelay`, in `plan.ts`, one line of arithmetic with its own test. A copy that stops is no exception here: `start` gives back how the copy ended, such as 'code 1', and the loop reads that as a value (idea 5).

WHAT TO CARRY AWAY

- **Count only crashes and hangs.** A database outage reaches both copies at once; if its 503s counted as failures, Nginx would rest both and lose the kept tiles' fallback for nothing.
- **Never let a restart spin.** A copy with a broken setting fails at once; the doubling wait keeps it from filling the log.

5.10 Part F · Clean places

Part F holds one story, worth 3 points, that cleans the places before they become a seed script, so faults in a publisher's data stop at the pipeline and never reach the map. The places Tessera ships are clean already, so the story waits until a publisher brings data of their own. All its code is Python, in `etl/`, and `npm run etl` runs it through uv.

5.10.1 US-36 Clean the places before they reach a script

A pipeline file lists the filters each table passes through, each a pure step that fixes, drops or stops, and every run writes a log of each row touched, so no faulty place reaches the map unseen.

Traces	FR-13, FR-14, FR-15, FR-16, FR-17, FR-18
Ideas	2 A pure core, effects at the edge · 3 Data is plain and never changes · 5 Failures are values · 6 A request is a chain of checks
Size	3 points · builds on US-24 · Part F, second plan
You know	Validation and verification of data · percentages · Python functions and lists
Files	`etl/src/tessera_etl/core/steps.py` · `core/filters/` · `core/pipeline.py` · `etl/tests/test_filters.py` · `etl/tests/test_pipeline.py` · `etl/tests/test_run.py`

> IN PLAIN WORDS Data from the field carries faults: a shape that is empty, an area whose edge crosses itself, longitude and latitude written the wrong way round, a name padded with spaces, a place with no name, or one place entered twice. The pipeline file lists ***filters*** for each table, run in turn. Each fixes what it safely can, and drops or stops at what it cannot. A ***stop rule*** ends the run when a table loses too much. Every run writes a log, a line for each row a filter touched, so the publisher sees what was left out and why. The repository holds no `db/seed-logs/` folder yet; the first run of `npm run etl` makes it.

> **As** a data publisher, **I want** my places cleaned by the filters I choose, with the run stopping on the faults I name and a log of every change, **so that** no faulty place reaches the map, and I know what the pipeline did to my data.

ACCEPTANCE CRITERIA

1. **Given** a pipeline file, **then** each table runs its filters in the order listed, each on the rows the one before kept: `dropEmptyShapes`, `repairShapes`, `swapCoordinates`, `insideArea`, `tidyText`, `requireFields`, `dropDuplicateIds` and `dropNearDuplicates`.
2. **Given** a row a filter cannot pass, **then** it is dropped, or, when the filter says `"onFail": "stop"`, the run ends with no script written.
3. **Given** a table that lost more than `droppedOverPercent` of its rows, 10 by default, or kept fewer than `rowsUnder`, 1 by default, **then** the run ends with no script written. `stopWhen` sets both for the whole file or for one table.
4. **Given** any run, finished or stopped, **then** it writes a log in `db/seed-logs/`, named after the script: each table's rows read and kept, each filter's counts, and one line for each row fixed, dropped or stopped at, with the reason.
5. **Given** a filter that is misspelt, lacks a setting it needs, or a key the file format lacks, **then** the pipeline names every fault at once and runs nothing.
6. **Given** a file geodatabase, **then** each table reads its feature class by `layer`, with curves made straight, heights dropped and coded values written as their descriptions.

TESTS TO WRITE FIRST

1. **Each filter alone.** Pure tests in `etl/tests/test_filters.py` (FR-13, FR-14): `test_drops_a_row_with_no_shape_or_an_empty_one`, `test_repairs_an_area_whose_edge_crosses_itself`, `test_swaps_a_point_whose_longitude_and_latitude_changed_places`, `test_tidies_text_into_one_form_with_single_spaces`, `test_drops_a_row_missing_a_required_field_or_holding_only_spaces`, `test_keeps_the_first_of_two_rows_with_one_id`, `test_drops_a_place_entered_twice_nearby_and_keeps_one_of_the_same_name_far_off`, and `test_stops_at_the_first_bad_row_when_told_to_and_keeps_every_row`.
2. **The order and the stop rules.** In `etl/tests/test_pipeline.py` (FR-13, FR-15): `test_runs_filters_in_order_and_ends_at_the_first_stop`; `test_stops_a_table_that_drops_more_than_ten_per_cent_by_default`, where 2 of 10 rows dropped gives '20 %, more than the 10 % allowed'; and `test_lets_a_table_set_its_own_rules`, which passes the same table at 25 %.
3. **Faults named at once, and the log.** `test_names_every_fault_at_once` (FR-17) finds five faults in one bad file; `test_logs_each_filter_and_each_row_it_touched` (FR-16) reads 'Row 10: dropped by requireFields: name is missing'. `test_extracts_one_layer_flat_and_in_longitude_and_latitude`, in the same file, checks the GDAL arguments that make curves straight and drop heights (FR-18).
4. **A geodatabase, end to end.** In `etl/tests/test_run.py` (FR-14, FR-16, FR-18), `test_cleans_a_geodatabase_into_one_script_and_logs_each_row` keeps 3 of 7 cafés, swaps one back, repairs a crossing park and logs each row; `test_stops_at_a_bad_row_writes_no_script_and_logs_why` writes no script and logs 'Row 2: stopped by insideArea'.

Every criterion of US-36 has a test: the requirements file marks none of FR-13 to FR-18 'No test yet' or 'in part'.

THE IDEA AT WORK

PYTHON

```python
    def step(cleaning: Cleaning) -> Cleaning:
        kept = len(cleaning.frame)
        percent = 0.0 if cleaning.read == 0 else 100.0 * cleaning.dropped / cleaning.read
        least, most = rules.rows_under, rules.dropped_over_percent
        if least is not None and kept < least:
            reason = f"only {kept} rows remain, fewer than the {least} required"
        elif most is not None and percent > most:
            reason = (
                f"{cleaning.dropped} of {cleaning.read} rows were dropped "
                f"({round(percent, 1):g} %), more than the {most:g} % allowed"
            )
        else:
            return cleaning
        return replace(cleaning, stop=f"{cleaning.table.table}: {reason}.")
```

Four ideas meet here, in the Python you know. This is the inner step of `stop_when` in `steps.py`, the last step of every table's chain. It is pure (idea 2): it reads a `Cleaning`, a plain record of the rows, the counts and the log (idea 3), and returns a new one with `replace`, never changing the old. A stop is no exception: it is a value, the field `stop`, holding the reason in words (idea 5), and a chain that holds a stop runs no later step. Each table's filters and this rule join with `>>` into one chain, so a table passes through its steps as a request passes through the service (idea 6).

> WATCH OUT **A new script, never an edited one.** DbUp, which runs the seed scripts, knows each script by its name alone and keeps no fingerprint of its text (ADR 0009). An edited `0002` that has already run is skipped without a word. When a cleaned table needs cleaning again, run the pipeline to a new, higher number, and commit the script and its log together, as ADR 0009 asks.

> WHAT TO CARRY AWAY
>
> - **Stop by default.** A run that drops more than a tenth of a table stops; a publisher who wants more loss must say so in the file.
> - **The log is the record.** A dropped row leaves no trace in the script, so the log, committed beside it, is the only account of what was left out.

5.11 After the release

With Part F done, the office runs all thirty stories, and the work turns from building to keeping. Tessera has no runbook yet; study plan 5, `docs/study/05-deployment-and-operations.md`, ends by asking you to write `docs/guides/runbook.md` from your own drills, and chapter 6 explains the daily work it must cover. When the map fails, ask `/health` first, read the log second and restart third. Updates arrive by themselves: each Monday ***Dependabot***, GitHub's helper for updates, proposes newer releases of the pinned packages, the .NET SDK and the images, and the build tests each one before you merge it (appendix E).

New work arrives as new stories, each test first; the 'Open to talk' lists in `docs/user-stories/user-stories.md` hold the next candidates, such as Brotli beside gzip and a second office network.

You should now be able to rebuild Tessera from an empty folder, story by story, test first; say where each of the seven ideas lives in its code; run the office machine, back it up and restore it; say why each wait fits inside the next; and show, requirement by requirement, which test proves each promise and which promise still lacks one.

> THINK IT OVER Nine requirements behind this chapter's last four parts still say 'No test yet', from NFR-04 to SR-24, and more say 'in part'. Which one would you write first, and what change to the code would it catch?

Check your understanding

1. Which `ServiceTests` case proves that a tile off the grid is refused, and what status does it receive?
2. Why does S-07 refuse a stranger before the limits and the tile store, not after?
3. N-24's check measures time, not a count of queries. What does a run of 1.8 to 4 seconds prove?
4. Why does a 503 from the database not make Nginx rest a copy of the service?
5. Started with a misspelt key, a pool size of 0 and no `DATABASE_URL`, what does the service name, and why does that meet N-21's criterion 4?

Answers. 1. `Answers_a_tile_outside_the_grid_with_404`; 404, as a problem document. 2. A check after the store would hand a stranger every tile the office had already asked for. 3. That twenty requests shared one two-second query; without the lock they would need three rounds of the eight connections, six seconds or more. 4. `proxy_next_upstream error timeout` counts only crashes and hangs, and a database outage reaches both copies at once. 5. All three, one to a line, then exit code 1: the pool, the misspelt key with its section, and the missing address. The binder sees only the keys that read cleanly, and `KeyCheck` names the rest beside the record's own checks, so one failed start names every problem at once.

6
Running it in the office

Tessera runs on one office PC as two Docker containers and two copies of the .NET service, which one command starts and keeps running; two Windows tasks repeat the start and the backup, and each script names what went wrong in its own words. You have built and tested Tessera on your own machine. This chapter moves it to the office PC, in the order you meet each part: what runs where (section 6.1), the first start (6.2), the jobs that repeat (6.3), a weekly check (6.4), and what each error tells you (6.5). Tessera has no runbook, the step-by-step guide an operator follows; study plan 5 asks you to write `docs/guides/runbook.md` as its capstone. Until then, this chapter draws only on the scripts' comments and messages and on the delivery plan, `docs/guides/delivery-plan.md`.

6.1 Two containers and two copies of the service

Four programs make up Tessera in the office, and an office machine can reach only one of them: Nginx. Docker runs two, each in a container, a sealed box with its own files made from an ***image***, a read-only template. The .NET runtime runs the other two as copies of one program, `src/Tessera/bin/Release/net10.0/Tessera.dll`. `npm start` starts all four, and fixed names let each start find what the last one left.

EXHIBIT 6.1 Four programs make up Tessera, and only Nginx answers the office

PROGRAM	RUNS AS	ITS JOB	WHERE IT LISTENS	COMES BACK BY ITSELF?
Nginx 1.29	Container `tessera-nginx`, built from `nginx/Dockerfile`	The door: the office rule, the limits, the tile store, gzip, two copies shared in turn	Port 80, on the PC's own address (`--network host`)	Yes: `--restart unless-stopped`
First copy of the service	.NET 10, `dotnet Tessera.dll`	Tiles, the map's page and files, `/health`	`127.0.0.1:7800`	Yes, while `npm start` runs
Second copy	The same program	The same work, so either copy can carry all of it	`127.0.0.1:7801`	Yes, while `npm start` runs
PostgreSQL 18 with PostGIS 3.6	Container `tessera-db`, built from `db/Dockerfile`; data in the volume `tessera-data`	The places, the two roles, DbUp's record of the seed scripts	`127.0.0.1:5432`	Yes: `--restart unless-stopped`

Source: `scripts/start.ts`; `scripts/lib/plan.ts`, `runArguments` *and* `nginxArguments`

Three of the four programs hide behind one address. `127.0.0.1` is the ***loopback address***: it means 'this machine alone', so a program bound there stays hidden from the network. The database and both copies

bind there; only Nginx opens a port to the office, so every office request passes through that one door, as Exhibit 2.5 drew in chapter 2. The shipped `tessera.json` sets the same rule for the service, `"Urls": "http://127.0.0.1:7800"`, and `SettingsTests` fails the build if the file ever names another address (SR-26).

Two layers lie beneath the programs: Ubuntu 24.04 in WSL2 with mirrored networking, so Nginx sees each office machine's own address; and Windows' two firewalls, which admit port 80 alone. US-35's page sets both up, and *Sixteen Tiles*, mini-guide M4, draws them one inside the next.

6.2 The first start

The first start takes five steps, run once in Ubuntu from a clone of the repository; each leaves something the next needs.

1. **Tools.** Install Docker Engine, the .NET 10 SDK, Node.js 26 and Git. `global.json` asks for SDK 10.0.100 or a later feature band of .NET 10, never .NET 11. The delivery plan gives this all of Monday 19 October, and ends the day when the worked answer's `npm test` passes on the machine.
2. **Settings.** Copy `.env.example` to `.env`, which Git ignores, and give each of the three passwords a value. Leave `TESSERA_HTTPPORT=80` and set `TESSERA_OFFICE_NETWORK` to the office's network, such as `192.168.10.0/24`.
3. **Start.** `npm install` fetches the scripts' tools, zx and TypeScript; `npm start` starts everything.
4. **Publish.** `npm run publish` grants the four schemas and their nine tables to the service's role.
5. **Open the door.** `npm run office-port`, then the firewall rules it prints, then a tile from another office machine.

The database makes its two roles on its first start alone. `db/initdb/20-roles.sql` runs once, on an empty volume, and makes `tessera_reader`, which may only read, and `tessera_publisher`, which owns the places' tables. Each password comes from the environment, and a missing one stops that first start with a message such as 'Set TESSERA_READER_PASSWORD before the database first starts.' Change a password later in `.env` alone and the database keeps the old one, so the service can no longer sign in. Change it in the database first, with `\password` in `psql`, then in `.env`; then start Tessera again.

`npm start` does six things in a fixed order, each waiting for the one before. It refuses to begin while a setting is missing, and `readStartSettings` names every missing variable at once, so one failed start tells you all there is to fix.

EXHIBIT 6.2 `npm start` brings Tessera up in six steps, each waiting for the last

STEP	WHAT RUNS	WHY IN THIS PLACE
1 Build	`node scripts/build.ts`: the map into `web/dist`; the release's old `wwwroot/` emptied, so no old map file ships; then `dotnet build --configuration Release`, which copies the map beside the program as `wwwroot/`	The release carries the map (US-23 c4); both copies and the seed run the one program
2 Database image	`docker build --quiet --tag tessera-db db`	The same image the tests start, so the office meets the tested database (US-01)
3 Database	`docker run` on the first start; `docker start` if it has stopped; nothing if it runs; then up to a minute's wait for `pg_isready`	The container keeps its data and passwords between starts

4 Seed	`dotnet Tessera.dll seed`, as `tessera_publisher`, then the seed's own report, line by line	DbUp runs each seed script it has not run before, so this is safe at every start (ADR 0009)
5 Two copies	The program twice, on 7800 and 7801, as `tessera_reader`	Either copy can carry every request while the other restarts (N-29)
6 Nginx	A fresh `tessera-nginx`, then a wait of up to a minute for `/health`, asked through Nginx, to say `UP`	A container from an earlier start may hold old ports; 'ready' means the whole path answers (N-25)

Source: `scripts/start.ts`; `scripts/build.ts`; `scripts/lib/build.ts`; `scripts/lib/database.ts`; `scripts/lib/plan.ts`, `seedReport`

The script prints `Tessera is ready` only once `/health` says `UP` through Nginx. The service answers at once but serves tiles only after its list of layers loads; until then `/health` says `OUT_OF_SERVICE` and the service tries again every five seconds. An empty list still counts as loaded: on a first start, before `npm run publish`, the service logs 'The service publishes 0 layers.', `/health` says `UP`, and the map shows nothing.

6.3 The scripts that repeat

Every job that repeats is a script, so it runs the same way each time and checks its own work before it reports success. Two run with no one at the PC: Windows' Task Scheduler starts Tessera when the PC switches on and takes a backup each night. Only an administrator may register a task, so each script prints the PowerShell for one to paste.

EXHIBIT 6.3 Five jobs repeat, and each script checks its own work

JOB	COMMAND	WHAT IT DOES, AND HOW IT CHECKS ITSELF	WHEN
Keep the copies running	`npm start`	Restarts a copy that stops, after 1 second, doubling the wait after each quick failure up to 30 seconds; a copy that ran a minute starts the count again (`restartDelay`, `keepRunning`)	While `npm start` runs
Start at switch-on	`npm run boot-task`, once	The task 'Tessera' runs `scripts/boot.ts` a minute after Windows starts, signed in or not; it writes to `start.log`, moves a log over 10 MB to `start.log.1`, and ends with `npm start`'s own exit code, so Windows retries a failed start up to three times	Each time the PC starts
Back up	`npm run backup`	Runs `pg_dump` into a `.part` file, reads it back with `pg_restore --list`, renames it only when it holds tables, and keeps the newest 14	02:00 each night, through 'Tessera backup' (`npm run backup-task`); and before any change
Restore	`npm run restore --FILE`	Replaces every table, grant and DbUp's record in one transaction; a restore that fails changes nothing	When needed
Publish	`npm run publish`	Grants the four schemas and their tables to `tessera_reader`; the service serves them within `Catalogue.RefreshSeconds`, 30 by default	After a seed script that replaces a table

Source: `scripts/start.ts`, `scripts/lib/plan.ts`, `scripts/lib/supervise.ts`, `scripts/boot.ts`, `scripts/lib/boot.ts`, `scripts/backup.ts`, `scripts/lib/backup.ts`, `scripts/restore.ts`, `scripts/publish.ts`

A ***backup*** is a copy of the data; a ***restore*** rebuilds the database from one. Each file's name holds the moment the backup began, in UTC, such as `tessera-2026-10-17T1800Z.dump`, so sorting the names sorts the backups by age, and `backupsToRemove` deletes the oldest beyond 14. Each run ends with one line, such

as 'Backed up 17 tables to … (0.1 MB); kept the newest 14, deleted 0.', which counts the tables whose rows the file holds.

> KEY IDEA A backup you have never read back is a hope, not a backup. `npm run backup` reads each file back the night it writes it, so a damaged file fails that night rather than on the day you need it. Only a restore proves the rest: study plan 5, pomodoro 9, asks you to back up, change data, restore and run `Tessera seed` on a spare database, timing each step.

A backup on the same disk dies with that disk. Set `TESSERA_BACKUP_FOLDER` in `.env` to a folder on another disk or a network share that Windows has mounted, such as `/mnt/d/tessera-backups`, the example in `scripts/backup.ts`. A backup holds the tables, the grants and DbUp's record, `tessera.schemaversions`; it leaves out the two roles, `.env` and `tessera.json`. Keep a copy of `.env` apart from the machine, since it holds the passwords.

6.4 A weekly check

A two-minute check each week finds most trouble before the office does. With no runbook to set it, this section proposes four questions, each with one place to look; carry them into the runbook you write in study plan 5.

1. **Is the service well?** Open `http://127.0.0.1/health` on the PC. `{"status":"UP"}` means both the database and the list of layers are well.
2. **Does the map work?** Open the PC's office name from another office machine, and zoom in until the places show.
3. **Are the backups there?** List the backup folder. The newest file should be from last night, and the sizes alike from night to night; a backup far smaller than the one before is a warning.
4. **Is the disk filling?** In Ubuntu, `df -h /` and `docker system df`. Note the free space each week, and act while it falls slowly.

`/health` gives one of three answers: 200 with `UP` when all is well; 503 with `OUT_OF_SERVICE` before the list of layers loads; 503 with `DOWN` when the database did not answer a ping within a second. N-25's page shows the two checks behind it.

> NOTE The check is weekly and done by hand, because Tessera has no metrics and no alerts: nothing wakes anyone when `/health` fails. Exhibit 9.1 names this gap, and study plan 5 closes it with OpenTelemetry's meters at `/metrics` and two alert rules in Prometheus.

6.5 What each error means

Tessera answers every error with a problem document (section 2.4) of at most four fields: `type`, always `about:blank`; `title`, the status's own phrase; `status`; and, from the service alone, a `detail` where one helps. Each carries `Cache-Control: no-store` and never echoes the address a stranger sent.

```json
{"type":"about:blank","title":"Service Unavailable","status":503,
 "detail":"The database cannot be reached. Try again shortly."}
```

Two programs write problem documents, and the status tells you which. Nginx writes 400, 403, 429 and its own 503, none with a `detail`. The service writes 404, 405, 500 and its own 503, and a 400 of its own for a

copy asked directly: `Common/Api/Problems.cs` shapes them all, `UseExceptionHandler` catches a crash, and `UseStatusCodePages` gives the same shape to an empty error, such as a 404 for an address no route knows.

EXHIBIT 6.4 Each status has one usual cause and one first action

STATUS	WHO ANSWERS	USUAL CAUSE	FIRST ACTION
304	Nginx, or ASP.NET Core	The browser's copy of the tile still matches its `ETag`	None: this is a saving, not an error
400	Nginx; the service, for a copy asked directly	A malformed or disguised address, such as one that climbs above the root, holds a stray `%` or `%00`, or hides a dot as `%2e` (S-06)	None on the PC: the client must correct its request
403	Nginx	The machine lies outside `TESSERA_OFFICE_NETWORK` (S-07)	Check the network in `.env`
404	The service	No published layer has that name; no tile has those coordinates; no route knows the address; or a grant was withdrawn since the list was read (US-03 c6)	If a layer should be there, run `npm run publish` and wait 30 seconds
405	The service	A method other than `GET` or `HEAD`, such as `POST`, on a tile, the map's page or `/health`; the answer names the two in `Allow`	None: the client must use `GET`
429	Nginx	One machine passed 100 tiles a second after a burst of 300, or held more than 32 requests at once (S-30); `Retry-After: 1`	Normal for a moment; if one machine does it all day, find what runs on it
503 with a `detail`	The service	The list of layers has not loaded; a seed script changed a layer's columns; the database cannot be reached, is busy, ran a query past 10 seconds, or gave no answer within 13, the connection wait and the query limit together (N-22); `Retry-After: 5`	`curl -s http://127.0.0.1/health`; if `DOWN`, `docker ps --all` and `docker logs tessera-db`
503 with no `detail`	Nginx	Neither copy answered and Nginx held no copy of the tile; `Retry-After: 5`	Is `npm start` running? Read its output, or `tail -n 100 start.log` after a start at switch-on
500	The service	A fault of the service's own; the answer says nothing more	Read the copies' output: the service logs a crash, and the cause of each refused query at error level; then write a failing test and fix it

Source: `nginx/default.conf.template`; `src/Tessera/Common/Api/Problems.cs`; `src/Tessera/TesseraHost.cs`; `src/Tessera/Modules/Tessera/Features/Tiles.cs`

> WATCH OUT A 500 says nothing to the browser, so read the copy's output for its cause. When the database refuses a query in a way waiting cannot cure, `TileServer` logs one line at error level, under the category `Tessera.Tiles`, with the layer, the tile, the SQLSTATE code and the database's message, such as `The tile food.cafes/15/26766/14284 failed (42883): function st_asmvt does not exist.` The search for the cause starts there, not in PostgreSQL's own log.

With no runbook, the scripts' own words are your failure table: each names what stopped it, and most name the next step. Exhibit 6.5 gathers them; follow the first line that matches what you see.

EXHIBIT 6.5 What each script or the program says, its usual cause, and what to do

WHAT YOU SEE	MOST LIKELY CAUSE	WHAT TO DO
`TESSERA_OFFICE_NETWORK: missing or empty`, or one such line for each setting	`.env` lacks a value	Fill in each named variable; one start names them all
`… is not a network such as 192.168.10.0/24, with a prefix from 8 to 32`	A misspelt network, or one as wide as `0.0.0.0/0` (S-07 c4)	Write the office's address, a slash and its prefix
`The database did not answer within a minute; read "docker logs tessera-db".`	A first start with a password missing, or Docker short of room	Read the log it names; the role script names a missing password
A seed line such as `0002-parks.sql failed: …`	A seed script that PostgreSQL refused	Nothing changed and nothing was recorded, so fix the script and start again (ADR 0009)
`The copy on port 7800 stopped (code 1); starting it again in 1 s.`, again and again	A broken setting, or a service that fails as it starts	Read the lines before it: the program names every refused setting, such as `… 'CacheSeconds' with the error: 'The field CacheSeconds must be between 0 and 2147483647.'`
`The service has not said UP within a minute; it keeps trying.`	The list of layers cannot load, or the database is down	Ask `/health`; read the copies' lines above it
The map shows no places; `/health` says `UP`	Nothing is published: the log said 'The service publishes 0 layers.'	`npm run publish`, then wait 30 seconds
A password failed for a role	The password in `.env` changed after the database's first start	Set it in the database first, then in `.env`
`The backup could not be read back, or held no tables; it was deleted.`	A full disk, or a database with nothing in it	`df -h /`; check the backup folder; run the backup again
`The restore failed and changed nothing.`	A damaged file, or one from another database	Read `pg_restore`'s message above it; try the backup before
`WSL2's networking mode is 'nat'; Tessera needs 'mirrored'.`	WSL2's default networking	Follow the two steps `npm run office-port` prints (US-35)
`Docker does not start with Ubuntu yet, and npm start needs it.`	systemd is off in WSL2, or Docker is not enabled	Follow the three steps `npm run boot-task` prints
The PC restarted and Tessera did not return	The start-up task failed, or was never registered	`tail -n 100 start.log`; run `npm run boot-task` again after moving the repository or changing Node.js

Source: `scripts/lib/plan.ts`, `scripts/lib/database.ts`, `scripts/lib/supervise.ts`, `scripts/start.ts`, `scripts/backup.ts`, `scripts/restore.ts`, `scripts/lib/office.ts`, `scripts/lib/boot.ts`; `src/Tessera/Program.cs`; `db/initdb/20-roles.sql`

> WATCH OUT zx keeps a command's output to itself unless asked, so `npm start` prints the seed's report itself: `seedReport`, in `scripts/lib/plan.ts`, passes on each line, such as 'Ran 0001-macau-places.sql' or 'The seed is up to date.', and says 'The seed ran, and printed no report.' when the seed printed nothing. A failed seed still stops the start.

Run the seed by hand to see why it failed, or to seed without starting the service. It needs one setting, the publisher's database address, and names each script it ran:

BASH

```
export TESSERA_PUBLISHER_DATABASE_URL=\
  'postgresql://tessera_publisher:PASSWORD@127.0.0.1:5432/tessera'
dotnet src/Tessera/bin/Release/net10.0/Tessera.dll seed
# The seed is up to date.
```

> COMMON MISTAKE A common mistake is to restart first and read second. A restart clears the state that would have told you the cause: the copy's last lines, the order of failures, the moment the database stopped. Study plan 0 sets the order for a failure at the office: `/health` first, then the runbook once you have written it. Read the log before any restart.

You cannot join the service's output to Nginx's log line by line. `Problems.Shape` removes the trace ID, so answers carry no request ID, and the service logs Microsoft's own lines at `Warning` (`tessera.json`), so it writes no line per request. To match a complaint to a request, use the time and the office machine's address in Nginx's log, `docker logs --tail 5 tessera-nginx`; mirrored networking lets Nginx see that address.

> THINK IT OVER The service answers 503 when the database is down, and Nginx answers 503 when the service is down. Both say `Retry-After: 5`. Why does it help the office to have one status for both, and why does it help you to tell them apart by the `detail`?

Check your understanding

1. Which one of the four programs listens on the office network, and why do the others bind to `127.0.0.1`?
2. You change a password in `.env` alone, and the service can no longer sign in. Why?
3. Why does `npm run backup` read each backup back the night it writes it?
4. After a first start, `/health` says `UP` but the map shows no places. What has not happened yet?
5. A tile answers 503 with no `detail`. Which program wrote it, and where do you look?

Answers. 1. Nginx, on port 80; the loopback address hides the others from the network, so every request passes the one door. 2. The database sets the roles' passwords once, on its first start, and keeps the old one; set the new one in the database first, then in `.env`. 3. So a damaged file fails that night, not on the day you need it. 4. No table is published: run `npm run publish` and wait up to 30 seconds. 5. Nginx, since neither copy answered; check that `npm start` runs, and read its output or `start.log`.

7

Proving it is secure, stable and fast

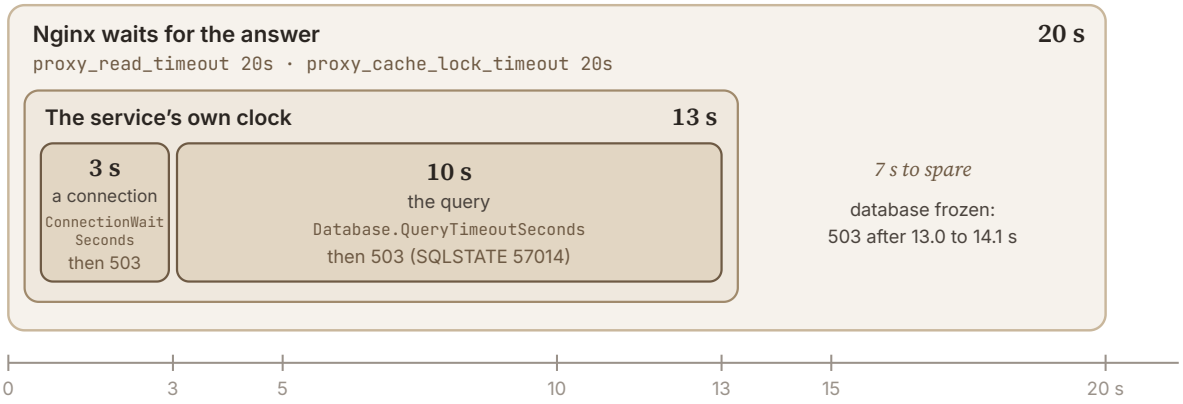
Tessera proves its door and its limits from outside, with fourteen edge checks and a requirement table that names the test behind each rule, and a floor on coverage fails the build when the tests leave too much unrun; its speed and its resistance to an attack scanner remain unproved, and this chapter turns those gaps into exercises. A test that passes on your machine proves the code, not the service. The office asks whether the map stays up under load and whether a stranger gets in. This chapter answers in the order of the argument: one model sets every limit (section 7.1); load is the first gap (7.2); an attack scan is the second (7.3); fourteen checks prove the edge (7.4); and three tables tie each requirement to its evidence (7.5).

7.1 One model for every limit

Every limit in Tessera follows one rule: each wait ends before the wait around it, so a client always hears a clear answer rather than a dropped connection. The limits live in three files, `tessera.json`, `nginx/default.conf.template` and `db/Dockerfile`, whose comments name one another. *Sixteen Tiles*, chapter 12, works out the arithmetic behind each number.

Think of a tile request as nested boxes. The request first waits for a connection from the pool, the few database connections each copy keeps open for its requests to share; holding one, it runs its query. Nginx waits outside both, for the whole answer. Exhibit 7.1 draws the boxes against a clock.

EXHIBIT 7.1 The waits nest: 3 s for a connection and 10 s for the query, 13 s on the service's own clock, fit inside Nginx's 20 s



Past 20 s Nginx counts a timeout against the copy, tries the other copy, and answers its own 503 if both fail.

Source: `tessera.json`, *section* `Database`; `Infrastructure/Settings.cs`, `DatabaseSettings`; `nginx/default.conf.template`, *the comment on* `proxy_read_timeout`; `Infrastructure/Sql/Database.cs`, `Run` *and* `Within`

Read the clock from 0 to 20 seconds. `Database.Open` hands two settings to Npgsql, the database client: `ConnectionWaitSeconds` becomes its `Timeout`, which covers both opening a connection and waiting for a free one, and `QueryTimeoutSeconds` becomes its `CommandTimeout`. A wait or a query past its time throws an

exception with a `TimeoutException` inside, and `Database.Error` turns it into SQLSTATE 57014. `DatabaseError.Of` sorts 57014 as `Unavailable`, and `TileServer` answers 503 with `Retry-After: 5`.

Npgsql's limits alone do not hold when the database freezes. Past the query's limit, Npgsql opens a second connection to ask the server to cancel the query, and waits on an answer that never comes; measured on 18 October 2026 with the database frozen by `docker pause`, a 10-second limit answered after 16 seconds, past Nginx's old 15. So `Database.Run` keeps a clock of its own, `Within`, set to the two settings together, 13 seconds as shipped: at that moment the request answers 503 and leaves the query to end out of sight. The same freeze then answered in 13.0 to 14.1 seconds, and Nginx's limit rose to 20, which leaves about 7 to spare; Nginx's own limit fires only for a copy that hangs.

> FOUNDATIONS Why does a service need limits set on purpose? Each request runs as an asynchronous method: while it waits for the database, its thread returns to the thread pool and serves another request (*Sixteen Tiles*, chapter 6). A handful of threads can keep thousands of requests waiting, so threads limit little. Something else must: the number of database connections, the requests each office machine may send, and the length of each wait. A limit nobody set is no limit.

NGINX

```
# Longer than the service's own worst case, so the service answers its own 503 first, and only
# a copy that hangs meets this limit. That worst case is the wait for a connection and the
# query's limit together, 3 + 10 = 13 seconds (Database settings); with the database frozen
# by 'docker pause', the service answered in 13.0 to 14.1 seconds (18 October 2026). The
# other 6 seconds leave room for a busy machine. Raise this with either setting.
proxy_read_timeout 20s;
```

> COMMON MISTAKE A common mistake is to raise one clock alone. Set `Database__QueryTimeoutSeconds=20` and leave Nginx at 20 seconds, and the service's worst case, 23 seconds, now outlasts Nginx. Nginx gives up first, counts the timeout as a failure of a healthy copy (`proxy_next_upstream error timeout`), and sends the same slow request to the other copy. Change the inner and outer clocks in one commit.

`FrozenDatabaseTests` proves the inner part of the nest: with a real database paused, a query under a one-second wait and a two-second limit gives up within three and a half seconds. No test checks the outer part, that the sum stays below Nginx's limit. That rule lives in comments, so a change to one file can break it while the build still passes. That gap is this chapter's first exercise.

> TRY IT Write a test in `scripts/lib` that reads `tessera.json` and `nginx/default.conf.template`, adds `ConnectionWaitSeconds` to `QueryTimeoutSeconds`, and fails unless the sum stays below `proxy_read_timeout`. Put the arithmetic in a pure function, as ADR 0002's first rule asks, so `node --test` can check it with no files.

The other limits bound counts rather than times, and each has a reason written beside it. Exhibit 7.2 gathers them.

EXHIBIT 7.2 Every count has a bound, and every bound has a reason

LIMIT	VALUE	WHERE	WHY
Connections in each copy's pool	4, so 8 for two copies	`Database.PoolSize` in `tessera.json`	Tiles build side by side, yet the database's load stays small (N-24)

Connections the database accepts	20	`max_connections` in `db/Dockerfile`	8 for the copies, the rest for the seed, `npm run publish` and `psql`
Tiles a second from one machine	100, with a burst of 300	`limit_req_zone`, `limit_req`	One fresh view of nine layers asks for about 100 tiles at once (ADR 0007)
Requests at once from one machine	32, then 429	`limit_conn connections 32`	One machine cannot take the whole door
Machines remembered by the limits	About 160,000 in 10 MB	`zone=visitor:10m`	Memory stays bounded however many addresses call
Tiles kept by Nginx	64 MB on disk; names of about 80,000 in 10 MB	`proxy_cache_path`	Far more than the office's places make, yet small beside the disk
Features in one tile	10,000	`Tiles.MaxFeatures`	A crowded square stays small and quick to build
A copy's failures before Nginx rests it	3 in 10 s, then 10 s of rest	`max_fails=3 fail_timeout=10s`	Passive health check over two copies (N-29)
Wait before restarting a copy	1 s, doubling to 30 s	`restartDelay` in `scripts/lib/plan.ts`	A copy with a broken setting does not spin
The database's answer to `/health`	1 s	`PingLimit` in `Database.cs`	`/health` answers within a second, whatever the pool holds (N-26 c3)

Source: `tessera.json`; `nginx/default.conf.template`; `db/Dockerfile`; `scripts/lib/plan.ts`; `Infrastructure/Settings.cs`; `Infrastructure/Sql/Database.cs`

7.2 Load: what exists and what does not

Tessera has no load test: no script in the repository measures how many tiles a second it serves or how long the slowest answers take, and the requirements set no number for speed. Three checks come nearest, and each proves that a limit refuses well, not how fast the service runs.

- **A flood.** `scripts/lib/edge.ts` sends 600 tile requests from one address over eight connections, and checks that at least 300 pass and the rest meet 429.
- **A crowd.** Twenty requests at once for a tile that takes two seconds to build all receive it in 1.8 to 4 seconds, because Nginx lets one query serve them all (N-24).
- **A cap.** Forty requests at once from one machine leave at least eight refused with 429 (S-30).

Two numbers for speed exist, both from runs whose scripts the repository does not keep. A comment in `Database.cs` records that prepared statements raised the rate from 1,580 to 2,408 tiles a second on 14 October 2026, with no tile store in front: one 30-second k6 run for each figure, random tiles at levels 13 to 18, on a machine of two processors that the service, PostGIS and k6 shared. The figures compare two settings, not an office server, and the notes survive in Git's history alone. ADR 0007 builds on them: about 2,400 tiles a second on two processors, so 24 machines at the full rate could fill the service. *Sixteen Tiles*, chapter 11, explains what a load test measures. Study plan 4, pomodoro 12, points to the phase-0 benchmark in the Project, `claude/phase-0/bench/tiles.js`, a script for ***k6***: a program that plays many users at once and records how long each answer takes.

> FROM SCHOOL You know the median: line up every answer's time from fastest to slowest and take the middle one. A ***percentile*** extends the idea. The 95th percentile, written ***p95***, is the time that 95 answers in 100 beat. The median says what a typical visitor sees; the p95 says what the unlucky one in twenty sees, and a map that asks for a hundred tiles a view meets that visitor every time.

> TRY IT **Exercise: the first load test.** Start from the phase-0 script and aim it at Nginx's tile route, as study plan 4 asks. Decide three things before you run it. *From where?* Requests from `127.0.0.1` meet no limit, since the `map` block gives this machine an empty key; requests from any other address meet the 100-a-second rule. *What to record?* The median, the p95, and the counts of 429 and 503. *What counts as a pass?* Write the target down before the run, then write the result beside it. Study plan 4's capstone keeps the script in the repository, as `load/tiles.js` with an `npm run load` command, and records the first numbers in `docs/design/`.

ADR 0007 says when you must measure rather than reason: when the map asks for many more tiles a view, or the office grows past 20 machines. Until then the limits rest on arithmetic and one measurement whose script was not kept.

7.3 Attack: a scanner, run by hand

No attack scanner runs in Tessera's build, and no study plan names one. The edge checks prove the rules the stories name; nothing tries the attacks no story foresaw. ZAP, the Zed Attack Proxy, is a free scanner that does: it sends requests, reads the answers and raises an ***alert*** for each weakness it recognises. This is ***black-box testing***, judging a system by what it does rather than by its code. A *passive* scan reads each answer and changes nothing; an *active* scan sends real attacks, such as SQL injection or path traversal (climbing out of a folder with `../`), and watches what comes back.

> TRY IT **Exercise: a baseline scan by hand.** This command comes from ZAP's own documents, not from the repository. With `npm start` running, run ZAP's baseline scan on the PC itself, through Nginx, as an office machine reaches it:
>
> ```
> docker run --rm --network host zaproxy/zap-stable \
>   zap-baseline.py -t http://127.0.0.1/ -r zap.html
> ```
> BASH
>
> The scan comes from `127.0.0.1`, which the office rule admits and the rate limit ignores, so Nginx answers every request. Sort each alert into one of two piles: it applies, so you write a failing test and fix it; or it does not, so you write down why. Never quieten a finding that applies.

An empty report proves less than it seems: ZAP found nothing it knows how to look for. It cannot read a binary tile, judge which tables the role may read, or stand outside the office. Other evidence answers those questions, as Exhibit 7.3 shows.

Expect some findings, and know where they come from. The map's page already carries a `Content-Security-Policy`, `X-Content-Type-Options: nosniff` and `Referrer-Policy: no-referrer`, set by `PageHeaders` in `Features/Map.cs`; `ServiceTests` checks the first two. Nginx, by contrast, names itself and its version in a `Server` header by default, such as `nginx/1.29.8` on the day of checking, and `default.conf.template` leaves that on. An active scan, the step after the baseline, would try SQL in a

layer's name; study plan 6 asks for the same by hand: PortSwigger's first three SQL injection labs at pomodoro 5, and an attack on your own copy at pomodoro 9.

EXHIBIT 7.3 A scanner answers one question; the tests answer the rest

QUESTION	WHAT ANSWERS IT
Can any value reach the SQL text?	`TileTests`' `Sends_the_layer_and_the_tile_as_parameters` and `Matches_the_approved_tile_query`; `SqlTests`' property `Quotes_every_name_safely`; `BoundaryTests`' `Only_Infrastructure_talks_to_the_database`; `SourceTests`' `Keeps_every_sql_text_in_queries_cs`; `ServiceTests`' `Answers_an_attack_with_404_and_keeps_every_row` and `Serves_a_granted_table_named_as_an_attack_and_keeps_every_row`
Can the service write to the database?	The role `tessera_reader`, which may only read, and whose sessions are read-only by default (`db/initdb/20-roles.sql`); `DatabaseTests`' `Keeps_the_reader_read_only` reads that setting, and no test yet tries a write, so a write is an exercise
Is the door shut to machines outside the office?	The office check in `scripts/lib/edge.ts`
Do the limits hold under a flood?	The flood and cap checks in `scripts/lib/edge.ts`
Does a malformed address reach a route?	The twelve raw addresses in `scripts/lib/edge.ts`; `ServiceTests`' `Refuses_a_dot_folder_in_disguise_with_400`, `Answers_a_backslash_with_404` and `Answers_an_unknown_address_with_a_problem_document`; `TargetTests`

Source: `docs/requirements/security-requirements.md`, *'Checked by';* `tests/Tessera.UnitTests`, `tests/Tessera.IntegrationTests`, `tests/Tessera.ArchitectureTests/BoundaryTests.cs`

7.4 The edge's fourteen checks

Fourteen checks prove from outside what Nginx adds in front of the service. The number is this book's grouping: `checkEdge`, in `scripts/lib/edge.ts`, is one function of `assert` calls, gathered here into fourteen, one for each thing they prove. Each check stages the visit or accident it guards against, then looks for the answer the requirement demands. `npm run e2e` runs them after the browser tests, with a database of its own, two copies of the service and an Nginx, and the office set to `127.0.0.4/30`. Linux answers every `127.x.x.x` address on the machine, so the checks can play an office machine (`127.0.0.5`), a stranger (`127.0.0.2`) or a flood (`127.0.0.6`, `127.0.0.7`).

EXHIBIT 7.4 Fourteen checks prove the edge, each against a requirement

#	WHAT IT DOES, AND WHAT IT PROVES	STORY
1	A tile answers 200, `application/vnd.mapbox-vector-tile`, `Cache-Control: public, max-age=60`; the second request says `X-Cache-Status: HIT` with the same bytes	N-18, N-19
2	The tile's `ETag` is weak (`W/`), and sending it back gets 304 with no body	N-19
3	With `Accept-Encoding: gzip` the tile arrives packed, with `Vary: Accept-Encoding`, and unpacks to the same bytes	N-20

4	`HEAD` gets 200 and the headers alone	US-07
5	The map's main script travels packed at under half its plain size	N-20
6	Twelve raw malformed or disguised addresses, among them `%2e%2e`, `../..`, `%zz`, `%00`, a control character, `%2F` and a backslash, each answer exactly 400 or 404 as problem documents with the matching title that never name the layer; `/malformed`, asked for by name, answers 404	S-06
7	A stranger gets 403, `no-store`, as a problem document; an office machine gets the tile	S-07
8	600 requests from one machine: at least 300 pass, the rest meet 429 with `Retry-After: 1`	S-30
9	Another office machine still gets 200 while the first is limited	S-30
10	Twenty requests at once for a two-second tile all succeed in 1.8 to 4 seconds: one query served them all	N-24
11	Forty requests at once from one machine: at least eight meet 429, the rest get the tile	S-30
12	With the first copy stopped, six fresh tiles at level 22 answer 200, `MISS`, and `/health` says `UP`	N-29
13	With both copies stopped, a kept tile still arrives with the same bytes	N-28, N-29
14	With both stopped, a tile never kept answers 503 with `Retry-After: 5`, as a problem document	N-28

Source: `scripts/lib/edge.ts`, `checkEdge`; `scripts/e2e.ts`

Two checks show the method. Check 10 needs a tile that is slow on purpose, so `scripts/e2e.ts` makes a view, `food.slow_cafes`, that pauses two seconds for each query. Without Nginx's lock, twenty queries would share the copies' eight connections and take three rounds, six seconds or more; the check passes only under four. Check 12 stops the first copy with the stop function `e2e.ts` hands in, then asks for six tiles Nginx has never held, so each must come from the second copy. Check 6 holds two guards. First, Nginx refuses `../..`, `%zz`, `%00`, a raw control character and any `%2e` itself with 400, while `%2F` and a backslash pass to the service, which finds no route for them and answers 404. Second, the service guards a copy reached directly: Kestrel decodes `%2e%2e` and removes the dot segment before routing, so `/x/%2e%2e/food.cafes/0/0/0.pbf` once reached the tile route, and `Targets.IsDisguised` now reads the target as sent and answers 400 before routing (S-06 c3, SR-11).

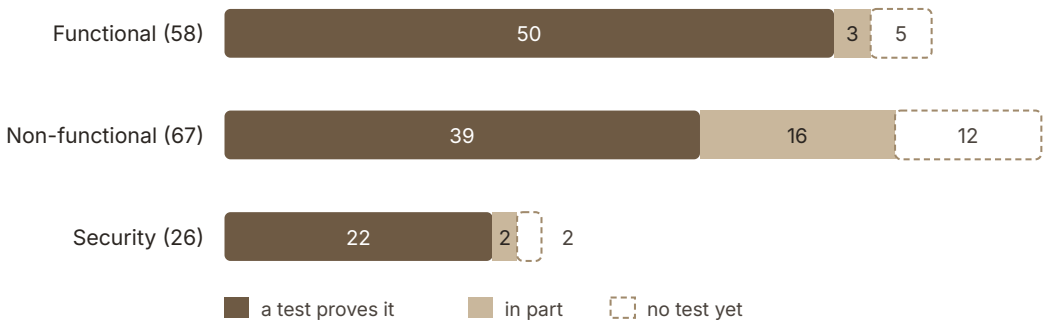
> WATCH OUT The checks run as one function of `assert` calls, so the first failure stops the rest, and a clean run prints one summary line. A red run names the first broken rule, not every one: fix it, then run again.

> TRY IT Run `npm run build`, then `npm run e2e -- --grep-invert "approved image"`. The words after `--` reach Playwright and leave out the screenshot comparisons, which the GitHub build runs inside Playwright's own image; the edge checks always run. The last line should begin `Nginx checks passed:`.

7.5 From requirement to evidence

Three files in `docs/requirements/` tie each of Tessera's 151 requirements to the test that proves it, and say where no test does. Each row holds an ID, the rule, the story criterion it comes from, and a 'Checked by' cell: a C# test class, a test file, a script, 'No test yet', or a test followed by 'in part' and what it leaves unproved.

EXHIBIT 7.5 Of 151 requirements, 111 have full evidence, 21 have part, and 19 have none yet



Source: `docs/requirements/functional-requirements.md`, `non-functional-requirements.md`, `security-requirements.md`, *the 'Checked by' column, counted for this book*

The gaps gather where tests are hardest to write. Seven of the 12 non-functional rows with no test describe Nginx over time: a store that fills, a tile that expires, a copy rested for ten seconds, a database outage that rests neither copy. Four more describe the map's design rules, and one the start script's ready line. Two rows show how a 'Checked by' cell reads. SR-16 (Nginx answers a machine inside the office's network) names `scripts/lib/edge.ts`, whose check 7 proves it. NFR-26 (the pool lends the next free connection to the request that has waited longest) names `PoolTests`, whose `Runs_queries_in_the_order_they_arrive` sends five tile queries, 50 milliseconds apart, through a pool of one connection, and checks that they finish in the order they were sent.

Two more kinds of evidence run on every change. The GitHub build, `.github/workflows/build.yml`, runs on each push to `main` and each pull request: every kind of check in Exhibit 1.4, on a clean machine, with the screenshot comparisons inside Playwright's own Docker image, a step no GitHub run has yet proved. ***Coverage***, the share of lines and branches the tests run, is the second. Each of the three test projects records the lines it runs, through Microsoft.Testing.Extensions.CodeCoverage; ReportGenerator merges the records into one report in `coverage/report/`; and `scripts/coverage.ts` fails `npm test` and the GitHub build below 94 per cent of lines or 84 of branches. The run for this book covered 96.1 per cent of the lines (648 of 674) and 85.5 per cent of the branches (267 of 312). Study plan 4 asks you to justify such a floor, at pomodoros 9 and 13 and in its capstone; *Sixteen Tiles*, chapter 10, explains coverage at greater length.

One link in the chain still needs a person. No script reads the 'Checked by' cells, so a renamed or deleted test leaves its row pointing at nothing while the build stays green. A script that checks each named class or file exists would close the gap; it suits study plan 1's pomodoro 8, which builds a table from story to file and test, or plan 7's capstone, which adds a check of story numbers to the GitHub build.

> KEY IDEA A requirement names a need; a story turns it into criteria; a test, written first, pins each criterion; and the 'Checked by' cell points from the requirement to that test. Where the cell says 'No test yet', the claim rests on reasoning alone. Count those cells, and you know how much of 'it works' is a result and how much is still a hope.

> THINK IT OVER Pick one requirement marked 'No test yet', such as NFR-36: after three failed requests to one copy within ten seconds, Nginx must rest that copy for ten seconds. What would a test of it need to start, stop and measure, and in which suite would it belong?

Check your understanding

1. Why must Nginx's 20 seconds stay longer than the service's 3 seconds for a connection plus 10 for a query?
2. Requests from `127.0.0.1` meet no rate limit. Why does that matter when you plan a load test?
3. An empty ZAP report: what does it prove, and what does it leave unproved?
4. How do the edge checks play a stranger and an office machine on one computer?
5. Of the 151 requirements, how many have full evidence, how many have part, and how many have none?

Answers. 1. So the service answers its own 503 first, and only a copy that hangs meets Nginx's limit; otherwise Nginx would count a slow but healthy copy as failed and resend the request. 2. A test from that address measures the service without the limits; aim from another address to meet the 100-a-second rule as the office does. 3. That ZAP found none of the weaknesses it knows; it cannot read a tile, judge the grants, or stand outside the office. 4. Linux answers every `127.x.x.x` address on the machine, and the test office is `127.0.0.4/30`, so `127.0.0.2` is a stranger and `127.0.0.5` an office machine. 5. 111, 21 and 19.

8
Decisions and risks

Ten decision records hold the reasons behind Tessera, and its security requirements and delivery plan hold what could still go wrong; read a record before you change the choice it made, and act on a risk's early sign rather than argue with it. Tessera's code shows what was chosen; its records show why; its requirements and plan show what the choices leave open. This chapter moves from the general to the particular: the form of a record (section 8.1), the ten records (8.2), the stack they chose, part by part (8.3), the threats to the running service (8.4), and the risks to your own path through the stories (8.5).

8.1 What a decision record is

A ***decision record***, often called an ADR (architecture decision record), is a short note of one choice: the forces that shaped it, the choice itself, and what becomes easier and harder because of it. Code shows that Tessera sends its SQL through Dapper and Npgsql; only a record says what else was weighed and why it lost. A developer working alone needs records most: a year on, no one remembers October, the author included.

> FROM SCHOOL A record works like the working you show under a maths answer. The answer alone earns a mark only if it is right; the working shows how you reached it, so a reader can find the wrong step.

Every record in `docs/adr/` takes the same shape, and yours should too: a small table, then the sections in a fixed order.

```text
# ADR NNNN: A short phrase naming the decision

| Status  | Accepted                                      |
| Date    | 16 October 2026                               |
| Decider | The maintainer                                |
| Covers  | The part of Tessera the decision governs      |
| Related | Other ADRs, and the stories, such as US-07    |

## Decision                what was decided, in the active voice
## Context  (or: Why)      the facts that forced it
## Options considered      each option, with what it offers and costs
## Consequences            what gets easier; what gets harder
## When to revisit         the events that reopen the decision
```

Some records add a section of their own between these, such as 'The parts' in ADR 0001, 'Rules' in ADR 0002 and ADR 0006, or 'How it works' in ADR 0009, and two name their stories in a 'Stories' row in place of 'Related'. Most records elsewhere lack the last section; Tessera's keep it. 'When to revisit' names the events that would make the decision wrong, such as 'the office grows past 20 machines' in ADR 0007, and so tells the next reader when to stop trusting the record.

Write a record when a choice is hard to reverse, when you chose between real options, when it touches security, or when a default surprised you. A choice you could undo in an afternoon needs a comment in the code instead. A record may still say its choice is cheap to undo: ADR 0002 notes that the scripts 'cost little to rewrite'.

8.2 The ten records behind Tessera

Ten records, accepted between 13 and 16 October 2026, settle how the service is shaped, where its limits live, and how the places reach the database. They fall into three groups: the shape of the code (0001, 0003, 0004, 0010), the edge and the map (0005 to 0008), and the scripts and the data (0002, 0009).

EXHIBIT 8.1 Ten records settle the code's shape, the edge and the map, and the data's path

NO.	DECISION	WHY, IN ONE LINE
0001	Build the back end as a modular monolith	One maintainer, one machine, one program; the seed is a command of the same program, signed in as a role of its own
0002	Write the scripts in functional TypeScript 6 with zx	One language with the map; pure decisions that `node --test` checks without a shell
0003	Read the service's settings from `tessera.json`	One file in sections, each time with its unit in its name; the options pattern refuses a bad value at start-up
0004	Write the code in a functional style, with `Result`	The maintainer comes from F#; CSharpFunctionalExtensions gives `Result`, and the compiler checks every `T?`
0005	Cluster the places in the browser, from the vector tiles	The tile route stays the only way a client loads places; the service's work stays one cached tile at a time
0006	The map lives in `web/` with a three-rule contract	One repository and one history; a contract small enough for another service to keep
0007	Guard the door in Nginx; keep resilience libraries out	A refusal at Nginx costs the service nothing; limits in each copy would count apart
0008	Put Nginx in front of the service	Caching, packing and watching are Nginx's daily work, so the service stays small enough to read
0009	Clean the places in a Python pipeline; seed with DbUp	GDAL reads many shapes of data; bad rows stop before they reach a script; DbUp, a package the service loads, runs each script once
0010	Lay the back end out as the modular-monolith template, guarded by tests	One project in three layers, and six ArchUnitNET rules check what project files would

Source: `docs/adr/0001-modular-monolith.md` *to* `docs/adr/0010-template-layout-guarded-by-tests.md`*, 'Decision' and 'Why' or 'Context'*

Three records weigh most for you as a builder. ADR 0010 draws the walls you meet in every story, `Domain`, `Infrastructure` and `Features`, with `BoundaryTests` guarding them. ADR 0004 sets the style of every line. ADR 0008 decides what the service need not do, because Nginx does it. Read these three before your first story, and the rest when a story names them.

> WATCH OUT A record can fall behind the code, and then it must be brought back. On 18 October ADR 0003 took up the program's real messages, such as `Tiles: 'CacheSecond' is not a setting; Tiles holds Resolution, …`, and ADR 0010 and the README the architecture tests' measured time: 8 to 13 seconds on a quiet machine, up to 45 on a busy one. ADR 0009 admits that the pipeline's path through GDAL's Docker image, used when no local `ogr2ogr` exists, is untested. Before a release, close each such gap or write it down where the next reader will look.

A record's 'Consequences' list is a list of work: check it against the code, not against your memory.

8.3 The stack, and why each part

Each part of the stack removes one kind of trouble and brings a risk with a remedy chosen in advance. Exhibit E.5 lists each part's version as the repository pins it. Dependabot, met in chapter 5's 'After the release', proposes each newer release on a Monday, a week after it appears, and the GitHub build tests it before anyone merges it.

EXHIBIT 8.2 The service's parts: each earns its place, and each brings a risk with a remedy

PART	WHY	THE RISK	THE REMEDY
C# 14 on .NET 10, with every analyser on	Records, nullable types and pattern matching, all settled (`LangVersion` 14); warnings as errors, Meziantou's rules and ExhaustiveMatching	An analyser's new release adds warnings, and each warning stops the build	Dependabot's pull request meets the build first; study plan 3's capstone brings Meziantou.Analyzer 3 to a state fit to merge
ASP.NET Core's minimal APIs	Settings checked at start-up, `/health`, 304 and problem documents, with little code	Code from the web, written for older versions, fails to compile, and few tutorials cover new features such as C# 14's extension members	Copy the pins from `Directory.Packages.props` and `global.json`; read the documents of the exact version (delivery plan)
CSharpFunctionalExtensions	`Result`, with `Map`, `Bind` and `Match`, tested by others	Its own `IResult` clashes with ASP.NET Core's	A `using` alias names ASP.NET Core's in `Features/Tiles.cs` (ADR 0004)
ArchUnitNET	Six border rules in `BoundaryTests`	A breach shows at test time, not as the code is typed, and a call through reflection stays out of sight	The rules run first in `npm test` and the GitHub build; if a breach slips past, the layers become projects (ADR 0010)
Dapper over Npgsql	Parameters beside the SQL, never inside; four connections a copy, lent in the order requests ask	Dapper fills a record by matching column names, so a misnamed column fails when the query runs, not when it compiles	The integration tests run every query against the packaged database (`CatalogueTests`, `ServiceTests`)

DbUp	Each seed script runs once, in the order of its name, as `tessera_publisher`	DbUp knows a script by its name alone, so a script changed after it ran is never run again, and nothing says so	Never change a script that has run; put corrections in a new one; the pipeline refuses a name that exists (ADR 0009)
PostgreSQL with PostGIS	The database cuts each tile itself, and a grant publishes a table	A new major version stores data the old one cannot read	Dependabot ignores major versions of `postgis/postgis`; upgrade by plan

Source: `Directory.Build.props`, `Directory.Packages.props`, `global.json`, `db/Dockerfile`, `.github/dependabot.yml`; *ADR 0004, ADR 0009, ADR 0010, 'Consequences' and 'When to revisit';* `docs/guides/delivery-plan.md`; `docs/study/03-programming-and-construction.md`

The service rests on C# and ASP.NET Core; the rest of the stack surrounds it, at the door, in the browser and in the build.

EXHIBIT 8.3 The edge, the map, the scripts, the pipeline and the tests, on the same terms

PART	WHY	THE RISK	THE REMEDY
Nginx	The office rule, the limits, the tile store, gzip, two copies shared	A withdrawn layer's tiles stay for up to 60 seconds; the free edition cannot clear one layer	Accept it, or move to NGINX Plus or a module that clears the store (ADR 0008)
Node.js, TypeScript, zx	One language for the map and the scripts	Node.js 26 enters maintenance on 20 October 2027	Move to the next long-term release before then (ADR 0002)
React, MapLibre GL JS, react-elmish	A map whose state lives in one model, changed by a pure update	Clusters count only loaded tiles, so a cluster at the screen's edge can change as you pan	If visitors notice, one GeoJSON file a layer removes the effect (ADR 0005)
Python pipeline: GeoPandas, Shapely, pyogrio, pydantic	GDAL reads every shape of source; filters clean rows before SQL	The path through GDAL's Docker image is untested	Install `gdal-bin`, as the GitHub build does
xUnit v3, FsCheck, Testcontainers, Playwright	The real database in tests; properties tried on a hundred made-up values; a real browser through Nginx	Screenshots drawn by another browser, or with other fonts, differ from the approved images; a part no test runs can break unseen	The build runs Playwright inside its own Docker image, a step no GitHub run has yet proved; Microsoft.Testing.Extensions.CodeCoverage and ReportGenerator measure coverage, and `npm test` and the build fail below 94 per cent of lines and 84 of branches

Source: `nginx/Dockerfile`, `package.json`, `web/package.json`, `.github/workflows/build.yml`; `scripts/lib/coverage.ts`; *ADR 0002, ADR 0005, ADR 0008, ADR 0009, 'Consequences' and 'When to revisit'*

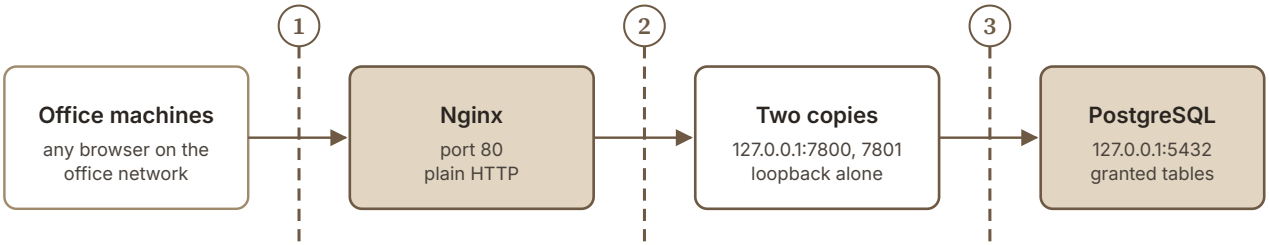
Read the last two columns together. A risk without a remedy is a worry; a remedy without a risk is decoration. Where the remedy is a test, it runs on every build, so the risk cannot return unseen.

8.4 The threat model in brief

A ***threat model*** names what could harm a system, where each threat would strike, and the ***control***, the defence, that answers it. Tessera has no threat-model file; its 26 security requirements in `docs/requirements/security-requirements.md` serve as one, each with the test that proves it (*Sixteen Tiles*, chapter 9, explains each threat). Study plan 6, pomodoro 13, asks you to draw the data's path and test each arrow against the six threats of STRIDE.

A ***trust boundary*** is a line across which data must be checked, as a passport is checked at a border. Tessera has three, and Exhibit 8.4 draws them.

EXHIBIT 8.4 Three borders: the office meets Nginx, Nginx meets the service, the service meets the database



1 Office network or 403; 100 tiles a second, burst 300; 32 at once; malformed addresses refused with 400

2 Nothing but Nginx reaches the copies; every error leaves as a problem document that names nothing

3 The role tessera_reader reads granted tables alone, read-only by default; every value travels as a parameter, every name as a quoted SqlName; only the Infrastructure layer talks to the database

Source: `nginx/default.conf.template`; `scripts/lib/plan.ts`; `db/initdb/20-roles.sql`; `README.md`, *'How the database keeps the service safe'*

EXHIBIT 8.5 Each threat meets a control, and most controls name the test that proves them

THREAT	CONTROL	PROVED BY
SQL injection through a layer's name or a coordinate	Values travel as parameters; names enter only as a quoted `SqlName`; every SQL text lives in `Queries.cs` (SR-05 to SR-08)	`TileTests`, `SqlTests`, `BoundaryTests`, `SourceTests`, `ServiceTests`
A malformed address written to slip past routing	Nginx refuses it with 400, and any `%2e`; the service refuses a dot folder in disguise or a raw control character with 400 before routing, since Kestrel decodes `%2e%2e` and removes the dot segment; `%2F` and a backslash reach no route and answer 404 (SR-09 to SR-12)	The edge checks; `ServiceTests`; `TargetTests`
An error that repeats the input or describes the server	Every error a problem document of four fields at most, with no trace ID and no stack trace (SR-13)	`Answers_an_unknown_layer_with_a_problem_document`; the edge checks
A machine outside the office	Nginx's `geo $office` rule, then 403; `npm start` refuses a network wider than `/8` (SR-16 to SR-19)	Edge check 7; `scripts/lib/plan.test.ts`

One machine floods the door	100 tiles a second, burst 300, 32 at once, then 429 (SR-20 to SR-24)	Edge checks 8, 9 and 11; SR-24, the machine's own freedom, has no test
Publishing a table by mistake	A table becomes a layer by a grant alone, and the seed grants nothing (SR-01 to SR-04)	`DatabaseTests`, `CatalogueTests`, `SeedingTests`
A password in the repository	Passwords live in `.env`, which Git ignores; the shipped `tessera.json` holds none (SR-26)	`SettingsTests` for `tessera.json`; `.env` has no story and no test
A library or image with a known flaw	Dependabot proposes each newer release for eight lists of pins	No story and no test

Source: `docs/requirements/security-requirements.md`, *'Checked by' and its closing paragraph;* `.github/dependabot.yml`

Three gaps stay open, and the study plans name them. First, every request travels in ***plain HTTP***, so any machine on the office network could read or change a tile or the map's script on the way; study plan 1 asks for an S- story for ***TLS***, the encryption behind HTTPS, and study plan 6 adds it at Nginx under ADR 0012. Second, Dependabot's security alerts: study plan 6 and `dependabot.yml`'s own comment agree that a flawed version gets its pull request at once only when 'Dependabot alerts' and 'Dependabot security updates' are switched on in the repository's settings, and nobody has checked that they are. Check the settings; until the alerts are on, a fix for a known flaw arrives only with the routine Monday updates, each a week behind its release. Third, Tessera has no sign-in: any office machine sees every published layer, so publish nothing some staff must not see.

> KEY IDEA A control without a test is a hope. Every row of the table above ends in a name you can run, or in 'no test', an invitation to write one.

8.5 Risks to your path, and the remedies

The largest risk to your rebuild lies in time, not in code: a week lost early costs more than any bug. A ***risk*** is something that would hurt if it happened, and its ***early sign*** is the first thing you would notice if it started. The delivery plan, `docs/guides/delivery-plan.md`, decides each response in advance, so on a bad day you act on the sign instead of debating it.

EXHIBIT 8.6 Seven risks threaten the plan, and each has an early sign and a remedy decided now

RISK	EARLY SIGN	REMEDY
The calendar holds 30 days for 52 points	Built in: the capacity sum shows it	Plan Parts A and B alone; move Parts C to F to a second plan; keep one day in hand
A story overruns	A story still open after its points in days, plus one	Stop; read the worked answer's version; adopt what you need and note the difference
US-35 needs the office's network	No reserved address, or WSL2 not mirrored, by the end of iteration 1	Run `npm run office-port` on the office machine in week 1; ask for a reserved address
Docker or WSL fails	The integration tests cannot start the packaged database	Prove the machine on the set-up day with the worked answer's `npm test`

New versions differ from tutorials	Code from the web fails to compile, such as C# 14's extension members or TypeScript 6's settings	Copy the version pins from the worked answer; check each answer against the exact version's documents
Unfamiliar ideas slow the work	A morning spent on one failing test	Read the matching step of the three reading guides before the story starts
A day lost to illness	Any day away	It uses the day in hand; a second lost day uses the first cut

Source: `docs/guides/delivery-plan.md`, *'Risks to delivery, and their remedies'*

Falling behind, the likeliest risk, has its own plan. The plan counts one point as about one working day, tests included, and checks a burn-up chart, the points done against the days gone, every Friday. If time runs short, it cuts stories in a fixed order, decided at a review rather than on the last day.

EXHIBIT 8.7 Three cuts, taken in this order, free four days and leave Part A whole

ORDER	STORY	POINTS	WHY IT GOES FIRST	WHAT THE VISITOR LOSES
1	S-07 Answer office machines alone	1	Planned only for the day in hand	Nothing they see; the office network alone can reach the machine in any case
2	UI-17 Keep place names from overlapping	2	Last in the build order; nothing in the plan builds on it	Names may overlap where places crowd
3	UI-11 Read place names when zoomed in	1	Only UI-17 builds on it	Names on the map; each pin still names its place to a screen reader

Source: `docs/guides/delivery-plan.md`, *'When time runs short'*

The list holds no story of Part A and no test, for two reasons. First, every map story rests on the service, and the two defences the release depends on, S-01 and S-06, live in Part A. Second, a cut story leaves the code as it stood before the story began, with no half-built control and no switched-off test. The release never slips past Monday 7 December 2026.

> THINK IT OVER Which of the seven risks have you met in another project, and what was its early sign? Would a burn-up checked every Friday have shown it sooner?

Check your understanding

1. What does a decision record hold that the code cannot show, and which of its sections tells you when to stop trusting it?
2. Which three records should you read before your first story, and why those three?
3. Name Tessera's three trust boundaries, and one check at each.

Answers. 1. The options weighed and why the others lost; 'When to revisit'. 2. ADR 0010, which draws the walls; ADR 0004, which sets the style of every line; ADR 0008, which decides what the service need not do. 3. Office to Nginx: the office network, or 403; Nginx to the service: nothing but Nginx reaches the copies; the service to the database: a read-only role, values as parameters and quoted names.

9
What comes next

Tessera ends this book with four honest gaps (TLS, metrics and alerts, a load test and a runbook), and the seven study plans in `docs/study/` close each one in the same small, tested steps that built the rest. You have rebuilt a service that does a few things and refuses everything else, and that narrowness makes it safe to grow. This chapter answers the question every finished project raises, 'What now?', in three steps: the gaps (section 9.1), the study plans that close them (9.2), and how to keep learning once the plans run out (9.3); one thought to carry away closes it (9.4).

9.1 The gaps

Four gaps stand between Tessera and a service the office could rely on for years, and the repository names each one. The opening page of the study plans lists five: 'no coverage report, no load test, no metrics or alerts, no runbook and no TLS'. The first has since closed: on 18 October 2026, `npm test` and the GitHub build began to measure coverage and to fail below a floor, as chapter 4 shows.

EXHIBIT 9.1 Four gaps, each with its risk today and the plan that closes it

GAP	WHAT IT COSTS TODAY	WHAT CLOSES IT	STUDY PLAN
TLS	Requests travel in plain HTTP on port 80, so any machine on the office network could read or change a tile or a script on the way	An S- story; ADR 0012, 'TLS on the office network'; port 80 sending browsers on to 443; an edge check of the redirect and the certificate; the renewal in the runbook	1, then 6
Metrics and alerts	Nothing tells anyone when `/health` fails or a backup does not run; a weekly check by hand is the only watch	An N- story; OpenTelemetry's meters and a Prometheus exporter at `/metrics`, kept to the office by Nginx; alert rules for a copy down and for 503s	1, then 5
A load test	No kept script says how many tiles a second Tessera serves, or how slow the slowest answer is; the limits rest on arithmetic	A k6 script in the repository, `load/tiles.js`, run by `npm run load`, its first numbers written down in `docs/design/`	4
A runbook	The steps for a failure live in the scripts' messages and in your memory; Exhibit 6.5 had to gather them	`docs/guides/runbook.md`, answer first, then one restore drill followed word for word, mending each step that misled you	5, then 7

Source: `docs/study/00-using-the-pomodoro-technique.md`; the capstones of study plans 1, 4, 5, 6 and 7; `scripts/lib/coverage.ts`

Smaller gaps lie beside the four. Nineteen requirements read 'No test yet' and 21 'in part' (Exhibit 7.5), and the outer edge of the nest of time limits, the sum that must stay below Nginx's, lives in comments, not in a test (Exhibit 7.1). Four faults that this book once named as open were closed in the code on 18 October 2026, and their pages now show the fix:

- **A disguised path reached a route.** Kestrel decodes `%2e%2e` and removes the dot segment before routing, so `/x/%2e%2e/food.cafes/0/0/0.pbf` once reached the tile route. Nginx now refuses any `%2e`, and `Targets.IsDisguised` reads the target as sent and answers 400 before routing (S-06).
- **A misspelt key stopped the start alone.** .NET's binder named it and nothing else. `Settings.Section` now hands the binder only the keys that read cleanly, and `KeyCheck` names a misspelt key beside every other problem (N-21).
- **N-26's known limit named the wrong signal.** The story now says a tile query carries its request's own cancel signal, and why the query still runs behind Nginx (N-26).
- **A 500 left no log.** `TileServer` now takes an `ILogger` and writes the layer, the tile, the SQLSTATE code and the message at error level (Exhibit 6.4).

Three small ones stay open: Dependabot's security alerts may still be off, since nobody has checked the repository's settings; no GitHub run has yet proved the screenshot comparisons inside Playwright's image; and the repository's early commits name no story, a history that stays as it is. Each gap is a story waiting to be written, in the same form as the thirty you built.

> KEY IDEA Every new feature arrives as a refusal lifted, never as a check removed. TLS does not remove port 80; it changes port 80's answer into 'go to 443', and a test proves the new answer. The rule that refused a request becomes a rule that admits a bounded one, and its test changes with it.

9.2 The study plans in `docs/study`

Seven study plans, one for each area of software work, take about 150 pomodoros, timed stretches of 25 minutes: some 75 hours, or eight or nine weeks at four a day. Each plan climbs four levels of four pomodoros, from 'orient' through 'understand' and 'apply' to 'master', and ends with a capstone of five or six more that closes a gap of section 9.1. `00-using-the-pomodoro-technique.md` sets the rules: a pomodoro cannot be split, a long break follows every four, and each question you answer becomes a card you answer again after a day, three days, a week and a fortnight.

EXHIBIT 9.2 Seven plans, worked in order, each ending in a capstone that leaves Tessera better

PLAN	POMODOROS	STARTS FROM	CAPSTONE
1 Requirements and analysis	16 + 5	The 30 stories, traced to their ADRs, files and tests	Stories for metrics and alerts, for TLS, and for coverage and load, placed in the delivery plan
2 Architecture and system design	16 + 5	The ten ADRs, one line each	ADR 0011 for a second module, with ArchUnitNET rules that keep the two apart
3 Programming and construction	16 + 5	The compiler's house rules, and a table from F# to C#	Meziantou.Analyzer 3 brought to a state fit to merge; a page on C# 15's unions
4 Testing and quality assurance	16 + 5	Each .NET test project, run and timed	Coverage with a justified floor, and a k6 script with `npm run load`
5 Deployment and operations	16 + 6	Every command an operator runs	The runbook, the metrics merged, and a restore drill from the runbook alone
6 Security and maintenance	16 + 6	The OWASP Top 10 of 2025, one line for each entry	ADR 0012 for TLS, carried out on a branch; Dependabot's alerts confirmed on; a policy for updates

7 Documentation	16 + 5	Every document sorted into Diátaxis's four kinds	A tutorial, a reference for `/health` and the problem documents, the runbook linked, and a check of story numbers in the build

Source: `docs/study/00-using-the-pomodoro-technique.md`; `docs/study/01-requirements-and-analysis.md` *to* `07-documentation.md`, *'Level 1' and 'Capstone'*

Work through the plans in order, because every later area traces its rules back to the stories and the ADRs. Each plan works on a branch of its own, named `study/<plan>-<topic>`, so an experiment never reaches `main` by accident.

> WATCH OUT The plans were written before Tessera measured coverage, and two still say it does not: the opening page and plan 4. Read plan 4's coverage pomodoros as a review of the floor `npm test` already checks, 94 per cent of lines and 84 of branches, and ask whether it suits; then spend the capstone's time on the load test.

> FROM SCHOOL A pomodoro works for the same reason a timed practice paper does. A short, fixed stretch with one clear question holds your attention on it, and the break afterwards lets the answer settle before you test yourself.

9.3 How to keep learning

The plans end; the method goes on: read code, write down your decisions, and retell what you learn in your own words. Each habit is small enough to keep for years, and together they turn a finished project into a way of working.

Read code as a reviewer reads it. Pick a file in the worked answer that you did not rebuild, such as `LiveCatalogue.cs` or `scripts/lib/supervise.ts`, and read it until you can explain each line to a friend. `docs/guides/reading-the-harder-files.md` gives the one idea behind three of the hardest: `Settings.cs`, `Database.cs` and `LiveCatalogue.cs`. Then read code you did not write at all: CSharpFunctionalExtensions' own `Result`, or the ArchUnitNET rule that checks your walls. You learn to read code as you learn to write it: by practice.

Write a decision record for your own choices. When you choose between two libraries, two designs or two ways to test, write a record in the shape of section 8.1, even for a project nobody else will see. In a month it will tell you why you chose, and writing its 'Consequences' will often change the choice before you make it.

Retell a story in your own words. Take one story from chapter 5, close the book, and explain it aloud or on paper: what the user wanted, which test you wrote first, and why the code takes the shape it does. Where your account stumbles, your understanding has a gap: go back to that page, and retell the story tomorrow.

> FOUNDATIONS Retelling works for the same reason that drawing an algorithm as a flowchart finds its faults: a picture or a spoken account cannot skip a step the way a memory can.

Prefer an official source to a summary of it. A summary is shorter, but its author wrote it for someone else's question, and it ages without telling you; the reference states what the current version does.

To go deeper, go to the sources the study plans name under 'Resources', at the end of each plan. Each is official or free, and each starts from a part of Tessera you already know, so you read it with a question of your own in hand.

EXHIBIT 9.3 Five sources take you past this book, each from a part you already know

SOURCE	WHAT IT TEACHES	WHERE TO START	PLAN
Microsoft Learn, 'ASP.NET Core metrics'	The numbers ASP.NET Core can count, and how Prometheus and Grafana read them	Its walk-through, beside the two health checks behind `/health`	5
RFC 9457, *Problem Details for HTTP APIs*	The shape of every error Tessera answers	The four fields `Problems.Shape` leaves	6
Nginx's documents for `ngx_http_limit_req_module`, `ngx_http_limit_conn_module` and `ngx_http_geo_module`	Exactly how the rate, the cap and the office rule behave	`burst` and `nodelay`	6
OWASP Top 10 of 2025, and OWASP's DotNet Security Cheat Sheet	The kinds of threat, and the defences .NET offers	The entries that touch Tessera; then `SqlName.cs` beside the cheat sheet's part on SQL injection	6
Site Reliability Engineering and its workbook, from Google, free at sre.google	How to run a service and learn from its failures	The chapters on service-level objectives and monitoring	5

Source: `docs/study/05-deployment-and-operations.md` *and* `docs/study/06-security-and-maintenance.md`, *'Resources'*

9.4 A closing thought

You built a service the office can trust, and you built it in small steps. Each story asked for one small thing, a failing test proved you understood it, and the tests at the borders stopped any step from reaching where it should not. Professionals build anything that must stay up the same way: in small steps, each one checked, each choice written down.

The ideas will outlast C#, ASP.NET Core and every version in this book: one program with walled modules; a pure core with effects at the edge; data that is plain and never changes; values that cannot be built wrong; failures as values; requests as chains of checks; and parts wired through one door. Add limits that nest and a door that refuses before it serves. Each idea answers one way programs go wrong, and programs will go wrong in those ways in any language you learn next.

Take the next step as you took the first: small, tested, and written down.

> THINK IT OVER Which of these ideas would you carry into a program that has nothing to do with maps, and what would it prevent there?

Check your understanding

1. Name the four gaps the study plans close, and a plan that closes each.
2. How does TLS arrive as 'a refusal lifted, never a check removed'?
3. Why prefer an official source to a summary of it?

Answers. 1. TLS, plans 1 and 6; metrics and alerts, plans 1 and 5; a load test, plan 4; a runbook, plans 5 and 7. 2. Port 80 stays, but its answer changes to 'go to 443', and a test proves the new answer. 3. A summary answers someone else's question and ages without telling you; the reference states what the current version does.

A
The requirements

Tessera must meet 151 requirements, 58 functional, 67 non-functional and 26 security, and a named test proves 111 of them in full, 21 in part and 19 not yet. Each requirement comes from one acceptance criterion of a story in chapter 5, so every promise below has an owner. This appendix shows how to read the three files and their IDs, picks the rows that matter most, and counts where the tests stand. The counts are this book's own, made from the 'Checked by' column; the files give only the totals.

The three files live in `docs/requirements/`. `functional-requirements.md` says what the service, the map, the pipeline and the scripts must do. `non-functional-requirements.md` says how well: how cheaply, how steadily, and how easily every visitor can use the map. `security-requirements.md` says what Tessera must refuse, and to whom. The tables here pick 36 rows; read the files for the rest.

A.1 How to read a requirement

Every row has four columns, each answering one question: which requirement, what it demands, where it comes from and what proves it. The word 'must' marks a requirement: Tessera meets it or fails it, with nothing between.

EXHIBIT A.1 One row, taken apart: SR-20 from the security file

COLUMN	IN SR-20	WHAT IT TELLS YOU
ID	`SR-20`	The file, by its letters (`FR` functional, `NFR` non-functional, `SR` security), and the row's number in that file
Requirement	Nginx must let one office machine ask for 100 tiles a second, with a burst of 300 at once	One sentence with 'must' in it, written so that a test can pass or fail it
Source	`S-30 c1`	The story and its criterion: story S-30, criterion 1
Checked by	`scripts/lib/edge.ts`	A C# test class, a test file, a script, or 'No test yet'; 'in part' adds what the test leaves unproved

Source: `docs/requirements/security-requirements.md`, *'How to read them' and section 6*

The story's letters say what kind of story it is. `US-` marks a story of the service or the map, such as US-07, 'Serve a tile'. `UI-` marks a story of how the map looks and behaves, such as UI-09, 'Zoom by web-map levels 0 to 22'. `N-` marks a story of quality, such as N-18, 'Let a browser keep a tile'; `S-` a story of security, such as S-01, 'Keep SQL out of every request'. Every row has a story behind it. Two safeguards have none, so the security file names them at its end without a requirement: Git ignores `.env`, and Dependabot proposes each newer release of what Tessera pins.

> FOUNDATIONS A requirement is the specification you write in the analysis stage of a project, before any design or code. You may have met two kinds at school: ***functional requirements***, which say what a system does, and ***non-functional requirements***, which say how well it does it, such as how fast or how easy to use. Tessera gives security a third file, because those rows deserve a reader of their own. ***Traceability*** joins each row to where it came from and to what proves it; here the 'Source' and 'Checked by' columns do that work.

A.2 What the service does

Fifty-eight functional requirements say what Tessera does, from the one command that builds it to the office machine that reaches it on port 80. Ten sections group them by part: building, publishing tables, the pipeline that cleans the places, the seed, tiles, the map, zooming and clusters, names, distance and position, and the office. Exhibit A.2 picks the rows a reader of this book meets most often.

EXHIBIT A.2 Twelve of the 58 functional requirements, in the order of the file

ID	REQUIREMENT, IN ONE LINE	SOURCE	CHECKED BY
FR-01	The build command builds the service from a fresh copy of the repository	US-00 c1	`.github/workflows/build.yml`
FR-04	The package carries the seed script `db/seed/0001-macau-places.sql`, made from `data/macau-pois.geojson`	US-01 c2	`DatabaseTests`
FR-06	A granted table with a typed geometry column and an SRID above 0 becomes a layer, with its SRID and geometry type	US-03 c1	`CatalogueTests`
FR-09	A granted function stays out of the catalogue	US-03 c5	`CatalogueTests`, both suites
FR-13	Each table runs the filters its pipeline file lists, in that order, each on the rows the one before kept	US-36 c1	`etl/tests/`, three files
FR-19	`Tessera seed` runs each new script in `db/seed` once, in the order of their names, and nothing on a second run	US-24 c4	`SeedingTests`
FR-20	A seed script that fails leaves the database as it was, and runs again once fixed	US-24 c6	`SeedingTests`; `etl/tests/test_run.py`
FR-21	`/{layer}/{z}/{x}/{y}.pbf` of a published layer answers 200 as `application/vnd.mapbox-vector-tile`	US-07 c1	`ServiceTests`, `edge.ts`
FR-23	The features of a layer stored in WGS 84 sit at the right positions in the tile's grid	US-07 c3	`ServiceTests`
FR-24	A tile of an unknown layer answers 404	US-07 c4	`ServiceTests`
FR-32	A tile that answers with a problem document shows its `detail`, or else its `title`, as text over the map	US-23 c8	`map.test.ts`, `App.test.tsx`, `map.spec.ts`
FR-55	An office machine that opens the Windows machine's office name reaches Nginx on port 80 and receives tiles	US-35 c1	No test yet

Source: `docs/requirements/functional-requirements.md`; *test files under* `tests/`, `web/src`, `web/e2e` *and* `etl/tests`

As Exhibit 7.5 showed in chapter 7, the functional rows stand best and the non-functional rows worst, because of what each kind asks for. A functional row asks for an answer to a request, which a test can

send and read. A non-functional row often asks how the whole system behaves over time, under load or when a part fails, and such a test needs two copies of the service, a stopped database or a clock.

Two kinds of functional gap stand out. Three of the four rows in ‘Opening the service to the office’, FR-55 to FR-57, need the office machine itself, a restart of Windows and a restart of WSL2; no automated test reaches that far. The delivery plan comes nearest on release day, when opening the map from another office machine proves FR-55 by hand. FR-37 and FR-48 ask what the map does as you zoom and where names fall at a tile’s edge, which a unit test cannot see and no browser test yet measures.

A.3 How well it does it

Sixty-seven non-functional requirements say how well Tessera works: how cheaply it serves a tile again, how it stays steady when the database is slow or down, how it reads its settings and reports its health, how calm the map is, and how well anyone can use it. Many name a number, such as a time in seconds or a size in megabytes, so a reviewer can hold the service to it.

EXHIBIT A.3 Twelve of the 67 non-functional requirements, and the numbers each one holds Tessera to

ID	REQUIREMENT, IN ONE LINE	SOURCE	CHECKED BY
NFR-01	A tile answered 200 carries `Cache-Control: public, max-age=N`, N being `Tiles.CacheSeconds`, 60 by default	N-18 c1	`TileTests`, `ServiceTests`, `SettingsTests`, `edge.ts`, `map.spec.ts`
NFR-06	A tile carries a weak `ETag`, `W/"…"`, made from its bytes alone	N-19 c1	`TileTests`, `ServiceTests`, `edge.ts`
NFR-08	Nginx answers a tile built within the last `Tiles.CacheSeconds` from its kept copy, says `X-Cache-Status: HIT`, and runs no query	N-19 c3	`edge.ts`
NFR-09	Kept tiles take no more than 64 MB, and the tiles used least go first	N-19 c4	No test yet
NFR-11	A tile, the page or a map file asked for with gzip arrives packed and unpacks to the plain bytes	N-20 c1	`edge.ts`, in part
NFR-15	Many requests at once for one unbuilt tile run one query, and every request receives the tile	N-24 c1	`edge.ts`
NFR-18	A tile asked for while the database cannot answer gets 503 with `Retry-After: 5`, as a problem document	N-22 c1	`CatalogueTests`, `DatabaseErrorTests`
NFR-24	With the database down at start, the service starts, serves the map, answers each tile 503 and tries again every 5 seconds	N-23 c3	`StartTests`, `CatalogueTests`, in part
NFR-26	With every connection busy, the pool lends the next free one to the request that has waited longest	N-26 c1	`PoolTests`
NFR-29	With the service or the database down or hanging, Nginx serves every tile it holds, even past its `max-age`	N-28 c1	`edge.ts`, in part
NFR-36	After three failed requests to one copy within ten seconds, Nginx rests it for ten seconds	N-29 c3	No test yet

NFR-46	`/health` answers 503 `DOWN` when the database fails, and 503 `OUT_OF_SERVICE` before the list loads	N-25 c2	`StartTests`, `CatalogueTests`, in part

Source: `docs/requirements/non-functional-requirements.md`

The section 'A calm page anyone can use' holds four of the twelve rows with no test: Material's elevation for each surface, its corners, the tint a control takes on hover and press, and pins that move out of a cluster within 0.3 seconds. Each describes how the page looks or moves, which a screenshot or a timed browser test could check. 'Accessibility' is the one non-functional section with every row proved.

A.4 How it defends itself

Twenty-six security requirements say what Tessera must refuse, and to whom. They guard four borders: the database's roles and secrets, the SQL each request sends, the shape of each address and error, and the door from the office, where Nginx stands.

EXHIBIT A.4 Twelve of the 26 security requirements, from the database outward to the office

ID	REQUIREMENT, IN ONE LINE	SOURCE	CHECKED BY
SR-01	The packaged database's role script makes the service's role, `tessera_reader`	US-01 c1	`DatabaseTests`
SR-02	A table never granted stays out of the catalogue	US-03 c3	`CatalogueTests`
SR-04	The seed publishes no table: each of the nine waits for its grant	US-24 c5	`SeedingTests`
SR-05	Every value in a tile address reaches the database as a bound parameter, and a property test proves it	S-01 c1	`TileTests`
SR-06	Every query's text lives in `Queries.cs`, and `Database` alone sends it, save the seed's journal schema	S-01 c2	`BoundaryTests`, `SourceTests`
SR-07	Each table or column name in a query comes from the catalogue, through the one function that quotes names	S-01 c3	`SqlTests`, `CatalogueTests`, `TileTests`
SR-09	Nginx answers 400 to a target that climbs above the root, or holds a stray `%`, `%00` or a raw control character	S-06 c1	`edge.ts`
SR-13	Every error is a problem document as RFC 9457 defines it, whose `status` matches the HTTP status code	US-07 c5	`ServiceTests`, `edge.ts`
SR-17	Nginx answers a machine outside the office's network with 403 and never passes its request on	S-07 c2	`edge.ts`
SR-20	One office machine may ask for 100 tiles a second, with a burst of 300	S-30 c1	`edge.ts`
SR-22	Nginx handles at most 32 of one machine's requests at once, and answers the 33rd with 429	S-30 c3	`edge.ts`
SR-25	The service reads its database from `DATABASE_URL`, the code names no machine, and a missing or malformed URL stops the start	US-00 c4	`SettingsTests`, `StartTests`, `SourceTests`

Source: `docs/requirements/security-requirements.md`

The security file has the fewest rows with no test, two of 26, and only two more proved in part. SR-14 needs the office machine, and SR-24, the machine's own freedom from every limit, has no check. The rows

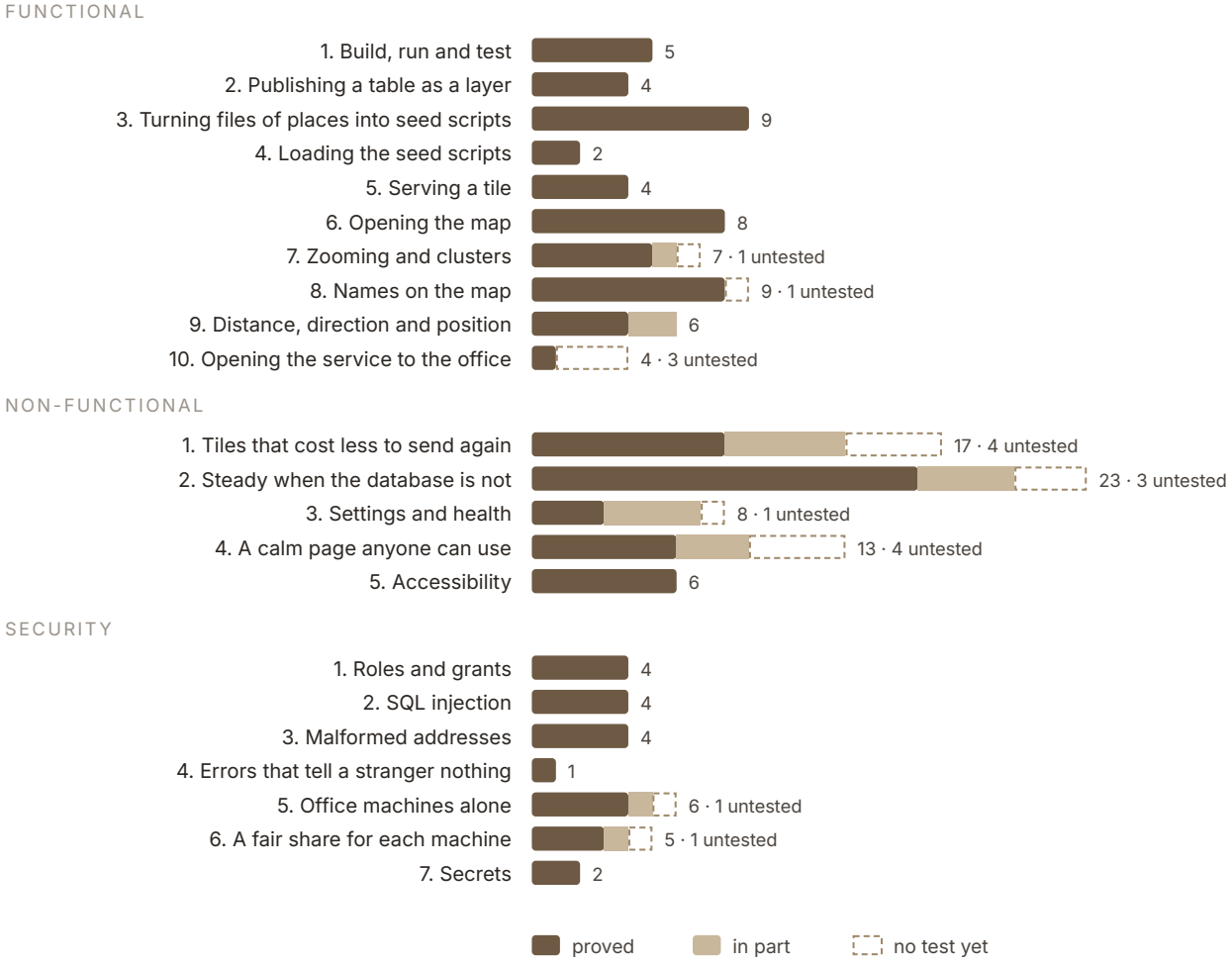
that guard against SQL injection, SR-05 to SR-08, all hold in full. SR-06 leans on two tests: `BoundaryTests` fails the build when any type outside the Infrastructure touches Npgsql, Dapper or DbUp, and `SourceTests` reads every C# file of the service for SQL text outside `Queries.cs`.

> WATCH OUT 'In part' hides real gaps, so read the words after it. SR-15's test checks the firewall steps the script prints, not the ports the machine leaves open, and SR-21's check reads `Retry-After` on the 429 but never `Cache-Control`. Six security rows once read 'in part' and now hold in full, because the tests grew. `edge.ts` expects the exact status for each malformed target and reads its title (SR-09, SR-10), and the service refuses a dot folder in disguise before routing, so `/x/%2e%2e/food.cafes/0/0/0.pbf` no longer reaches the tile route (SR-11). The attack test counts every table and reads the content type (SR-08); `SourceTests` searches for SQL text outside `Queries.cs` (SR-06) and for a machine named in the code (SR-25).

A.5 Where the tests stand

Nineteen rows have no test yet, spread over 9 of the 22 sections; Exhibit A.5 counts them section by section. The other thirteen sections have none untested, among them roles and grants, SQL injection, malformed addresses and errors: the borders an attacker tries first.

EXHIBIT A.5 Twenty-two sections, each a bar of its rows: proved, proved in part and with no test yet



Source: `docs/requirements/functional-requirements.md`, `non-functional-requirements.md`, `security-requirements.md`, *counted for this book*

The 19 fall into four groups, each with its own reason, and the reason tells you which test to write first.

1. **Nginx over time.** NFR-04 and NFR-10 (every copy expired once `max-age` passes after a new seed), NFR-09 (64 MB, least used first), NFR-16 (a query that goes on when one browser leaves), NFR-30 (an expired tile served while a fresh one is fetched), NFR-36 (a copy rested for ten seconds) and NFR-38 (a database outage rests neither copy) need a clock, a full store or a failure on cue.
2. **The map's look and movement.** FR-37, FR-48, NFR-51, NFR-52, NFR-53 and NFR-58 describe what the eye sees, and need a screenshot or a timed browser test.
3. **The office itself.** FR-55 to FR-57 and SR-14 need real machines on the office network, a restart of Windows and a restart of WSL2, and only the release day's check by hand comes near them.
4. **The start and the machine itself.** NFR-48 (`npm start` says 'ready' only once `/health` says `UP` through Nginx) and SR-24 (the machine Tessera runs on meets no limit) need only the scripts that already run, so they are the cheapest gaps to close.

> TRY IT Close one gap. SR-24 says no limit may apply to the machine Tessera runs on. The `map` block in `nginx/default.conf.template` gives `127.0.0.1` an empty key, so the rule already holds. Open `scripts/lib/edge.ts` and copy the flood check, 600 requests over eight connections. Leave out the address it sends from, so the requests leave from `127.0.0.1`, and assert that none meets 429. Run `npm run e2e -- --grep-invert "approved image"`, then change SR-24's 'Checked by' to `scripts/lib/edge.ts`.

> THINK IT OVER A requirement with no test is a promise nobody checks. Which is worse: a row that says 'No test yet', or a row whose test proves it 'in part' and so looks safe at a glance?

> WHAT TO CARRY AWAY 151 requirements, each from one criterion of one story, each with the test that proves it or the plain words 'No test yet'. Read an ID as file and number, a source as story and criterion. The gaps gather where a test would need time, failure, the eye or a real office, and the cheapest to close are the two the scripts can reach.

B

Nginx, the front door

Nginx is the one door between the office and Tessera: every office machine talks to Nginx on port 80, never to the service, and Nginx admits the office alone, caps what each machine asks, keeps and packs tiles, shares the work between two copies of the service, and answers when they cannot. The whole door is two files in `nginx/`: a `Dockerfile` of one instruction, `FROM nginx:1.29-alpine`, and `default.conf.template`, 188 lines with their comments. This appendix lists Nginx's jobs, each with the story page that explains it; shows the parts of the file no story page quotes; and names what the checks leave unproved. *Sixteen Tiles*, chapter 4, walks through the whole file block by block.

ADR 0007 and ADR 0008 record the decisions: the limits live in Nginx, not in the service, and Nginx stands in front of two copies of it. Read this appendix beside the two files.

B.1 What Nginx does for Tessera

A ***reverse proxy*** is a server that stands in front of other servers and passes requests to them, as a school's reception desk stands between visitors and the classrooms. Nginx is that desk: a server to the browser on port 80, and a client to the two copies of Tessera on ports 7800 and 7801. It speaks plain HTTP on port 80 alone, with no certificate, as US-35 asks; study plans 1 and 6 set TLS as an exercise.

Nginx does thirteen jobs at the door. Some protect the service from the office's machines, others save the service work, and the last few keep the map calm when the service is down. Exhibit B.1 lists the jobs with the story each one answers, and Exhibit B.2 follows one request past each rule. Each story's page in chapter 5 quotes the lines that do its job: S-06 the malformed address, S-07 the office, S-30 the two limits, N-19 and N-24 the store and its lock, N-20 gzip, N-28 the kept tiles and the 503.

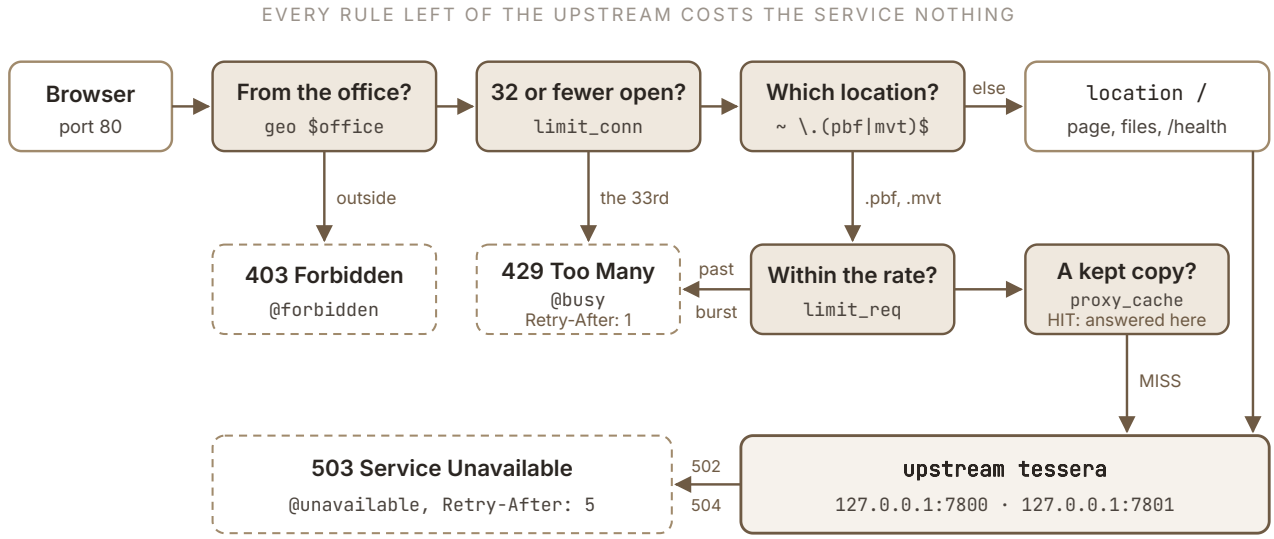
EXHIBIT B.1 Thirteen jobs at the door, each for a story and its requirements

JOB	HOW NGINX DOES IT	FOR
Admit the office alone	`geo $office`; anyone else gets 403, as a problem document	S-07; SR-16, SR-17
Let the machine itself in, free of limits	`127.0.0.1` counts as office, and its key is empty	S-07, S-30; SR-18, SR-24
Cap each machine's rate	`limit_req`: 100 tiles a second, a burst of 300, then 429	S-30; SR-20, SR-21, SR-23
Cap each machine's requests at once	`limit_conn`: 32, then 429	S-30; SR-22
Share the work between two copies	`upstream tessera`, two servers, taken in turn	N-29; NFR-34

Step round a copy that fails	`proxy_next_upstream` ; `max_fails=3 fail_timeout=10s`	N-29; NFR-35, NFR-36, NFR-38
Keep tiles	`proxy_cache tiles`, 64 MB on disk, for as long as `Cache-Control` allows	N-19; NFR-08, NFR-09
Serve a crowd from one query	`proxy_cache_lock` ; `proxy_ignore_client_abort`	N-24; NFR-15 to NFR-17
Answer 304	Its own check of `If-None-Match` against the stored `ETag`	N-19; NFR-07
Pack answers	`gzip`, with `Vary: Accept-Encoding`	N-20; NFR-11 to NFR-14
Serve kept tiles while the service or database is down	`proxy_cache_use_stale` , `proxy_cache_background_update`	N-28; NFR-29, NFR-30, NFR-39
Refuse a malformed or disguised address	Its own parser, a `map` on `$request_uri` for any `%2e`, then `error_page 400`	S-06; SR-09 to SR-11
Answer for an absent service	502 or 504 becomes a 503 problem document with `Retry-After: 5`	N-28; NFR-31

Source: `nginx/default.conf.template` *; ADR 0007; ADR 0008;* `docs/requirements/`

EXHIBIT B.2 One request through the door, and where each rule may stop it



On the way out, gzip packs tiles, JSON, CSS and scripts, and each tile carries X-Cache-Status: HIT, MISS or STALE

Source: `nginx/default.conf.template`, *lines 15–188*

B.2 The parts no story page quotes

Four terms describe the file. A ***directive*** is one instruction, ending in a semicolon, such as `gzip on;`. A ***block*** groups directives in braces: `server` for one site on one port, and `location` for the paths it matches. An ***upstream*** names the servers that requests go on to. A ***zone*** is a piece of memory that every Nginx worker shares, where Nginx counts requests or keeps the names of stored tiles. The Nginx image fills each `${NAME}` in the template from the environment as the container starts. `npm start` mounts the template rather than building it into the image, so a change needs only the fresh container that every `npm start` makes.

NGINX

```
upstream tessera {
    server 127.0.0.1:${TESSERA_SERVICE_PORT} max_fails=3 fail_timeout=10s;
    server 127.0.0.1:${TESSERA_SECOND_SERVICE_PORT} max_fails=3 fail_timeout=10s;
    keepalive 16;
}
# … in the server block:
proxy_http_version 1.1;
proxy_set_header Connection "";
proxy_set_header Host $host;
proxy_connect_timeout 1s;
proxy_read_timeout 20s;
proxy_next_upstream error timeout;
```

Nginx takes the two copies in turn and watches them with a ***passive health check***: it judges a copy only through the requests it passes on, and after three failures within ten seconds rests that copy for ten seconds and sends everything to the other. Nginx's free edition has no active check, which would ask `/health` on a timer. `keepalive 16` keeps up to 16 idle connections to the copies open in each worker, and HTTP/1.1 with an empty `Connection` header lets Nginx reuse them. `proxy_connect_timeout 1s` rests on one fact: a copy on the same machine accepts a connection in far less than a second, so one silent after a second is down. The read limit, 20 seconds, outlasts the service's own worst case, 13 seconds (Exhibit 7.1). `proxy_next_upstream error timeout` sends a request to the other copy after a crash or a hang, and only then: a 503 from a database outage, which both copies share, never counts against a copy. The file sets no `proxy_cache_key`, so Nginx keys each tile by its scheme, the upstream's name and the address; it sets no `X-Forwarded-For` either, since the service reads no client's address.

> WATCH OUT `add_header` without `always` adds its header to successful answers alone, and an error is no success. Leave out `always`, and the 503 loses its `Retry-After` and its `Cache-Control: no-store`, so a cache may keep the error and a browser may ask again at once. Every error location writes `always` on every line for that reason.

B.3 Which check proves which job

Most jobs in Exhibit B.1 have a check in `scripts/lib/edge.ts`, listed in Exhibit 7.4 in chapter 7. Four have no check of their own. Nothing fills the store past 64 MB (NFR-09), waits ten seconds to see a copy rested and returned (NFR-36), watches an expired tile served while a fresh one is fetched (NFR-30), or proves that the machine itself meets no limit (SR-24). The malformed-address check, by contrast, now names the exact answer for each of its twelve targets, 400 from Nginx or 404 from the service, so it tells the two apart; it sends `/x/%2e%2e/food.cafes/0/0/0.pbf`, which Nginx refuses, and asks for `/malformed` by name, which meets the 404 of any unknown address through a named location, `@unknown`. Each of the four is a test worth writing; appendix A lists the four among the rows with no test yet.

TRY IT With `npm start` running, ask Nginx for the example tile twice from the machine itself, and read the store's verdict in the headers:

```bash
for i in 1 2; do
  curl -s -o /dev/null -D - http://127.0.0.1/food.cafes/12/3339/1788.pbf \
    | grep -i x-cache-status
done
```

The first line should read `MISS`, unless someone asked for that tile in the last minute, and the second `HIT`: the second answer never reached the service. Wait a minute and run it again. The tile has expired, so the first word changes: which word do you see, and which line of the tile location explains it?

THINK IT OVER Nginx keeps a tile for 60 seconds. A publisher who withdraws a layer's grant sees its tiles leave the map only once those seconds pass, and Nginx's free edition cannot clear one layer on demand (ADR 0008). Is a minute too long for your office?

WHAT TO CARRY AWAY Nginx is a reverse proxy: a server to the browser and a client to the service. In Tessera it guards the door (office alone, 100 a second, 32 at once), saves work (a store of tiles, one query for a crowd, gzip, 304) and keeps the map calm (two copies, kept tiles, a 503 that says when to come back). Every rule sits in one template, and `edge.ts` proves most of them from outside.

C

C# for Python programmers

C# 14 says almost everything Python says, but it says it before the program runs: the compiler checks every type and every value that may be missing, an analyser checks every case of a closed `switch`, and Tessera's build stops on any warning. You already write functions, loops and classes in Python. This appendix shows the C# forms Tessera's service uses, each beside its Python cousin, in the order you meet them in the code.

Static types come first; then records, nullable types and pattern matching; then lambdas and LINQ, `async` and `await`, and `Result`, which together carry a request; then immutable collections; and last the features Tessera leaves out on purpose. The Python samples are this book's illustrations; every C# sample comes from `src/Tessera`, trimmed and marked where cut. *Sixteen Tiles*, chapters 1, 2 and 6, explains the same ideas at more length.

EXHIBIT C.1 Most of the Python you know has a C# counterpart in Tessera

IN PYTHON	IN C# 14	WHERE TESSERA USES IT
A type hint, checked by nobody unless you run a tool	A static type, checked by the compiler on every build	Everywhere
`@dataclass(frozen=True)`	`record`	`TileAddress`, `Layer`, `Query`, `CatalogueRow`, `TileSettings`
`Layer | None`	`Layer?`, a nullable type the compiler makes you check	`LayerCatalogue.Find`, `TileAddress.Of`, `LiveCatalogue.Current`
`match` with patterns	A `switch` expression; `[Closed]` records, checked for every case	`Program.cs`, `DatabaseError`, `TileServer.Answer`
`lambda`, comprehensions, `itertools`	Lambdas and LINQ	`CatalogueReader.FromRows`, `Program.Report`
`async def` and `await`	`async Task` and `await`	`Database`, `LiveCatalogue`, `TileServer.Answer`
A raised exception, or a returned pair of value and error	`Result<T, E>`, from CSharpFunctionalExtensions	`Program.Seed`, `CatalogueReader.Load`, `TileServer.Answer`
`tuple`, `frozenset`, `MappingProxyType`	`ImmutableArray`, `ImmutableHashSet`, `ImmutableDictionary`	`Layer`, `MapRoutes.LayerIds`, `LayerCatalogue`
f-strings and triple-quoted strings	`$"…"` and raw strings, `"""…"""`	`Queries.cs`, `Program.cs`

Source: `src/Tessera`; `Directory.Build.props`; ADR 0004

C.1 Static types, and `var`

In C# every variable, parameter and return value has a type, and the compiler refuses the program until the types agree. Python checks types while the program runs, and a hint such as `-> int` is a note for

readers and tools. C# checks them as it compiles, before anything runs, and a mismatch stops the build. Tessera goes further: `Directory.Build.props` sets `TreatWarningsAsErrors`, so every warning stops the build too, and `AnalysisMode All` with Meziantou's analyser adds hundreds of rules, each a possible warning. `.editorconfig` switches off a few, each with its reason beside it.

PYTHON
```python
# an illustration in Python, not a file in the repository
def of(z: int, x: int, y: int, min_zoom: int = 0, max_zoom: int = 22) -> TileAddress | None:
    ...
```

CSHARP
```csharp
// Domain/Tiles.cs
public static TileAddress? Of(int z, int x, int y, int minZoom = 0, int maxZoom = 22) =>
```

The two lines say the same thing: three whole numbers, two more with defaults, and an address or nothing back. C# writes the type before the name, and `static` means the function belongs to the type, not to one value of it, as a Python `@staticmethod` does. C#'s whole numbers come in fixed sizes, where Python's `int` grows as large as it needs: an `int` holds 32 bits, so `Of` refuses any level above 30 before it works out `1 << z`. *Sixteen Tiles*, chapter 2, works through that sum.

Every type written in full makes long lines, so C# has `var`. Inside a method, `var` asks the compiler to work out a local variable's type from its first value: in `TileServer.Answer`, `var built = await read(…)` makes `built` a `Result<byte[]?, DatabaseError>`. The variable is still typed and still checked; `var` saves typing, not checking. Tessera writes types in full where a reader needs them: on fields, parameters and return values.

> FOUNDATIONS Data types and casting. C#'s `int`, `double`, `bool`, `char` and `string` match the integer, real, Boolean, character and string types you know. A cast converts one to another: `Queries.Tile` writes `(double)limits.Buffer / limits.Resolution`, since 256 / 4096 between two `int` values is integer division and gives 0. Python's `/` always gives a real number, and `//` gives the whole part.

C.2 Records

A record is a class that holds named values and compares by them, written in one line. Python's nearest cousin is a frozen dataclass; both give you a constructor, equality by value and a readable printout.

PYTHON
```python
# an illustration in Python, not a file in the repository
@dataclass(frozen=True)
class Layer:
    id: str
    table: SqlName
    geometry: SqlName
    srid: int
    geometry_type: str
    properties: tuple[SqlName, ...]
    order: tuple[SqlName, ...]
```

CSHARP
```csharp
// Domain/Layers.cs
public sealed record Layer(
    string Id, SqlName Table, SqlName Geometry, int Srid, string GeometryType,
    ImmutableArray<SqlName> Properties, ImmutableArray<SqlName> Order);
```

This is a *positional* record: the list in brackets makes a constructor and a property for each name, so `layer.Id` reads the id, as `layer.id` does in Python. Each property is set only as the record is made, so no code can change a `Layer` afterwards; `sealed` forbids any other type to extend it. Two records with equal properties are equal, so a test can compare whole values with one `Assert.Equal`. A `with` expression makes a changed copy, as `dataclasses.replace` does: `settings with { CacheSeconds = 0 }` would copy a `TileSettings` with one value changed.

Settings records take another form: a property on each line, with a default and a limit. Python's pipeline in `etl/` writes the same idea with pydantic, whose `Field(ge=0)` plays the part of `[Range]`:

CSHARP

```
// Domain/Settings.cs, TileSettings (trimmed and rewrapped)
/// <summary>How long a browser and Nginx may keep a tile; 0 means they check each
/// time.</summary>
[Range(0, int.MaxValue)]
public int CacheSeconds { get; init; } = 60;
```

`get; init;` lets ASP.NET Core set the value once, from the `Tiles` section of `tessera.json`, and never again. The square brackets hold an attribute (section 3.4), a label the compiler stores with the code for a library to read, much as a Python decorator marks a function; .NET reads `[Range]` and refuses to start on a value outside it.

Two records go one step further. `TileAddress` and `SqlName` make their constructors `private`, so only their own static methods can make one, and those make only good values: a tile on the grid, a name in quotes. Python cannot forbid a constructor this way; a leading underscore only asks politely.

C.3 Nullable types, in place of `None`

A question mark after a type says the value may be missing, and the compiler refuses to use it until the code has checked. Python's `None` can stand anywhere, and a hint such as `Layer | None` warns only a tool you choose to run. C#'s `null` plays the same part, but `Directory.Build.props` switches on `<Nullable>enable</Nullable>`, so a plain `Layer` may never hold one, and a `Layer?` must be checked before use.

PYTHON

```
# an illustration in Python, not a file in the repository
def find(self, id: str) -> Layer | None:
    return self.layers.get(id)
```

CSHARP

```
// Domain/Layers.cs
public Layer? Find(string id) => Layers.GetValueOrDefault(id);
```

`=>` here begins a body of one expression, like a one-line Python `return`. Three short forms check a nullable value. `is not { } found` tests for a value and names it in one step; `?.` reads a property only if the value exists; `??` gives a fallback. `SeedScripts.Run` uses the last two at once, where Python would write `result.error_script.name if result.error_script else "The seed"`:

CSHARP

```
// Infrastructure/Seeding/SeedScripts.cs
$"{result.ErrorScript?.Name ?? "The seed"} failed: {result.Error.Message}"
```

The `$` before the quotes makes an interpolated string, Python's f-string. ADR 0004 chose `T?` for a missing value, and Tessera never uses CSharpFunctionalExtensions' `Maybe<T>`, a box for the same idea.

C.4 Pattern matching, and closed unions

A `switch` expression picks a result by the shape of a value, as Python's `match` does, and an analyser checks that a switch over Tessera's closed records misses no case. The program's first lines read almost as Python:

```python
# an illustration in Python, not a file in the repository
match sys.argv[1:]:
    case []: return serve()
    case ["seed"]: return seed()
    case _: return usage()
```

```csharp
// Program.cs
return args switch
{
    [] => await Serve().ConfigureAwait(false),
    ["seed"] => Seed(),
    _ => Usage(),
};
```

Each arm is a ***pattern***: `[]` and `["seed"]` are list patterns, and `_` matches anything. Tessera uses three other kinds. A type pattern tests a type and names the value: `PostgresException refusal`. A property pattern looks inside: `{ InnerException: TimeoutException }` matches an error that carries a timeout. A guard adds a condition with `when`, as Python's `case s if …` does: `DatabaseError.Of`, which N-22's page quotes, sorts every code that starts `08` with `_ when sqlState.StartsWith("08", …)`.

A ***union*** is a type whose values come in a fixed set of cases, as F# writes `MissingTable | MissingColumn | Unavailable | Other`. C# 14 has no unions, so Tessera builds one from records: an `abstract record DatabaseError` with four `sealed record` cases inside it, and an attribute, `[Closed(…)]`, that names all four. The ExhaustiveMatching analyser then reads every `switch` over a `DatabaseError` and stops the build if one misses a case. Python's `match` has no such check: a forgotten case returns `None`, and nobody notices until it runs. The `switch` in `TileServer.Answer`, quoted in section 3.6, ends with an arm that never runs, `_ => throw ExhaustiveMatch.Failed(error)`. C# demands it, since the compiler alone cannot see that the four cases are all there are; the analyser guards the rest.

> COMMON MISTAKE A common mistake is to add a case to a union and expect the code to carry on. By design, the opposite happens: every `switch` that misses the new case stops the build, and the analyser lists each one. That list is the work the new case needs.

C.5 Lambdas and LINQ

A lambda is a function written as a value, and LINQ chains such functions over any sequence, as Python's comprehensions and `itertools` do. C#'s lambda `layer => layer.Id` is Python's `lambda layer: layer.id`, and a method's name alone, such as `ToLayer`, passes the method as a value, as `to_layer` does in Python. ***LINQ***, Language-Integrated Query, gives every sequence methods such as `Select` (Python's `map`), `Where` (a comprehension's `if`), `GroupBy` and `Single`. Each step waits, as a generator does, until a last step such as `ToImmutableDictionary` asks for results.

```csharp
// Infrastructure/Catalogue.cs, CatalogueReader (last line rewrapped)
public static LayerCatalogue FromRows(IEnumerable<CatalogueRow> rows) =>
    new(rows.Select(ToLayer)
        .GroupBy(layer => layer.Id, StringComparer.Ordinal)
        .Where(sharing => sharing.Count() == 1)
        .ToImmutableDictionary(
            sharing => sharing.Key, sharing => sharing.Single(), StringComparer.Ordinal));
```

```python
# an illustration in Python, not a file in the repository
layers = [to_layer(row) for row in rows]
counts = Counter(layer.id for layer in layers)
catalogue = MappingProxyType({layer.id: layer for layer in layers if counts[layer.id] == 1})
```

Read the C# from the top: turn each row into a layer, group the layers by id, keep the groups of one, and build a dictionary that cannot change. Two tables whose ids clash both drop out, since a request could not say which it meant. `new(…)` leaves out the type's name, which the return type, `LayerCatalogue`, already gives.

A variable that holds a function needs a type: a delegate (section 3.2). `Func<LayerCatalogue?>` is a function of no arguments that returns a catalogue or nothing, like Python's `Callable[[], LayerCatalogue | None]`; `TileServer` takes one, so a test can hand in a catalogue of its own. Square brackets with `..` spread a sequence into a new list, as `*` does in Python:

```csharp
// Program.cs, Report (trimmed)
string.Join(Environment.NewLine, [.. ran.Select(name => $"Ran {name}"),
    "A new table waits for its grant: npm run publish."])
```

C.6 `async` and `await`

An `async` method can pause at each `await` while the database works, and give its thread to another request in the meantime. Python's `asyncio` uses the same two words for the same idea. An `async def` returns a coroutine; an `async` C# method returns a `Task`, a value standing for work still to finish, or a `Task<T>` when the work gives back a `T`.

```csharp
// Infrastructure/LiveCatalogue.cs (trimmed)
public async Task Refresh(CancellationToken cancel)
{
    var loaded = await load(cancel).ConfigureAwait(false);
    if (loaded.IsFailure)
    {
        // … log a warning, keep the last good list
        return;
    }

    var old = Interlocked.Exchange(ref _current, loaded.Value);
    // …
}
```

```python
# an illustration in Python, not a file in the repository
async def refresh(self) -> None:
    loaded = await self.load()
    if isinstance(loaded, Failure):
        return
    self.current = loaded.value
```

Four differences matter. First, a `Task` starts as soon as the method is called, where a Python coroutine waits until something awaits it. Second, a C# method that resumes after `await` may run on another thread from .NET's thread pool, so two requests can truly run at the same moment, while `asyncio` runs everything on one thread. So `LiveCatalogue` swaps its whole list in one step with `Interlocked.Exchange` and reads it with `Volatile.Read`, where code on `asyncio` would need neither. Third, every `await` in Tessera ends in `.ConfigureAwait(false)`, which lets the code resume on any thread; analyser rule CA2007 demands it. Fourth, cancelling is explicit: a `CancellationToken` travels as a parameter, from the request through `Database.Run` to Npgsql, so a request that gives up leaves the pool's queue at once. The loop that rereads the list every 30 seconds, quoted on N-23's page, reads like an `asyncio` task with `asyncio.sleep`: `await Task.Delay(…)` is `await asyncio.sleep(…)`. `await using`, in `Database.RunOnce`, closes the connection when the block ends, as Python's `async with` does.

C.7 `Result`, and the generics behind it

`Result<T, E>` holds either the answer, of type `T`, or the reason there is none, of type `E`, so a function's type says it may fail. Python raises an exception, which a caller cannot see in the function's hints, or returns a pair the caller may forget to check. Tessera takes `Result` from the CSharpFunctionalExtensions library (ADR 0004) and turns each database exception into one in `Database.Run`, the one place they meet.

```python
# an illustration in Python 3.12, not a file in the repository
@dataclass(frozen=True)
class Success[T]:
    value: T

@dataclass(frozen=True)
class Failure[E]:
    error: E

type Result[T, E] = Success[T] | Failure[E]
```

The letters in angle brackets are ***type parameters***, placeholders each use fills in, as in Python's `list[int]`. `Result<ConnectionString, string>` is a connection string or a sentence saying why not; `Result<LayerCatalogue, DatabaseError>` is a list of layers or the database's refusal. Four methods carry a result along a chain: `Map` changes the answer, `MapError` the reason, `Bind` runs a next step that may fail too, and `Match` turns either side into one value at the end. A failure at any step skips the rest, as on F#'s railway. The seed command, `Program.Seed`, is the whole chain in four lines, which section 3.5 quotes and Exhibit 3.3 draws. `MapError` there names its three types because, as the comment above it in `Program.cs` says, the library would otherwise pick a form that returns its plain `Result<T>`, and `Bind` would find no step to run. Where the compiler can work the types out, the code leaves them unsaid: `CatalogueReader.Load` ends `.Map(rows => FromRows(rows))`. The library also names a type `IResult`, as ASP.NET Core does, so `Features/Tiles.cs` opens with an alias, `using IResult = Microsoft.AspNetCore.Http.IResult;`, to say which one it means.

C.8 Immutable collections

Tessera keeps every list and map it holds in a collection that cannot change, so no code can alter a value after it is built. Python offers a `tuple` for a fixed list, a `frozenset` for a set, and `types.MappingProxyType` for a read-only view of a dictionary. .NET's `System.Collections.Immutable` offers `ImmutableArray`, `ImmutableHashSet` and `ImmutableDictionary`: a method that would change one returns a new copy instead.

```csharp
// Features/Map.cs, MapRoutes (trimmed)
public static readonly ImmutableHashSet<string> LayerIds =
[
    "food.cafes", "food.restaurants", "food.markets",
    // … six more
];
```

The square brackets are a collection expression, C#'s form of a list display, and the declared type decides what they build. Two details differ from Python. A dictionary built with `StringComparer.Ordinal` compares keys letter for letter, case included, so `food.cafes` is a layer and `Food.Cafes` is not. And the one value in Tessera that does change, the current list of layers, changes by swapping in a whole new `LayerCatalogue`, never by editing the old one.

C.9 What Tessera leaves out

Tessera uses the settled features of C# 14 alone, and leaves out every preview feature. A preview feature is one .NET ships for trial: complete, but free to change or vanish in the next release. `Directory.Build.props` sets `LangVersion` to 14 and says why in a comment: code built on a preview feature can break without warning. C# 15's unions are such a feature today; ExhaustiveMatching stands in for them, and the comment in `Tessera.csproj` says the analyser can go once C# 15's closed records do the same work.

Tessera also does without some settled features by choice (ADR 0004): no exception for a failure it expects, no `Maybe<T>`, and no tree of classes of its own.

> TRY IT Prove the closed union's check for yourself, on a copy of the repository. In `Infrastructure/Sql/Database.cs`, add `typeof(WriteRefused)` to the `[Closed(…)]` list, and add a fifth case below `Other`: `public sealed record WriteRefused(string SqlState, string Message) : DatabaseError(SqlState, Message);`. Run `dotnet build` and read the error, EM0003, 'Subtype not handled by switch': it names the line in `Features/Tiles.cs` whose `switch` no longer covers every case. Name the case `ReadOnly` instead, and a second rule, CA1716, objects to a name that is a keyword in another .NET language. Then remove the lines.

> WHAT TO CARRY AWAY C# says before the program runs what Python finds out while it runs. Records hold values; a question mark marks a value that may be missing, and the compiler makes you check it; `switch` matches shapes, and an analyser checks every case of a closed union; lambdas and LINQ chain steps over a sequence; `async` and `await` free a thread while the database works; `Result` carries a failure as a value; immutable collections never change. Tessera keeps to settled features and keeps failures as values.

D

.NET attributes, and how ASP.NET Core wires everything together

Tessera writes 220 attributes of nine kinds, and not one of them wires the program: ASP.NET Core builds every object from the calls Tessera writes in `AddTessera`, so the wiring reads as plain code from top to bottom. The attributes give the settings their limits, name the cases of one closed type, silence one rule in one method, and mark the tests. The wiring lives in two short files, `TesseraHost.cs` and `TesseraModule.cs`; follow their calls and you have followed every object the program makes. *Sixteen Tiles*, chapter 5, follows a request through ASP.NET Core in brief; this appendix goes down to each registration and its lifetime, and proves each step with what the built `Tessera.dll` printed.

The appendix takes three steps. Sections D.1 to D.6 take the attributes: what one is, every kind the code uses, family by family, and those the build writes for you. Sections D.7 to D.12 take the wiring: the start, the service collection, the settings, the hosted services, the endpoints and the middleware, and last one request for `12/3339/1788` through the objects they built. Section D.13 lists the attributes Tessera avoids.

D.1 What an attribute is, and who reads it

An attribute is a label fixed to a piece of code; it does nothing itself, and some other program must read it and act. You write it in square brackets above a class, a method, a property or a parameter, such as `[Fact]` or `[Range(0, 30)]`. The compiler keeps the label in the built `.dll`, beside the code, and the code beneath it runs exactly as it would without it. An attribute is a message to a reader, and the reader decides what it means. Java calls the same idea an annotation, written with `@`.

Think of a parcel labelled 'Keep upright'. The label lifts nothing; the courier who reads it carries the parcel with care, and a courier who cannot read ignores it. Tessera's labels have four readers, each at its own time:

1. **An analyser or a generator, at build time.** An analyser (section 3.3) is a plug-in that runs inside the C# compiler and reports a fault as the code builds. ExhaustiveMatching reads `[Closed]` on `DatabaseError` and stops the build when a `switch` misses one of its cases; .NET's own analysers read `[SuppressMessage]` and stay silent where it asks. A source generator writes code as the program builds: .NET's reads `[GeneratedRegex]` and writes the code of a regular expression.
2. **The validator, at start-up.** .NET's data-annotations validator reads `[Range]` and `[Required]` on the settings records, once, before the service answers a single request.
3. **xUnit and FsCheck, when the tests run.** xUnit reads `[Fact]`, `[Theory]` and `[InlineData]`; FsCheck, a library for property tests, reads `[Property]`.
4. **ASP.NET Core, when it maps a route.** It collects every attribute on a route's method as the endpoint's ***metadata***, a list of facts about the endpoint. Tessera writes none there, but the compiler does (section D.6).

The validator and xUnit read their labels by ***reflection***: a program's power to look at its own types while it runs, to list a class's properties, read their labels and call them. The two lines below, an illustration outside Tessera, ran on 18 October 2026 against the built service; the validator does the same for every property of every settings record:

```csharp
// an illustration, not in Tessera: read a label at run time, as the validator does
var range = typeof(TileSettings).GetProperty("MaxZoom")!.GetCustomAttribute<RangeAttribute>()!;
Console.WriteLine($"{range.Minimum} to {range.Maximum}");   // prints: 0 to 30
```

FOUNDATIONS A `.dll` file is data, as a text file is. Reflection reads that data while the program runs, much as your own code might loop over a record's fields and print each name. An attribute is one more item in that data: a name, such as `Range`, and its values, such as `0` and `30`. The validator is a loop over a record's properties, with a selection statement for each kind of label it knows.

WATCH OUT A label without its reader fails in silence. On a scratch copy, the author deleted the one line `.ValidateDataAnnotations()` from `Settings.Section`, built it, and started it with `Tiles__CacheSeconds=-1`, `Tiles__MinZoom=10` and `Tiles__MaxZoom=2`. The code still compiled and every `[Range]` stayed on its property, but no program read them, and the service started and answered requests. With the line in place, the same start stops with exit code 1 and names both faults.

D.2 Every attribute in the code, counted

The source holds nine kinds of attribute: four in the service, which the validator and the analysers read, and five in the tests, which xUnit, FsCheck and a source generator read. Exhibit D.1 counts each kind; the sections after it take the families in turn.

EXHIBIT D.1 Every attribute written in src/ and tests/, by kind, reader and count

ATTRIBUTE	ON	READ BY, AND WHEN	USES	WHERE
`[Range]`	Property	The validator, at start-up	18	`TileSettings` 6, `AreaSettings` 7, `DatabaseSettings` 4, `CatalogueSettings` 1
`[Required]`	Property	The validator, at start-up	1	`AreaSettings.Name`
`[Closed]`	Class	The ExhaustiveMatching analyser, at build time	1	`DatabaseError`
`[SuppressMessage]`	Method	.NET's analysers, at build time	1	`Database.Within`, written in full as `[System.Diagnostics.CodeAnalysis.SuppressMessage]`
`[Fact]`	Method	xUnit, when the tests run	77	Architecture 6, unit 34, integration 37
`[Theory]`	Method	xUnit	26	Unit 16, integration 10
`[InlineData]`	Method	xUnit, one row for each	87	Unit 57, integration 30

`[Property]`	Method	FsCheck, through xUnit	3	`SqlTests` 1; `TileTests` 2, written in full as `[FsCheck.Xunit.Property]`
`[GeneratedRegex]`	Method	.NET's regular-expression generator, at build time	6	`SourceTests`, each pattern with a time limit of a second
In all			**220**	21 in `src/`, 199 in `tests/`

Source: a search of every `.cs` file in `src/` and `tests/`, outside `bin/` and `obj/`, on 18 October 2026. Lines that open with `[` but fill a dictionary or a list, such as `[HealthStatus.Healthy] = …` in `Features/Health.cs` and `["DATABASE_URL"] = …` in the integration tests, are no attributes.

No attribute in the table wires the program. No label tells ASP.NET Core to make an object, find a class or fill a field; section D.8 shows that each of those steps is a method call. Here Tessera differs most from a Spring program, where labels such as `@Bean` and `@Import` carry the wiring.

The one `[SuppressMessage]` breaks a habit, and says why in its own words. Every other rule switched off sits in an `.editorconfig` with its reason: 11 for all code, 2 for `TesseraHost.cs`, 6 for the tests. `Database.Within` hands a clock to work that may outlive the method, and releases it only when the work ends; rule CA2000 expects it released before the method returns, so the label silences that rule for this one method, with the justification 'The continuation disposes the clock when the work ends, which may come after Within returns.'

D.3 Settings: limits the validator reads at start-up

Nineteen attributes, on four of the five settings records, give each value its limits, and three records add a `Validate` method for a rule that ties two values together. Both forms come from `System.ComponentModel.DataAnnotations`, part of .NET's base library rather than ASP.NET Core, so `TileSettings` and `AreaSettings` may live in the Domain, where the first architecture rule forbids the web. Exhibit D.2 lists every one.

EXHIBIT D.2 The limits on every setting, and the rules that tie two together

RECORD, AND ITS SECTION	ATTRIBUTES	RULE IN `VALIDATE`
`TileSettings`, `Tiles` (Domain)	`[Range(1, int.MaxValue)]` on `Resolution` and `MaxFeatures`; `[Range(0, int.MaxValue)]` on `Buffer` and `CacheSeconds`; `[Range(0, 30)]` on `MinZoom` and `MaxZoom`	'MaxZoom must not be below MinZoom.'
`AreaSettings`, `Area` (Domain)	`[Required(AllowEmptyStrings = false)]` on `Name`; `[Range(-180, 180)]` on the three longitudes; `[Range(-85.06, 85.06)]` on the three latitudes; `[Range(0, 22)]` on `HomeLevel`	'The box must run west to east and south to north.'
`DatabaseSettings`, `Database` (Infrastructure)	`[Range(1, 1000)]` on `PoolSize`; `[Range(1, int.MaxValue)]` on `ConnectionLifetimeSeconds`; `[Range(0, int.MaxValue)]` on `QueryTimeoutSeconds`; `[Range(1, 600)]` on `ConnectionWaitSeconds`	None
`CatalogueSettings`, `Catalogue` (Infrastructure)	`[Range(0, int.MaxValue)]` on `RefreshSeconds`	None
`MapSettings`, `Map` (Features)	None: `ShowPreview` is true or false	None

`DatabaseConnection`, from `DATABASE_URL` (Infrastructure)	None	The URL parsed into a connection string, or 'DATABASE_URL: missing or empty'

Source: `Domain/Settings.cs`; `Infrastructure/Settings.cs`; `Features/Settings.cs`; `Infrastructure/Sql/Connection.cs`

A `[Range]` checks one value; `Validate`, the one method of the interface `IValidatableObject`, checks a rule across two, such as a highest zoom no lower than the lowest. The validator calls both and gathers every failure before it reports. Each property takes its default on the same line, as in `public int MaxZoom { get; init; } = 22;`, so a key the file leaves out keeps that value. Section D.9 shows when the validator runs and what it prints.

The limit 30 on the zooms has a reason in binary: `TileAddress.Of` computes `1 << z`, the number of columns at level z, and in a 32-bit `int`, `1 << 31` is already negative. The limit 85.06 on the latitudes has a reason in the map: Web Mercator, the web map's projection, stops near 85 degrees north and south.

D.4 [Closed]: a label for the analyser

One attribute, `[Closed]`, turns `DatabaseError` into a closed set of four cases, and the ExhaustiveMatching analyser refuses any `switch` over it that misses one. C# 14 has no union type, a type that is exactly one of a fixed list of cases, so Tessera writes an abstract record with four records nested inside and names them in the label:

```csharp
// Infrastructure/Sql/Database.cs (trimmed)
[Closed(typeof(MissingTable), typeof(MissingColumn), typeof(Unavailable), typeof(Other))]
public abstract record DatabaseError(string SqlState, string Message)
{
    public sealed record MissingTable(string SqlState, string Message)
        : DatabaseError(SqlState, Message);
    // … MissingColumn, Unavailable, Other
}
```

`TileServer.Answer` turns the four cases into 404, 503, 503 and 500. Its last arm, `_ => throw ExhaustiveMatch.Failed(error)`, tells the analyser that the `switch` means to cover every case. Add a fifth record to the label and leave the `switch` alone, and the build stops with EM0003, 'Subtype not handled by switch' (appendix C's Try it). The analyser checks only the cases the label names, so a record that derives from `DatabaseError` unnamed would pass. A test closes that gap and so becomes the label's second reader: `DatabaseErrorTests.Lists_every_kind_as_a_case` reads `[Closed]` itself by reflection, lists every record that derives from `DatabaseError`, and fails when the two lists differ. `Tessera.csproj` says when the label can go: C# 15's closed records will do the same work in the language itself.

D.5 Test attributes

Four attributes in `tests/` tell xUnit which methods are tests and what data each runs on, and a fifth builds the patterns one test class searches with; no attribute starts or shares the database. The three test projects hold 106 test methods and run 167 cases, 6 architecture, 94 unit and 67 integration, since each row of a theory counts as a case.

- **`[Fact]`, 77 methods.** A test with no data: it passes or fails as written. All six architecture rules are facts.
- **`[Theory]` with `[InlineData]`, 26 methods and 87 rows.** A test run once for each row. `TileTests.Refuses_a_tile_outside_the_grid` has four rows, from `[InlineData(-1, 0, 0)]` to

`[InlineData(1, 0, -1)]`; `ServiceTests.Answers_an_attack_with_404_and_keeps_every_row` has two attacking names.

- `[Property]`**, 3 methods.** FsCheck makes up a hundred inputs and checks one rule against each: any string comes out of `SqlName.Quote` as one quoted name; any three numbers give a `TileAddress` on the grid or none; any tile leaves the query's text unchanged. FsCheck counts each property as one case.
- `[GeneratedRegex]`**, 6 patterns.** `SourceTests` reads the service's files as text and searches them for a machine's name or for SQL outside `Queries.cs`. Each pattern sits on a `static partial` method, and the generator writes its body while the tests build, with the time limit the attribute names.

The integration tests get their database from interfaces, not labels. A class that implements `IClassFixture<DatabaseFixture>` gets one `DatabaseFixture` for all its tests, made before the first and disposed after the last; `IAsyncLifetime` gives the fixture its `InitializeAsync`, which builds the image from `db/Dockerfile` and starts it with Testcontainers. `xunit.runner.json` sets `maxParallelThreads` to 2, so two classes, and two databases, run at once. `StartTests` needs no database: it starts the built program as a separate process, with only the variables each test gives it.

D.6 Attributes the build writes for you

The SDK and the compiler add attributes of their own to every build, so the built `Tessera.dll` carries more labels than the source shows. None changes what the service does, but three matter to a reader.

EXHIBIT D.3 Attributes no .cs file in the repository holds, and where each comes from

ATTRIBUTE	WRITTEN BY	WHAT IT SAYS
`[assembly: InternalsVisibleTo("Tessera.UnitTests")]`, and the same for `Tessera.IntegrationTests`	The SDK, from two `<InternalsVisibleTo>` items in `Tessera.csproj`	The two test projects may call `internal` members, such as `MapRoutes.Answer` and `MapRoutes.PageHeaders`
`[assembly: AssemblyTitle]`, `AssemblyVersion`, `AssemblyInformationalVersion` and four more	The SDK, into `obj/…/Tessera.AssemblyInfo.cs`	The program's name and version, 1.0.0, with the Git commit after a `+`
`[assembly: TargetFramework(".NETCoreApp,Version=v10.0")]`	The SDK	The runtime the program was built for
`[assembly: AssemblyMetadata("Microsoft.Testing.Platform.Application", "true")]`	The SDK, in each test project	This program runs its own tests (appendix E)
`AsyncStateMachine`, `DebuggerStepThrough`, `NullableContext`	The C# compiler	How it turned an `async` method into a state machine, and which values may be null

Source: `obj/Release/net10.0/` *after* `dotnet build --configuration Release` *on a scratch copy, 18 October 2026; the endpoints' metadata, listed by the scratch program of section D.8*

The compiler's labels show where ASP.NET Core reads attributes. The author listed the metadata of each endpoint the built service maps: the tile endpoint carries `AsyncStateMachineAttribute` and `DebuggerStepThroughAttribute`, because `TileServer.Answer` is an `async` method, and the map's page

endpoints carry `NullableContextAttribute`. A label such as `[Authorize]` on a route's method would land in the same list, for a middleware step to read. Tessera puts none of its own there; section D.13 says why.

D.7 From Program.cs to TesseraHost.Build

The program starts in `Program.cs`, which picks one of two commands, and the command that serves hands the whole build to `TesseraHost.Build`, a short method in two halves. `Program.cs` holds no class: its first statement is a `switch` on the arguments, which C# runs as the program's entry point.

```csharp
// Program.cs (trimmed)
return args switch
{
    [] => await Serve().ConfigureAwait(false),
    ["seed"] => Seed(),
    _ => Usage(),
};
```

With `seed`, the program reads one setting, `TESSERA_PUBLISHER_DATABASE_URL`, runs DbUp, and builds no web service. With any other word it prints its two commands and ends with exit code 2. With no argument, `Serve` calls `TesseraHost.Build()` and runs what it returns until told to stop:

```csharp
// TesseraHost.cs (trimmed)
public static WebApplication Build(IReadOnlyDictionary<string, string?>? overrides = null)
{
    var builder = WebApplication.CreateBuilder();
    Settings.AddFile(builder.Configuration);
    if (overrides is not null)
        builder.Configuration.AddInMemoryCollection(overrides);
    builder.Services.AddProblemDetails(
        options => options.CustomizeProblemDetails = Problems.Shape);
    builder.AddTessera();

    var app = builder.Build();
    Settings.Check(app.Services);
    app.UseExceptionHandler();
    app.UseStatusCodePages();
    app.UseDisguisedTargetRefusal();
    app.UseRouting();
    app.MapTessera();
    return app;
}
```

`builder.Build()` divides the method. Above it, the code fills the ***service collection***: a list of instructions, each saying 'when someone asks for this type, make it this way, and keep it this long'. Nothing is made yet. `builder.Build()` turns the list into a ***service provider***, the object that makes each part on request and hands it to whatever asks for its type. Below it, the code lays the steps every request passes, then the routes. The tests call the same method with a dictionary of overrides, such as a free port and the test database's URL, so they test the program the office runs.

`CreateBuilder` takes no arguments, on purpose: the program's own arguments choose a command and nothing else, so every setting comes from `tessera.json` or the environment, never the command line.

D.8 The service collection: every registration and its lifetime

The door, `AddTessera`, adds 30 registrations to the 114 that ASP.NET Core and `AddProblemDetails` make, and every object of Tessera's own among them is a singleton, made once and shared by every request. Each registration carries a ***lifetime***, saying how long the object it makes lives:

- a singleton is made once, the first time anyone asks, and that one object serves the whole program;
- a ***scoped*** object is made once for each scope, such as one request, and dropped when the scope ends;
- a ***transient*** object is made new each time anyone asks.

> FOUNDATIONS A lifetime answers the question every program asks of a variable: where is it declared, and when does it vanish? A singleton is like a global variable set once at the start; a scoped object is like a local variable of one subroutine call; a transient one is like a new value built inside a loop on each pass. Sharing one value across many threads is safe only if no thread can change it half-way, so Tessera's singletons hold either a pool built for sharing or a list swapped whole in one step.

Here is the door, trimmed to its registrations, which read as a list of everything the module adds:

CSHARP

```csharp
// Modules/Tessera/TesseraModule.cs (trimmed)
public static IHostApplicationBuilder AddTessera(this IHostApplicationBuilder builder)
{
    AddSettings(builder.Services);  // Database, Tiles, Map, Catalogue, Area
    DatabaseConnections.Register(builder.Services, builder.Configuration,
        DatabaseConnections.ReaderDatabase);  // DATABASE_URL
    builder.Services.AddSingleton(services => Database.Open(
        services.GetRequiredService<IOptions<DatabaseConnection>>().Value.Parsed.Value.Text,
        services.GetRequiredService<IOptions<DatabaseSettings>>().Value));
    builder.Services.AddSingleton(services => new LiveCatalogue(
        cancel => CatalogueReader.Load(services.GetRequiredService<Database>(), cancel),
        TimeSpan.FromSeconds(services.GetRequiredService<IOptions<CatalogueSettings>>()
            .Value.RefreshSeconds),
        TimeSpan.FromSeconds(Problems.RetryAfterSeconds),
        services.GetRequiredService<ILoggerFactory>().CreateLogger("Tessera.Catalogue")));
    builder.Services.AddHostedService(
        services => services.GetRequiredService<LiveCatalogue>());
    builder.Services.AddHealthChecks()
        .AddCheck<DatabaseAnswers>("database")
        .AddCheck<CatalogueReady>("catalogue");
    return builder;
}
```

To see the real list, the author wrote a scratch program of thirty lines that references `Tessera.csproj`, calls `CreateBuilder` and `AddTessera` as the host does, and prints each registration `AddTessera` added, with its lifetime. Exhibit D.4 groups the output. Run again after `Settings.Section` gained its own key check on 18 October 2026, it printed the same totals, 111, 3 and 30: one registration per section changed its kind, and none was added.

Three facts stand out. First, Tessera registers nothing scoped or transient of its own: the transient entries are the framework's small steps for building settings. Second, a factory, the lambda `services => …`, makes each of Tessera's singletons, so each object's needs show as calls of `GetRequiredService` in one line of the door, not as labels in another file. Third, the two health checks are absent. `AddCheck<DatabaseAnswers>`

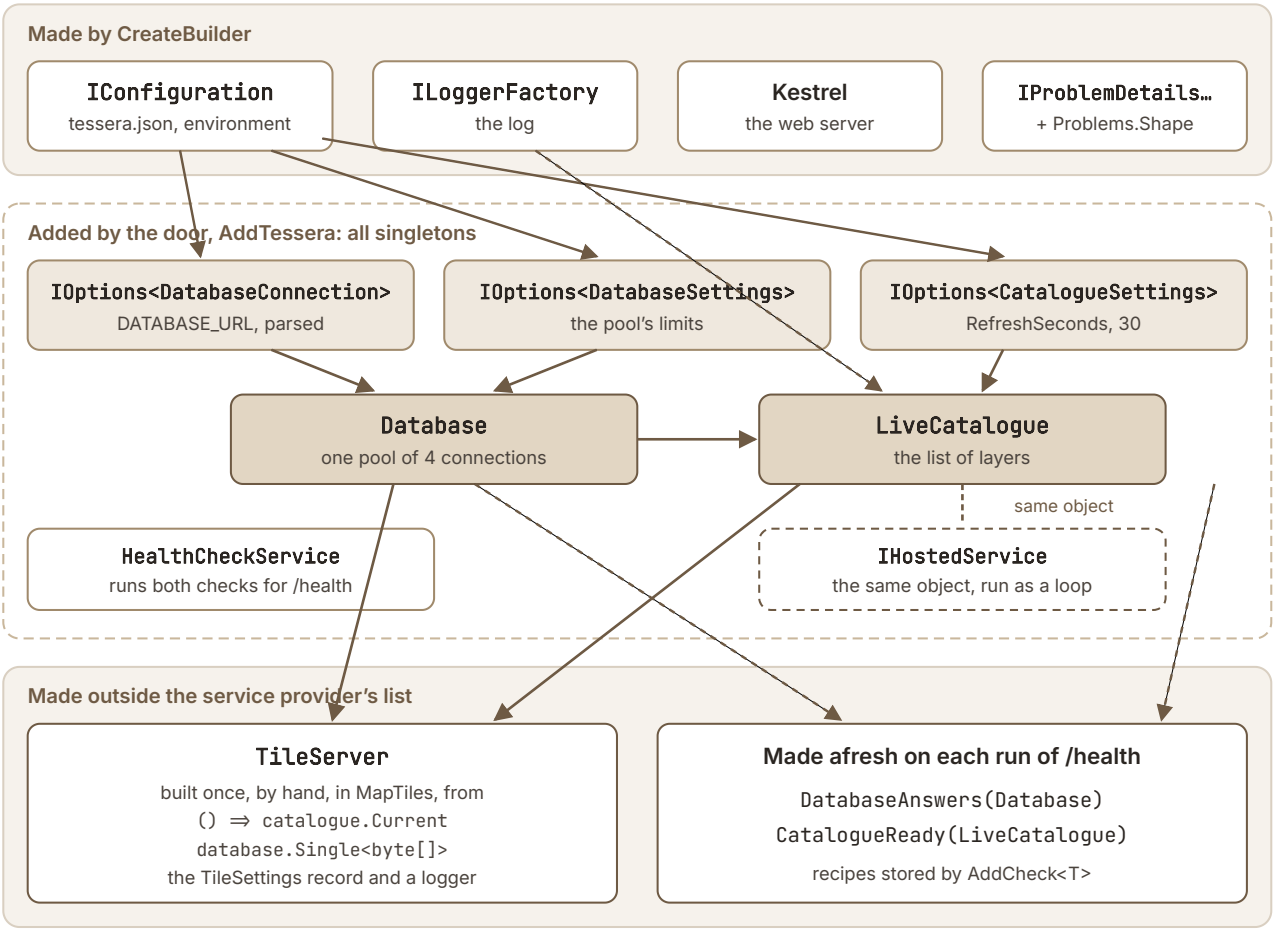
stores a recipe, and the health-check service makes a new `DatabaseAnswers` each time `/health` runs, handing its constructor the one `Database`; ASP.NET Core does this, not a line of Tessera's.

EXHIBIT D.4 The 30 registrations AddTessera adds, with the lifetimes the service provider reported

CALL IN THE DOOR	REGISTRATIONS	LIFETIME	WHAT EACH MAKES
`AddSettings`: five calls of `Settings.Section<T>`	4 for each section, 20 in all	Transient: `IConfigureOptions<T>`, which copies the section's clean keys into the record, and one for the start-up check. Singleton: two `IValidateOptions<T>`, the validator of section D.3 and `KeyCheck`, which names each key it could not place	The rules for building each settings record; the record itself comes from `IOptions<T>`, which `CreateBuilder` already registers as a singleton
`DatabaseConnections.Register`	3	Singleton: the step that parses `DATABASE_URL`, and its validator. Transient: its start-up check	A `DatabaseConnection`, parsed once
`AddSingleton(… Database.Open …)`	1	Singleton, from a factory	One `Database`, which holds the one pool of connections
`AddSingleton(… new LiveCatalogue …)`	1	Singleton, from a factory	One list of layers, which every request reads
`AddHostedService(…)`	1	Singleton `IHostedService`, from a factory	The same `LiveCatalogue`, run as a loop (section D.10)
`AddHealthChecks()` and two `AddCheck`	4	Singleton: `HealthCheckService`; a second `IHostedService`, `HealthCheckPublisherHostedService`; one options step for each check	The service `/health` asks; the two checks themselves are not registered (below)

Source: `TesseraModule.cs`; a scratch program run with SDK 10.0.401 on 18 October 2026, which printed 'CreateBuilder registers 111; AddProblemDetails adds 3; AddTessera adds 30', then each registration

EXHIBIT D.5 The objects the service provider makes, and who needs whom; each arrow runs from an object to the one that needs it



Source: `TesseraModule.cs`; `Features/Tiles.cs`; `Infrastructure/HealthChecks.cs`. *Dashed arrows: a need met when* `/health` *runs, or the logger. The settings records for* `Tiles`, `Map` *and* `Area` *are left out;* `MapTessera` *reads each once.*

As Exhibit D.5 shows, one chain carries the module's weight: `DATABASE_URL`, then `DatabaseConnection`, then `Database`, then `LiveCatalogue`, then `TileServer`. The service provider makes each object only when something first asks for it, and each factory asks for the objects it needs, so the pool exists before the list, and the list before the tiles. This handing of each part what it needs is dependency injection (section 3.7) in its plainest form: each part receives its needs through its constructor or its factory, and nothing fills a field later.

- **One object serves several readers.** The scratch program asked the provider for `Database` twice and printed 'Same Database twice: True'. `MapTiles`, `LiveCatalogue` and each `DatabaseAnswers` share that one pool.
- **The tile route is no registered service.** `MapTessera` asks the provider for `LiveCatalogue`, `Database` and the `TileSettings` record once, at start-up, and `MapTiles` builds one `TileServer` from them by hand, with a logger named `Tessera.Tiles` from the provider's `ILoggerFactory`. `TileServer` takes two functions, not the objects, so a test can hand it stand-ins with no provider, and `NullLogger.Instance` for a log that writes nowhere.
- **The constructor tells the whole story.** `DatabaseAnswers(Database database)` and `CatalogueReady(LiveCatalogue catalogue)` each name in one line everything they depend on.

> TRY IT .NET 10 runs a single `.cs` file as a program, and a line such as `#:project ../src/Tessera/Tessera.csproj` at its head lets the file use Tessera's code. Outside the repository, write such a file that calls `WebApplication.CreateBuilder()`, notes `builder.Services.Count`, calls `builder.AddTessera()`, and prints `Lifetime` and `ServiceType.Name` for each registration after the count. Run it with `dotnet run dump.cs`. If `MapTessera` later complains that 'Reflection-based serialization has been disabled', add `#:property PublishAot=false`: a single-file program turns that serialisation off by default.

D.9 Options: binding tessera.json and the environment, and checking them

Each section of `tessera.json` becomes a checked record through .NET's *options pattern*, and the program checks every record before it answers, naming every faulty value in one start. Four steps take a value from the file to the code.

1. Gather the sources. `CreateBuilder` lays down ASP.NET Core's own sources; `Settings.AddFile` then adds `tessera.json` from beside the program, and the environment again after it; a test may add its overrides last. A later source wins. The scratch program printed the list, weakest first:

```text
MemoryConfigurationProvider
EnvironmentVariablesConfigurationProvider Prefix: 'ASPNETCORE_'
MemoryConfigurationProvider
EnvironmentVariablesConfigurationProvider Prefix: 'DOTNET_'
JsonConfigurationProvider for 'appsettings.json' (Optional)
JsonConfigurationProvider for 'appsettings.Production.json' (Optional)
EnvironmentVariablesConfigurationProvider
Microsoft.Extensions.Configuration.ChainedConfigurationProvider
JsonConfigurationProvider for 'tessera.json' (Optional)
EnvironmentVariablesConfigurationProvider
```

The repository ships no `appsettings.json`, so the two optional files read nothing. The environment appears twice, and the second copy, after `tessera.json`, is the one that counts: `Tiles__CacheSeconds=0` overrides the file's 60, with `__` for each step down a level. *Sixteen Tiles*, chapter 5, counts the three sources an operator meets: the record's defaults, the file and the environment.

2. Bind each section. `Settings.Section<T>` makes each binding with four choices:

```csharp
// Common/Infrastructure/Settings.cs; line breaks moved to fit the page
public static void Section<T>(IServiceCollection services, string name) where T : class
{
    services.AddOptions<T>()
        .Configure<IConfiguration>((options, configuration) =>
            Bind(options, configuration.GetSection(name)))
        .ValidateDataAnnotations()
        .ValidateOnStart();
    services.AddSingleton<IValidateOptions<T>>(provider =>
        new KeyCheck<T>(provider.GetRequiredService<IConfiguration>().GetSection(name)));
}
```

`Bind` copies the section into the record, property by property, through each `init` setter, but hands .NET's binder only the keys that read cleanly, so the binder never throws. `KeyCheck`, a validator of Tessera's own,

names each key the record lacks and each value of the wrong kind, such as a word where a number belongs. `ValidateDataAnnotations` registers the validator of section D.3, and `ValidateOnStart` adds the record to the list that `IStartupValidator` checks, so both validators report together. The database's URL takes another road: `DatabaseConnections.Register` reads `DATABASE_URL` itself, parses it with `ConnectionString.FromUrl`, and lets `DatabaseConnection.Validate` refuse a missing or malformed one.

3. Check them all, at once. Straight after `builder.Build()`, `TesseraHost.Build` calls `Settings.Check`, which asks `IStartupValidator` to build and check every record. A failed check throws inside `Build`, before a single step of the request line exists.

4. Report and stop. `Serve` catches two kinds of exception, one section's problems or several sections' at once, prints each problem on a line of its own, and returns 1. The author ran the built program on a scratch copy:

```
$ env -i PATH=$PATH HOME=$HOME Tiles__MinZoom=10 Tiles__MaxZoom=2 Area__Name= \
    dotnet Tessera.dll
DataAnnotation validation failed for 'TileSettings' members: '' with the error:
  'MaxZoom must not be below MinZoom.'.
DataAnnotation validation failed for 'AreaSettings' members: 'Name' with the error:
  'The Name field is required.'.
DataAnnotation validation failed for 'DatabaseConnection' members: '' with the error:
  'DATABASE_URL: missing or empty'.
$ echo $?
1
```

TEXT

Three records, three faults, one start. A misspelt key joins the same list: with a valid `DATABASE_URL`, `Tiles__CacheSecond=5` and `Tiles__MaxZoom=31` printed two lines, the zoom's range first and then `Tiles: 'CacheSecond' is not a setting; Tiles holds Resolution, Buffer, MaxFeatures, MinZoom, MaxZoom, CacheSeconds.`, and the program stopped with exit code 1. The record's validator names the record, `TileSettings`; `KeyCheck` names the section, `Tiles`, as the file does.

> COMMON MISTAKE A misspelt section name passes unseen. `KeyCheck` names an unknown key inside `Tiles`, but a section called `Tile` belongs to no record, so no binder reads it and every tile setting keeps its default. `SettingsTests.Leaves_a_misspelt_section_at_its_defaults` pins that gap, and ADR 0003 records it. When a setting seems to have no effect, check the section's spelling first.

D.10 Hosted services: work that starts and stops with the program

Two objects run beside the requests as hosted services, and one of them, the list of layers, is the same object every tile request reads. A hosted service is an object that starts and stops with the program; `BackgroundService`, the base class `LiveCatalogue` extends, turns one method, `ExecuteAsync`, into a loop that runs on its own.

`LiveCatalogue`'s loop tries every five seconds until the first list of layers loads, then rereads it every `Catalogue.RefreshSeconds`, or stops when that is 0. Two registrations name it: `AddSingleton` makes it, and `AddHostedService(services => services.GetRequiredService<LiveCatalogue>())` hands that same object to the host to run. The scratch program asked the provider for both and printed 'Hosted service is the LiveCatalogue: True'. So the loop that swaps in a new list and the requests that read it share one field, `_current`, which `Interlocked.Exchange` writes in one step and `Volatile.Read` reads afresh each time (*Sixteen Tiles*, chapter 6).

The second hosted service, `HealthCheckPublisherHostedService`, comes with `AddHealthChecks`. It would send health reports to a publisher on a timer, but Tessera registers no publisher; `/health` runs the checks only when asked.

The debug log of a real start shows the loop at work. Kestrel listened first, and a watcher asked `/health` before the list had loaded; these are the four lines that matter:

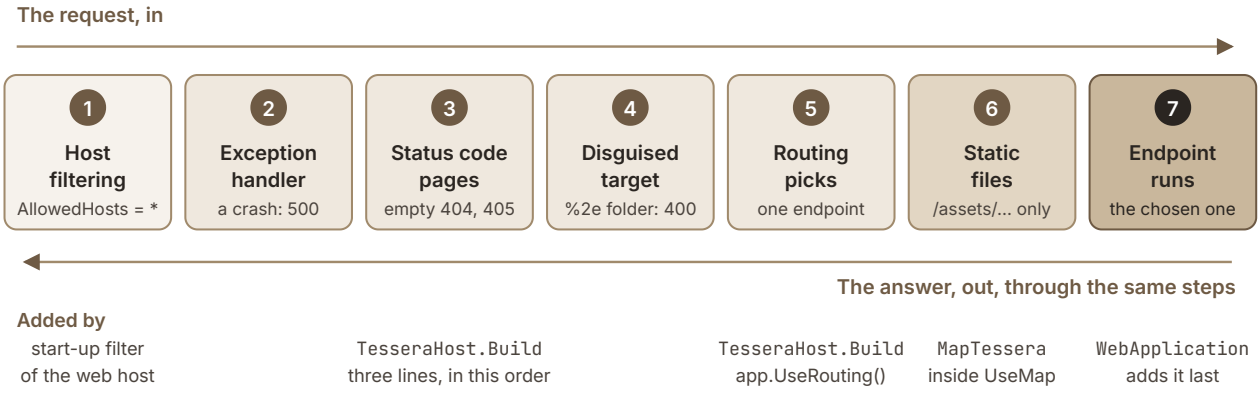
```text
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://127.0.0.1:7890
warn: Microsoft.Extensions.Diagnostics.HealthChecks.DefaultHealthCheckService[103]
      Health check catalogue with status Degraded completed after 0.5934ms
info: Microsoft.AspNetCore.Hosting.Diagnostics[2]
      Request finished HTTP/1.1 GET http://127.0.0.1:7890/health - 503 - application/json
info: Tessera.Catalogue[0]
      The service publishes 9 layers.
```

That is N-23 c3 in action: the service answers from its first moment, says `OUT_OF_SERVICE` with 503 until the list loads, and then serves.

D.11 Endpoints, filters and the order of the middleware

Every request passes seven steps of *middleware* in a fixed order, and only the last one runs the *endpoint* that routing chose, one of five that `MapTessera` maps. Middleware is a step that sees every request on its way in and every response on its way out; an endpoint is the one method chosen to answer. Exhibit D.6 draws the line and names who added each step.

EXHIBIT D.6 The seven steps every request passes, in order, and who added each



Source: `TesseraHost.cs`; `Common/Api/Targets.cs`; `Features/Map.cs`; the service's log at the level Debug, 18 October 2026, which named each of the framework's steps as it ran

Steps 2 to 5 are the four lines Tessera writes in `TesseraHost.Build`, in that order, so the two error steps wrap every request whatever answers it. Step 4, `UseDisguisedTargetRefusal` from `Common/Api/Targets.cs`, is a step written as one lambda: it reads the target as sent, from `IHttpRequestFeature.RawTarget`, and answers 400 with a problem document when `Targets.IsDisguised` finds a dot folder hidden as `%2e` or a raw control character; otherwise it calls the next step (S-06). It stands before routing because Kestrel has already tidied the disguise out of the path that routing reads. Step 1 comes from the web host's defaults: the log opened with 'Middleware configuration started with options: {AllowedHosts = *, …}', and with no `AllowedHosts` setting it lets every host name through. Step 6 is the odd one: `MapRoutes.UseMap` calls `app.UseStaticFiles` from inside `MapTessera`, so it sees each request after routing has chosen. If routing chose an endpoint, it stands aside, as the log said on every tile: 'Static files was skipped as the request

already matched an endpoint.' It serves a file only for a path no endpoint matches, such as `/assets/index-….js`. Step 7 is no line of Tessera's: `WebApplication` adds it at the end by itself.

The scratch program also listed the five endpoints `MapTessera` maps, by the names ASP.NET Core gives them; every one, `/health` among them, takes `GET` and `HEAD` alone:

```text
Health checks
HTTP: GET, HEAD /{layer}/{z:int}/{x:int}/{y:int}.pbf => Answer
HTTP: GET, HEAD /{layer}/{z:int}/{x:int}/{y:int}.mvt => Answer
HTTP: GET, HEAD /
HTTP: GET, HEAD /{file}
```

The patterns, not the order of the calls, decide which endpoint answers. `:int` is a ***route constraint***: a part of the path that must read as a whole number. ASP.NET Core fills each parameter of `TileServer.Answer(string layer, int z, int x, int y, CancellationToken cancel)` from the part of the path with the same name, and the `CancellationToken` from the request itself, so no `[FromRoute]` label is needed. A `POST` to a tile matches no method of the endpoint, so routing picks an endpoint of its own, '405 HTTP Method Not Supported'; status code pages, step 3, then fill the empty answer with a problem document and `Allow: GET, HEAD`. `/health` takes the same two methods through the metadata `MapHealth` adds, `WithMetadata(new HttpMethodMetadata([HttpMethods.Get, HttpMethods.Head]))`, so a `POST` there meets the same 405.

The two page endpoints share a group, `app.MapGroup("")`, which carries one ***endpoint filter***, `MapRoutes.PageHeaders`: a step that runs around those two endpoints and no others. It adds the page's Content Security Policy, `no-cache`, `nosniff` and `no-referrer` when the answer is the page itself; a 404 keeps the problem document's own headers.

D.12 One request for 12/3339/1788, through the objects DI built

A request for `/food.cafes/12/3339/1788.pbf` passes the seven steps, reaches the one `TileServer`, reads the one list of layers and borrows a connection from the one pool, and leaves as 17,575 bytes with a weak ETag. Exhibit D.7 follows it, with what the log said at each step. The author ran the built `Tessera.dll` on a scratch copy against the shipped database image, seeded as `tessera_publisher` and granted as `npm run publish` grants, with `Logging__LogLevel__Microsoft.AspNetCore=Debug`, and asked it directly on port 7890, past Nginx.

EXHIBIT D.7 One tile request, step by step, from Kestrel to the bytes

STEP	OBJECT	WHAT IT DOES	WHAT THE LOG SAID
1	Kestrel	Reads the request from its connection; in the office, Nginx opens it	'Request starting HTTP/1.1 GET http://127.0.0.1:7890/food.cafes/12/3339/1788.pbf'
2	Host filtering, exception handler, status code pages, the refusal of a disguised target	Let it pass, and wait for the answer; the target holds no `%2e` folder	Nothing for this request

3	Routing	Finds two candidates, rejects the `.mvt` pattern, keeps the `.pbf` one	`2 candidate(s) found`; `… .mvt … was rejected by complex segment '{y:}.mvt'`; `Request matched endpoint 'HTTP: GET, HEAD /{layer}/{z:int}/{x:int}/{y:int}.pbf => Answer'`
4	Static files	Stands aside	'Static files was skipped as the request already matched an endpoint.'
5	The endpoint	Fills `layer` with `food.cafes`, `z`, `x` and `y` with 12, 3339 and 1788, and calls `TileServer.Answer`	'Executing endpoint …'
6	`TileServer`, made once in `MapTiles`	Reads `LiveCatalogue.Current`; `Find("food.cafes")`; `TileAddress.Of(12, 3339, 1788, 0, 22)`	Nothing: pure steps
7	`Queries.Tile`	Builds the SQL text, with every value as a parameter	Nothing
8	`Database`, the singleton	`Single<byte[]>` borrows a connection from the pool; PostGIS builds the tile	Nothing at this level
9	`TileResult`	Sets `Cache-Control`; `TypedResults.Bytes` writes the bytes and the ETag	`Sending file with download name ''.`
10	Kestrel	Sends the answer	'Request finished … - 200 17575 application/vnd.mapbox-vector-tile 228.2920ms'

Source: the service's log at level Debug for `Microsoft.AspNetCore`, 18 October 2026; `Features/Tiles.cs`; `Infrastructure/Sql/Database.cs`

> THE PROOF The wiring in this appendix is the wiring the program follows. Started with no variables, the built `Tessera.dll` printed `DataAnnotation validation failed for 'DatabaseConnection' members: '' with the error: 'DATABASE_URL: missing or empty'.` and ended with exit code 1, before Kestrel listened. Given the reader's URL, it answered the tile with `200 OK`, `Content-Length: 17575`, `Cache-Control: public, max-age=60` and `ETag: W/"1aa11bddf9a69bf3e8f7b34258303866"`. Asked again with that tag in `If-None-Match`, it answered `304 Not Modified` in 4 ms, with no body. `/food.cafes/12/3339/9999.pbf` got a 404 problem document whose detail read 'No tile has those coordinates.'; a `POST` got 405 with `Allow: GET, HEAD`; `/health` said `{"status":"UP"}`. Together with the scratch program's three lines in sections D.8 and D.10 ('AddTessera adds 30', 'Same Database twice: True' and 'Hosted service is the LiveCatalogue: True'), these runs prove each step this appendix describes.

> KEY IDEA Read `TesseraHost.Build` and `AddTessera`, and you have read the wiring. Every object Tessera makes is named in a factory or a constructor, no field is filled after an object is made, and no attribute moves a part out of sight. A test can therefore make the same objects with `new`: the unit tests build a `LiveCatalogue` from a stand-in loader and a `TileServer` from two lambdas, with no service provider at all.

D.13 Attributes Tessera avoids, and why

Tessera leaves out every attribute that would wire a part out of sight, or do in a label a job that a line of code, a setting or Nginx already does where you can read it. A search of the source finds none of those in Exhibit D.8; for each, the code does the job in a form you can read and test.

EXHIBIT D.8 Common .NET attributes absent from Tessera, and what the code does instead

ABSENT	WHAT IT WOULD DO	WHAT TESSERA DOES INSTEAD
`[ApiController]`, `[Route]`, `[HttpGet]`	Mark a controller class and its routes	Minimal APIs: one `MapMethods` line joins a path to a method (`Features/Tiles.cs`)
`[FromRoute]`, `[FromServices]`	Say where a parameter's value comes from	A parameter named as the route's part is filled from it; `MapTiles` builds `TileServer` by hand, so the route asks for no service
`[Authorize]`	Demand a signed-in user	No one signs in; Nginx answers the office's network alone (S-07, ADR 0007)
`[OutputCache]`, `[ResponseCache]`	Keep answers in the service's memory, or set caching headers	Nginx keeps tiles (ADR 0008); `TileResult` writes `Cache-Control`, and `TypedResults.Bytes` the ETag
`[EnableRateLimiting]`	Cap how often one client may ask	Nginx caps each machine at 100 tiles a second, with a burst of 300 (ADR 0007)
`[JsonPropertyName]`	Rename a property in JSON	`MapRoutes.Render` writes an anonymous object whose names are already the JSON's, such as `bounds`; `/health` writes its one line by hand
`[LoggerMessage]`	Generate fast logging code	Plain `LogError`, `LogWarning` and `LogInformation`; the root `.editorconfig` switches off CA1848 and CA1873, since 'the service writes a few lines an hour'
`[OptionsValidator]`	Generate a settings validator while building, with no reflection	`ValidateDataAnnotations`, which reads the attributes by reflection, once, at start-up
`[Collection]`, `[CollectionDefinition]`	Share one fixture among several test classes	Each integration class gets its own database through `IClassFixture<DatabaseFixture>`, so classes never share rows
`[MemberData]`, `[Trait]`	Feed a theory from a method; tag tests to run some alone	Rows sit in `[InlineData]`, and FsCheck makes up the rest; three test projects divide the tests by kind

Source: a search of `src/` and `tests/`; the root `.editorconfig`; ADR 0004, ADR 0007, ADR 0008, ADR 0010; `docs/design/template-mapping.md`

Two of these absences shape the whole program. With no controller and no `[From…]` label, a route is one line, and its method a plain method that a unit test calls directly, as `TileTests.Answers_503_for_a_dropped_column` calls `TileServer.Answer`. And with no scan for labelled classes, every link between two parts shows as a call in the door, where the architecture tests and a reader both see it; `docs/design/template-mapping.md` refuses 'a reflection scan to find modules, since such a scan hides who calls whom'. The cost is a door that must list each new part; the gain, a whole module shown in the 85 lines of `TesseraModule.cs`.

> THINK IT OVER `AddHostedService<LiveCatalogue>()`, with no factory, is the form most guides show. Would it work here? Read `LiveCatalogue`'s constructor first, then ask how many lists of layers the program would hold, and which one the tiles would read.

Check your understanding

1. An attribute 'does nothing itself'. Name the program that reads `[Range(0, 30)]` on `MaxZoom`, and say when it reads it.
2. Why may `TileSettings`, with its `[Range]` attributes, live in the Domain, where the first architecture rule forbids ASP.NET Core?
3. What lifetime does `Database` have, and what would go wrong if each request made a new one?
4. `DatabaseAnswers` appears nowhere in the list of registrations. How does `/health` get one?
5. A request for a tile matches an endpoint. Why does the static files step, which comes after routing, not try to serve it as a file?
6. An operator starts the service with `Tiles__CacheSecond=5` and `Tiles__MaxZoom=31`. How many problems does the first start name, and why?

Answers. 1. .NET's data-annotations validator, which `ValidateDataAnnotations` registers; it runs when `Settings.Check` calls `IStartupValidator.Validate`, straight after `builder.Build()`, before the service answers anything. 2. The attributes come from `System.ComponentModel.DataAnnotations`, in .NET's base library, not from ASP.NET Core. 3. Singleton. It holds the one pool of four connections; a new one for each request would open a new pool each time, losing the pool's whole point and the limits of N-22. 4. `AddCheck<DatabaseAnswers>` stores a recipe, and the health-check service makes a new one on each run, handing its constructor the singleton `Database`. 5. It stands aside whenever an endpoint was chosen, as the log says on every tile; it serves only a path no endpoint matches, such as a file under `/assets/`. 6. Two, with a valid `DATABASE_URL`: the zoom out of range and the misspelt key, each on a line of its own. The binder sees only the keys that read cleanly, and `KeyCheck` names the rest beside the record's own validator, so one start names every problem.

E

The toolchain

Every tool Tessera needs stands on one base, Ubuntu 24.04 in WSL2, and four commands reach them all: `dotnet`**,** `docker`**,** `npm` **and** `uv`**.** The file that uses each tool names its version, so a version changes only on purpose: `global.json` names the .NET SDK, `Directory.Packages.props` every .NET package, `dotnet-tools.json` the one .NET tool, each Dockerfile its image, `package.json` Node.js. A missing Docker stops the most work; a fresh machine most often lacks the SDK.

A ***toolchain*** is the set of tools that turns source code into a running, tested service, each resting on the ones beneath it. This appendix draws the whole picture (section E.1), then takes the .NET SDK and `global.json` (E.2), the shared build files and their analysers (E.3), the test platform (E.4), Docker (E.5), and Node.js and uv for the parts outside the service (E.6). It ends with every version in one place (E.7) and what stops when a tool is missing (E.8). *Sixteen Tiles*, chapter 10, explains why each check exists; this appendix says which tool runs it, and at which version.

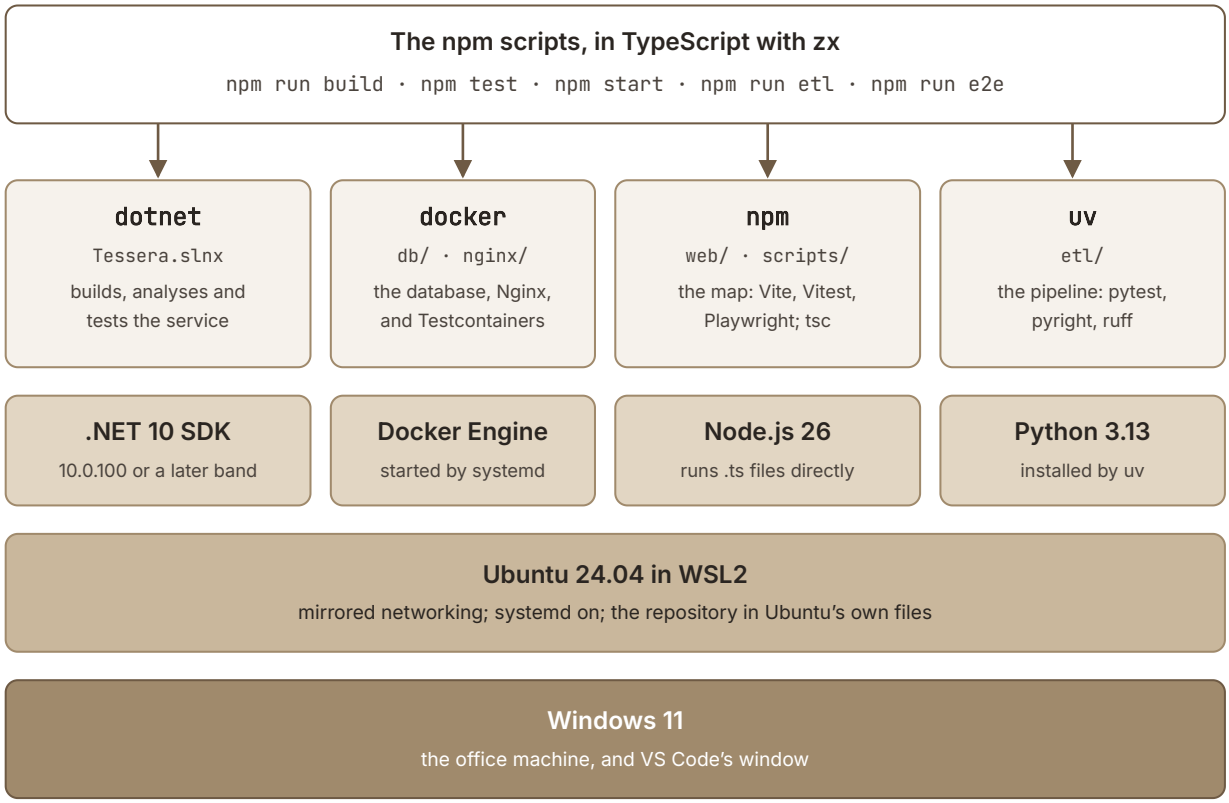
E.1 The toolchain in one picture

Four runtimes sit on Ubuntu, each carries one command, and the npm scripts call all four in the right order. Exhibit E.1 stacks them as a building: Windows at the foundation, the npm scripts on the top floor.

Each command owns one part of the repository. `dotnet` builds and tests the C# service from `Tessera.slnx`, the solution that lists its four projects. `docker` builds the database image from `db/` and the Nginx image from `nginx/`, and runs both. `npm` runs the scripts at the root and the map in `web/`. `uv`, Python's project tool, runs the pipeline in `etl/`. The scripts join the four into the commands you type each day: `npm run build` installs and builds the map, then runs `dotnet build --configuration Release`, which copies `web/dist` into the program's `wwwroot/`. `npm test` runs every check, from the scripts' types to the browser tests.

> KEY IDEA The four commands are the way in. No Makefile or wrapper stands around `dotnet`; `README.md` lists the commands, and each npm script is a short TypeScript file in `scripts/`. Learn `dotnet`, `docker`, `npm` and `uv`, and you can rebuild, test and run everything in this book.

EXHIBIT E.1 The toolchain from Windows up to the daily commands



Source: `README.md`, *'What you need'*; `global.json`; `package.json`; `etl/.python-version`; `scripts/build.ts`; `scripts/test.ts`

E.2 The .NET SDK and global.json

One small file, `global.json`, chooses the SDK that builds Tessera and the platform that runs its tests, so every machine and the build on GitHub build with the same tools. The .NET SDK (section 4.4) is the toolkit behind the command `dotnet`: it holds the C# compiler, MSBuild, the build engine that reads each project file, NuGet, which downloads packages, and the runtime that runs the result. JSON allows no comments, so `Directory.Build.props` explains the file in a comment of its own:

JSON

```
{
  "sdk": {
    "version": "10.0.100",
    "rollForward": "latestFeature"
  },
  "test": {
    "runner": "Microsoft.Testing.Platform"
  }
}
```

An SDK version has three parts: 10 is the major release, 0 the minor, and the last number holds a feature band in its hundreds (100, 200 and on) and a patch in the rest. `"rollForward": "latestFeature"` lets `dotnet` use the newest SDK of .NET 10 the machine holds, from 10.0.100 up, and never one of .NET 11. This book's author built and ran Tessera with SDK 10.0.401, whose .NET and ASP.NET Core runtimes are 10.0.12. Section E.4 covers the `test` section.

With no SDK of .NET 10 on the machine, every `dotnet` command stops before it reads a project. The author pointed a scratch folder's `global.json` at a missing SDK to see the message a reader would meet:

```text
A compatible .NET SDK was not found.

Requested SDK version: 10.0.500
global.json file: …/global.json

Installed SDKs:

Install the [10.0.500] .NET SDK or update […/global.json] to match an installed SDK.
```

`dotnet --list-sdks` prints the SDKs a machine holds; read its answer before you read anything else into a failed build. The build on GitHub runs `actions/setup-dotnet@v4` with `global-json-file: global.json`, so the runner installs the SDK the file names.

> FOUNDATIONS A build tool runs other programs in order, as a recipe orders its steps. MSBuild reads each project file as a list of targets, such as restore, compile and copy, each depending on the ones before, and runs them in an order that meets every dependency. A NuGet package is a zipped library with a name and a version, like a module you import in Python, fetched from a shared store on the web the first time a build asks for it.

E.3 The shared build files, and the analysers they switch on

Two files at the root, `Directory.Build.props` and `Directory.Packages.props`, give all four projects the same settings and the same package versions, and three `.editorconfig` files switch off, with a reason each, the few rules that clash with the code. MSBuild applies a file named `Directory.Build.props` to every project beneath it, so no project repeats a setting. Exhibit E.2 lists what it sets.

EXHIBIT E.2 What Directory.Build.props sets for every project, and why

SETTING	VALUE	WHAT IT DOES
`TargetFramework`	`net10.0`	Builds for .NET 10, a release with long-term support
`LangVersion`	14	C# 14, with no feature still marked preview
`Nullable`	`enable`	The compiler checks every value that may be null
`ImplicitUsings`	`enable`	Common namespaces, such as `System.Linq`, need no `using` line
`TreatWarningsAsErrors`	true	Every warning stops the build
`EnforceCodeStyleInBuild`	true	The editor's style rules, such as IDE0130, run in every build
`GenerateDocumentationFile`	true, with CS1591 off	The compiler checks each `///` comment that exists, and asks for none
`ManagePackageVersionsCentrally`	true	Versions live in `Directory.Packages.props` alone
`InvariantGlobalization`	true	No culture data, so 22.19 never reads as 22,19
`AnalysisLevel`; `AnalysisMode`	`latest`; `All`	Every rule of .NET's own analysers, at the newest level

`PackageReference`	`Meziantou.Analyzer`, `PrivateAssets="all"`	Meziantou's rules in every project, used while building and never shipped

Source: `Directory.Build.props`, *whose comments give each reason*

One version of each package. With ***central package management***, a project names a package with no version, as `<PackageReference Include="Npgsql" />`, and `Directory.Packages.props` names its version once for the whole solution, in two groups: six for the service and six for the tests, and every project that needs one of the twelve names it. Beside the packages stands one .NET tool: `dotnet-tools.json` pins ReportGenerator 5.5.11, which `dotnet tool restore` fetches and `scripts/coverage.ts` runs.

Four sets of analysers. An analyser runs inside the compiler (appendix D, section D.1), and with `TreatWarningsAsErrors` each of its warnings stops the build:

- **.NET's own**, the CA rules for quality and the IDE rules for style, all of them at `AnalysisMode` `All`;
- **Meziantou.Analyzer** 2.0.295, the MA rules, which add checks .NET lacks, such as naming the comparer a set of strings uses;
- **ExhaustiveMatching.Analyzer** 0.5.0, referenced by the service and reaching the tests through it, which reads `[Closed]`;
- **xunit.analyzers** 2.2.0, which arrives with xUnit in each test project and checks the tests' own shape.

The root `.editorconfig` switches off 11 rules for all code and 2 more for `TesseraHost.cs`; `tests/.editorconfig` switches off 6 for the tests; `src/Tessera/.editorconfig` raises one, IDE0130, to an error, so each file's namespace must match its folder, as the architecture tests need. Each rule switched off carries its reason in a comment above it.

E.4 The test platform: xUnit v3 on Microsoft.Testing.Platform

Each of the three test projects builds into a program of its own that runs its tests, so `dotnet run --project` runs one project and `dotnet test` runs all three. xUnit, version 3, is the test framework; it reads the attributes of appendix D, section D.5. Microsoft.Testing.Platform, version 2.5.1 here, is the ***test platform***: the small runner that xUnit v3 builds into each test project, replacing the older separate runner, `vstest`. Three lines make it so: `"runner": "Microsoft.Testing.Platform"` in `global.json`, which tells `dotnet test` to speak to it; `<OutputType>Exe</OutputType>` in each test project, which makes the project a program; and `<UseMicrosoftTestingPlatformRunner>true</UseMicrosoftTestingPlatformRunner>`, which keeps the platform in charge even under `dotnet run`, so the program takes the platform's options and not xUnit's alone. Options follow a `--`: `--filter-class Tessera.UnitTests.TileTests` runs one class, where xUnit's own runner took `-class`, and `--coverage` records which lines the tests run. Each project names the package behind that option, Microsoft.Testing.Extensions.CodeCoverage 18.11.2, which writes its record as Cobertura, an XML format that ReportGenerator reads.

EXHIBIT E.3 The three test projects, the libraries each adds, and one run of each

PROJECT	LIBRARIES BESIDE XUNIT	CASES	TIME ON ONE RUN
`Tessera.ArchitectureTests`	ArchUnitNET 0.13.4, through `TngTech.ArchUnitNET.xUnitV3`	6	8.1 s
`Tessera.UnitTests`	FsCheck 3.4.0, through `FsCheck.Xunit.v3`; an approved file, `approved/tile-query.sql`	94	0.8 s

`Tessera.IntegrationTests`	Testcontainers 4.16.0; Npgsql 10.0.3; `NetTopologySuite.IO.VectorTiles.Mapbox` 1.1.0, which reads a tile back into points	67	29 s

Source: the three project files; `dotnet list package --include-transitive`; one run of each project on a scratch copy built in Release, 18 October 2026, with the platform's own count of time.

The commands, from the repository's root, after one build:

```bash
# Build once, in Release, then run each test project as the build on GitHub does
dotnet build --configuration Release
dotnet run --no-build --configuration Release --project tests/Tessera.ArchitectureTests
dotnet run --no-build --configuration Release --project tests/Tessera.UnitTests
dotnet run --no-build --configuration Release --project tests/Tessera.IntegrationTests

# Or all three at once, through global.json's test runner
dotnet test
```

Each run ends with the platform's summary, 'Test run summary: Passed!', then the counts, such as `total: 94`, `failed: 0` and `succeeded: 94` for the unit tests. The integration tests need Docker running, since `DatabaseFixture` builds the database image for every class but `StartTests`. `npm test` and the GitHub build add `--coverage --coverage-output-format cobertura` to each run, so each suite writes a record to `coverage/`; ReportGenerator then merges the three into `coverage/report/index.html`, and `scripts/coverage.ts` fails below 94 per cent of lines or 84 of branches. The run for this book covered 96.1 per cent of the lines and 85.5 per cent of the branches.

E.5 Docker: the images and what each runs

Docker runs two images built from the repository, PostgreSQL with PostGIS and Nginx, and Testcontainers builds the same database image for the tests; the C# service itself runs outside Docker. An image is a packed file system with one program ready to start; a container is one running copy of an image. Each Dockerfile names its base by a fixed tag, so every machine and every test meets the same release until someone changes that line.

EXHIBIT E.4 Four base images, what each runs, and where each is named

BASE IMAGE	BUILT INTO	WHAT IT RUNS	STARTED BY
`postgis/postgis:18-3.6`	`db/Dockerfile`: the image `tessera-db`; `tessera-db-test:latest` for the tests	PostgreSQL 18 with PostGIS 3.6, the two roles from `db/initdb/20-roles.sql`, five server settings such as `jit=off`	`npm start`, as the container `tessera-db` with the volume `tessera-data`; `npm run e2e`, as `tessera-e2e`; `DatabaseFixture`
`nginx:1.29-alpine`	`nginx/Dockerfile`: the image `tessera-nginx`	Nginx in front of the two copies of the service; its settings, `default.conf.template`, mounted from outside the image	`npm start`, as the container `tessera-nginx`; `npm run e2e`
`ghcr.io/osgeo/gdal:alpine-small-3.11.4`	Nothing; run as it is	GDAL's `ogrinfo`, to describe a source of places	`npm run inspect`, on a machine with no `ogrinfo` of its own

`mcr.microsoft.com/playwright:v1.56.0-noble`	Nothing; run as it is	Playwright with the Chromium and fonts its version was tested with, so the approved images of UI-17 match on any machine	`npm run e2e` with `TESSERA_PLAYWRIGHT_IN_DOCKER` set, as the GitHub build does

Source: `db/Dockerfile`; `nginx/Dockerfile`; `scripts/start.ts`; `scripts/e2e.ts`; `scripts/lib/plan.ts`, `playwrightImage`; `scripts/lib/inspect.ts`; `tests/Tessera.IntegrationTests/DatabaseFixture.cs`

The database's Dockerfile is short, and its comments carry the reasons:

```text
# db/Dockerfile (trimmed)
FROM postgis/postgis:18-3.6

# The image runs these scripts once, on a new, empty data folder, after its own 10_postgis.sh.
COPY initdb/ /docker-entrypoint-initdb.d/

CMD ["postgres", \
     "-c", "jit=off", \
     "-c", "max_connections=20", \
     "-c", "shared_buffers=256MB", \
     "-c", "effective_cache_size=2GB", \
     "-c", "random_page_cost=1.1"]
```

The service alone runs on no image. `npm start` builds it once and runs two copies of `src/Tessera/bin/Release/net10.0/Tessera.dll` with `dotnet`, on ports 7800 and 7801, straight on Ubuntu (N-29); Nginx reaches them on the host's own network. The same program, run as `dotnet …/Tessera.dll seed`, loads the places first.

Testcontainers starts the database for the tests, as US-01's page shows. `DatabaseFixture` builds the image once for the whole run, names it `tessera-db-test:latest` and keeps it, so a later run starts in seconds; each test class then starts its own container on a free port, clear of `tessera-db` on 5432. For the same reason, the build on GitHub needs no database service.

> WATCH OUT The image runs the scripts in `/docker-entrypoint-initdb.d/` only when its data folder is empty. Change `db/initdb/20-roles.sql` and restart `tessera-db`, and nothing changes: the volume `tessera-data` still holds the old database. The tests never meet this trap, since each test class starts a new container with an empty folder.

E.6 Node.js and uv, for the parts outside the service

Node.js 26 runs the scripts and builds the map, and uv runs the Python pipeline; neither touches the C# service, but `npm test` needs both. ADR 0002 chose TypeScript for the scripts, and Node.js 26 runs a `.ts` file directly by stripping its types, so `package.json` asks for `"node": ">=26"` and runs each script as `node scripts/….ts`. Node.js strips the types without checking them, so `npm test` runs `tsc --noEmit` first. The map in `web/` has its own `package.json`, its own lock file and its own tools: Vite builds it, Vitest tests its logic, Playwright drives Chromium for the browser tests, on the machine itself or inside Playwright's own Docker image, named after the version `web/package.json` pins.

uv, Python's project tool, reads `etl/pyproject.toml` and `etl/uv.lock`, installs Python 3.13, which `etl/.python-version` names, and the pipeline's packages, then runs each check: pyright for types, ruff for style, import-linter for the rule that the pipeline's pure code reads no file, and pytest for the tests. The

repository pins no version of uv itself; the build on GitHub installs it with `astral-sh/setup-uv@v6`. Some pipeline tests call GDAL's `ogr2ogr`, so the build on GitHub installs Ubuntu's `gdal-bin` first. *Sixteen Tiles*, mini-guides M1 to M3, explains the pipeline, the map and the scripts.

E.7 Every version in one place

Each tool has one version, named in the file that uses it, and every .NET package takes its version from `Directory.Packages.props`. Exhibit E.5 lists the tools; Exhibit E.6 lists the packages NuGet resolved for the service and its tests.

EXHIBIT E.5 Every tool, its version, and the file that names it

TOOL	VERSION	NAMED IN	WHAT NEEDS IT
Windows	11	US-35	WSL2's mirrored networking
Ubuntu in WSL2	24.04	`build.yml` (`runs-on: ubuntu-24.04`); US-35	Everything below
.NET SDK	10.0.100 or a later band of .NET 10; checked with 10.0.401	`global.json`	Building and testing the service
.NET and ASP.NET Core runtimes	10.0.12, with SDK 10.0.401	Nowhere: they come with the SDK	Running `Tessera.dll`
C#	14	`Directory.Build.props` (`LangVersion`)	The compiler
Microsoft.Testing.Platform	2.5.1, through xunit.v3	`global.json` names it; the version comes with xUnit	Running the tests
ReportGenerator	5.5.11	`dotnet-tools.json`	Merging the three coverage records into one report
Docker Engine	Not pinned	`README.md`, 'What you need'	The database, Nginx, the integration tests
PostgreSQL; PostGIS	18; 3.6	`db/Dockerfile`	The places and the tiles
Nginx	1.29, on Alpine	`nginx/Dockerfile`	The office's door on port 80
Node.js	26	`package.json` and `web/package.json` (`"node": ">=26"`); `build.yml`	The scripts and the map's build
TypeScript; zx	6.0.3; 8.8.5	`package.json`; TypeScript also in `web/package.json`	The scripts' types and runner
React; MapLibre GL JS; react-elmish	19.3.0; 6.14.0; 10.7.1	`web/package.json`	The map
Vite; Vitest; Playwright	8.3.4; 5.0.4; 1.56.0	`web/package.json`	The map's build, unit tests and browser tests

Python; uv	3.13; not pinned	`etl/.python-version`; `build.yml` (`setup-uv@v6`)	The pipeline
pytest; ruff; pyright	9.1.1; 0.17.1; 1.1.414	`etl/uv.lock`	The pipeline's checks
GDAL, as an image	3.11.4	`scripts/lib/inspect.ts`	`npm run inspect`

Source: `global.json`; `Directory.Build.props`; `dotnet-tools.json`; `package.json`; `web/package.json`; `etl/`; `db/Dockerfile`; `nginx/Dockerfile`; `.github/workflows/build.yml`; `dotnet --list-runtimes`

EXHIBIT E.6 The .NET packages, as NuGet resolved them

PACKAGE	VERSION	ITS JOB
ASP.NET Core	10.0.12, the shared runtime	The web host, Kestrel, routing, options, health checks, problem documents
Npgsql	10.0.3	Talks to PostgreSQL, and keeps the pool of connections
Dapper	2.1.89	Reads rows into records
dbup-postgresql; dbup-core	7.0.1; 6.1.1	The seed
CSharpFunctionalExtensions	3.7.0	`Result`
ExhaustiveMatching.Analyzer; Meziantou.Analyzer	0.5.0; 2.0.295	Analysers, while building only
xunit.v3; xunit.analyzers	4.0.2; 2.2.0	The tests, and the analyser that checks them
FsCheck.Xunit.v3	3.4.0	The property tests
Testcontainers.PostgreSql	4.16.0	The database for the integration tests
NetTopologySuite.IO.VectorTiles.Mapbox	1.1.0	Reads a tile back into points
TngTech.ArchUnitNET.xUnitV3	0.13.4	The six rules in `BoundaryTests`
Microsoft.Testing.Extensions.CodeCoverage	18.11.2	Records the lines and branches each test suite runs

Source: `Directory.Packages.props`; `dotnet list Tessera.slnx package --include-transitive`, *run on a scratch copy on 18 October 2026*

Dependabot keeps these versions moving. Each Monday at 06:00, Taipei time, the same as Macau's, it opens a pull request for each of eight lists that has a release at least seven days old: the .NET packages, the SDK in `global.json`, the scripts, the map, the pipeline, each Dockerfile and the build's own actions. It never proposes a new major release of PostgreSQL, whose data an older release cannot read. Minor and patch releases arrive grouped, one pull request for each list, so a week's updates make one review; a major release arrives alone. A version with a known flaw gets its pull request at once only when 'Dependabot alerts' and 'Dependabot security updates' are switched on in the repository's settings, and nobody has yet checked that they are. *Sixteen Tiles*, chapter 9, tells the rest.

E.8 What stops when a tool is missing

Each missing tool stops a known part of the work, and Docker stops the most. When a command fails without naming a line of your code, read Exhibit E.7: the cause is often a tool, not the program, and the table names the tool to install or start.

EXHIBIT E.7 Each missing tool stops a known part of the work

MISSING OR WRONG	WHAT STOPS	WHAT STILL WORKS
The .NET 10 SDK, or one older than 10.0.100	Every `dotnet` command: 'A compatible .NET SDK was not found.'; so `npm run build`, `npm start` and `npm test`	The map's own tests in `web/`, the scripts' tests, the pipeline
Docker Engine, or Docker stopped	Six of the seven integration classes, which start a database, and so the coverage check; `npm start`; `npm run e2e`	The architecture and unit tests; `StartTests`, which starts the program with no database
Node.js 26	Every npm script, and the map's build	`dotnet build` and the architecture and unit tests; with no `web/dist`, the release carries no map, `/` answers 404, 'This release carries no map.', and the map's cases in `ServiceTests` fail
uv	The pipeline's four checks, so `npm test`; `npm run etl`	The service and the map
Playwright's browser	The browser tests in `npm run e2e`	Everything else; and the browser tests themselves inside Playwright's image, with `TESSERA_PLAYWRIGHT_IN_DOCKER=1`
The network to NuGet	A first build, or one after a version changes; the first `dotnet tool restore`, so the coverage report	Builds from NuGet's own store in `~/.nuget/packages`
The `.git` folder	The integration tests that start a database: `DatabaseFixture` finds `db/` through it	Everything else

Source: `README.md`; `global.json`; `scripts/build.ts`; `scripts/test.ts`; `Features/Map.cs`; `DatabaseFixture.cs`; `StartTests.cs`

Check your understanding

1. A machine holds SDK 10.0.401 and SDK 11.0.100. Which one builds Tessera, and which line decides?
2. Which file fixes the version of Npgsql, though `Tessera.csproj` never names a version?
3. Why does `dotnet run --project tests/Tessera.UnitTests` run tests at all, when `dotnet run` usually starts a program?
4. Which one program of Tessera's runs on no Docker image, and how does it start?
5. Docker is down for the day. Which commands still check the borders, the domain and the settings?

Answers. 1. SDK 10.0.401: `global.json` asks for 10.0.100 with `"rollForward": "latestFeature"`, which allows any later band of .NET 10 and never .NET 11. 2. `Directory.Packages.props`, through central package management, which `ManagePackageVersionsCentrally` in `Directory.Build.props` switches on. 3. Each test project is a program: `OutputType` `Exe`, with xUnit v3 and Microsoft.Testing.Platform built in, so running it runs its tests. 4. The service: `npm start` runs two copies of `src/Tessera/bin/Release/net10.0/Tessera.dll` with `dotnet`, on ports 7800 and 7801. 5. `dotnet run --project tests/Tessera.ArchitectureTests` and `dotnet run --project tests/Tessera.UnitTests`; `SettingsTests` is a unit test class.

F

Ten sessions in VS Code: set up and rebuild the skeleton

Ten sessions take you from a bare PC to a tile served by code you wrote yourself: one builds the machine, eight rebuild US-00, US-01 and US-07 test-first with the .NET SDK and the C# Dev Kit, and the last compares your work with the worked answer. Each session ends with one piece of evidence you can show, and the next session starts from it.

Code marked 'your rebuild' is yours to write; the repository holds none of it. This book's author built a scratch rebuild along these sessions on 18 October 2026, with SDK 10.0.401, and the marked code compiled and passed there; yours will differ in its details, as it should. The sessions take two named short cuts. They skip US-03 and hard-code one layer, so a tile reaches your screen sooner; US-03 later replaces that layer with the database's own list. And they borrow US-24's seed command in its plainest form, since a tile needs places to draw. A session is one sitting of about three hours: six pomodoros of 25 minutes with their breaks, as `docs/study/00-using-the-pomodoro-technique.md` describes them. The delivery plan sizes the three stories at 4 points, about four working days, after its set-up day; US-07's two points cross from iteration 1 into iteration 2.

F.1 The ten sessions at a glance

Each session ends with a test or a command that proves it done, and the next session starts from that proof. Exhibit F.1 runs in time order.

EXHIBIT F.1 One session builds the machine, eight rebuild three stories, and the tenth reviews them

SESSION	STORY	THE WORK	DONE WHEN
1	The machine	WSL2, Docker Engine, .NET 10 SDK, Node.js 26, uv; the worked answer cloned	The worked answer's three test projects pass
2	US-00	An empty solution whose service answers on its port	`Answers_on_its_port` passes; `curl` reaches `/health`
3	US-00	The worked answer's build rules; the door; the first wall	`Common_never_reaches_a_module` passes, after you have seen it fail
4	US-01	`db/Dockerfile` and the two roles, proved by hand	`tessera_reader` reads `on` and cannot write
5	US-01	The same image under Testcontainers, for every test	Your reader's test passes with no database of your own running
6	US-00	`tessera.json`, `TileSettings`, `DATABASE_URL`, and the start refused	Your settings tests and `Refuses_to_serve_and_names_each_missing_variable` pass

7	US-07	`TileAddress`, `SqlName`, and the Domain's wall	`Refuses_a_tile_outside_the_grid`, a property test and `Domain_stands_alone` pass
8	US-07	`Layer`, `Query`, `Queries`, `DatabaseError`, `Database`	`Matches_the_approved_tile_query` and two more rules pass
9	US-07	The problem documents, the route, the seed and the door	`curl` fetches `/food.cafes/12/3339/1788.pbf` with 200 from your service
10	Review	A clean run, the comparison, and a decision record of your own	4 points done; your notes written

Source: this book; `docs/user-stories/user-stories.md`; `docs/guides/delivery-plan.md`; `tests/`

Keep two folders apart. The worked answer, `~/src/tessera`, is for reading and running; never edit it during the rebuild. Your rebuild, `~/src/tessera-rebuild`, begins empty in session 2, and every line you type goes there. Both serve on port 7800, and both name their database container `tessera-db` when started by hand, so run one at a time.

F.2 Three facts about VS Code

VS Code splits itself between Windows and Ubuntu, and the C# Dev Kit reads the solution through the same compiler and analysers as `dotnet build`, yet the terminal has the last word; three habits follow.

1. **VS Code runs in two halves.** The window runs on Windows. The WSL extension installs VS Code's server inside Ubuntu, where every other extension runs with it, beside the SDK and Docker. Open each folder from an Ubuntu terminal with `code .`; the corner of the status bar then reads 'WSL: Ubuntu-24.04'. The ***integrated terminal***, opened with Ctrl+`, is then an Ubuntu shell in the project's folder, and every command in this appendix runs there.
2. **One extension, installed inside Ubuntu, and three settings.** The C# Dev Kit brings the C# extension and its ***language server***, which reads `Tessera.slnx` and its projects and marks mistakes as you type, with a Solution Explorer, the debugger and the Testing view. It runs the real C# compiler and the analysers the projects name, so it shows the errors `dotnet build` reports; by default, though, only for the files you have open. Put these in the rebuild's `.vscode/settings.json`:

   ```json
   {
     "dotnet.defaultSolution": "Tessera.slnx",
     "dotnet.backgroundAnalysis.analyzerDiagnosticsScope": "fullSolution",
     "dotnet.testWindow.useTestingPlatformProtocol": true
   }
   ```

   The first picks the solution, the second analyses every file, and the third lets the Testing view talk to Microsoft.Testing.Platform, the runner that `global.json` names (appendix E, section E.4).
3. **The Testing view runs tests; the terminal judges them.** Run a single test from the arrow beside it while you work. But the architecture tests read the built program, the integration tests need Docker, and only a full build in the terminal runs every analyser on every file. Before every commit, build and run the test projects in the terminal: their verdict counts.

F.3 The shape of every session

Every session runs through the same five steps: read, red, green, refine, commit. They are chapter 4's test-first loop, with a reading step before it and a commit after it. Read the story and the worked answer's

test for today, one pomodoro at most, until you can name the first test you will write. Write that test and watch it fail for the reason you expect. Spend most of the session on the least code that makes it pass, then tidy names and shape while every test stays green, and commit one change with every test green in the terminal.

Then loop through red, green and refine for each test the session names. Commit at every green, never at red, and name the story in each message, as the delivery plan asks:

```bash
# Commit at green: the terminal's verdict first, then one change with one message
dotnet build
dotnet run --project tests/Tessera.UnitTests
git add -A && git commit -m "Refuse a tile off the grid (US-07)"
```

Run the test project the session works in, and from session 3 the architecture tests too; from session 5 the integration tests need Docker running. `dotnet run --project` builds the project first, so one command builds and tests.

> WATCH OUT Read the worked answer's test before you write yours, but not its code. Once you have seen the code, your hands copy it, and the session teaches typing instead of design. Where a session says 'copy', copy; everywhere else, write.

F.4 Session 1 · The machine

Session 1 builds a machine on which every later command works, and writes no code. It ends when the worked answer's tests pass on your PC: a green run there proves the machine, so a red run later points at your code.

1 Mirrored networking and systemd. In WSL's default mode, Ubuntu sits behind a private address that Windows translates; mirrored mode lets it share the PC's own network connections, as the office machine needs (US-35). systemd, Linux's service manager, starts Docker whenever Ubuntu starts. `npm run office-port` checks the first later; set both now. In Windows, create or edit `%UserProfile%\.wslconfig`; in Ubuntu, edit `/etc/wsl.conf`:

```text
# %UserProfile%\.wslconfig, on Windows
[wsl2]
networkingMode=mirrored

# /etc/wsl.conf, in Ubuntu
[boot]
systemd=true
```

Then run `wsl --shutdown` in PowerShell, and open Ubuntu 24.04 again.

2 Docker Engine. Install Docker Engine, the program that runs containers, from Docker's own apt repository, as Docker's guide 'Install Docker Engine on Ubuntu' describes; use the engine, not Docker Desktop. Then let your user run it without `sudo`:

```bash
# Docker Engine for your user, started now and at every boot
sudo usermod -aG docker "$USER"
sudo systemctl enable --now docker
newgrp docker                        # or open a new terminal
docker run --rm hello-world
docker pull postgis/postgis:18-3.6  # the base of the one database image
```

3 The .NET 10 SDK. Microsoft's install script puts the SDK in your home folder, with no administrator's rights. `global.json` asks for 10.0.100 or a later band of .NET 10, and the channel `10.0` fetches the newest:

```bash
# The .NET 10 SDK under ~/.dotnet, on the PATH of every new terminal
cd /tmp && curl -fsSLO https://dot.net/v1/dotnet-install.sh
bash dotnet-install.sh --channel 10.0
echo 'export DOTNET_ROOT="$HOME/.dotnet"' >> ~/.bashrc
echo 'export PATH="$DOTNET_ROOT:$DOTNET_ROOT/tools:$PATH"' >> ~/.bashrc
source ~/.bashrc
dotnet --list-sdks                   # 10.0.… [~/.dotnet/sdk]
```

4 Node.js 26 and uv. The worked answer's scripts and map need Node.js (`package.json` asks for `"node": ">=26"`), and its pipeline needs uv; the sessions themselves need Node.js once, in this session, to build the map. Install nvm with the command its README gives, open a new terminal, then run `nvm install 26`; `node --version` must print a version that starts with `v26`. Install uv with the command Astral's guide gives; `uv --version` must answer.

5 The worked answer. Clone it into Ubuntu's own files, never under `/mnt/c`, where every build would run slowly across Windows' file sharing. Then build it with the map inside, and run its three test projects:

```bash
# The worked answer, read and run but never edited
mkdir -p ~/src && cd ~/src
git clone https://github.com/nathanwh71/Tessera.git tessera
cd tessera
npm run build                        # the map, then dotnet build --configuration Release
for suite in ArchitectureTests UnitTests IntegrationTests; do
  dotnet run --no-build --configuration Release --project tests/Tessera.$suite
done                                 # 6, 94 and 67 cases; Docker must be running
```

The first build downloads every package from NuGet, and the integration tests build the database image, so the first run may take several minutes; later runs take about one and a half. Testcontainers starts PostgreSQL for the integration tests, so a green run proves Docker as well as .NET.

6 VS Code. On Windows, install VS Code and its WSL extension. Then, in Ubuntu, run `code .` in `~/src/tessera`, and install the C# Dev Kit from the Extensions view; it installs inside Ubuntu, with the C# extension it needs. Open `tests/Tessera.UnitTests/TileTests.cs` and run it from the Testing view.

> THE PROOF A new Ubuntu terminal must pass all five checks: `docker run --rm hello-world` works without `sudo`; `dotnet --list-sdks` names a 10.0 SDK; `node --version` prints `v26`; `uv --version` answers; and the three test projects pass in `~/src/tessera`, `total: 6`, `total: 94` and `total: 67`, each with `failed: 0`, while VS Code runs `TileTests` green.

F.5 Session 2 · US-00, an empty solution that answers

Session 2 makes a solution that builds and whose service answers an HTTP request on its port: US-00's first two criteria. Read `global.json`, `Directory.Build.props`, `src/Tessera/Tessera.csproj`, `Program.cs` and `TesseraHost.cs` in the worked answer first.

BASH

```
# The rebuild begins as an empty repository, with the worked answer's versions
mkdir -p ~/src/tessera-rebuild && cd ~/src/tessera-rebuild
git init -b main
cp ~/src/tessera/{.gitignore,global.json,Directory.Packages.props} .
dotnet new sln --name Tessera                          # writes Tessera.slnx
dotnet new web --output src/Tessera --name Tessera     # an empty ASP.NET Core project
rm -r src/Tessera/Properties src/Tessera/appsettings*.json
code .
```

Write a short `Directory.Build.props` of your own today; session 3 swaps in the worked answer's. Give `Tessera.csproj` the two lines that matter to the walls later, `<AssemblyName>Tessera</AssemblyName>` and an empty `<RootNamespace></RootNamespace>`, and write the host:

CSHARP

```
// Your rebuild, session 2: src/Tessera/TesseraHost.cs, the host, with no route yet
public static class TesseraHost
{
    public static WebApplication Build(IReadOnlyDictionary<string, string?>? overrides = null)
    {
        var builder = WebApplication.CreateBuilder();
        if (overrides is not null)
            builder.Configuration.AddInMemoryCollection(overrides);
        builder.Services.AddHealthChecks();

        var app = builder.Build();
        return app;
    }
}
```

`Program.cs` holds one line, `await TesseraHost.Build().RunAsync();`; the seed's second command waits for session 9.

Red. Make `tests/Tessera.IntegrationTests` with a project file copied from the worked answer's, cut down to `xunit.v3` and the reference to the service, and add both projects to the solution with `dotnet sln add`. The worked answer's nearest test, `Starts_from_the_run_command_and_answers`, needs a database; yours asks less:

```csharp
// Your rebuild, session 2: not in the worked answer
public sealed class ServiceTests
{
    // US-00 test 1: the skeleton answers on its port.
    [Fact]
    public async Task Answers_on_its_port()
    {
        using var listener = new TcpListener(IPAddress.Loopback, 0);
        listener.Start();
        var port = ((IPEndPoint)listener.LocalEndpoint).Port;
        listener.Stop();

        await using var app = TesseraHost.Build(new Dictionary<string, string?>
        {
            ["Urls"] = $"http://127.0.0.1:{port}",
        });
        await app.StartAsync(TestContext.Current.CancellationToken);
        using var client = new HttpClient
        {
            BaseAddress = new Uri($"http://127.0.0.1:{port}"),
        };
        using var answer =
            await client.GetAsync("/health", TestContext.Current.CancellationToken);

        Assert.Equal(HttpStatusCode.OK, answer.StatusCode);
    }
}
```

`dotnet run --project tests/Tessera.IntegrationTests` fails with 'Expected: OK, Actual: NotFound', for the right reason: the service runs, but no route answers `/health`. **Green.** Add `app.MapHealthChecks("/health");` after `builder.Build()`. Then run the program itself, as US-00 c2 asks:

```bash
# Build, run, and ask the running service
dotnet build
Urls=http://127.0.0.1:7800 dotnet src/Tessera/bin/Debug/net10.0/Tessera.dll   # Ctrl+C stops it
curl -s http://127.0.0.1:7800/health          # in a second terminal: Healthy
```

> TRY IT Start the program with no `Urls` and read the log's 'Now listening on': with no setting, ASP.NET Core listens on its default, `http://localhost:5000`. The worked answer moves the port into `tessera.json`; you will do the same in session 3, and then the plain command listens on 7800.

F.6 Session 3 · US-00, the build rules, the door and the first wall

Session 3 swaps in the worked answer's build rules, gives the module its one door, and writes the test that keeps `Common` out of the module. Read the worked answer's `Directory.Build.props`, the three `.editorconfig` files, `Common/Infrastructure/Settings.cs`, `TesseraModule.cs` and the first lines of `BoundaryTests.cs`, and appendix E, section E.3.

The rules. Copy `Directory.Build.props`, `.editorconfig`, `src/Tessera/.editorconfig` and `tests/.editorconfig` from the worked answer, and build. It stops at once:

```text
Program.cs(1,7): error CA2007: Consider calling ConfigureAwait on the awaited task
```

Every warning is now an error, and every rule of .NET's analysers is on. Write `await TesseraHost.Build().RunAsync().ConfigureAwait(false);` and the build passes. Then watch the compiler refuse a null: add `string name = null;` and `Console.WriteLine(name.Length);` to `TesseraHost.Build`. The build stops twice, with CS8600, 'Converting null literal or possible null value to non-nullable type', and CS8602, 'Dereference of a possibly null reference'. Delete both lines.

The door and the settings file. Copy `Common/Infrastructure/Settings.cs` whole; today you use its `AddFile`, session 6 the rest. Write a `tessera.json` at the root that holds one key, `"Urls": "http://127.0.0.1:7800"`, and copy it beside the program with the `<None Include="../../tessera.json" … >` line of the worked answer's `Tessera.csproj`. Then move `/health` behind the module's door:

```csharp
// Your rebuild, session 3: Modules/Tessera/TesseraModule.cs, holding /health alone
namespace Modules.Tessera;

public static class TesseraModule
{
    public static IHostApplicationBuilder AddTessera(this IHostApplicationBuilder builder)
    {
        builder.Services.AddHealthChecks();
        return builder;
    }

    public static WebApplication MapTessera(this WebApplication app)
    {
        app.MapHealthChecks("/health");
        return app;
    }
}
```

`TesseraHost.Build` now calls `Settings.AddFile(builder.Configuration)` first, `builder.AddTessera()` before the build and `app.MapTessera()` after it, as the worked answer's does. Appendix D, section D.8, explains why the door names each part by hand.

Red. Make `tests/Tessera.ArchitectureTests` with the worked answer's project file, and write `BoundaryTests` with the fields `Ours` and `Program`, the helpers `In` and `OursIn`, and one rule copied whole, `Common_never_reaches_a_module`. It passes at once, so make it fail: add to `Settings` a line naming the door, such as `public static string Door => typeof(Modules.Tessera.TesseraModule).Name;`. The rule fails and names the breach: `Common.Infrastructure.Settings does depend on "Modules.Tessera.TesseraModule"`. **Green.** Delete the line.

> WATCH OUT From now on every test carries a comment naming the criterion it proves, such as `// US-07 c4`, as the worked answer's tests do, so `grep -rn 'US-07' tests` finds them all. Add each ArchUnitNET rule in the session that gives it a class to check: ArchUnitNET fails a rule whose types match nothing, with 'The rule requires positive evaluation'. `Domain_stands_alone` waits for session 7, when the Domain holds `TileAddress`.

F.7 Session 4 · US-01, one database image and two roles

Session 4 writes the database image the service signs in to, and proves by hand that its reader can read and cannot write. Read the worked answer's `db/Dockerfile` and `db/initdb/20-roles.sql`, all of both, and US-01 in `docs/user-stories/user-stories.md`.

Copy `db/Dockerfile`, and write `db/initdb/20-roles.sql` yourself with the worked answer open beside you. It makes two roles: `tessera_reader`, the service's, whose every transaction is read-only; and `tessera_publisher`, the data publisher's, which may create schemas. Each password comes from the environment through psql's `\getenv`, and a missing one stops the first start. Then build the image, run it, and ask it:

BASH

```
# A database that runs the roles script once, when it is first made
cd ~/src/tessera-rebuild
docker build --tag tessera-db db
docker run --detach --name tessera-db --publish 127.0.0.1:5432:5432 \
    --env POSTGRES_DB=tessera --env POSTGRES_PASSWORD=owner \
    --env TESSERA_READER_PASSWORD=reader --env TESSERA_PUBLISHER_PASSWORD=publisher \
    tessera-db
docker logs -f tessera-db      # wait for the second 'ready to accept connections'; Ctrl+C
docker exec tessera-db psql -U tessera_reader -d tessera \
    -c 'SHOW default_transaction_read_only;'
docker exec tessera-db psql -U tessera_reader -d tessera \
    -c 'CREATE TABLE public.t (id int);'
```

The first query must answer `on`; the second must be refused, since every transaction of the reader's is read-only. The image starts PostgreSQL twice, once to run its set-up scripts and once to serve, so wait for the second message.

> WATCH OUT The image runs the scripts in `/docker-entrypoint-initdb.d/` only when its data folder is empty. After you change `20-roles.sql`, remove the container with `docker rm -f tessera-db` and run it again; a restart runs nothing new.

F.8 Session 5 · US-01, the same database for every test

Session 5 makes every database test start the image you wrote in session 4, so no test depends on whatever is running on your PC. Read the worked answer's `DatabaseFixture.cs`, all of it, and `DatabaseTests.cs`, and appendix E, section E.5.

Add `Testcontainers.PostgreSql` and `Npgsql` to the integration tests' project file; `Directory.Packages.props` already names their versions. Copy `DatabaseFixture.cs` whole: it builds the image from `db/Dockerfile`, starts it with both passwords, and hands each test class its own copy. It finds `db/` through the repository's `.git` folder, which `git init` made in session 2.

Red. US-01's first test asks the tests to sign in as the role the shipped script makes. The worked answer's `Signs_in_as_the_shipped_reader_role` proves it through the list of layers, which you have not built; write your own:

```csharp
// Your rebuild, session 5: not in the worked answer
public sealed class DatabaseTests(DatabaseFixture database) : IClassFixture<DatabaseFixture>
{
    // US-01 test 1: sign in as the role the shipped script makes, and find it read-only.
    [Fact]
    public async Task Signs_in_as_the_reader_and_cannot_write()
    {
        var cancel = TestContext.Current.CancellationToken;
        await using var connection = new NpgsqlConnection(database.Reader);
        await connection.OpenAsync(cancel);

        await using var show =
            new NpgsqlCommand("SHOW default_transaction_read_only", connection);
        Assert.Equal("on", await show.ExecuteScalarAsync(cancel));

        await using var write =
            new NpgsqlCommand("CREATE TABLE public.t (id int)", connection);
        await Assert.ThrowsAsync<PostgresException>(() => write.ExecuteNonQueryAsync(cancel));
    }
}
```

Leave the `ALTER ROLE … SET default_transaction_read_only = on` line out of `20-roles.sql`, and watch the test fail. **Green.** Put the line back. Then copy the worked answer's `Keeps_the_reader_read_only` whole: it reads the role's settings as the owner, a second view of the same lock.

> THE PROOF Prove the test needs nothing running. Stop your own database with `docker rm -f tessera-db`, then run `dotnet run --project tests/Tessera.IntegrationTests`. The tests must pass: Testcontainers builds the image, starts its own PostgreSQL on a free port, and removes it afterwards. In the author's scratch rebuild, the class took about 12 seconds, most of it the container's start.

F.9 Session 6 · US-00, the settings and the start refused

Session 6 binds `tessera.json`'s `Tiles` section into a checked record, reads `DATABASE_URL`, and proves the service refuses to start on a bad value or with no database address: US-00's fourth criterion. Read `Domain/Settings.cs`, `Infrastructure/Sql/Connection.cs`, `SettingsTests.cs`, `StartTests.cs` and ADR 0003 in the worked answer, and appendix D, section D.9.

Copy `TileSettings` into `Modules/Tessera/Domain/Settings.cs`, with its attributes and its `Validate`, and `Connection.cs` whole into `Modules/Tessera/Infrastructure/Sql/`; it needs `CSharpFunctionalExtensions` and `Npgsql` in the service's project file. In the door, bind the section and the URL, as the worked answer's `AddSettings` and `AddTessera` do: `Settings.Section<TileSettings>(services, "Tiles")` and `DatabaseConnections.Register(…)`. In `TesseraHost.Build`, call `Settings.Check(app.Services)` straight after `builder.Build()`, and copy `Serve` and `Fail` into `Program.cs`, so a failed check prints each problem and returns 1.

Red. Make `tests/Tessera.UnitTests` with the worked answer's project file. Copy `SettingsTests`' helpers, `Host`, `Settings` and `Problems`, with `TesseraModule.AddSettings` cut to the one section you have, and four of its tests: `Names_a_missing_or_bad_database_url`, `Refuses_a_key_it_does_not_know`,

`Leaves_a_misspelt_section_at_its_defaults` and `Lets_a_variable_override_a_key`. Into the integration tests, copy `StartTests` with its two helpers and its first test alone, `Refuses_to_serve_and_names_each_missing_variable`. They fail to compile until the code exists, which counts as red.

Green. When they pass, run the integration tests again. Your session 2 test now fails: `TesseraHost.Build` refuses to build without `DATABASE_URL`, as US-00 c4 asks. Give its dictionary one more entry, `["DATABASE_URL"] = "postgresql://tessera_reader:x@127.0.0.1:9/tessera"`; any well-formed URL will do, since the service starts while the database is down.

> COMMON MISTAKE A common mistake is to pass settings to the program as arguments. `dotnet Tessera.dll --Urls=http://127.0.0.1:0` works in many ASP.NET Core programs, but Tessera's `Program.cs` accepts only `seed`, and prints its usage, with exit code 2, for anything else. Set a value through the environment instead, such as `Tiles__CacheSeconds=0 dotnet Tessera.dll`; the tests pass a dictionary to `TesseraHost.Build`, which skips `Program.cs`.

F.10 Session 7 · US-07, a tile that cannot be off the grid

Session 7 builds the first values that cannot be wrong: a tile address that exists only inside its grid, and a name that reaches SQL only quoted. Read `Domain/Tiles.cs`, `Domain/SqlName.cs`, the first half of `TileTests.cs`, `SqlTests.cs`, and the rule `Domain_stands_alone` in `BoundaryTests`.

Write `TileAddress` in `Modules.Tessera.Domain`, test-first. **Red.** Copy `Accepts_a_tile_that_exists` with its three rows and `Refuses_a_tile_outside_the_grid` with its four. Begin with an `Of` that always returns `null`, so the rows that expect a tile fail. **Green.** Make the constructor private and write `Of`, which checks the zoom against `minZoom`, `maxZoom` and 30 before it computes `1 << z`, and keeps x and y inside the grid. Then add FsCheck, through `FsCheck.Xunit.v3`, and copy the property `Gives_only_addresses_on_the_grid`: a hundred made-up triples, each either refused or on the grid.

> FROM SCHOOL `1 << z` is the binary shift you met at school: shifting 1 left by z places gives 2 to the power z, the number of tiles along one side of the grid at zoom z. In a 32-bit `int`, `1 << 31` is already negative, one reason `Of` checks the zoom first.

Then `SqlName`, with a private constructor and the two ways to make one, `Quote` and `Qualified`, driven by `SqlTests`' two tests, the second a property test over any string. Last, the wall that guards them: copy the fields `Database`, `Web` and `Workings` and the rule `Domain_stands_alone`. See it fail once: let `SqlName.Quote` build its text with an `Npgsql.NpgsqlConnectionStringBuilder`, and the rule names the breach, `Modules.Tessera.Domain.SqlName does depend on "Npgsql.NpgsqlConnectionStringBuilder"`; then take it out.

F.11 Session 8 · US-07, the SQL and the one door to the database

Session 8 builds the pure middle of the tile route, a layer and its address in and one query out, and the one class allowed to send it. The longest session, it splits well into two: the query first, then the database. Read `Domain/Layers.cs`, `Infrastructure/Sql/Queries.cs`, `Infrastructure/Sql/Database.cs`, the second half of `TileTests.cs`, `DatabaseErrorTests.cs`, and appendix D, section D.4.

The query. Write `Layer` and `LayerCatalogue` in the Domain, and `Query` and `Queries` in `Infrastructure/Sql`: `Queries.Tile` fills the tile's SQL text with each `SqlName` and puts every value in an

anonymous object of parameters. Leave out `Layers` and `JournalSchema`; US-03 and session 9 bring them. **Red.** Copy the approved file, `approved/tile-query.sql`, with its line in the project file, and the tests `Matches_the_approved_tile_query` and `Sends_the_layer_and_the_tile_as_parameters`. The worked answer builds its test layer with `CatalogueReader.ToLayer`, which belongs to US-03; build yours with `new Layer(…)`, with the same four properties, `id`, `layer`, `group` and `name`, and the same key, `fid`. The test fails until your text matches the approved one to the character.

The database. Copy `DatabaseError` with its `[Closed]` label and add `ExhaustiveMatching.Analyzer` to the service; `DatabaseErrorTests`' three tests drive `Of`. Then write `Database`, with `Open`, `Single`, the private `Run`, the one place where Npgsql's exceptions become a `Result`, the two helpers it calls, `Within`, which keeps the query to its own clock, and `RunOnce`, and `Error`; `Rows` and `Ping` wait for their stories. Add Dapper. Then the two rules that guard the door: `Only_Infrastructure_talks_to_the_database` and `Infrastructure_knows_no_web`. Red: give `Queries` a method that returns `Results.Ok()`. Green: delete it.

> WATCH OUT A test that builds its expected SQL with the same code as `Queries.Tile` passes for the wrong reason the day both drift together. The worked answer's test compares the text with a file someone read and approved, so you see every change to the SQL.

F.12 Session 9 · US-07, the route, the seed and the walking skeleton

Session 9 joins the pure core to the world, with a route thin enough to read in one sitting, and ends with a tile travelling from PostGIS to your terminal. Read `Common/Api/Problems.cs`, `Features/Tiles.cs`, `TesseraModule.cs`, `TesseraHost.cs`, `Seed` in `Program.cs`, `SeedScripts.cs`, and the tests of `ServiceTests` that serve a tile.

The answers. Copy `Problems` into `Common/Api`, and give `TesseraHost.Build` the worked answer's three lines: `AddProblemDetails` with `Problems.Shape`, `UseExceptionHandler` and `UseStatusCodePages`. Then write `Features/Tiles.cs`: `TileServer` as the worked answer has it, since it already takes the list of layers as a function and a logger for the cause of a 500, and `TileRoutes.MapTiles` with one change: it takes a `Func<LayerCatalogue?>`, not a `LiveCatalogue`.

The door. Register `Database` as a singleton, as the worked answer's door does, add `DatabaseSettings` with its section, and give the route one layer typed by hand:

CSHARP

```csharp
// Your rebuild, session 9: one layer, until US-03 reads the list from the database
private static readonly Layer Cafes = new(
    "food.cafes", SqlName.Qualified("food", "cafes"), SqlName.Quote("geom"), 4326, "POINT",
    [SqlName.Quote("id"), SqlName.Quote("name")], [SqlName.Quote("fid")]);

private static readonly LayerCatalogue OneLayer =
    new(ImmutableDictionary.Create<string, Layer>(StringComparer.Ordinal)
        .Add(Cafes.Id, Cafes));

public static WebApplication MapTessera(this WebApplication app)
{
    app.MapHealthChecks("/health");
    app.MapTiles(
        () => OneLayer,
        app.Services.GetRequiredService<Database>(),
        app.Services.GetRequiredService<IOptions<TileSettings>>().Value);
    return app;
}
```

Build the dictionary without its comparer, and Meziantou stops the build with MA0002: a collection keyed by strings must say how it compares them. The ordinal comparer matches ids letter for letter, case included, as US-03 asks.

The seed. Add `dbup-postgresql`, copy `Seeding/SeedScripts.cs` and `Queries.JournalSchema`, and copy `Seed`, `Report` and `Usage` into `Program.cs` with the `switch` that chooses among them. Copy `db/seed/0001-macau-places.sql`, and the line of the worked answer's project file that ships it beside the program as `seed/`.

Red, then green. Copy four tests from `ServiceTests`, with `ServiceFixture` cut down to seed, grant and start: `Serves_a_tile_as_a_mapbox_vector_tile`, `Serves_the_same_body_for_mvt`, `Answers_an_unknown_layer_with_a_problem_document` and `Answers_a_tile_outside_the_grid_with_404`. The fixture's wait for the list of layers can go, since your one layer never loads late. Then run it all by hand:

BASH

```
# First terminal: your database from session 4, seeded, granted, then the service
cd ~/src/tessera-rebuild
docker run --detach --name tessera-db --publish 127.0.0.1:5432:5432 \
    --env POSTGRES_DB=tessera --env POSTGRES_PASSWORD=owner \
    --env TESSERA_READER_PASSWORD=reader --env TESSERA_PUBLISHER_PASSWORD=publisher \
    tessera-db      # again, since session 5 removed it; wait for the second 'ready'
dotnet build
TESSERA_PUBLISHER_DATABASE_URL=\
postgresql://tessera_publisher:publisher@127.0.0.1:5432/tessera \
    dotnet src/Tessera/bin/Debug/net10.0/Tessera.dll seed
docker exec tessera-db psql -U tessera_publisher -d tessera \
    -c 'GRANT USAGE ON SCHEMA food TO tessera_reader;' \
    -c 'GRANT SELECT ON food.cafes TO tessera_reader;'
DATABASE_URL=postgresql://tessera_reader:reader@127.0.0.1:5432/tessera \
    dotnet src/Tessera/bin/Debug/net10.0/Tessera.dll
```

BASH

```
# Second terminal, once the service listens: a tile, an unknown layer, a tile off the grid
curl -s -o tile.pbf -w '%{http_code} %{size_download}\n' \
    http://127.0.0.1:7800/food.cafes/12/3339/1788.pbf
curl -s -w '\n%{http_code}\n' http://127.0.0.1:7800/food.nowhere/12/3339/1788.pbf
curl -s -w '\n%{http_code}\n' http://127.0.0.1:7800/food.cafes/1/2/0.pbf
```

The seed must print 'Ran 0001-macau-places.sql', then 'A new table waits for its grant: npm run publish.' The three requests must print 200, 404 and 404, the last two each after a problem document whose detail reads 'No published layer has that name.' and 'No tile has those coordinates.' In the author's scratch rebuild the tile held 3,977 bytes; the worked answer's holds 17,575, since its layer carries every column of simple type and yours only `id` and `name`. That is your walking skeleton: a request in, one query to PostGIS, a vector tile out.

> WATCH OUT `SeedScripts.Folder` is the folder `seed` beside the program, not beside your terminal. If `Tessera seed` prints 'The seed is up to date.' on a new database, the build has not copied the script: check the `<None Include="../../db/seed/*.sql" …>` line in the project file, and build again. Without the grant, every tile answers 404, since PostgreSQL's refusal, 42501, means a layer has left.

F.13 Session 10 · Review, and compare with the worked answer

Session 10 proves the rebuild from nothing, measures it against the worked answer, and writes down what you learnt; it adds no feature.

BASH

```
# Everything from nothing, after your last commit: no build output, no database, every test
cd ~/src/tessera-rebuild
docker rm -f tessera-db
git clean -xdf --exclude=.vscode      # deletes every file Git does not track
dotnet build
for suite in ArchitectureTests UnitTests IntegrationTests; do
  dotnet run --no-build --project tests/Tessera.$suite
done

# Which files differ between the worked answer and the rebuild
git diff --no-index --stat ~/src/tessera/src ~/src/tessera-rebuild/src
```

To read a pair of files side by side, let VS Code show them: `code --diff ~/src/tessera/src/Tessera/Modules/Tessera/Domain/Tiles.cs src/Tessera/Modules/Tessera/Domain/Tiles.cs`. Read four pairs closely: `Tiles.cs` in the Domain, `Queries.cs`, `Features/Tiles.cs` and `BoundaryTests.cs`. For each story, write a note of three lines outside the repository: what matched, what differed, and why. Then write your first decision record, in the shape of the worked answer's ADRs (Decision, Options considered, Consequences, When to revisit), for the choice these sessions made: one hard-coded layer until the database's list arrives. Its 'When to revisit' should name the story that undoes it, US-03.

Count the points. US-00 and US-01 make 1 point each, and US-07 makes 2: 4 points of the 29 that Parts A and B plan. If the ten sessions took more than two weeks, compare the count with milestone M1, 'The places are published by grant', and read the delivery plan's 'When time runs short' before you go on.

F.14 Where your rebuild may differ

Your rebuild will differ from the worked answer in shape and names, and that is fine wherever the tests and the walls agree. A test pins behaviour, not wording: two programs that pass the same tests and architecture rules do the same job. Exhibit F.2 lists where the rebuild of ten sessions differs on purpose, and which story closes each gap.

EXHIBIT F.2 After ten sessions the rebuild differs in eight places, and a later story closes most of them

WHERE	THE WORKED ANSWER	YOUR REBUILD AFTER TEN SESSIONS	CLOSED BY
Which tables are layers	`LiveCatalogue`, a hosted service, rereads the database's list every 30 seconds	One hard-coded layer, `food.cafes`, with two properties	US-03, N-23
The places	Nine layers, granted by `npm run publish`	The seed borrowed; one grant typed by hand	US-24
Settings	Five sections: `Database`, `Tiles`, `Map`, `Catalogue`, `Area`	`Tiles` and `Database`	US-23, N-22, N-23
A tile's headers	`Cache-Control` from `Tiles.CacheSeconds`, and a weak ETag; 304 for a kept tile	The same, since you copied `TileServer`; untested	N-18, N-19

`/health`	Two checks; `UP`, `OUT_OF_SERVICE` or `DOWN`, with `no-store`	ASP.NET Core's plain 'Healthy', with no check	N-25
The map page	`MapRoutes` at `/` and one page for each layer, with its endpoint filter	None	US-23
Architecture rules	Six	Four: the two about `PublicApi` wait for a second module	Copy them when you like; they pass with nothing to judge
Two tests of yours	`Starts_from_the_run_command_and_answers` and `Signs_in_as_the_shipped_reader_role`, each through parts you have not built	`Answers_on_its_port`; `Signs_in_as_the_reader_and_cannot_write`	Keep yours beside them

Source: this appendix; `src/Tessera`*;* `tests/`*;* `docs/user-stories/user-stories.md`

Two things must never differ. The first is the walls: your `BoundaryTests` holds the worked answer's rules, each seen red once, so your layers lean on one another only as the worked answer's do. The second is the SQL: no value ever enters a query's text, and a name enters it only as a `SqlName`. Match the worked answer there, line for line if you must, because these two guard the security of everything you build next.

> TRY IT US-01's second test asks that `data/macau-pois.geojson` match its recorded SHA-256 fingerprint, and that the seed script's first line name the file. The worked answer's `Ships_the_seed_made_from_the_places` does both. Write your own before you read it: compute the hash with `SHA256.HashData`, as `TileServer.Tag` does, and compare it with a value you record today with `sha256sum`.

Check your understanding

1. VS Code shows no error in the Problems view, yet `dotnet build` stops with an analyser's error in a file you have not opened. Why, and which setting changes it?
2. Why does session 5 remove `tessera-db` before it runs the integration tests?
3. After session 6, why does your session 2 test fail, and what is the smallest fix that keeps it honest?
4. After a seed that printed 'Ran 0001-macau-places.sql', session 9's tile of `food.cafes` answers 404. Name the likely cause.

Answers. 1. By default the C# Dev Kit runs the analysers on open files alone, while the build runs them on every file; `"dotnet.backgroundAnalysis.analyzerDiagnosticsScope": "fullSolution"` makes the editor analyse them all. 2. To prove the tests need nothing running: Testcontainers builds and starts its own database. 3. `TesseraHost.Build` now refuses to build without `DATABASE_URL`, as US-00 c4 asks; give the test a well-formed URL, since the service starts while the database is down. 4. No one ran the grant, so the reader may not read the table, and PostgreSQL's refusal, 42501, becomes 404.

G
Glossary

Every concept the book marks where it first explains it appears here, in alphabetical order, in one or two plain sentences. A word in brackets after a term says which of two meanings it takes, such as a Docker container or a C4 one.

Alert
A warning that a scanner such as ZAP raises for each weakness it recognises in a service's answers.

Analyser
A small program that runs inside the C# compiler and adds checks of its own, such as ExhaustiveMatching, which stops the build when a `switch` misses a case.

Architecture test
A test that reads the built program and checks its shape, such as which layer may use which. `BoundaryTests` holds Tessera's six.

ArchUnitNET
A .NET library that tests rules about which types may use which, and fails the tests when code breaks one.

ASP.NET Core
Microsoft's library for web services, which answers each address with a short function of your own; Kestrel, its web server, receives each request.

Assertion
One check inside a test, such as `Assert.Null(TileAddress.Of(z, x, y))`, that stops the test and turns it red when the answer differs.

Async method
A C# method that can pause at each `await` while it waits, such as for the database, and give its thread to other work meanwhile.

Attribute
A label in square brackets fixed to a piece of C# code, such as `[Range(0, 30)]`. It does nothing itself; the tool that reads it, such as the settings' validator, an analyser or xUnit, decides what it means.

Back-off
A wait before trying again that grows after each failure. `npm start` waits a second before restarting a stopped copy, and doubles the wait after each quick failure, up to 30 seconds.

Backup
A copy of the database's data, kept so a lost or damaged database can be rebuilt. Tessera takes one each night and reads it back before keeping it.

Behaviour
What a caller of some code can see: the answer, the status code, the text of a query. A good test checks behaviour, never the inner workings.

Black-box testing
Judging a system by what it does from outside, through its answers, without reading its code.

Block (Nginx)
A group of Nginx directives in braces, such as `server` for one site or `location` for the paths it matches.

Bound parameter
A value sent to the database beside a SQL statement, matched to a named slot such as `@z`, so the database never reads it as SQL.

Boundary data
Test data at the edge of what a rule accepts: the last value inside the range, or the first outside it.

Branded type
A TypeScript type that adds a mark, seen only by the compiler, to a plain number or string, so two kinds of number, such as a web-map level and MapLibre's zoom, cannot be mixed.

Burn-up chart
A chart of two lines that climb towards the total work: the points you planned to finish by each week, and the points you did finish.

Burst
The number of requests above the steady rate that Nginx lets through at once, such as the hundred tiles of a fresh view. Tessera allows a burst of 300.

C4 model
Simon Brown's way of drawing software in diagrams that zoom in step by step, from the system and its users to the parts inside it.

Cache lock

A rule that lets the first of many requests for the same new tile build it, while the rest wait for its answer, so a crowd costs one query.

Cache-Control

The HTTP header that says whether, and for how long, a browser or a proxy may keep an answer. Tessera's tiles carry `public, max-age=60`; its errors carry `no-store`.

Catalogue

The list of layers the service reads from the database: every table or view it may read with a typed geometry column. It reloads every 30 seconds.

Central package management

Naming each .NET package's version once, in `Directory.Packages.props`, so every project uses the same one.

Cluster

One round marker that stands for several places close together on the map, showing their number. It breaks back into its pins as you zoom in.

Collision box

The box on screen that a map label or icon claims. MapLibre moves or hides a later label whose box would cover one already placed.

Colour roles

Material 3's names for colours by their job, such as `surface` or `on-surface`, all grown from one seed colour.

Commit

A saved snapshot of the project in Git, with a message saying what changed.

Connection pool

A few database connections kept open and lent to requests in turn, so no request pays to open one. Npgsql keeps four for each copy of Tessera.

Container (C4)

In a C4 diagram, a part of a system that runs on its own or holds data, such as the service, Nginx or the database.

Container (Docker)

One running copy of a Docker image: a packaged program, sealed off from the rest of the machine, that starts in seconds.

Context

The first C4 diagram: the system as one box, the people who use it, and what it relies on outside.

Contrast ratio

A measure of how far apart two colours are in lightness, from 1:1 for the same colour to 21:1 for black on white. Text needs at least 4.5:1 to read with ease.

Control

A defence that answers one threat, such as a rate limit against a flood of requests.

Coverage

The share of a program's lines and branches that the tests run. Tessera measures it on every `npm test` and fails below 94 per cent of lines or 84 of branches.

Decision record

A short note, often called an ADR, of one choice: the forces behind it, the options weighed, what it makes easier and harder, and when to revisit it.

Definition of done

The checklist every story must meet besides its own criteria, such as every test passing and the walls holding, before it counts as finished.

Delegate

A C# type for a function, so a variable or a parameter can hold one, such as `TileReader`, which `TileServer` takes in place of the database.

Dependabot

GitHub's robot for updates. Each Monday it proposes newer releases of the versions the repository pins, and the build tests each one before anyone merges it.

Dependency injection

Handing each part of a program the other parts it needs, rather than letting it build them itself.

Directive

One instruction in an Nginx file, ending in a semicolon, such as `gzip on;`.

Discard

The name `_` in C#, which says the code needs nothing from a value, such as a matched case of `DatabaseError`.

Docker

A program that runs other programs in sealed boxes, called containers, each made from an image. Tessera runs its database and Nginx in Docker.

Early sign

The first thing you would notice if a risk began to happen, such as a story still open a day after its points.

Easing

The curve of speed a movement follows on screen. Material's emphasised easing starts fast and settles slowly.

Endpoint

The one method that ASP.NET Core's routing chooses to answer a request, such as `TileServer.Answer` for a tile.

Endpoint filter

A step that runs around certain endpoints alone, such as `MapRoutes.PageHeaders`, which adds the map page's security headers.

Endpoint metadata

The list of facts ASP.NET Core keeps about an endpoint, including every attribute on its method, for later steps to read.

Environment variable

A named value the operating system hands a program as it starts, such as `DATABASE_URL`, so the code itself names no machine and holds no password.

Erroneous data

Test data of the wrong kind altogether, such as letters where a number belongs.

ETag

A short label for one version of an answer. A browser sends it back in `If-None-Match`, and a matching tag earns 304, 'Not Modified', with no body.

Exception

C#'s usual way to report a failure: an object thrown out of a method that jumps up the chain of callers until some code catches it.

Filter

One step of the Python pipeline that cleans a table of places: it fixes what it safely can, and drops or stops at what it cannot.

Firewall

A guard on a machine that lets in network traffic only on the ports its rules allow. On the office machine, Windows' firewall and Hyper-V's each admit port 80.

Functional core, imperative shell

A design in which pure functions make every decision, and a thin outer layer of code acts on the world as they decide.

Functional requirement

A requirement that says what a system does, such as 'serve a tile of a published layer'.

Generic type

A type that takes other types as parameters, so one definition, such as `Result<T, E>`, serves values of any type while the compiler still knows which.

Grant

A database command that gives a role the right to read a table. In Tessera a grant publishes a table as a layer, and a revoke withdraws it.

Guard clause

A check at the start of a method that returns at once when it fails, so the code after it reads straight down.

gzip

A lossless way to pack a file into fewer bytes. Nginx packs tiles and the map's files with gzip for every browser that accepts it.

Health check

An address that a watcher, a script or a person asks to learn whether a service can do its job. Tessera answers at `/health`.

Hosted service

An object that starts and stops with the program and runs beside the requests, such as `LiveCatalogue`, which rereads the list of layers.

Hostile input

Any request built to make a service misbehave, such as SQL hidden in a layer's name or a disguised address.

Image (Docker)

A read-only template from which Docker starts containers: a packed file system with one program ready to run.

Immutable collection

A list, set or dictionary that offers no way to add, remove or replace an item, such as `ImmutableArray`; a change returns a new copy.

Integrated terminal

The command line built into VS Code; in this build, an Ubuntu shell inside WSL2.

Integration test

A test that runs the code against a real PostgreSQL, started in Docker; the fullest ones start the whole service and talk to it over HTTP.

Iteration

A block of about two weeks of work that ends with a review on its last working day. Four iterations carry Parts A and B to the release.

Journal

The table `tessera.schemaversions`, where DbUp records each seed script it has run, so it never runs one twice.

k6

A program that plays many users at once against a service and records how long each answer takes.

Kestrel

The web server inside every ASP.NET Core program, which reads requests from the network and sends the answers back.

Lambda

A function written as a value, without a name, such as `why => $"{variable}: {why}"`.

Language server

The program behind an editor that reads the project, such as `Tessera.slnx`, and marks mistakes as you type.

Layer

A published table or view of shapes, named by its schema and table, such as `food.cafes`; a map asks for its tiles one at a time.

Layer (of code)

One of the three parts the Tessera module is cut into: `Domain` for plain values and rules, `Infrastructure` for the database, `Features` for the web routes.

Lifetime

How long an object made by ASP.NET Core's container lives: once for the program, once for each request, or anew each time.

LINQ

Language-Integrated Query: methods such as `Select`, `Where` and `GroupBy` that chain steps over any sequence in C#.

Loopback address

The address `127.0.0.1`, which means 'this machine alone'; a program that listens there stays hidden from the network.

Making illegal states unrepresentable

Shaping types so that a value breaking a rule, such as a tile off the grid, cannot be held in a variable at all.

Malformed URL

An address written so that it looks clean only after a web server tidies it, such as one that climbs above the root with `%2e%2e`. Nginx refuses one with 400, and so does the service, before it picks a route.

Mapbox Vector Tile

The compact binary format of Tessera's tiles, whose files end `.pbf` or `.mvt`.

Max-age

The part of `Cache-Control` that says how many seconds an answer stays good; Tessera's tiles carry 60.

Middleware

A step of ASP.NET Core that sees every request on its way in and every answer on its way out, such as the exception handler.

Milestone

A state of the product that someone could see and test, named in a sentence, such as M1, 'The places are published by grant'.

Mirrored networking

A mode of WSL2 in which Ubuntu shares the Windows machine's own network address, so Nginx sees each office machine's own address.

Model (Elmish)

The single value that holds everything the map's page knows. Only the pure update function may replace it.

Modular monolith

One program, run as one process and shipped as one file, divided inside into modules with walls between them.

Module

A walled part of a program with its own inside, which other modules may not reach into. Tessera has one, `Modules.Tessera`, beside the shared `Common`.

Namespace

A dotted name that groups C# types, such as `Modules.Tessera.Domain`. Tessera makes each match its folder.

.NET SDK

The set of tools behind the command `dotnet`: the C# compiler, the build engine, the package manager and the runtime. `global.json` asks for .NET 10.

Network prefix

The number after the slash in a network such as `192.168.10.0/24`: how many leading bits every address in the range shares.

Nginx

A web server that receives every request first and passes it on. In Tessera it is the front door on port 80, with the office rule, the limits, the tile store and two copies of the service behind it.

Non-functional requirement

A requirement that says how well a system works, such as how fast, how steady or how easy to use.

Normal data

Test data that a rule should accept, well inside its range.

Nullable type

A C# type with a question mark, such as `Layer?`, whose value may be missing; the compiler makes every caller check it before use.

Options pattern

.NET's way of binding a section of settings, such as `Tiles` in `tessera.json`, to a record and checking it before the program serves.

p95

The 95th percentile of answer times: the time that 95 answers in 100 beat.

Parameter

A slot in a SQL statement, such as `@z`, that the database fills with a value sent beside the statement, never pasted into its text.

Passive health check

Watching a server only through the requests passed to it. Nginx rests a copy of the service for ten seconds after three failed requests within ten seconds.

Pattern

In a C# `switch`, a case that tests a value's shape, such as its type, a property or a list, and, if it matches, may name the value.

Percentile

The value below which a given share of measurements fall; the 50th is the median.

Persona

A short portrait of one kind of user, named by the work they do, so a story can say whose need it meets. Tessera's stories speak for five.

Pipeline

Tessera's Python program in `etl/` that turns files of places into SQL seed scripts, cleaning them on the way.

Plain HTTP

HTTP with no encryption, so any machine on the network path could read or change a request or an answer.

Port (Elmish)

A function passed into the map's update from outside, such as one that moves the map, which a test can swap for a fake.

PostGIS

The extension to PostgreSQL that stores shapes on the globe and answers questions about them, such as which places lie in a square.

PostgreSQL

The database that holds Tessera's places, its roles and DbUp's record of the seed scripts it has run.

Private constructor

A constructor that only code inside its own type may call, so every value must pass through the type's checked door.

Problem document

A short error answer in JSON, in the shape RFC 9457 sets: a type, a title, the status and, where it helps, a sentence of detail.

Property test

A test that states a rule for every input and lets a library, FsCheck, try a hundred made-up inputs against it.

Pull request

A proposed change to a repository on GitHub, which the build tests and someone reviews before it is merged.

Pure function

A function that gives the same output for the same inputs and changes nothing outside itself: no database, no clock, no network, no file.

Pyramid principle

Barbara Minto's order for writing: the answer first, then the supporting points, then the detail.

Railway-oriented programming

Scott Wlaschin's name for joining steps that can fail, so the first failure switches a request to a second track that runs past every later step.

Raster tile

A tile that holds a picture, a grid of coloured pixels, rather than shapes.

Rate limit

A cap on how many requests one machine may make in a second. Nginx allows each office machine 100 tiles a second.

Record

A C# class whose main job is to hold named values. C# writes its constructor, its properties and its comparison by value.

Reflection

A program's power to look at its own types while it runs, to list their properties, read their attributes and call them. .NET's validator and xUnit read attributes this way.

Repository

The folder of Tessera's finished code, tests and documents, kept in Git: your worked answer.

Requirement

A sentence that the finished service must make true, with 'must' in it. Tessera keeps 151, each traced to a story's criterion and to the test that proves it.

Restore

Rebuilding a database from a backup. Only a restore proves that a backup works.

Reverse proxy

A server that stands in front of other servers and passes requests to them: a server to the browser, and a client to the servers behind it.

Risk

Something that has not happened yet but would hurt if it did.

Role

A database user with its own rights. The service signs in as `tessera_reader`, which may only read; the publisher as `tessera_publisher`.

Round robin

Sharing requests among servers in turn, one after another. Nginx shares them between the two copies of the service this way.

Route constraint

A mark in a route, such as `{z:int}`, that lets a part of the path match only when it reads as a whole number.

Scale bar

A bar on a map that shows how far a stretch of the map spans on the ground.

Schema

A named folder of tables in a database, such as `food`, which holds the cafés, restaurants and markets.

Scoped service

An object ASP.NET Core's container makes once for each scope, such as one request, and drops when the scope ends.

Seed script

A numbered SQL file in `db/seed` that loads places into the database. DbUp runs each one once, in the order of its name.

Service collection

The list of instructions, built as the program starts, that says how to make each part and how long to keep it.

Service container

ASP.NET Core's table of 'when code asks for this type, give it this object', which makes the parts and hands them to the parts that need them.

Service provider

The object that `builder.Build()` makes from the service collection; it makes each part on request.

Session

One sitting at the keyboard, of about three hours.

Singleton

An object the container makes once and hands to every part that asks, such as Tessera's one `Database`.

Smart constructor

A function that checks its inputs and returns either a valid value or no value, such as `TileAddress.Of`.

Sort key

A number that decides the order in which MapLibre places labels, so the same name wins a clash every time.

Spatial index

A guide to where each shape in a table lies, like the index at the back of a book, so the database finds the shapes in a square without reading the whole table.

SQL injection

An attack in which text from a request ends up as part of a SQL statement, in the hope that the database runs it.

SQLSTATE

PostgreSQL's five-character code for each kind of refusal, such as `42P01` for a missing table.

SRID

The code number of a coordinate system, such as 4326 for longitude and latitude (WGS 84) or 3857 for Web Mercator.

Stale tile

A kept tile older than its `max-age`, which Nginx still serves while the service or the database cannot answer.

Stop rule

A rule of the pipeline that ends a run, with no script written, when a table loses too many rows.

Story page

One page of chapter 5 that sets out a story: its point, its card, the story and its criteria, the tests to write first, and the worked answer's code.

Story point

The measure of a story's size: about one working day, tests included.

Symbol layer

A layer of MapLibre's style that draws an icon or a text at each point.

Test

A small method that calls code with chosen inputs and checks the answer.

Test platform

The small runner built into each test project, Microsoft.Testing.Platform, which makes the project a program that runs its own tests.

Testcontainers

A library that starts a container from inside a test, such as a real PostgreSQL, and removes it afterwards.

Test-first development

Writing a test and watching it fail before writing the code that makes it pass, then tidying the code while the test stays green.

Theory (xUnit)

An xUnit test marked `[Theory]`, run once for each row of data above it.

Threat model

A list of what could harm a system, where each threat would strike, and the control that answers it.

Tile

One square of the map at one zoom level, named by three numbers, z/x/y, such as 12/3339/1788.

Tile pyramid

The tiles of every level stacked together: one at level 0, four at level 1, sixteen at level 2, and so on.

TLS

The encryption behind HTTPS, which stops anyone on the way from reading or changing a request. Study plan 6 adds it to Tessera.

Toolchain

The set of tools that turns source code into a running, tested service, each resting on the ones beneath it.

Traceability

The links from each requirement back to where it came from and forward to the test that proves it.

Transient service

An object ASP.NET Core's container makes anew each time any part asks for it.

Trust boundary

A line across which data must be checked, as a passport is checked at a border.

Type parameter

A placeholder for a type, such as `T` in `Result<T, E>`, that each use fills in.

Union

A type whose values come in a fixed set of cases. C# 14 has none, so Tessera builds one from records and the `[Closed]` attribute.

Unit test

A test that calls one function with plain values and needs nothing but the language itself.

Update function

The pure function at the heart of the map's page: given the old model and a message, it returns the new model.

Upstream

In Nginx, the named group of servers that requests go on to; Tessera's holds the two copies of the service.

User story

A short statement of one thing a person needs, in the form 'As …, I want …, so that …', with criteria that say when it is done.

Vector tile

A tile that holds shapes and their properties rather than a picture, so the browser draws them in any style.

Walking skeleton

The thinnest version of the whole service that works from end to end, on which every later story builds.

Web Mercator

The projection nearly every web map uses, which unrolls the globe into one square; its code number is EPSG:3857.

WGS 84

The system of longitude and latitude in degrees that GPS uses; its code number is 4326.

Worked answer

The finished repository, read like the answers at the back of a mathematics book: after you have tried a story, never before.

WSL2

The Windows Subsystem for Linux, a light virtual machine that runs Ubuntu on Windows.

xUnit

The test framework for .NET that finds the methods marked `[Fact]` or `[Theory]` and runs them.

Zone

A piece of memory that every Nginx worker shares, where Nginx counts requests or keeps the names of stored tiles.

Zoom level

How many times the map splits the world into four. At level z the world is 2^z tiles wide and 2^z tiles tall. Every web map counts levels this way, and Tessera serves levels 0 to 22.